

链上套利残酷共学 精选打卡笔记汇总

从 635 篇参与者打卡中精选 54 篇深度笔记

涵盖：套利框架 · AMM 深度 · LI.FI 跨链 · MEV · 实盘验证 · Agent 方法论

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-06

目录

01. Day02打卡链接： (FireLoveInIce, 19426字)
02. 任务目标1： (allen123wzh, 13243字)
03. 开营打卡：我的 21 天链上、DeFi 与套利强化学习计划 (tobin, 12766字)
04. 在看到群里讨论这个方法的时候，我也好奇心驱使，问了AI这个方法，然后用数据验证了一下。以下是本人的记录： (starkxun, 10049字)
05. 7BN 钱包 Pump AMM 交易策略逆向研究 (z0y1, 9916字)
06. 基本概念 (Galaxy9, 9262字)
07. 一、DEX 内套利 (ChnPsych, 9035字)
08. 今天做了套利学习计划书1. Hermes 做了总结 以及推荐路线 (gt66-66, 8802字)
09.
 (ChenXuan1985, 8397字)
10. 链上套利统一判断框架与 21 天学习大纲 (blimnwind-bot, 8146字)
11. 聚合器 1inch Aqua Case study (jonascyang, 8085字)
12. Day 2 · 2026-08-06 · HYPE 美股永续合约 vs 美股非交易时段套利调查 (rslofty, 7660字)
13. 昨晚根据Bruce的教程配置了hermes，然后连接飞书。 (richardxv, 7442字)
14. D3 打卡 · 2026-08-06 (HOU RUI, 7024字)
15. LI.FI 跨链可执行价差 120 轮实测 (cfanboy, 6637字)
16. 链上套利残酷共学 · 5 阶段弹性自学计划 (0xChainWorld, 6624字)
17. 链上套利类型、经典案例与行业现状 (X, 6327字)
18. 从价差到系统工程：理解加密市场套利的路径、架构与完整分类 (x-tan-dev, 5763字)
19. Day 2 · 自己实现 AMM 定价 (Junfan Chen, 5258字)
20. 一、用我自己的话理解"套利" (GeoGu0, 4903字)
21. 今天的整理 (youchen-design, 4488字)
22. 问题：DEX 内、DEX 间、跨链、三角、稳定币、清算与 MEV 等常见套利机会结构分别是什么？它们之间有什么区别？ (A9xYau, 4345字)
23.
 (wyy2221, 3930字)
24. 链上套利学习笔记 Day 1：AMM 第一性原理 —— $x \cdot y = k$ 到底在说什么 (倪响, 3762字)
25. 实验A:只改金额(Arbitrum → Optimism,USDC → USDC) (Kevin Jiang, 3363字)
26. Day 2 | 从 439 条共学笔记中提炼"可证伪的链上套利研究框架" (Lizzie, 3331字)
27. Day 2 · LI.FI 跨链 Route 只读评估 (dyhbrewer, 3168字)
28. 1. 用一句话说清「链上套利」的钱从哪来 (bzd111, 3066字)
29. Day 2 | 从 Transfer 日志到跨链资产净流量 (Zacteh, 2983字)
30. 有。你如果从零开始，建议先学习“区块链和以太坊”，再学DeFi，最后才进入套利。不要一开始买很多所谓“量化套利秘籍”， (zhangjunhaihhh-pixel, 2810字)
31. 一、今日目标 (rakisa, 2793字)
32. 链上套利残酷共学-DAY01 (Theo, 2742字)
33. Day 2 / 21 · AMM 与恒定乘积：把名词变成数字 (clare ma, 2536字)
34. Day 2 学习笔记：LI.FI API 实操 + Route 比较 + 成本模型 (journeyanhk, 2091字)
35. 链上套利共学 Day 2 | 跨所套利框架 M1-M5 全部落地 + 真实套利信号验证 (Cz07cring, 2082字)
36. 今天主要搭建了一套独立、可复用的 Hermes 智能分析平台基础架构。它不属于某一个具体业务，而是可以同时服务多个系统和 (wang wanlu, 1994字)

37. 学习计划 (Leon, 1982字)
38. Day 2 | LI.FI 真實成本實測：25 bps 如何殺死紙上價差 (HenryChang, 1980字)
39. 我今天整理的几篇 Flashbots 文档，分别讲 Auction、交易入口和 searcher Quick Start (bai · 小白, 1727字)
40. RobinHood Chain 套利机会监控与反馈 (gm365, 1672字)
41. 今天把“看见价差”推进到了“可验证、可拒绝、可持续前向观察”的阶段。【完整处理群内信息】通过 CipherTalk 按 (fanchen621, 1649字)
42. 三明治攻击 (hellingercheri-gif, 1602字)
43. 1. 套利的经济学定义 (qingzhou, 1567字)
44. 链上套利地图 (SimonQuant, 1469字)
45. 今日打卡 (elipitic, 1451字)
46. 链上套利共学 Day 1：老鼠仓指纹与光模块制裁 (solaren, 1261字)
47. Day 1 | “币有” Meme 机制复盘：从预测热点转向识别流动性瓶颈 (dami.eth, 1227字)
48. Day 2 | 2026.08.05 | 跨链价差实证：2小时×3资产×5链全扫描 (xzone911, 1191字)
49. 一、什么是 MEV (Rui Pan, 1097字)
50. 今日成就： (Freetrip, 892字)
51. 链上套利残酷共学 Day 3 打卡 (2026-08-06) (KuTear, 850字)
52. Day 2 | 把“高 APR”还原成做市风险：MARS/USDT V3 案例 (Joe, 842字)
53. 一句话：LI.FI 是一个「跨链流动性聚合与编排层」(routing & orchestration layer) —— (Joneloo, 828字)
54. Day 2 — AMM 机制深度解析 (pennchester, 814字)

笔记 01

作者：FireLoveInIce | 时间：2026-08-06 13:12 UTC | 字数：19426

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day02打卡链接：

<<https://github.com/FireLoveInIce/21DaysCoLearnNote/tree/main/day02/note>>

Day 02：Uniswap V2/V3 从建池到撤池的完整调用链

=====

本日主线是 Uniswap V2，因为 Pair、Router02、LP ERC-20 都属于 V2。V3 不存在，但是V3继承了相关思想。

Pair 和同质化 LP Token；对应概念是 Pool、SwapRouter、NonfungiblePositionManager 和代表仓位的 NFT。两套流程不能混用。

1. 今日产出与边界

追踪代币创建后，V2 的 createPair → approve → addLiquidity → quote → swap → removeLiquidity。

整理每一步的外部账户、合约账户、授权对象、资产流和事件。

解释 Factory、Pair、Router02、ERC-20 函数的参数和内部原理。

对照 V3 的建池、初始化、集中流动性、报价、交换和撤出流程。

理清直接调用 Pair、skim/sync、Flash Swap、手续费和 K 的边界。

为后续使用真实交易和 Python 复算链上结果建立验证清单。

必须先纠正一个表述：上面的生命周期通常不是“一笔交易”，而是多笔按顺序执行的交易。

创建两个代币本身还会再增加两笔部署交易。approve 也是独立交易，除非代币支持 permit，或者使用自定义执行合约把允许组合的步骤原子化。一次普通 EOA 调用 Router02 不能同时完成建池、加池、交换和撤池。

2. 账户、角色与信任边界

| 角色/账户 | 类型 | 持有什么 | 调用什么 | 是否有特权 |

| :----- | :----- | :----- | :----- | :----- |

| Token 部署者 | EOA/部署合约 | 初始 Token、ETH | 部署 ERC-20 | 取决于 Token 是否保留 owner/mint/tax 权限 |

| Pair 创建者 | 任意 EOA/合约 | Gas 代币 | Factory.createPair | 没有；谁先创建都不能占有 Pair |

| LP | EOA/合约 | tokenA、tokenB | approve、addLiquidity、removeLiquidity | 没有；按 LP 份额拥有池中资产索取权 |

| Trader | EOA/合约 | 输入 Token | approve、Router swap | 没有 |

| Factory | 合约账户 | Pair 注册表 | 部署和登记 Pair | feeToSetter 可管理协议费接收设置 |

| Pair | 合约账户 | 两种底层 Token；自身也是 LP ERC-20 | mint、burn、swap、skim、sync | Core 状态机；只校验余额和不变量，不信任 Router |

| Router02 | 合约账户 | 正常情况下不长期留存资产 | 编排创建 Pair、转账、mint/burn/swap | 无 owner；用户只授权本次需要的 Token |

| WETH | 合约账户 | 托管 ETH 并发行 WETH | deposit、withdraw | 解决原生 ETH 不是 ERC-20 的问题 |

| 套利者/MEV 搜索者 | EOA/合约 | 执行资金 | 调用任意 Router 或直接调用 Pair | 将偏离的池价拉回；也可能抢跑/夹击宽松滑点交易 |

| RPC/前端 | 链下服务 | 不托管链上状态 | 报价、构造 calldata、广播 | 可能给出错误或过期信息，最终以链上执行为准 |

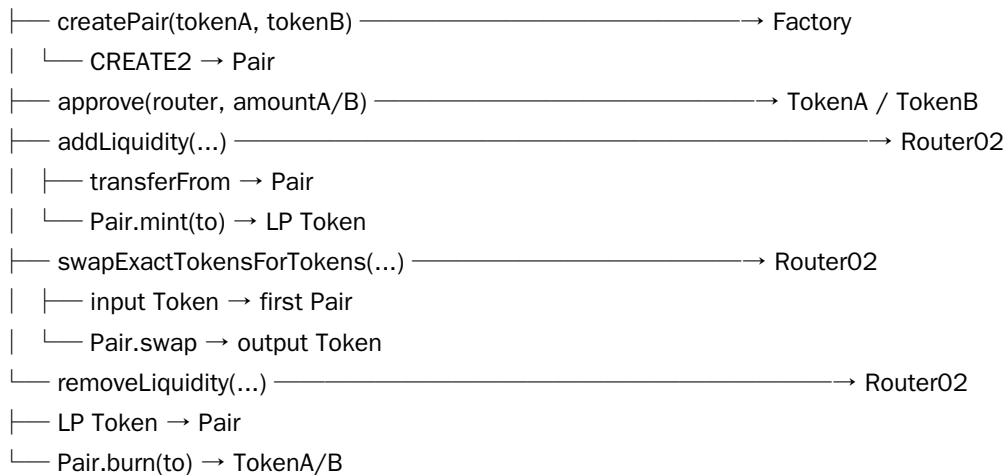
关键安全边界：approve(spender, amount) 的 spender 是 Router02，不是 Factory，也通常不是 Pair。

Router 随后调用 Token 的 transferFrom(user, pair, amount)，Token 合约检查的是 allowance[user][router]。

3. V2 合约关系与资产流

text

用户 EOA



Router02 是无状态编排器，不是资金池。Pair 的 ERC-20 `balanceOf(pair)` 是实际余额，`reserve0/reserve1` 是上次 `_update` 固化的会计储备。Pair 利用两者的差值判断本次输入。

4. 步骤一：代币创建

典型 ERC-20 构造参数包括 `name`、`symbol`、初始供应量和接收者。真正影响 DEX 的不是名称，而是以下行为：

`decimals()`：只影响人类显示和链下换算，合约始终处理最小单位整数。

`balanceOf` / `transfer` / `transferFrom` / `approve`：Router 所依赖的基本接口。

是否收转账税、rebasing、黑名单、暂停交易、最大钱包、可增发：这些都可能破坏标准 Router 的假设。

token 地址：V2 用地址数值升序决定 `token0/token1`，与创建时参数顺序无关。

创建池前应做只读检查：代码是否已验证、总供应量、owner 权限、转账能否正常执行、是否为代理合约，以及是否存在转账税或动态余额。

5. 步骤二：Factory 创建 Pair

5.1 外部调用

solidity

`Factory.createPair(address tokenA, address tokenB) returns (address pair)`

参数：

`tokenA/tokenB`：两个不同的非零 ERC-20 地址；顺序不重要。

`msg.sender`：仅支付 Gas，不成为 owner，也不会自动获得 LP Token。

内部过程：

1. 按地址排序成 `token0 < token1`。
2. 检查两地址不同、`token0 != address(0)`、`getPair[token0][token1] == 0`。
3. 令 `salt = keccak256(abi.encodePacked(token0, token1))`。
4. 用 CREATE2 部署 `UniswapV2Pair`，因此地址可在部署前确定。
5. 调用 `pair.initialize(token0, token1)`；此函数只允许 Factory 调用一次。
6. 双向写入 `getPair`，追加 `allPairs`，发出 `PairCreated(token0, token1, pair, index)`。

创建后 `reserve0 = reserve1 = totalSupply = 0`；“有 Pair”不等于“有流动性”。

Router 的 `_addLiquidity` 在 Pair 不存在时也会代为调用 `createPair`，所以显式创建不是必需步骤。

5.2 同币对能否创建多个 Pair

同一 Factory 中，排序后的 `(token0, token1)` 只能对应一个 Pair；再次调用会以 `UniswapV2: PAIR_EXISTS` 回滚。换一个 Factory 则可以为相同币对创建另一个独立 Pair。

不同 Factory 可以近似理解为不同的 V2 市场或 DEX 部署，但不严格等于不同品牌：同一项目可能在不同链或不同时期部署多个 Factory，任何人也都能部署 V2 Fork。监控时应使用

chainId + factory + pair 作为身份，而不是只看 DEX 名称。

[!IMPORTANT]

套利相关：同币对的独立市场。不同 Factory 下的 A/B Pair 拥有独立储备和价格，因此可能形成跨 DEX 套利。同一 Factory 内也可能通过 $A \rightarrow B$ 与 $A \rightarrow C \rightarrow B$ 的路径差形成三角套利。

V3 在同一 Factory 内就允许 (token0, token1, fee) 多池；V4 在同一 PoolManager 内允许由 currency0/currency1/fee/tickSpacing/hooks 区分多个 Pool。套利的本质不是“品牌不同”，而是独立流动性状态给出了足以覆盖手续费、Gas、价格影响、MEV 与失败风险的不同可成交价格。

6. 步骤三：授权与添加 V2 流动性

6.1 授权

solidity

```
tokenA.approve(router, amountADesired)
```

```
tokenB.approve(router, amountBDesired)
```

授权只是修改 allowance，不会移动资产。无限授权减少后续交易次数，但扩大 Router/集成被攻击时的风险面；学习和测试时建议精确授权。某些旧 Token 修改非零 allowance 前必须先设为零。

6.2 Router02.addLiquidity

solidity

```
addLiquidity(
```

```
tokenA, tokenB,
```

```
amountADesired, amountBDesired,
```

```
amountAMin, amountBMin,
```

```
to, deadline
```

```
) returns (amountA, amountB, liquidity)
```

| 参数 | 原理 |

| :----- | :----- |

| amountADesired/BDesired | 最多希望投入的最小单位数量，不保证全部使用 |

| amountAMin/BMin | 状态变化后仍能接受的最低实际投入量；防止按过差比例加池 |

| to | 接收 LP Token 的地址，可以不同于付款人 |

| deadline | Unix 秒；矿工执行时必须 $\text{block.timestamp} \leq \text{deadline}$ |

Router 的 _addLiquidity：

1. 若 Pair 不存在则创建。
2. 读取储备，并按用户参数顺序映射 reserveA/reserveB。
3. 空池使用全部 desired，初始 LP 用自己的比例定义初始价格。
4. 非空池计算 $\text{amountBOptimal} = \text{amountADesired} \times \text{reserveB} / \text{reserveA}$ ；若合规则使用它，否则反向计算 amountAOptimal。多余 Token 留在用户钱包。
5. 检查两个 minimum；任一不满足则整笔回滚。
6. transferFrom 两种 Token：用户直接转到 Pair。
7. 调用 Pair.mint(to)。

amountBOptimal 的准确含义是：如果完整使用 amountADesired，为了保持当前储备比例应搭配的 B 数量。它不是 Swap 输出，也不是“市场最优价格”。例如储备为 1000 A / 2000 B：

text

用户 desired = 100 A / 250 B

$\text{amountBOptimal} = 100 \times 2000 / 1000 = 200 \text{ B}$

实际投入 = 100 A / 200 B，50 B 留在钱包

用户 $\text{desired} = 100 \text{ A} / 150 \text{ B}$

$\text{amountBOptimal} = 200 \text{ B} > 150 \text{ B}$

$\text{amountAOptimal} = 150 \cdot 1000 / 2000 = 75 \text{ A}$

实际投入 = $75 \text{ A} / 150 \text{ B}$ ，25 A 留在钱包

amountDesired 是上限， amountMin 是执行保护。Router 总是尝试完整使用一侧，再按储备比例减少另一侧，使增加流动性本身不改变池价。

6.3 Pair.mint 的内部原理

Pair 不接收 amount 参数，而是读取实际余额并计算：

text

$\text{amount0} = \text{balance0} - \text{reserve0}$

$\text{amount1} = \text{balance1} - \text{reserve1}$

首次注入：

text

$\text{liquidity} = \text{floor}(\text{sqrt}(\text{amount0} \cdot \text{amount1})) - \text{MINIMUM_LIQUIDITY}$

$\text{MINIMUM_LIQUIDITY} = 1000$

1000 份 LP 永久铸给零地址，使 totalSupply 永不归零并降低某些份额操纵风险。后续注入：

text

```
liquidity = min(  
  floor(amount0 totalSupply / reserve0),  
  floor(amount1 totalSupply / reserve1)  
)
```

最后更新储备，发出 $\text{Mint}(\text{sender}, \text{amount0}, \text{amount1})$ 和 $\text{Sync}(\text{reserve0}, \text{reserve1})$ 。

Mint.sender 通常是 Router，不一定是实际 LP；LP 接收者要结合 LP Token Transfer 判断。

6.4 是否必须通过 Router 添加流动性

不是。Pair 本身公开提供 $\text{mint}(\text{to})$ ，任何地址都可以调用。底层顺序是：

text

$\text{token0} \rightarrow \text{Pair}$

$\text{token1} \rightarrow \text{Pair}$

$\text{Pair.mint}(\text{LP接收者})$

Pair 不接收 $\text{desired}/\text{min}/\text{deadline}$ ，也不会主动从用户钱包拉取 Token，只根据 $\text{balance} - \text{reserve}$ 识别本次两侧增量。Router 的价值是创建/查找 Pair、排序 Token、按当前比例选择实际投入、执行 transferFrom 、检查 $\text{minimum}/\text{deadline}$ 以及处理 ETH/WETH。

不能把直接调用拆成多笔公开交易。Pair 不记录是谁转入 Token；如果两侧都出现未记账增量，任何人都能抢先 $\text{mint}(\text{attacker})$ 把 LP 铸给自己。安全的直接调用必须由自定义执行合约在同一笔交易中完成 $\text{transferFrom} \rightarrow \text{mint} \rightarrow \text{minLiquidity}$ 检查，任一步失败则全部回滚。

$\text{skim}(\text{to})$ 同样是任何人可调用的底层函数：它把 $\text{balance0}-\text{reserve0}$ 和 $\text{balance1}-\text{reserve1}$ 发送给指定地址，但不更新储备； $\text{sync}()$ 则不转账，直接把 reserve 更新为当前 balance 。因此，裸转进 Pair、尚未被 $\text{mint}/\text{swap}/\text{sync}$ 计入储备的 Token 不属于某个地址的存款，可能被第三方取走或利用。

[!IMPORTANT]

套利相关：不平衡加池与未记账余额。绕过 Router 按不同比例转入两种 Token 时，LP 铸造量取两侧份额的较小值，多余一侧不会获得对应 LP，实质上部分捐赠给全体 LP，并会改变池价、制造套利机会。若 $\text{transfer} \rightarrow \text{mint}$ 被拆成多笔交易，搜索者还可能抢先 mint 或 skim ；专业执行必须原子完成并校验 Pair 来源、比例和最低 LP 数量。

7. 步骤四：报价 amountIn / amountOut

V2 单跳、0.3% 输入手续费的精确输入公式：

```
text
amountInWithFee = amountIn 997
amountOut = floor(
amountInWithFee reserveOut
/ (reserveIn 1000 + amountInWithFee)
)
```

已知精确输出反推最少输入：

```
text
amountIn = floor(
reserveIn amountOut 1000
/ ((reserveOut - amountOut) 997)
) + 1
```

末尾 +1 很重要：整数除法向下取整，少这一单位可能无法通过 K 校验。

Router/Library 接口：

quote(amountA, reserveA, reserveB)：不含手续费的同比例换算，用于加池。

getAmountOut(amountIn, reserveIn, reserveOut)：单跳精确输入。

getAmountIn(amountOut, reserveIn, reserveOut)：单跳精确输出。

getAmountsOut(amountIn, path)：逐池把上一跳输出作为下一跳输入。

getAmountsIn(amountOut, path)：从最后一跳向前反推。

这些函数只使用读取时的储备，不是价格承诺，也不是 Oracle。签名到上链之间可能发生状态变化。

quote 的实现只是保持比例：

```
text
amountA / reserveA = amountB / reserveB
amountB = amountA reserveB / reserveA
```

它是 pure 数学函数，不会自己读取 Pair，也不包含 0.3% 手续费、恒定乘积价格影响或多跳逻辑。

因此 quote(100, 1000, 2000) = 200 表示“加池时 100 A 应搭配 200 B”，不表示把 100 A

卖给池子能收到 200 B。真实 Swap 必须使用 getAmountOut，在同样储备下输出会因为手续费和价格影响低于 200 B。

8. 步骤五：通过 Router02 执行 Swap

8.1 精确输入

```
solidity
swapExactTokensForTokens(
amountIn, amountOutMin, path, to, deadline
) returns (uint[] amounts)
```

amountIn：固定支付数量。

amountOutMin：全路径最终至少收到多少；不要简单设 0。

path：至少两个 Token 地址，例如 [A, WETH, B] 代表两跳、两个 Pair。

to：最终输出接收者。

deadline：执行过期保护，不是“必须多久确认”的保证。

内部链路：

1. getAmountsOut 用当前储备计算每一跳 amounts。
2. 要求最终值 $\geq$ amountOutMin。

3. `transferFrom(msg.sender, firstPair, amounts[0])`。
4. `_swap` 遍历 `path`；根据 `token0` 顺序把输出填到 `amount0Out` 或 `amount1Out`。
5. 中间跳的 `to` 是下一 `Pair`，最后一跳的 `to` 才是用户指定接收者。
6. 每个 `Pair` 执行 `swap`、更新储备并发出 `Swap`、`Sync`。

单跳调用栈的职责边界如下。它们都是同一笔 EVM 交易，不是七笔交易：

步骤	执行者	所在函数
计算输出	Router/Library	<code>swapExactTokensForTokens</code> → <code>getAmountsOut</code>
检查最低输出	Router	<code>require(amounts[last] >= amountOutMin)</code>
输入转入 <code>Pair</code>	Router → Token	<code>transferFrom(user, pair, amountIn)</code>
发起底层交换	Router → Pair	<code>Pair.swap(...)</code> 外部调用边界
反推实际输入	Pair	同一个 <code>Pair.swap()</code> 内部
手续费与 <code>K</code> 校验	Pair	同一个 <code>Pair.swap()</code> 内部
更新储备与事件	Pair	<code>Pair.swap()</code> 内调用私有 <code>_update()</code>

`Token` 转账虽然是跨合约调用，仍处在同一调用栈。任意 `require` 失败时，`Router`、`Pair` 和 `Token` 的所有状态变化及日志全部回滚，用户只承担已消耗 `Gas`。只有人为把 `transfer(Pair)` 和 `Pair.swap()` 拆成两笔交易时才失去原子性。

8.2 `Pair.swap`

`solidity`

```
swap(amount0Out, amount1Out, to, data)
```

底层参数是输出而不是输入。`Pair` 先乐观转出 `Token`；若 `data` 非空，再回调 `uniswapV2Call`，这就是 `Flash Swap`。随后读取实际余额，反推 `amount0In/amount1In`：

`text`

```
amount0In = max(balance0 - (reserve0 - amount0Out), 0)
amount1In = max(balance1 - (reserve1 - amount1Out), 0)
```

再校验手续费调整后的恒定乘积：

`text`

```
(balance01000 - amount0In3)
(balance11000 - amount1In3)
= reserve0 reserve1 10002
```

校验失败则包括之前 `Token` 转账在内全部回滚。成功后储备变为实际余额。手续费没有转给 `Router`，而是留在池中，使 `LP` 每份对应的底层资产增长。

`Pair` 不维护“谁转入了多少”的个人存款账本。`Token` 的 `Transfer` 日志能显示来源，但 `Pair` 只观察当前余额、旧 `reserve` 和本次输出。`Pair.swap` 也不替用户决定最小输出：调用者先指定 `amount0Out/amount1Out`，`Pair` 只判断该输出是否得到足够输入支持。请求太多会因 `K` 回滚；请求太少通常成功，额外价值在 `_update` 后成为正式储备，归全体 `LP` 份额所有者，不能再被 `skim`。

8.3 手续费、`K` 与 `kLast`

`Router` 用 `997/1000` 公式预计算输出；`Pair` 则根据真实 `amountIn` 执行调整后余额的 `K` 校验。

所以直接调用 `Pair` 不会绕过 `0.3%` 手续费。手续费不是单独转给 `Router` 或逐个 `LP`，而是留在池中。

每次 `Swap` 的右侧基准是函数开始时读取的 `_reserve0 _reserve1`，已经包含此前交易积累的手续费。成功后 `_update` 写入新储备，下一笔交易以新的乘积为基准。在连续 `Swap`、没有加减流动性时，原始 `reserve0 reserve1` 通常因手续费和整数取整增长；手续费调整后的不变量被要求不能下降。

`kLast` 是另一个变量：它记录最近一次 `mint/burn` 流动性事件后的储备乘积，只用于可选协议费 `_mintFee`，不是每次 `Swap` 的校验基准。`V2 LP` 也没有单独 `collectFees`：手续费与本金混在储备中，

只能通过 burn LP 按比例取回。撤池会同时减少两侧储备，所以绝对 K 下降约为 $K(1 - \text{withdrawnShare})^2$ ，但不会“回到初始 K”；剩余手续费仍按 LP 份额留在池中。

8.4 直接调用 Pair.swap

不强制经过 Router。自定义合约可以在一笔交易中完成：

text

验证可信 Factory 与 Pair

→ 读取 token0/token1 和 reserve

→ 使用整数公式计算 amountOut

→ 输入 Token 转入 Pair

→ Pair.swap

→ 检查实际输出或最低利润

同样储备、实际输入、路径和手续费下，直接调用与 Router 得到的最大输出相同；直接调用可能减少通用编排 Gas，但必须自己承担 Pair 验证、排序、多跳、WETH、deadline、滑点、税币与回滚保护。

[!!IMPORTANT]

套利相关：直接 Pair 执行。套利与清算合约经常直接调用 Pair，以固定路径、减少通用 Router 开销，并让第一池输出直接进入下一池。但优势主要是原子组合和潜在 Gas 节省，不是绕过手续费或获得更优 AMM 价格。绝不能用两笔交易完成 transfer → swap，否则未记账输入可能被抢先利用。

8.5 Flash Swap

Flash Swap 不是另一个函数，而是官方 Pair.swap(amount0Out, amount1Out, to, data) 在

data.length > 0 时的内置模式：Pair 先乐观转出 Token，再调用接收合约的

uniswapV2Call；回调结束前必须支付足够的一种或两种 Token，随后仍执行同一套余额、手续费和 K 校验。不足则连同最初输出一起原子回滚。

Uniswap 部署的 Pair 提供能力，套利者或协议自行部署实现 callback 的执行合约。普通钱包不能直接承担 callback。

[!!IMPORTANT]

套利相关：Flash Swap。它允许执行合约先使用 Pair 的资产在另一 DEX 套利、执行清算或完成抵押品置换，再在同一交易回调中偿还。它降低预置本金要求，但不会创造利润；净收益仍必须覆盖 Pair 费用、另一市场费用、Gas、价格影响、MEV 和失败风险。callback 必须验证 msg.sender 是可信 Factory 登记的真实 Pair，防止伪造回调骗取资产。

8.6 精确输出

swapTokensForExactTokens(amountOut, amountInMax, path, to, deadline) 固定最终输出、限制最大输入。

Router 用 getAmountsIn 逆向报价，但实际执行仍从第一跳向最后一跳转账。

9. 步骤六：移除 V2 流动性

先授权 LP Token：

solidity

pair.approve(router, liquidity)

再调用：

solidity

removeLiquidity(

tokenA, tokenB, liquidity,

amountAMin, amountBMin,

to, deadline

) returns (amountA, amountB)

内部过程：

1. Router 用 `pair.transferFrom(user, pair, liquidity)` 把 LP Token 送回 Pair 自身。
2. 调用 `Pair.burn(to)`。
3. Pair 以实时 Token 余额和 LP 总供应量计算：
 $\text{amount0} = \text{liquidity balance0} / \text{totalSupply}$ ，另一边同理。
4. 销毁 Pair 自身持有的 LP，向 to 转出两种 Token。
5. 再读余额、更新储备，发出 Burn 和 Sync。
6. Router 把 `token0/token1` 结果映射回用户的 A/B 顺序，并检查 minimum。

撤池拿回的是当前储备构成，不是最初存入数量；其差异来自交易、手续费、价格变化和无常损失。

10. V3 对照：相同目标，不同模型

目标	V2	V3
池身份	<code>(token0, token1)</code>	<code>(token0, token1, fee)</code> ；同一币对可有多费率池
创建	<code>Factory.createPair</code>	<code>Factory.createPool(tokenA, tokenB, fee)</code>
首次定价	首次按两币注入比例	<code>Pool.initialize(sqrtPriceX96)</code> ，必须显式初始化
LP 凭证	Pair 自身发行同质化 ERC-20	<code>PositionManager</code> 发行 NFT <code>tokenId</code>
流动性范围	全价格区间	<code>[tickLower, tickUpper)</code> 集中流动性
添加	<code>Router02 addLiquidity</code>	<code>NonfungiblePositionManager.mint / increaseLiquidity</code>
移除	<code>removeLiquidity</code> 调 Pair burn	<code>decreaseLiquidity</code> 后还要 collect
Swap 入口	<code>Router02</code>	<code>SwapRouter.exactInputSingle/exactInput/...</code>
报价	<code>Router view</code> 公式	<code>Quoter/QuoterV2</code> 通过模拟 swap 并 revert 返回结果；不应在链上业务中调用
Core 收款	输入先转入 Pair	<code>Pool</code> 先算 delta，再在 callback 中要求付款

没有变化的基础包括：Factory/Core/Periphery 分层、token0/token1 地址排序、ERC-20 授权、精确输入与精确输出、多跳、最低输出保护、WETH、原子回滚和套利驱动的价格收敛。发生根本变化的是：Pair 政名为 Pool；同币对可按 fee 多池；全局 reserve/K 模型改为 $\text{sqrtPriceX96} + \text{tick} + \text{liquidity}$ ；同质化 LP ERC-20 改为官方 PositionManager 管理的仓位 NFT；手续费可按仓位单独 collect。

V3 没有 V2 的 `getReserves/skim/sync`。误转 Token 到 Pool 不会自动成为某个仓位的 liquidity 或手续费，也没有标准 skim 入口。V3 Core 仍然持有真实 Token，但定价不再由两个总余额的比值决定。

10.1 V3 建池与初始化

solidity

```
factory.createPool(tokenA, tokenB, fee)
pool.initialize(sqrtPriceX96)
```

常用外围函数 `createAndInitializePoolIfNecessary(token0, token1, fee, sqrtPriceX96)` 可以组合两步。

fee：百万分比，例如 $3000 = 0.3\%$ ；必须是 Factory 已启用的费率档。

`sqrtPriceX96`： $\text{sqrt}(\text{token1}/\text{token0}) \cdot 2^{96}$ ，比例使用最小单位，方向严格是 `token1/token0`。

初始价格一旦设置，不能再次 initialize；随后价格由 swap 移动。

10.2 V3 创建仓位

solidity

```
NonfungiblePositionManager.mint(MintParams)
```

核心参数：`token0`、`token1`、`fee`、`tickLower`、`tickUpper`、`amount0Desired`、`amount1Desired`、`amount0Min`、`amount1Min`、`recipient`、`deadline`。

tick 必须落在该费率池 `tickSpacing` 的整数倍上。

当前价格低于区间时仓位只需要 `token0`；高于区间时只需要 `token1`；区间内需要两者。

`amountDesired` 是上限，实际使用量由当前 `sqrtPrice` 和区间决定。

Pool 在 mint 过程中回调 `uniswapV3MintCallback`，PositionManager 验证调用者是正确 Pool 后付款。

返回 tokenId、增加的 liquidity 和实际支付 amount0/amount1。

V3 的 liquidity L 不是 Token 数量，也不能与 V2 LP totalSupply 直接比较。区间内常用关系为：

text

$$\text{amount0} = L (\text{sqrtB} - \text{sqrtP}) / (\text{sqrtP} \text{ sqrtB})$$
$$\text{amount1} = L (\text{sqrtP} - \text{sqrtA})$$

实际合约使用 Q64.96 定点整数和严格的向上/向下取整。

10.3 Tick 与有效流动性 L

Tick 是对数价格坐标：原始单位下 $\text{price}(\text{token1}/\text{token0}) = 1.0001^{\text{tick}}$ ，而 Pool 保存的精确价格是 $\text{sqrtPriceX96} = \text{sqrt}(\text{price}) \cdot 2^{48}$ 。价格可在 Tick 之间连续移动；只有被 LP 选作区间边界的 Tick 才需要初始化，tickSpacing 限制哪些 Tick 可以作为边界。

仓位在 [tickLower, tickUpper) 内才 active。L 不是 Token 数量或美元价值，而是当前价格区间的交易深度；当前有效 L 等于所有覆盖当前 Tick 的仓位 liquidity 之和。L 越大，同样 amountIn 推动价格越少。区间内常用变化关系为：

text

$$\Delta \text{token0} = L (1/\text{sqrtP_after} - 1/\text{sqrtP_before})$$
$$\Delta \text{token1} = L (\text{sqrtP_after} - \text{sqrtP_before})$$

两个初始化 Tick 之间 L 保持不变；跨过边界时应用该 Tick 的 liquidityNet，得到新的有效 L，再计算下一段。因此 V3 没有一个描述整个 Pool 的固定全局 K；仅在单个有效区间内可用虚拟储备理解为 $x_{\text{virtual}} y_{\text{virtual}} = L^2$ 。

[!!IMPORTANT]

套利相关：V3 报价不能只比较显示价格。两个 V3 Pool 即使当前 sqrtPriceX96 接近，当前 Tick 附近的 L 和后续初始化 Tick 分布也可能完全不同。套利计算必须逐段模拟跨 Tick 后的 liquidityNet、费率、价格限制与整数取整，不能仅用池中 Token 总余额或瞬时显示价判断利润。

10.4 V3 Swap 与 callback 付款

solidity

```
SwapRouter.exactInputSingle({
  tokenIn, tokenOut, fee, recipient, deadline,
  amountIn, amountOutMinimum, sqrtPriceLimitX96
})
```

Pool 的底层 swap(recipient, zeroForOne, amountSpecified, sqrtPriceLimitX96, data)：

1. 正数 amountSpecified 表示 exact input，负数表示 exact output。
2. 沿价格方向逐 tick 计算；跨越已初始化 tick 时改变当前有效 liquidity。
3. 收取该池 fee，更新 fee growth、tick、观察值和价格。
4. 先把输出转给 recipient，再回调 Router 的 uniswapV3SwapCallback 收取输入。
5. Router 必须验证 callback 来自 Factory 对应的真实 Pool。

调用关系：

text

用户授权并调用 SwapRouter

→ Router 调用 Pool.swap

→ Pool 逐 Tick 计算精确 amount0Delta/amount1Delta

→ Pool 先发送负数 delta 对应的输出 Token

→ Pool 调用 Router.uniswapV3SwapCallback

→ Router 为正数 delta 付款

→ Pool 比较 callback 前后余额，付款不足则全部回滚

callback 的好处是：Pool 可在完成 Tick 穿越后要求精确付款；无需像 V2 那样预先裸转 Token；付款来源可以是用户、Router、上一跳输出或其他原子化协议；同时支持高效多跳和 exact-output 的反向递归结算。Pool 不绑定官方 Router，任何实现 callback 的合约都可调用，但 callback 合约必须验证调用者是真实 Pool，否则攻击者可伪造 callback 诱导付款。

V3 仍由 Core 强制执行手续费，直接调用 Pool 不能绕过；不同费率 Pool 用 fee growth 为 active LP 记录手续费。V3 普通 swap callback 与独立的 Pool.flash(...) 不同：后者是明确的闪电借贷入口，通过 uniswapV3FlashCallback 归还本金和费用。

$\text{sqrtPriceLimitX96} = 0$ 在外围 Router 中通常被替换为协议允许的极限；生产交易可以设置更严格的价格边界。它是池内价格限制，amountOutMinimum 是最终收到数量限制，二者不能互相替代。

11. 今日核心结论

1. V2 同一 Factory 的同币对只能有一个 Pair；不同 Factory、路径或版本都可能形成独立市场和价差。
2. Router 是无状态外围编排器，Pair 才是持有资产并执行 AMM 校验的 Core；两者都没有普通用户特权。
3. transfer 移动调用者自己的 Token，transferFrom 依赖 owner 对当前 spender 的 allowance。
4. quote 只做储备比例换算；getAmountOut/getAmountIn 才包含手续费和恒定乘积价格影响。
5. Pair 不记录转账者存款；标准 Router 在同一交易中完成输入转账、Pair.swap、K 校验和储备更新。
6. Router 负责手续费报价和滑点保护，Pair 通过调整后余额强制执行手续费与 K，直接调用无法绕过。
7. Swap 手续费进入储备，LP 没有独立 collect；每次 Swap 使用包含此前手续费的新储备乘积作为基准。
8. skim/mint/sync 都可公开调用，裸转 Pair 的未记账余额没有个人归属；直接执行必须原子化。
9. V3 用 Pool、Tick、有效 L 和 Position NFT 取代 Pair、全局储备 K 和同质化 LP Token。
10. V3 callback 让 Pool 在逐 Tick 计算后精确收款，并支持多跳、exact-output 和原子组合；回调必须认证。

笔记 02

作者：allen123wzh | 时间：2026-08-05 15:42 UTC | 字数：13243

来源：<https://intensivecolearn.org/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

任务目标1：

1. Uniswap v3 pool分tick的扫矿，具体为：输入pool address，通过查询合约的sqrtPriceX96与L值，计算当前tick的流动性，并推导从0到 $+\infty$ 范围内所有tick的流动性。需要详细了解每个pool的流动性分布来查询让价格变动到底需要多大资金，会导致多少滑点

2. 对于ve33类矿池，由于rewards每周总量固定，并且按比例线性发放给当前active tick内的所有liquidity，所以可以推导投入资金后池子的apr怎么变化

3. 将上述扫矿+ apr推导做成小工具网站，输入pool address，输入预计投放的资金，计算apr

4. 研究vfat.io (<http://vfat.io>)的开源资料，毕竟是自己用了几年的最好用的defi跨链扫矿小工具

1. 2. 已了解原理并完成代码sketch，目前可以通过脚本获取tick, tickSpacing, sqrtPriceX96, liquidity这四个值，然后推导每个tick的流动性。具体见chatgpt总结如下：

Uniswap V3 / CL Pool：Per-Tick 流动性查询与计算方法

目的：从任意 Uniswap V3 / Slipstream / 兼容 CL Pool 的链上状态，推导当前 tick 及其前后若干 tick 区间的 Active Liquidity、TOKEN0 数量、TOKEN1 数量和当前 USD spot value。

本文使用通用占位符，不包含任何具体 Pool Address、交易对或私人交易信息。

1. 推导 Per-Tick 流动性需要查询哪些值

设目标池地址为：

POOL="`<POOL_ADDRESS>`"

RPC="`<RPC_URL>`"

核心需要读取以下链上数据。

1.1 tickSpacing()

tickSpacing() returns (int24)

用途：确定每个可初始化 tick 的间隔。

例如：

tickSpacing = 200

则区间依次为：

[..., 101000–101200, 101200–101400, 101400–101600, ...]

1.2 slot0()

典型接口：

slot0()

returns (

uint160 sqrtPriceX96,

int24 tick,

uint16 observationIndex,

uint16 observationCardinality,

uint16 observationCardinalityNext,

bool unlocked

)

实际计算主要需要：

sqrtPriceX96

tick

用途：

tick：判断当前价格位于哪个 active tick 区间；

sqrtPriceX96：取得当前精确价格，不要只使用整数 tick 近似当前价格。

当前平方根价格：

\$\$

$$S = \frac{\text{sqrtPriceX96}}{2^{96}}$$

\$\$

其中 S 是基于 token raw units 的：

\$\$

$$S = \sqrt{\frac{\text{TOKEN1}}{\text{TOKEN0}}}$$

\$\$

1.3 liquidity()

liquidity() returns (uint128)

它返回：

当前价格所在 tick 区间的 Active Liquidity L_0

这是推导整个附近 tick liquidity distribution 的锚点。

注意：

liquidity() 不是 Pool TVL；

liquidity() 不是 TOKEN0 或 TOKEN1 数量；

liquidity() 也不是某个 LP Position 的 NFT liquidity；

它表示当前价格处所有 active positions 聚合后的 liquidity。

1.4 ticks(int24)

典型接口：

ticks(int24)

returns (

```
uint128 liquidityGross,  
int128 liquidityNet,  
uint256 feeGrowthOutside0X128,  
uint256 feeGrowthOutside1X128,  
int56 tickCumulativeOutside,  
uint160 secondsPerLiquidityOutsideX128,  
uint32 secondsOutside,  
bool initialized
```

)

推导 tick distribution 时，最重要的是：

liquidityGross

liquidityNet

initialized

其中真正用于逐格还原 Active Liquidity 的是：

liquidityNet

liquidityGross 与 liquidityNet 的区别

不要把 liquidityGross 当成某个 tick 区间的 liquidity。

liquidityGross(t)：

所有以 tick t 作为 lower 或 upper boundary 的 Position，其 liquidity 绝对值之和。

liquidityNet(t)：

当价格从低 tick 向高 tick 穿过 t 时，Active Liquidity 的净变化量。

因此：

\$\$

$$L_{\text{after crossing } t} = L_{\text{before crossing } t} + \text{liquidityNet}(t)$$

\$\$

这是还原 per-tick liquidity distribution 的核心关系。

1.5 TOKEN0 / TOKEN1 和 decimals

为了把 liquidity 换算成真实币数，还应读取：

token0()

token1()

然后分别查询两个 ERC-20：

decimals()

symbol()

原因是 Uniswap V3 的数学使用 raw token units。

最终需要：

TOKEN0 address

TOKEN1 address

TOKEN0 decimals = d0

TOKEN1 decimals = d1

如果两个 token 恰好都是 18 decimals，计算和展示会比较直观；如果 decimals 不同，则必须分别按各自 decimals 还原数量。

2. 如何使用 cast 查询这些值

推荐使用 Foundry cast，只读调用：

不需要私钥；

不消耗 gas；

不发送交易。

2.1 固定同一个最新 Block

所有读取必须尽量来自同一个 block：

```
BLOCK=$(cast block-number --rpc-url "$RPC")
```

```
echo "BLOCK=$BLOCK"
```

原因：

如果分别用 latest 查询，而中间刚好发生 Swap / Mint / Burn，则可能把不同 block 的：

slot0

liquidity

ticks

混在一起，产生错误结果。

2.2 查询 tickSpacing

```
cast call "$POOL"
```

```
"tickSpacing()(int24)"
```

```
--block "$BLOCK"
```

```
--rpc-url "$RPC"
```

2.3 查询 slot0

```
cast call "$POOL"
```

```
"slot0()(uint160,int24,uint16,uint16,uint16,bool)"
```

```
--block "$BLOCK"
```

```
--rpc-url "$RPC"
```

重点记录前两行：

```
sqrtPriceX96
```

```
currentTick
```

2.4 查询当前 Active Liquidity

```
cast call "$POOL"
```

```
"liquidity()(uint128)"
```

```
--block "$BLOCK"
```

```
--rpc-url "$RPC"
```

记为：

```
$$
```

```
L_0
```

```
$$
```

2.5 查询 TOKEN0 / TOKEN1

```
TOKEN0=$(cast call "$POOL"
```

```
"token0()(address)"
```

```
--block "$BLOCK"
```



```
--rpc-url "$RPC")
TOKEN1=$(cast call "$POOL"
"token1()(address)"
--block "$BLOCK"
--rpc-url "$RPC")
```

```
echo "TOKEN0=$TOKEN0"
echo "TOKEN1=$TOKEN1"
```

查询 decimals :

```
cast call "$TOKEN0" "decimals()(uint8)" --rpc-url "$RPC"
cast call "$TOKEN1" "decimals()(uint8)" --rpc-url "$RPC"
```

可选：查询 symbol :

```
cast call "$TOKEN0" "symbol()(string)" --rpc-url "$RPC"
cast call "$TOKEN1" "symbol()(string)" --rpc-url "$RPC"
```

3. 确定当前 Active Tick

设：

```
currentTick = T
tickSpacing = s
```

当前区间：

```
$$
[text{LOWER},text{UPPER})
$$
```

其中：

```
$$

$$\text{LOWER} = \text{leftlfloor} \frac{T}{s} \text{righttrfloor } s$$

$$
$$

$$\text{UPPER} = \text{LOWER} + s$$

$$
```

注意：Bash 的整数除法对负数是向 0 截断，不是数学意义上的 floor。

为了兼容负 tick，推荐：

```
Q=$(( TICK / SPACING ))
R=$(( TICK % SPACING ))
if (( TICK < 0 && R != 0 )); then
Q=$(( Q - 1 ))
fi
LOWER=$(( Q * SPACING ))
UPPER=$(( LOWER + SPACING ))
```

4. 如何从 liquidityNet 推导每个 Tick 的 Active Liquidity

假设当前 active range 为：

tick 0 = [LOWER, LOWER + s]

且：

\$\$

$L_0 = \text{liquidity}()$

\$\$

4.1 往上推导

下一格：

\$\$

$L_{+1} = L_0 + \text{liquidityNet}(\text{LOWER} + s)$

\$\$

再下一格：

\$\$

$L_{+2} = L_{+1} + \text{liquidityNet}(\text{LOWER} + 2s)$

\$\$

一般形式：

\$\$

$L_{+n} = L_{+(n-1)} + \text{liquidityNet}(\text{LOWER} + ns)$

\$\$

4.2 往下推导

前一格：

\$\$

$L_{-1} = L_0 - \text{liquidityNet}(\text{LOWER})$

\$\$

再前一格：

\$\$

$L_{-2} = L_{-1} - \text{liquidityNet}(\text{LOWER} - s)$

\$\$

一般形式：

\$\$

$L_{-n} = L_{-(n-1)} - \text{liquidityNet}(\text{LOWER} - (n-1)s)$

\$\$

减号的原因是：

$\text{liquidityNet}(t)$ 定义的是从低价格向高价格穿越 boundary 时的变化量。

我们从当前区间反向走到低价格区间，因此要逆运算。

5. 查询当前 Tick ± 5 格所需的 Boundary

如果目标是：

tick -5

tick -4

tick -3

tick -2

tick -1

CURRENT

tick +1

tick +2

tick +3

tick +4

tick +5

总计 11 个区间。

已知当前 L_0 后，只需要读取 10 个 boundary 的 liquidityNet。

设当前 lower boundary 为 LOWER。

需要查询：

LOWER - 4s

LOWER - 3s

LOWER - 2s

LOWER - 1s

LOWER

LOWER + 1s

LOWER + 2s

LOWER + 3s

LOWER + 4s

LOWER + 5s

也就是：

```
for OFFSET in {-4..5}; do
```

```
BOUNDARY=$(( LOWER + OFFSET SPACING ))
```

```
cast call "$POOL"
```

```
"ticks(int24)(uint128,int128,uint256,uint256,int56,uint160,uint32,bool)"
```

```
"$BOUNDARY"
```

```
--block "$BLOCK"
```

```
--rpc-url "$RPC"
```

```
done
```

为了减少输出，可以只提取前三个关键字段：

```
echo "BOUNDARY | liquidityGross | liquidityNet | initialized"
```

```
for OFFSET in {-4..5}; do
```

```
BOUNDARY=$(( LOWER + OFFSET SPACING ))
```

```
OUT=$(cast call "$POOL"
```

```
"ticks(int24)(uint128,int128,uint256,uint256,int56,uint160,uint32,bool)"
```

```
"$BOUNDARY"
```

```
--block "$BLOCK"
```

```
--rpc-url "$RPC")
```

```
GROSS=$(echo "$OUT" | sed -n '1p' | awk '{print $1}')
```

```
NET=$(echo "$OUT" | sed -n '2p' | awk '{print $1}')
```

```
INIT=$(echo "$OUT" | sed -n '8p' | awk '{print $1}')
```

```
echo "$BOUNDARY | $GROSS | $NET | $INIT"
```

```
done
```

6. 从 L 换算每个 Tick 区间内 TOKEN0 / TOKEN1 数量

这是第二个核心步骤。

对于任意 tick 区间：

\$\$

[a,b]

\$\$

定义：

\$\$

$S_a = \sqrt{1.0001^a}$

\$\$

\$\$

$S_b = \sqrt{1.0001^b}$

\$\$

当前精确平方根价格：

\$\$

$S = \frac{\text{sqrtPriceX96}}{2^{96}}$

\$\$

所有公式首先计算的是 raw token amounts。

6.1 当前价格以下的 Tick

如果整个区间都位于当前价格下方：

\$\$

$b \leq \text{currentTick}$

\$\$

那么该区间对应的 liquidity 已经全部表现为 TOKEN1：

\$\$

$\text{amount0}_{\{\text{raw}\}} = 0$

\$\$

\$\$

$\text{amount1}_{\{\text{raw}\}} = L(S_b - S_a)$

\$\$

6.2 当前 Active Tick

如果：

\$\$

$a \leq \text{currentTick} < b$

\$\$

那么这个区间同时持有 TOKEN0 与 TOKEN1：

\$\$

$\text{amount0}_{\{\text{raw}\}} = L \left(\frac{1}{S} - \frac{1}{S_b} \right)$

\$\$

\$\$

$\text{amount1}_{\{\text{raw}\}} = L(S - S_a)$

\$\$

注意：

当前 Active Tick 必须使用 sqrtPriceX96 得到的精确 S，不要直接用整数 tick 推算当前价格。

整数 tick 只能说明价格位于：

\$\$

$1.0001^{\text{tick}} \leq P < 1.0001^{\text{tick}+1}$

\$\$

而不能给出 tick 内部的精确位置。

6.3 当前价格以上的 Tick

如果整个区间位于当前价格上方：

\$\$

$a > \text{currentTick}$

\$\$

则该区间对应 liquidity 全部表现为 TOKEN0：

\$\$

$\text{amount0}_{\text{raw}} = L \cdot \left(\frac{1}{S_a} - \frac{1}{S_b} \right)$

\$\$

\$\$

$\text{amount1}_{\text{raw}} = 0$

\$\$

7. Token Decimals 处理

得到 raw amount 后：

\$\$

$\text{amount0} = \frac{\text{amount0}_{\text{raw}}}{10^{d_0}}$

\$\$

\$\$

$\text{amount1} = \frac{\text{amount1}_{\text{raw}}}{10^{d_1}}$

\$\$

其中：

$d_0 = \text{TOKEN0 decimals}$

$d_1 = \text{TOKEN1 decimals}$

关于 Liquidity L 的展示

liquidity() 本身是 Uniswap V3 的数学 liquidity 数值，不应机械理解成某种 token 数量。

如果 TOKEN0 与 TOKEN1 decimals 相同，例如都为 18，则为了阅读方便，可以把：

$L_{\text{raw}} / 1e18$

作为展示值。

但如果两个 token decimals 不同，则不存在一个通用的“除以 1e18”规则。

这时建议：

内部计算始终使用 L_{raw} ；

表格中的 Active Liquidity (L) 可以保留 raw integer，或明确注明使用了何种 display normalization；

不要把经过人为缩放的 L 当成协议层原始 liquidity。

8. 如何计算当前 Spot Value

首先从 sqrtPriceX96 得到 raw price：

\$\$

$P_{\text{raw}} = S^2$

\$\$

即：

\$\$

$$P_{\text{raw}} = \frac{\text{TOKEN1}_{\text{raw}}}{\text{TOKEN0}_{\text{raw}}}$$

\$\$

考虑 decimals 后，人类可读价格：

\$\$

$$P_{\text{TOKEN1 per TOKEN0}} = P_{\text{raw}} \times 10^{d_0 - d_1}$$

\$\$

如果已知 TOKEN0 的 USD spot price：

\$\$

USD_0

\$\$

则 TOKEN1 的池内理论 USD price：

\$\$

$$\text{USD}_1 = \frac{\text{USD}_0}{P_{\text{TOKEN1 per TOKEN0}}}$$

\$\$

这样做的优点：

TOKEN0 / TOKEN1 的 USD value 与该 block 的池内价格完全自治，不会因为两个外部行情源更新时间不同产生小幅价差。

分别计算：

\$\$

$$\text{Value}_0 = \text{amount}_0 \times \text{USD}_0$$

\$\$

\$\$

$$\text{Value}_1 = \text{amount}_1 \times \text{USD}_1$$

\$\$

总值：

\$\$

$$\text{Value}_{\text{total}} = \text{Value}_0 + \text{Value}_1$$

\$\$

也可以表达成 TOKEN0 spot equivalent：

\$\$

$$\text{TOKEN0}_{\text{equiv}} = \text{amount}_0 + \frac{\text{amount}_1}{P_{\text{TOKEN1 per TOKEN0}}}$$

\$\$

9. 最终建议表格格式

输出格式统一为：

TOKEN0 类资产：保留 2 位小数；

TOKEN1：取整数；

Active Liquidity L：取整数；

USD Value：取整数；

只有 USD Value 加粗；

USD Value 与 token 数量换行显示；

USD Value 放在括号内，例如 (\$3,786)。

公开文档使用匿名 TOKEN0 / TOKEN1 :

Tick 区间	Active Liquidity L	TOKEN0	TOKEN1	当前 spot 等值
tick -2 · [A-2s, A-s]	x x x	0	xxx (\$)	$\approx$ TOKEN0 (\$)
tick -1 · [A-s, A]	x x x	0	TOKEN1 (\$)	$\approx$ TOKEN0 (\$)
当前 · [A, A+s]	x x x	TOKEN0 (\$)	TOKEN1 (\$)	$\approx$ TOKEN0 (\$)
tick +1 · [A+s, A+2s]	x x x	TOKEN0 (\$)	0	$\approx$ TOKEN0 (\$)
tick +2 · [A+2s, A+3s]	x x x	TOKEN0 (\$)	0	$\approx$ TOKEN0 (\$)

其中 :

A = 当前 active range 的 lower tick

s = tickSpacing

10. 推荐的一次性查询脚本

下面这段脚本可以一次取得推导 ± 5 tick distribution 所需的主要链上数据。

RPC="<RPC_URL>"

POOL="<POOL_ADDRESS>"

固定同一个 block

BLOCK=\$(cast block-number --rpc-url "\$RPC")

tick spacing

SPACING=\$(cast call "\$POOL"

"tickSpacing()(int24)"

--block "\$BLOCK"

--rpc-url "\$RPC" | awk '{print \$1}')

slot0

SLOT0=\$(cast call "\$POOL"

"slot0()(uint160,int24,uint16,uint16,uint16,bool)"

--block "\$BLOCK"

--rpc-url "\$RPC")

SQRT_PRICE=\$(echo "\$SLOT0" | sed -n '1p' | awk '{print \$1}')

TICK=\$(echo "\$SLOT0" | sed -n '2p' | awk '{print \$1}')

正确处理负 tick

Q=\$((TICK / SPACING))

R=\$((TICK % SPACING))

if ((TICK < 0 && R != 0)); then

Q=\$((Q - 1))

fi

LOWER=\$((Q SPACING))

UPPER=\$((LOWER + SPACING))

当前 liquidity

CURRENT_L=\$(cast call "\$POOL"

"liquidity()(uint128)"

--block "\$BLOCK"

--rpc-url "\$RPC" | awk '{print \$1}')

token addresses

TOKEN0=\$(cast call "\$POOL"

"token0()(address)"

```

--block "$BLOCK"
--rpc-url "$RPC")

TOKEN1=$(cast call "$POOL"
"token1()(address)"
--block "$BLOCK"
--rpc-url "$RPC")

echo "====="
echo "BLOCK=$BLOCK"
echo -n "TIMESTAMP="
cast block "$BLOCK" --field timestamp --rpc-url "$RPC"

echo "tickSpacing=$SPACING"
echo "currentTick=$TICK"
echo "sqrtPriceX96=$SQRT_PRICE"
echo "currentLiquidity=$CURRENT_L"
echo "activeRange=[$LOWER,$UPPER]"
echo "TOKEN0=$TOKEN0"
echo "TOKEN1=$TOKEN1"

echo -n "TOKEN0 decimals="
cast call "$TOKEN0" "decimals()(uint8)" --rpc-url "$RPC" | awk '{print $1}'

echo -n "TOKEN1 decimals="
cast call "$TOKEN1" "decimals()(uint8)" --rpc-url "$RPC" | awk '{print $1}'

echo "====="
echo
echo "BOUNDARY | liquidityGross | liquidityNet | initialized"

±5 ranges 需要的 10 个 boundary
for OFFSET in {-4..5}; do
BOUNDARY=$(( LOWER + OFFSET SPACING ))

OUT=$(cast call "$POOL"
"ticks(int24)(uint128,uint128,uint256,uint256,int56,uint160,uint32,bool)"
"$BOUNDARY"
--block "$BLOCK"
--rpc-url "$RPC")

GROSS=$(echo "$OUT" | sed -n '1p' | awk '{print $1}')
NET=$(echo "$OUT" | sed -n '2p' | awk '{print $1}')
INIT=$(echo "$OUT" | sed -n '8p' | awk '{print $1}')

echo "$BOUNDARY | $GROSS | $NET | $INIT"
done

```

把这段输出保存下来，就具备了重建当前 tick ± 5 格 liquidity distribution 所需的全部核心链上状态。

11. 方法论总结

完整流程可以压缩成：

1. 固定 BLOCK

↓

2. 读取 tickSpacing

↓

3. $\text{slot0} \rightarrow \text{currentTick} + \text{sqrtPriceX96}$
↓
4. $\text{liquidity}() \rightarrow$ 当前区间 $L0$
↓
5. $\text{ticks}(\text{boundary}).\text{liquidityNet}$
↓
6. 从 $L0$ 向上下逐格递推 L_i
↓
7. 用 tick boundary 计算 S_a / S_b
↓
8. 用 sqrtPriceX96 计算当前精确 S
↓
9. $L_i + S_a/S/S_b \rightarrow \text{TOKEN0} / \text{TOKEN1 raw amount}$
↓
10. 根据 token decimals 转成人类可读数量
↓
11. 用同一时点 USD spot price 标记 USD value
↓
12. 输出 Per-Tick Liquidity Distribution 表

三个最重要的原则

1. 所有状态读取尽量固定在同一个 block。
2. Per-tick Active Liquidity 必须使用 $\text{liquidity}() + \text{liquidityNet}$ 递推，不能用 liquidityGross 代替。
3. 当前 active tick 的资产数量必须使用 sqrtPriceX96 ，不要仅靠整数 tick 反推当前精确价格。

按这套方法，可以从链上状态直接重建任意兼容 Uniswap V3 集中流动性池在当前价格附近的 per-tick liquidity distribution。

笔记 03

作者：tobin | 时间：2026-08-05 02:35 UTC | 字数：12766

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

开营打卡：我的 21 天链上、DeFi 与套利强化学习计划

今天完成了学习目标梳理和 21 天执行计划。以下是后续每天的学习、编码与验收安排。

21 天链上、DeFi 与套利强化学习计划

适用对象：有资深通用编程经验，但链上、DeFi 和 MEV 经验较少的开发者。

时间预算：每天 5–6 小时，共约 115 小时。学习主线采用 TypeScript + viem；合约和主网 fork 使用 Solidity + Foundry。

21 天后的目标

完成本计划后，应该能够：

1. 准确解释 EVM 交易从签名、广播、执行、排序、打包到确认/最终性的全过程。
2. 使用 JSON-RPC 和 ABI 读取区块、交易、日志及合约状态，并理解同一区块状态一致性。
3. 计算 Uniswap V2 类 AMM 的报价、手续费、价格影响和双池套利净利润。
4. 理解 Uniswap V3 的集中流动性、tick、流动性区间及跨 tick 交易，不再把 AMM 与订单簿混淆。
5. 理解借贷、超额抵押、预言机、清算、闪电贷，以及稳定币、LST、金库、衍生品、桥和治理的基本结构。
6. 独立实现一个只读、实时、可回放的 V2-compatible 跨 DEX 套利扫描器。
7. 理解 frontrun、backrun、sandwich、builders、bundles 和私有交易通道对真实利润的影响。

8. 使用 Foundry 在固定主网 fork 上验证原子双池交易和最小利润保护。

这不是“21 天成为成熟实盘搜索者”的承诺。生产级 V3 跨 tick 报价器、跨链套利、私有 order flow、低延迟基础设施、合约审计和稳定实盘仍需要后续训练。

每日固定节奏

90 分钟：必读资料，边读边写自己的术语和公式。

150–180 分钟：编码或链上实验。

45–60 分钟：测试、对照真实链上结果、修正理解。

30 分钟：回答当日验收题，提交当天代码。

有额外时间时再看“选读”，不要挤占编码和验收。

学习时只使用自己申请的 RPC URL。私钥不得写入源码、命令历史或提交到 Git。

第 1 天：账户、交易和 Gas

必读

Ethereum Accounts (<https://ethereum.org/developers/docs/accounts/>)

Ethereum Transactions (<https://ethereum.org/developers/docs/transactions/>)

Ethereum Gas and Fees (<https://ethereum.org/developers/docs/gas/>)

交易是由账户签名的状态变更指令；交易需要费用并进入有效区块。Gas 费用按实际使用的 gas 计费，即使交易失败也要支付已经消耗的执行成本。Ethereum Gas (<https://ethereum.org/developers/docs/gas/>)

动手

任选三笔成功交易和一笔失败交易，记录：

from、to、nonce、value、input

gas、gasUsed、effectiveGasPrice

maxFeePerGas、maxPriorityFeePerGas

status 和实际手续费

验收

能解释 gas limit、gas used、base fee、priority fee 的区别。

能明确回答：revert 会回滚状态，但不会退回已经消耗的 gas 费用。

第 2 天：区块、PoS、确认和重组

必读

Ethereum Blocks (<https://ethereum.org/developers/docs/blocks/>)

Proof of Stake (<https://ethereum.org/developers/docs/consensus-mechanisms/pos/>)

Block Proposal (<https://ethereum.org/developers/docs/consensus-mechanisms/pos/block-proposal/>)

动手

连续读取 20 个区块，输出 number、hash、parentHash、timestamp、gasUsed、baseFeePerGas 和交易数量，检查父子哈希关系。

验收

能区分 proposed、safe、finalized。

能解释 mempool 为什么不是“链上的 pool”，并说明有些交易可能通过私有通道而不进入公开 mempool。

能说明扫描器为什么必须处理漏块、重复块和小概率重组。

第 3 天：EVM、调用栈、ABI、日志和回滚

必读

Ethereum Virtual Machine (<https://ethereum.org/developers/docs/evm/>)

Solidity: Introduction to Smart Contracts

(<https://docs.soliditylang.org/en/latest/introduction-to-smart-contracts.html>)

Solidity ABI Specification (<https://docs.soliditylang.org/en/latest/abi-spec.html>)

动手

手工识别一笔 ERC-20 transfer 的 4-byte selector 和 calldata 参数。

从交易 receipt 的 logs 中解码 Transfer(address,address,uint256)。

找一笔 revert 交易，对照 receipt 与 trace 判断它执行了什么。

可使用 Geth 官方的 EVM Tracing (<https://geth.ethereum.org/docs/developers/evm-tracing>) 理解 trace 是如何通过重放交易得到的。

验收

能区分 transaction、message call、eth_call、event/log 和 storage。

能解释外层调用 revert 时，内层已经产生的状态变更为什么也会回滚。

第 4 天：Solidity 最小必需集

必读

Solidity Contracts (<https://docs.soliditylang.org/en/latest/contracts.html>)

Solidity Error Handling (<https://docs.soliditylang.org/en/latest/control-structures.html#error-handling-assert-require-revert-and-exceptions>)

Solidity Security Considerations (<https://docs.soliditylang.org/en/latest/security-considerations.html>)

动手

写一个 AtomicExecutor.sol 骨架，先不接 DEX，只包含：

owner 或明确的调用权限

两个外部调用依次执行

开始余额和结束余额检查

minProfit 不满足时 revert

一个成功事件和自定义错误

验收

能解释 msg.sender、msg.value、allowance、external call 和 reentrancy。

测试“两步都成功”和“第二步失败导致全部回滚”。

第 5 天：JSON-RPC 与 viem

必读

Ethereum JSON-RPC API (<https://ethereum.org/developers/docs/apis/json-rpc/>)

viem Getting Started (<https://viem.sh/docs/getting-started>)

viem Public Client (<https://viem.sh/docs/clients/public>)

动手

初始化 TypeScript 项目，实现 inspect-tx：输入交易哈希，输出交易、receipt、手续费、调用目标和已解码 ERC-20 Transfer 日志。

验收

至少直接调用一次原始 JSON-RPC，再用 viem 完成同一查询。

所有链上整数使用 bigint，不得用 JavaScript number 表示 token 数量。

第 6 天：合约读取、事件订阅和同块 Multicall

必读

viem readContract (<https://viem.sh/docs/contract/readContract>)

viem multicall (<https://viem.sh/docs/contract/multicall>)

viem watchBlocks (<https://viem.sh/docs/actions/public/watchBlocks>)

readContract 最终使用 eth_call ; multicall 可以批量读取多个合约，并允许指定
blockNumber，这是保证两个池子状态来自同一区块的关键。viem readContract
(https://viem.sh/docs/contract/readContract) viem multicall (https://viem.sh/docs/contract/multicall)

动手

读取两个 ERC-20 的 symbol、decimals 和 totalSupply。

在固定 blockNumber 下批量读取两个池子的状态。

订阅新区块并带上 emitMissed 处理漏掉的区块。

验收

能证明一次计算使用的所有状态来自同一个 block number。

知道 WebSocket 只负责及时通知，不自动保证数据正确或永不丢失。

第 7 天：Uniswap V2 AMM 数学

必读

Uniswap V2 Whitepaper (https://app.uniswap.org/whitepaper.pdf)，重点读第 1–3 节

Uniswap V2 Pools (https://developers.uniswap.org/docs/protocols/v2/concepts/pools)

V2 池是两个 ERC-20 的智能合约交易场所，不使用订单簿；交易沿 constant-product 曲线改变
reserves，套利者会把偏离外部市场的池内价格推回去。Uniswap V2 Pools
(https://developers.uniswap.org/docs/protocols/v2/concepts/pools)

动手

推导并实现整数版本：

text

amountInWithFee = amountIn 997

amountOut = amountInWithFee reserveOut /
(reserveIn 1000 + amountInWithFee)

同时计算 mid price、execution price、price impact 和 fee。

验收

使用至少 10 组边界测试验证方向、舍入和极小/极大输入。

能说明为什么“大额成交变差”不是因为 V2 有订单簿档位。

第 8 天：阅读 V2 合约源码

必读

UniswapV2Pair.sol (https://github.com/Uniswap/v2-core/blob/master/contracts/UniswapV2Pair.sol)

UniswapV2Library.sol

(https://github.com/Uniswap/v2-periphery/blob/master/contracts/libraries/UniswapV2Library.sol)

Uniswap V2 Architecture (https://developers.uniswap.org/docs/protocols/v2/concepts/architecture)

动手

通过 factory 找到 pair，读取 token0、token1、getReserves。

将本地 getAmountOut 与官方 library 和真实 router quote 对比。

解码 Pair 的 Swap 和 Sync 日志。

验收

报价差异只能来自自己解释的整数舍入或配置差异。

能解释 reserve0/reserve1 与 token 地址排序的关系。

第 9 天：Uniswap V3 集中流动性

必读

(<https://developers.uniswap.org/docs/get-started/concepts/liquidity-providers/concentrated-liquidity>)

Uniswap V3 Whitepaper (<https://app.uniswap.org/whitepaper-v3.pdf>)，重点读第 1、2、6 节

Uniswap V3 Architecture (<https://developers.uniswap.org/docs/protocols/v3/concepts/architecture>)

V3 允许 LP 把资金放进自选价格区间；tick 是离散价格边界，swap 跨越活跃 tick 时可用流动性会变化。Uniswap Concentrated Liquidity

(<https://developers.uniswap.org/docs/get-started/concepts/liquidity-providers/concentrated-liquidity>)

动手

读取一个 V3 pool 的 token0、token1、fee、liquidity、slot0/tick。

将 sqrtPriceX96 转成考虑 decimals 后的可读价格。

使用官方 Quoter 对三个不同输入量报价，观察跨 tick 后的变化。

验收

能准确解释 tick、tick spacing、active liquidity、fee tier。

不要求在本计划内自己完成生产级跨 tick 报价器。

第 10 天：DeFi 全景与可组合性

必读

Ethereum DeFi Introduction (<https://ethereum.org/defi/>)

ERC-20 (<https://eips.ethereum.org/EIPS/eip-20>)

ERC-4626 Tokenized Vaults (<https://eips.ethereum.org/EIPS/eip-4626>)

Chainlink Data Feeds (<https://docs.chain.link/data-feeds>)

Ethereum Bridges (<https://ethereum.org/developers/docs/bridges/>)

建立协议地图

稳定币：法币储备、超额抵押、算法/混合模型。

DEX：AMM、集中流动性、聚合器、intent/RFQ。

借贷：供给、借款、利率、抵押率、清算。

预言机：spot、TWAP、外部数据 feed、延迟与操纵风险。

质押/LST：质押收益、流动性凭证、脱锚和 slashing 风险。

收益金库：ERC-4626 shares/assets、策略和 first-depositor 风险。

衍生品：永续合约、资金费率、保证金和清算。

跨链桥：锁定/铸造、流动性网络、验证与最终性风险。

治理：token voting、timelock、代理升级和参数风险。

动手与验收

选 5 笔真实交易，各自归类协议原语，画出 token 流和信任假设。能够回答：“收益从哪里来、坏账由谁承担、价格由谁提供、谁有升级权限？”

第 11 天：Aave 借贷、清算和闪电贷

必读

Aave Borrow Tokens (<https://aave.com/help/borrowing/borrow-tokens>)

Aave Health Factor and Liquidations (<https://aave.com/help/borrowing/liquidations>)

Aave V3 Pool Contract (<https://aave.com/docs/aave-v3/smart-contracts/pool>)，重点看 borrow、liquidationCall、flashLoanSimple

Aave 的 health factor 是抵押价值、加权清算阈值与借款价值的函数；低于 1 时可被清算。清算人替借款人偿还债务，并获得相应抵押品和清算奖励。Aave Liquidations

(<https://aave.com/help/borrowing/liquidations>)

动手

手算三组仓位的 health factor，并改变 ETH 价格找出清算点。

画出 flash loan：借出 → 两次 swap → 偿还本金和 premium → 剩余利润。

验收

能解释超额抵押为何保护 lender。

能说明闪电贷解决临时本金，不解决滑点、gas、MEV、机会消失、合约 bug 或被抢先。

第 12 天：双池套利数学与最优输入量

必读

重读 Uniswap V2 Whitepaper (<https://app.uniswap.org/whitepaper.pdf>) 的 constant product 与 flash swaps 部分。

阅读官方

getAmountOut

实现

(<https://github.com/Uniswap/v2-periphery/blob/master/contracts/libraries/UniswapV2Library.sol>)。

动手

实现：

text

```
outA = quote(poolA, tokenIn, amountIn)
```

```
outB = quote(poolB, tokenMid, outA)
```

```
grossProfit = outB - amountIn
```

```
netProfit = grossProfit - gasCost - borrowPremium
```

两个方向都要计算。

先用网格搜索，再用一维优化或数学推导寻找最大净利润输入量。

为每个机会输出 break-even gas price。

验收

能证明“两个池子的现价不同”不等于“存在可交易净利润”。

测试费用、储备量、方向和整数舍入的边界条件。

第 13 天：扫描器设计与数据边界

必读

重读 Ethereum JSON-RPC (<https://ethereum.org/developers/docs/apis/json-rpc/>) 的 block parameter。

重读 viem multicall (<https://viem.sh/docs/contract/multicall>) 的 blockNumber 与 allowFailure。

设计

第一版严格限制为：

一条 EVM 链。

一个 token pair。

两个 V2-compatible pools。

token 白名单，不支持 fee-on-transfer/rebasing token。

只读，不保存私钥，不发送交易。

模块：block source、pool reader、quote engine、optimizer、gas estimator、opportunity recorder、replay runner。

验收

写出每个模块的输入/输出、失败方式和可测试边界。所有机会必须记录 block number、block hash、reserves、input、outputs、fees、gas 和 net profit。

第 14 天：实现实时扫描器 V1

动手

watchBlocks 接收新区块。

在该块高度 Multicall 两个池的 reserves。

对两个方向运行输入量优化。

输出净利润为正的机会到 JSONL 或 SQLite。

验收

扫描器连续运行 2 小时不崩溃。

每条记录可以被离线函数重新计算出完全一致的结果。

RPC 失败会明确报错或重试，不会悄悄使用旧状态。

第 15 天：净利润、数据质量和异常条件

动手

加入：

decimals 和 token 顺序标准化

DEX fee 配置

gas units 与 native-token gas price 换算

基础资产和 gas token 的价格换算

stale block、漏块和重组检测

最小净利润和安全折扣

验收

人为注入 RPC 超时、漏块、错误 decimals 和反向 token 顺序，扫描器不得产生静默假利润。

分别输出 grossProfit、gasCost、netProfit，不能只输出价差百分比。

第 16 天：MEV、交易排序和私有通道

必读

Ethereum MEV (<https://ethereum.org/developers/docs/mev/>)

Flashbots:

Understanding

Bundles

(<https://docs.flashbots.net/flashbots-auction/advanced/understanding-bundles>)

Flashbots JSON-RPC Endpoints (<https://docs.flashbots.net/flashbots-auction/advanced/rpc-endpoint>)

Flashbots Protect (<https://docs.flashbots.net/flashbots-protect/overview>)

DEX 套利是高度竞争的 MEV；公开 profitable calldata 可能被 generalized frontrunner 复制并以更高优先级提交。Flashbots 类通道允许交易或 bundles 不暴露给公开 mempool，但仍不保证一定包含或盈利。Ethereum MEV (<https://ethereum.org/developers/docs/mev/>) Flashbots Docs (<https://docs.flashbots.net/>)

动手

找一个包含套利或 sandwich 的真实区块，按 transaction index 重建：前置状态 → 用户 swap → 后置状态 → 搜索者利润。

验收

能逐项解释 frontrun、backrun、sandwich、bundle、builder、relay、private transaction。

能说明“扫描到机会”和“赢得排序并成交”之间缺少哪些环节。

第 17 天：历史回放和真实交易分析

必读

Geth EVM Tracing (<https://geth.ethereum.org/docs/developers/evm-tracing>)

viem Public Actions (<https://viem.sh/docs/actions/public/introduction>)

动手

给扫描器增加 [fromBlock, toBlock] 回放模式。

固定 block hash/number 读取历史 reserves。

对扫描到的机会检查下一块是否出现对应池交易，并分类：已被套利、状态变化、gas 后不盈利、数据错误、未知。

验收

回放同一范围两次，输出应确定一致；如果 RPC 不支持历史状态，必须明确失败并换用 archive-capable RPC。

第 18 天：Foundry 测试与原子执行合约

必读

Foundry Installation (<https://getfoundry.sh/introduction/installation/>)

Foundry Tests (<https://getfoundry.sh/forgedocs/tests/overview/>)

Solidity Security Considerations (<https://docs.soliditylang.org/en/latest/security-considerations.html>)

动手

完善 Day 4 的合约：

从调用者获取本金或接收 flash loan callback。

在两个 router/pair 上依次 swap。

使用 deadline、amountOutMin 和最终余额 minProfit 检查。

只把利润返给指定 recipient。

验收

测试成功、第一跳失败、第二跳失败、利润不足、未授权调用和 token 顺序错误。任何失败路径都必须原子回滚。

第 19 天：固定区块主网 Fork 模拟

必读

Foundry Fork Testing (<https://getfoundry.sh/forgedocs/tests/fork-testing/>)

viem simulateContract (<https://viem.sh/docs/contract/simulateContract>)

simulateContract 使用 eth_call 验证合约交互且不改变链上状态；固定 block number 的 mainnet fork 则能让测试结果可复现。viem simulateContract (<https://viem.sh/docs/contract/simulateContract>)

动手

固定 RPC 和 block number 创建 fork。

复现一组真实 pool reserves。

运行原子双池交易并计算实际 token balance delta 与 gas used。

将扫描器预测值与 fork 执行值对比。

验收

预测输出与实际合约输出的差异必须被整数舍入、router 行为或 gas 解释。

改变 fork block 后机会消失时，合约应因 minProfit 而 revert。

第 20 天：Shadow Run 与实盘门槛

动手

让扫描器运行至少 6 小时，对每条机会记录：

发现时间和 block

预测毛利润、gas、净利润

安全折扣后的利润

下一块实际池状态

fork 模拟是否成功

机会是否被其他交易消费

验收

输出一份数据质量报告：机会总数、可复现数、fork 成功数、假阳性原因分布。不得把“理论净利润为正”计为已经赚到的钱。

第 21 天：综合考试、代码审查和下一阶段

理论考试

不看资料回答：

1. 交易失败后哪些状态回滚，哪些成本不会返还？
2. V2 的价格和大额 price impact 如何产生？
3. V3 tick 与订单簿价位有什么相似和不同？
4. Aave health factor 为什么会下降，谁执行清算并获得什么？
5. 闪电贷能和不能解决什么？
6. 为什么两个 DEX 有价差仍可能无法盈利？
7. 为什么所有池状态必须固定到同一 block？
8. 公开 mempool 会给套利者带来什么风险？
9. eth_call、本地 fork 和真实成交分别能证明什么？
10. 原子合约中的最终 minProfit 检查为什么不能省？

项目终验

全部单元测试通过。

固定历史区块的回放结果可复现。

扫描器至少稳定运行 6 小时。

至少一条机会完成“实时发现 → 数据落盘 → 历史复算 → fork 模拟”的闭环。

README 能让另一位工程师从零运行，但不包含任何密钥。

21 天后才允许考虑的小资金测试门槛

1. 继续 shadow run 1-2 周，而不是完成第 21 天就发真钱交易。
2. 使用独立、资金上限明确的热钱包。
3. 代码内有链上 minProfit、max gas、deadline、pause/kill switch。
4. 每笔交易先模拟；明确私有提交方式和失败/未包含处理。
5. 只使用完全可损失且不会影响生活的金额。

笔记 04

作者：starkxun | 时间：2026-08-06 01:58 UTC | 字数：10049

来源：<https://intensivecolearn.org/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

在看到群里讨论这个方法的时候，我也好奇心驱使，问了AI

这个方法，然后用数据验证了一下。以下是本人的记录：

1. 先搞懂这个玩法

1.1 什么是「集中流动性」

传统 AMM(Uniswap V2)里,你存进去的钱铺在 0 到 ∞ 的整个价格轴上。绝大部分钱永远用不到 —— 没人会在 ETH 涨到 100 万美元时来跟你成交。

Uniswap V3 让你指定一个价格区间,钱只铺在这段里。同样的钱,在这段区间里的深度就厚得多,赚的手续费也多。代价是:价格跑出区间,你就一分钱不赚。

1.2 单边存入 = 一张限价单

假设某币现价 1.00 USDT,你把区间设在 0.80 ~ 0.98,并且只存 USDT。

因为区间完全在现价下方,此刻它是 100% 的 USDT。只有当币价跌进 0.98 以下,才会开始有人拿币来换走你的 USDT。

这等价于:你在 0.80~0.98 之间挂了一串限价买单。

区别只是你还额外收一笔手续费。

原文说的「相当于在现价下方挂了一张收费的买单」——这句话完全正确,而且是全文最重要的一句。但原文没往下推一步。

5%

1.3 谁是你的对手方

原文的框架是「机器人经过你的池,交过路费给你」。听起来你是收费站老板。

但机器人为什么要经过你的池?

原文第二节自己回答了:「机器人只比较最终到手金额……剩下的利润仍然为正,它才会成交。」

也就是说——机器人只在「跟你交易对它有利」的时候才来。它在外边买到便宜币,卖给你换走 USDT,赚走的就是你的报价和外部市场之间的差。

它赚的钱 = 你的报价 与 真实市场价 的差 ← 从你口袋里出

你赚的钱 = 手续费 ← 从它口袋里出

这两笔钱方向相反,净值可正可负。原文只写了第二项。

这个损失有正式名字:LVR(loss versus rebalancing),中文常译「相对再平衡损失」,直白说就是逆向选择——你系统性地在被信息更快的人挑着打。

换个比喻:你不是收费站,你是外汇兑换窗口,而排队的全是知道下一秒汇率的人。

手续费是你的点差收入,被挑着换是你的成本。生意能不能做,看点差够不够覆盖被挑着打的损失。

2. 验证计算

要证伪一个假设,先得把它写成能算的式子:

净收益 = ① 手续费收入

② 存货盈亏(撤池拿回的东西,比投进去的少多少)

③ gas

三项都能从历史数据算出来:

| 项 | 数据来源 | 怎么算 |

| :--- | :----- | :----- |

| ① 手续费 | 逐小时成交额 × 费率 × 我的份额 | 只在价格落在我区间内的小时才计 |

| ② 存货 | 逐小时价格 + V3 数学 | 用 (L, 区间, 当前价) 精确算出仓位构成 |

| ③ gas | 手动估 | 建池 + 撤池 + 换回稳定币 |

这套算法不含任何拟合和猜测——V3 的仓位构成完全由「流动性 L、区间 [Pa,Pb]、当前价 P」四个量决定,是精确公式。

3. 手动复现

这里我让AI写了一脚本: graph.py ,

可以在这里找到: graph.py

(<https://github.com/starkxun/MEV-LIFI/blob/main/lib/graph.py>) 。

第 1 步:准备 The Graph

需要一个 API key(免费额度足够)。去 <<https://thegraph.com/studio/apikeys/>> 建一个,写进 .env:

```
bash
```

```
echo 'THE_GRAPH_KEY=你的key' >> .env
```

验证一下:

```
bash
```

```
python3 graph.py
```

应该看到三行。

第 2 步:先看看高费率池到底有没有流量

在做复杂回测之前,先问个更基础的问题:高费率池能不能拿到流量?

Base 上 WETH/USDC 四个费率档,24 小时实测:

fee100 (0.01%) \$3,030,371 35,727 笔

fee500 (0.05%) \$7,691,203 13,843 笔

fee3000 (0.30%) \$31,456,452 19,523 笔

fee10000 (1.0%) \$9,249 78 笔 ← 高费率档几乎吃不到流量

1% 档一天只做了 9,249 美元,是 0.3% 档的 0.03%。

这不能直接否定收费站(它打的正是「没有低费率竞品」的场景),但它说明:聚合器路由只认最终到手金额,费率一高立刻被绕开。你的池能有流量,唯一原因是当时没有更便宜的路——而这个条件会自我毁灭。

第 3 步:找可测的池

这里准备了一个脚本工具,扫描高费率的池子,脚本在这里:lp_backtest.py

(https://github.com/starkxun/MEV-LIFI/blob/main/lp_backtest.py)

拿到脚本后运行:

```
bash
```

```
python3 lp_backtest.py --list --chain BNB
```

会列出 BSC 上费率 $\geq 1\%$ 的池,按累计成交额排序。运行结果如下:

```
bash
```

池对 费率 累计成交 累计费 TVL 费/成交 地址

```
-----
CAT/WBNB 1.00% $ 42,495,616,379 $ 424,956,164 $ 16,495 1.000% 0xa6e64fa67f... 成交额/TVL
过高,疑似刷量
COAI/USDT 1.00% $ 315,333,759 $ 3,153,338 $ 1,674,483 1.000% 0x4610464356...
FET/WBNB 1.00% $ 227,735,307 $ 2,277,353 $ 508,669 1.000% 0x41bf5eeae0...
USDT/memes 1.00% $ 75,712,721 $ 757,127 $ 458,789 1.000% 0x7231d17fa8...
ASTER/USDT 1.00% $ 63,846,198 $ 638,462 $ 457,045 1.000% 0x9d66f536b5...
NMT/USDT 1.00% $ 58,700,874 $ 587,009 $ 87,917 1.000% 0x96530dac78...
USDT/1 1.00% $ 54,911,214 $ 549,112 $ 379,607 1.000% 0x012a51fbcc...
USDT/TWoN 1.00% $ 50,114,226 $ 501,142 $ 91,005 1.000% 0x1648781b4a...
USDT/uSPE 1.00% $ 49,962,263 $ 499,623 $ 88,419 1.000% 0xb13e746630...
WBNB/VELO 1.00% $ 37,540,720 $ 375,407 $ 180,529 1.000% 0x42d749f736...
USDT/STBL 1.00% $ 37,441,614 $ 374,416 $ 208,461 1.000% 0x6b86794e11...
USDT/我踏马来了 1.00% $ 34,121,590 $ 341,216 $ 259,891 1.000% 0xe818624dbf...
USDT/RIVER 1.00% $ 33,451,654 $ 334,517 $ 302,642 1.000% 0x2e066d83ad...
USDT/币安人生 1.00% $ 33,383,941 $ 333,839 $ 868,658 1.000% 0x65d5b99ba4...
USDT/龙虾 1.00% $ 33,137,664 $ 331,377 $ 821,322 1.000% 0x8cda56bb79...
USDT/WARD 1.00% $ 32,822,007 $ 328,220 $ 276,759 1.000% 0xf391ba6522...
Freedom of/USDT 1.00% $ 32,425,903 $ 324,259 $ 255,752 1.000% 0x13c17791f8...
INJ/WBNB 1.00% $ 30,708,112 $ 307,081 $ 158,137 1.000% 0x33e59edd32...
USDT/LAB 1.00% $ 29,626,955 $ 296,270 $ 207,337 1.000% 0xc7e3ca54c3...
USDT/BCH 1.00% $ 29,614,519 $ 296,145 $ 141,445 1.000% 0x1fd22fa727...
USDT/BULLA 1.00% $ 29,514,501 $ 295,145 $ 306,450 1.000% 0x3ec74c0b38...
USDT/GUA 1.00% $ 28,964,603 $ 289,646 $ 160,025 1.000% 0xaeed9a1a98...
NAORIS/USDT 1.00% $ 25,774,291 $ 257,743 $ 246,910 1.000% 0xaa133749ab...
USDT/MYX 1.00% $ 25,677,510 $ 256,775 $ 182,538 1.000% 0x9e1cfd8eb2...
AKE/USDT 1.00% $ 25,382,559 $ 253,826 $ 823,482 1.000% 0x83fcd80d79...
```

提示:TVL 极小而成交额极大的池基本是刷量,回测它们没有意义。

注意第一行:

CAT/WBNB 1.00% 累计成交 \$42,495,616,379 TVL \$16,495 成交额/TVL 过高,疑似刷量
425 亿美元成交额,1.6 万 TVL。这是刷量,不是真流量。脚本会自动标记。

第 4 步:回测一个良性场景

bash

```
python3 lp_backtest.py --chain BNB
```

```
--pool 0x4610464356ba6bba15eec558619d84b72fea260f
```

```
--capital 10000 --range-lo -20 --range-hi -2
```

```
--since 2026-07-29T08:00:00Z --hours 168
```

--since 锁死时间窗,不加的话是「从现在往回 168 小时」,过几天跑结果就变了。

要可复现就一定要给 --since。

输出:

回测结果 COAI/USDT 费率 1.00%

建仓价 0.33414492 USDT/COAI 结束价 0.34299299 (+2.6%)

区间 [0.26731593, 0.32746202] 期间价格 0.317165 ~ 0.345327

时长 100 小时,其中 19 小时在区间内(19%),穿越 12 次

投入 10,000.00 USDT(单边)

撤池拿回 10,000.00 USDT + 0 COAI 折合 10,000.00 USDT

① 手续费收入 +4.29 USDT (+4.3 bps)

② 存货盈亏 +0.00 USDT (+0.0 bps) ← 逆向选择在这里

③ gas -5.00 USDT

净收益 -0.71 USDT -0.7 bps

解读:币涨了 2.6%,价格从上方穿过区间又回去了,所以撤池时仓位又变回全 USDT,存货盈亏 = 0。四天赚了 4.29 USDT 手续费,还不够 gas。

这是最好的情况之一 —— 没亏钱,只是白忙。

第 5 步:回测下跌场景

bash

```
python3 lp_backtest.py --chain BNB
```

```
--pool 0x65d5b99ba46b29005856ddb7d5c6675aab77b204
```

```
--capital 10000 --range-lo -20 --range-hi -2
```

```
--since 2026-07-29T07:00:00Z --hours 168
```

输出:

回测结果 币安人生/USDT 费率 1.00%

建仓价 0.64250149 USDT/币安人生 结束价 0.51352711 (-20.1%)

区间 [0.53541791, 0.62990342] 期间价格 0.513527 ~ 0.646521

时长 135 小时,其中 88 小时在区间内(65%),穿越 8 次

投入 10,000.00 USDT(单边)

撤池拿回 0.00 USDT + 17,219.3 币安人生 折合 8,842.60 USDT

① 手续费收入 +180.60 USDT (+180.6 bps)

② 存货盈亏 -1,157.40 USDT (-1157.4 bps) ← 逆向选择在这里

③ gas -5.00 USDT

净收益 -981.80 USDT -981.8 bps

> 手续费覆盖了 15.6% 的存货亏损——覆盖不住

解读:

88/135 小时在区间内(65%) —— 按原文标准,这个收费站开张得非常成功

手续费 +180.60 USDT —— 四天 180 bps,年化看着很香

撤池拿回 0 USDT + 17,219 个币 —— 你的 USDT 全部变成了已经跌 20% 的币

净 981.80 USDT —— 亏掉本金的 9.8%

最反直觉的一点:手续费和存货亏损不是独立的两项,它们由同一个原因驱动。

成交越频繁 → 说明价格穿越越剧烈 → 你被填的单越多 → 存货亏得越狠。

「生意好」和「亏钱」在这里是同一件事。

4. 区间扫描

bash

python3 lp_backtest.py --chain BNB

--pool 0x4610464356ba6bba15eec558619d84b72fea260f

--capital 10000 --since 2026-07-29T08:00:00Z --hours 168 --sweep

区间 在区间小时 穿越 手续费 存货盈亏 净收益bps

[-5%,-1%] 42 22 +6.37 +0.00 +1.4

[-10%,-2%] 19 12 +4.38 +0.00 -0.6

[-20%,-2%] 19 12 +4.29 +0.00 -0.7

[-30%,-5%] 1 2 +0.42 +0.00 -4.6

[-50%,-10%] 0 0 +0.00 +0.00 -5.0

[-20%,-10%] 0 0 +0.00 +0.00 -5.0

[-40%,-20%] 0 0 +0.00 +0.00 -5.0

只有最窄的 [-5%,-1%] 是正收益(+1.4 bps),稍微放宽就转负,再宽就是 0 小时在区间内。

最后三行的 0 小时 / 0 手续费 是收费站最常见的失败模式 —— 不是亏钱,是根本没开张,资金白占着。

于是矛盾出现了:

要有流量 → 区间必须贴近现价(窄)

不被砸穿 → 区间必须远离现价(宽)

这两个要求直接冲突,没有中间解。窄区间有流量,但也最容易被一根大阴线扫穿变成全仓接盘。

6. 本次回测的局限

诚实地讲,以下几点会影响结论的精度 —— 但都不足以翻转结论,因为差距是 6 倍量级:

1. 手续费份额是近似的。我按「我的 L / (池子活跃 L + 我的 L)」分摊,而 liquidity 字段只是当前 tick 的活跃流动性。真实 V3 流动性按 tick 分布不均,在分布很偏的池上会有偏差。

2. 只测了两个池、一段行情。样本远远不够下普适结论,只能说「在这两个案例里不成立」。

3. 没有模拟动态调仓。原文暗示可以「流量结束以后撤出来」,而回测是买入并持有到期末。主动管理可能改善结果,但那要求你比机器人反应快 —— 而这个玩法的出发点恰恰是「卷不过机器人」。

4. subgraph 的 TVL 不可信(实测比链上真实余额虚高 3~6 倍),所以脚本完全不用 TVL 做计算,只用价格、成交额、手续费、流动性。

5. 不含新币特有的风险:合约后门、抽池、转账税。这些只会让结论更差。

附:完整复现清单

bash

cd ~/MEV-LIFI

0. 确认 The Graph 可用

```
python3 lib/graph.py
```

1. 高费率池能不能拿到流量(Base 对照)

```
python3 pool_price.py --chains BAS --base USDC --quote WETH
```

2. 列出 BSC 高费率池

```
python3 lp_backtest.py --list --chain BNB
```

3. 良性场景:币价小涨

```
python3 lp_backtest.py --chain BNB
```

```
--pool 0x4610464356ba6bba15eec558619d84b72fea260f
```

```
--capital 10000 --range-lo -20 --range-hi -2
```

```
--since 2026-07-29T08:00:00Z --hours 168
```

4. 下跌场景:关键案例

```
python3 lp_backtest.py --chain BNB
```

```
--pool 0x65d5b99ba46b29005856ddb7d5c6675aab77b204
```

```
--capital 10000 --range-lo -20 --range-hi -2
```

```
--since 2026-07-29T07:00:00Z --hours 168
```

5. 区间扫描

```
python3 lp_backtest.py --chain BNB
```

```
--pool 0x4610464356ba6bba15eec558619d84b72fea260f
```

```
--capital 10000 --since 2026-07-29T08:00:00Z --hours 168 --sweep
```

想自己找更多案例:改 --pool,或者用 --list 里的其它地址。

想测别的链:--chain BAS / ARB / BNB-V4(注册表见 [lib/graph.py])。

笔记 05

作者: z0y1 | 时间: 2026-08-06 15:21 UTC | 字数: 9916

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

7BN 钱包 Pump AMM 交易策略逆向研究

目标钱包: 7BNaxx6KdUYrjACNQZ9He26NBFoFxujQMAfNLnArLGH5

研究时间: 2026-08-06 (北京时间)

结论

这个钱包的主体不是“高 tip 抢首包”,也不是单纯复制某个鲸鱼卖出的 dump sniper。它更像一个持续 24 小时运行的 Pump AMM 池级轮询系统:

1. 以池子为单位,以约 120 slot (约 50 秒) 为状态更新节拍。
2. 在成交活跃度和价格波动突然上升时入场,其中约三分之二是承接近 120 slot 的净卖压,也包含少量正向动量入场。
3. 成交数量极度标准化:常规时买出交易前池子约 1.89729889% 的 base reserve,最大报价支出封顶 15 SOL。
4. 卖出几乎都是全平。正常退出高度集中于买入后 120 slot 的整数倍,并有 20000 slot (约 140 分钟) 的硬超时。
5. 退出不是简单的固定止盈/止损。对配对市场路径的最优单一价格边界仅能解释 9.5% 的抽样退出,说明还有隐藏状态、分池持有期模型或随机化。
6. 它的优势是“小幅正期望 × 极高周转 × 广泛分散”,而不是胜率极高或偶尔抓到百倍币。

因此,低 tip 是“对极致抢速不敏感”的证据,不是策略优秀的独立证明。直接监听该钱包后跟单不能复制它:它的单笔买入本身就会改变池价,跟单者只能获得更差的价格。应复制它的“前置市场信号”,而不是复制已经发生的交易。

。

数据口径与校验

全量主分析覆盖 slot 431194398—437557766，对应北京时间 2026-07-07 00:01:53—2026-08-06 17:27:28，约 30.7 天。

ClickHouse 中全量读取该钱包作为 fee payer 的交易，并与 Pump AMM trade event 逐笔匹配。

全部查询使用 max_threads=1 和 max_memory_usage=3500000000。

入场上下文在全量交易上按时间均匀抽取 600 个样本，对每次买入前的当前 120 slot 和前一个 120 slot 做配对对照。

退出上下文抽取 600 个正常 cadence 退出，对退出前两个 120 slot 窗口做配对对照；共读取 596247 个相关市场事件。

GMGN 30 天与 7 天钱包统计用于独立交叉校验。

一个重要数据陷阱：ClickHouse 的部分买入 tip_lamports 字段存的是第一笔 System transfer，即买入资金，不是 tip。通过原始 RPC 交易解码和 GMGN activity 交叉验证后，成功买入 tip 按约 301350 lamports 校正；卖出 tip 可直接使用链上表中的实际值。该估计的单笔误差只有几十 lamports 级别，不会改变本报告结论。

Pump AMM 的 user_quote_amount_in/out 是钱包实际现金流，已内含 DEX、protocol 和 creator 费用影响，因此这些费用不能在 PnL 中再扣一遍。

实盘表现

指标 结果
:----- -----:
fee-payer 交易尝试 43,309
成功 / 失败 40,438 / 2,871
链上成功率 93.3709%
完整买卖回合 20,197
交易池数 3,375
买入周转 60,033.4083 SOL
DEX 费后、链上费前现金流 +1,317.9128 SOL
runtime fee 5.6302 SOL
成功交易 tip 12.1763 SOL
保守窗口净收益 +1,300.1064 SOL
净收益 / 买入周转 2.1657%
回合净胜率 50.4184%
Profit factor 1.3425
最大已实现回撤 34.59 SOL
最大同时仓位 49
最大同时买入成本 162.43 SOL

保守窗口净收益用完整回合现金流减去窗口内全部成功/失败交易的链上执行成本，因此还包含未配对边界交易的执行费，比仅计算完整回合更保守。

收益的稳定性确实很强：

31 个日历日全部为正；日净收益中位数 36.81 SOL，最低约 8.57 SOL，最高约 95.78 SOL。

按自然周分组的周转净利率介于 1.99%—2.37%，未发现只在某几天有效。

单回合 PnL 中位数只有 +0.002416 SOL；p1 约 -2.079 SOL，p99 约 +3.123 SOL，最差 -8.356 SOL，最好 +21.031 SOL。

前 5 个最赚池总贡献约 183.48 SOL；删除它们仍有约 1,117 SOL 净收益，不是少数幸运尾部驱动。

钱包与 coin creator、pool creator 直接重合都为 0，不像依赖自己的 creator 返佣获利。

完整日几乎每天 24 个小时都在运行，且 30 天合计的每个小时时段都为正。

GMGN 交叉校验给出：30 天 realized profit 约 \$107,358.68，PnL 约 2.537%，18,902 次买入、18,912 次卖出；7 天 realized profit 约 \$23,508.64，PnL 约

2.456%。两套数据的时间边界、币种聚合和美元折算口径不同，不应要求数值完全一致，但收益规模和 2%—2.5% 周转利率相互支持。

交易场所和候选池

在主分析窗口内，40,438 笔成功交易与 40,438 个 Pump AMM trade event 一一对应；报价币全部为 WSOL，未见 USDC 或 Pump.fun bonding curve 交易。这是一个纯 Pump AMM 执行钱包。

它并不只做刚创建的池子。入场时池龄分布如下：

池龄 回合数
:----- ----:
≤ 1 分钟 1,696
1—5 分钟 289
5—15 分钟 787
15—60 分钟 2,517
1—6 小时 4,602
6—24 小时 4,292
1—3 天 2,988
3—7 天 1,874
> 7 天 1,152

所以“新币首包狙击”无法解释它的大部分交易。

入场逻辑

可直接确认的行为

600 个均匀抽样入场中，买入前 120 slot 全部有外部成交，86.67% 有外部卖出。

最近外部卖出距买入不超过 1 / 5 / 20 / 120 slot 的比例分别是 15.83% / 36.00% / 61.50% / 86.67%。

买入前最后一笔外部成交是卖出的比例只有 37.83%，因此策略不是“看见一笔 sell 就立即 buy”。

买入前 120 slot 的价格中位变化是 -4.73%，前一对照窗口只有 -0.06%。

当前窗口外部净报价流中位数是 -2.544 SOL，对照窗口为 -0.003 SOL。

当前窗口外部事件数中位数为 16，对照窗口为 9，说明它偏好交易活跃度突然放大的池。

65.0% 的入场窗口价格下跌，64.67% 是净卖出流；另外约三分之一是正向流量/正向动量。

用当前窗口和紧邻对照窗口构造的浅层诊断树，时间外推 AUC 约 0.708；它主要依赖“外部 buy 数、窗口价格变化和净流量”。这证明用简单的实时市场特征可以近似其入场分布，但 AUC 还远未达到“已还原源码规则”的程度。

最合理的入场解释

它在每个候选池的 120-slot 节拍上计算活跃度、价格变化和买卖流量的综合分数。主分支是在活跃度放大时承接短时净卖压，赌均值回归；次分支则在强正向波动时跟随动量。池龄、fee configuration、流动性和风险过滤大概还会参与候选池打分。

这一解释也说明为什么低 tip 仍能成交：它大多数时候不是在与其它 bot 竞争同一 slot 的第一个成交。

仓位管理

最强的可复制指纹是 base 数量。在三个独立时间分段中，base_amount_out / pre_trade_pool_base_reserve 的中位数都是：

text

$0.0189729889 \approx 1.89729889\%$

14,408 笔买入几乎精确落在这个比例上。对常规池，指令的 max_quote_amount_in 约等于交易前 quote reserve 的 2%；对高储备池，报价最高封顶 15 SOL。低储备和不同 fee configuration 存在虚拟储备/费率差异，因此不应用“实际 SOL 支出等于储备的固定 2%”取代 base 比例规则。

实际买入规模分位数：

分位	SOL
p10	0.659
p25	1.004
p50	1.826
p75	3.535
p90	6.609
p95	10.942
p99	14.629
最大	14.998

买入 $\text{max_quote} / \text{actual_quote}$ 的中位余量约 216.9 bps，卖出 $\text{actual_quote} / \text{min_quote}$ 的中位余量约 210.1 bps，即两边都使用约 2.1% 的报价容忍区间。

仓位几乎不拆分：20,197 个完整回合中只有 41 个存在非零 base residual，且其中还包含窗口边界和极少量重叠买入导致的 FIFO 配对偏差。策略语义可视为“一次买入，一次全平”。

退出与复入

退出的时间指纹非常清晰：

排除硬超时后，87.25% 的退出 slot 差精确是 120 的整数倍。

最常见持有 slot 是 240、360、480、600、720 等。

持有秒数 p50 / p75 / p90 / p95 约为 657 / 1995 / 8189 / 8410 秒。

1,924 个回合精确在 20000 slot 卖出；考虑出块抖动和失败重试，持有不少于 19,950 slot 的回合共 2,425 个。

这 2,425 个长持有/超时回合的胜率只有 12.83%，净亏约 630.5 SOL；硬超时是该策略最明显的结构性尾部风险。

为验证退出是否是固定 TP/SL，对 600 个 cadence 退出比较了退出前两个 120-slot 窗口：

最优单一 TP/SL 边界的新穿越覆盖率只有 9.5%。

事件数、buy 数、sell 数、净流量、价格变化区分“退出窗口”与“前一未退出窗口”的最佳单特征 AUC 不超过 0.569。

仅用入场时可知的池龄、储备、仓位、复入次数和费率特征预测 log 持有期，时间外推 $R^2 \approx 0.277$ 、Spearman 约 0.508；池龄是最强特征，但仍不足以精确还原每次退出。

因此，可确认的是“120-slot 检查节拍 + 20000-slot 硬超时”；不能确认的是每个检查点的隐藏决策函数。它可能是分池龄的 hazard/持有期模型，也可能包含本地状态和随机化。

复入行为也保留同一节拍：

每个池回合数中位数为 3，最多 99。

窗口内有 16,822 次复入；排除极少量重叠后，前一次卖出到下一次买入的间隔中位数约 7,463 slot。

80.77% 的正间隔是 120 的整数倍，而同池两次买入间隔有 87.05% 是 120 的整数倍。

这说明它会持续维护池级状态，而不是交易完一次就丢弃该池。

费率配置与收益来源

两组主要 Pump AMM fee configuration 都是正收益，但收益质量差异明显：

LP / protocol fee bps	回合	池	买入周转	净收益	周转净利率	胜率	PF	正收益日
20 / 5	10,410	1,502	49,882.64 SOL	1,090.29 SOL	2.1857%	53.74%	1.381	31/31
2 / 93	9,787	3,000	10,150.77 SOL	210.21 SOL	2.0709%	46.89%	1.225	27/31

20 / 5 组贡献了约 84% 的净收益，并且在前、中、后三个时间段都保持正收益和 PF > 1.34。这是复现时最值得优先研究的子市场，但 fee configuration 也可能是池龄、流动性和用户结构的代理变量，不应直接解释为费率本身造成收益。

执行模板与低 tip

该钱包的交易模板几乎完全固定：

参数	值
Compute unit limit	250,000
Compute unit price	500,000 micro-lamports/CU
Priority fee	125,000 lamports
每笔 runtime fee	130,000 lamports
成功交易 tip	约 300,231—301,380 lamports，中心约 301,000
单笔 runtime + tip	约 0.000431 SOL
买入 CU consumed p50	112,184
卖出 CU consumed p50	97,840.5

一个完整回合的 runtime + tip 约 0.000862 SOL。以 1.826 SOL 的中位仓位计算，约为 4.7 bps。与之相比：

完整配对回合的 DEX 费率影响约 1,266.17 SOL，约等于买入周转的 2.11%。

DEX 费率影响约是全部链上 runtime + tip 成本的 71 倍。

低 tip 对利润有帮助，但决定策略能否为正的是入场前的毛 edge 能否覆盖约 2% 的往返 DEX 费率和价格冲击。

失败交易中，2,358 笔是 Pump AMM ExceededSlippage (6004)，472 笔是 base ATA/余额类错误，40 笔是 Anchor constraint 2006，另有 1 笔其他错误。失败交易仍支付 runtime fee，但不支付 tip；30.7 天失败执行费合计只有 0.3732 SOL。策略接受一定失败率，用报价保护避免恶劣成交。

可复制部分与不可见部分

组件	结论	信心
:-----	:-----:	:
交易场所	Pump AMM + WSOL	高
轮询节拍	池级 120 slot	高
硬超时	20000 slot	高
买入 base 数量	pre-trade base reserve 的约 1.89729889%	高
最大报价支出	15 SOL	高
报价余量	买卖两边约 2.1%	高
卖出数量	几乎全平	高
主入场风格	活跃度放大后偏向承接净卖压	中高
次入场风格	活跃度放大后的正动量	中
候选池完整过滤	不可从成交输出唯一反推	低
每个检查点的退出函数	不是简单 TP/SL，仍有隐藏状态	低
执行模板	固定 CU、priority fee 和约 0.000301 SOL tip	高

最小可复现实验方案

下面是用于 paper trade 和严格回测的候选规范，不是已证实可直接实盘的完整克隆。

1. 只监听 Pump AMM WSOL 池，按 pool 保存状态，每 120 slot 生成一次特征。
2. 优先验证 lp_fee=20 bps / protocol_fee=5 bps 子市场，但不把费率当因果特征。
3. 最小入场特征集：当前/上一 120-slot 窗口的事件数、buy/sell 数、净 quote flow、价格变化、池龄、quote reserve、fee configuration。
4. 先只测试主分支：当前窗口活跃度明显高于上一窗口，且净 quote flow < 0。对正动量分支单独建模，不与均值回归分支混在一起。
5. 买入 base 目标使用 pre_pool_base_reserve 0.0189729889，调用实时 Pump AMM quote 计算实际支出，max_quote_amount_in <= 15 SOL，报价余量初始使用约 2.1%。
6. 同池同时只允许一个仓位；以 120 slot 为检查节拍，硬超时使用 20000 slot。
7. 退出不要误设为某个固定 TP/SL 并声称是原钱包规则。第一版应使用按池龄分层的经验持有期/生存模型，并在独立前推期内重新拟合。

8. 执行参数可以从 `CU limit=250000`、`CU price=500000` `micro-lamports`、`tip` 约 `0.000301 SOL` 开始。保留失败即放弃、不无限抬高报价的保护。
9. 以约 `162.43 SOL` 的历史最大同时成本和 `34.59 SOL` 回撤估算，原尺度复现的风险资金不宜低于约 `200 SOL`，还应另留报价和交易费缓冲。资金不足时应等比降低仓位和最大并发，不要只删掉亏损池后宣称可获得同样收益。
10. 先做时间分割回测，再连续 `paper trade` 至少 `7—14` 天；验收应同时满足成交覆盖率、周转净利率、硬超时损失、并发资金和真实失败率，不只看总 `PnL`。

主要风险和尚未识别的信息

1. 主样本只有约 `30.7` 天。虽然 `7` 天、周级和三段时间外推一致，仍不能证明边际永久存在。
2. 交易输出不能唯一还原候选池集合、私有黑名单、社交信号、开发者风险分数或其他链下特征。
3. 退出函数仍未完全识别；硬把历史持有期分布当作源码规则会产生模型风险。
4. 硬超时仓位在样本中亏损约 `630.5 SOL`，是主要尾部风险；新市场状态下超时比例可能显著上升。
5. 原钱包的买入量接近池子 `base reserve` 的 `1.9%`，会产生明显自身价格冲击。任何事后跟单都可能把该钱包的 `edge` 变成跟单者的亏损。
6. 收益来自大量小 `edge`，近 `2%` 的往返 `DEX` 费率已经很高。即使信号质量只下降少量 `bps`，净利也可能迅速消失。

可复现性

核心全量抽取脚本：

```
bash
uv run python research/pump-amm-dump-execution/src/analyze_wallet_july_live.py
--cache-dir /tmp/solana-inspect-7bn-30d
--wallet 7BNaxx6KdUYrjACNQZ9He26NBFoFxujQMAfNLnArLGH5
--start-slot 431194398
--end-slot 437557766
--buy-tip-lamports 301350
```

配对上下文抽样脚本：

```
bash
uv run python
research/wallet-behavior-reverse-engineering/experiments/2026-08-06_7BN钱包策略逆向/analyze_window_context.py
--cache-dir /tmp/solana-inspect-7bn-30d
```

原始缓存和中间结果在 `/tmp/solana-inspect-7bn-30d`，不纳入 `Git`。

笔记 06

作者：Galaxy9 | 时间：2026-08-05 13:32 UTC | 字数：9262

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

基本概念

`DEX` 内交易交易在同一个去中心化交易所完成，价格由 `AMM` 池或订单簿决定。`AMM` 常用公式 $x \times y = k$ ，买入一种资产会使其价格上升。用户在 `Uniswap` 用 `USDC` 买 `ETH`；聚合器比较多个交易池，选择滑点和手续费较低的路径。

`DEX` 内套利同一 `DEX` 内不同交易池、交易对或路径之间存在短暂价差，通过连续兑换赚取差额。`ETH/USDC` 池和 `ETH/DAI` 池价格不一致，机器人执行 `USDC → ETH → DAI → USDC`，若收入超过手续费和 `Gas`，就产生套利利润。

`DEX` 间套利同一种资产在不同 `DEX` 的价格不同，在低价市场买入、高价市场卖出，使价格重新趋于一致。`Uniswap` 上 `ETH` 为 `3,000 USDC`，`SushiSwap` 上为 `3,020 USDC`，套利者在前者买入、后者卖出。

跨链套利同一资产在不同区块链上的价格不同，需要跨链转移资产，或提前在各链准备资金后同时交易。ETH 在 Ethereum 比 Arbitrum 贵 1%，套利者在 Arbitrum 买入、Ethereum 卖出；实际通常提前在两条链备好资金，避免等待桥接。

三角套利在同一市场中利用三个交易对的汇率不一致，绕一圈回到原资产。在某 DEX 执行 $USDC \rightarrow ETH \rightarrow WBTC \rightarrow USDC$ 。如果最终 USDC 数量增加，扣除手续费后即为利润。

稳定币机制稳定币试图让价格贴近某个目标，例如 1 美元。机制包括法币储备、超额抵押、算法调节或混合模式。USDC 通过储备资产维持接近 1 美元；DAI 由抵押 ETH 等资产铸造；当 DAI 低于 1 美元时，套利者买入并赎回或交易获利，帮助价格回归。

稳定币套利利用稳定币之间或稳定币与美元目标价之间的偏差获利。 $USDT = 0.998$ 美元、 $USDC = 1.002$ 美元时，买入便宜的 USDT、卖出较贵的 USDC。

清算借贷协议要求借款人的抵押品价值保持在安全范围内。抵押率过低时，任何人都可偿还部分债务并获得折价抵押品。用户抵押 10,000 美元 ETH 借出 6,000 DAI。ETH 下跌后抵押率不足，清算人偿还 DAI，获得打折后的 ETH。

清算套利清算人利用协议提供的清算奖励获利，同时承担价格波动、Gas 和交易失败风险。Aave 给清算人 5% 奖励。机器人发现某账户可清算，调用清算接口并立即卖出获得的 ETH。

MEV（最大可提取价值）由于区块生产者或区块构建者可以决定交易排序、插入或取消交易，因此能从用户交易顺序中提取额外价值。大额买单会推高价格，机器人先买入、再让用户买入、最后卖出，形成“三明治攻击”。

抢跑监控待确认交易，在目标交易之前提交相似交易，以获取价格或清算优势。看到某账户即将被清算，机器人提高 Gas 费，争取让自己的清算交易先打包。

反向交易目标交易完成后，利用其造成的价格变化立即交易。用户的大额 ETH 买单推高池内 ETH 价格，机器人随后卖出 ETH，赚取价格变化。

三明治攻击在用户交易前后各插入一笔交易，从用户造成的价格滑动中获利。用户准备买入大量 ETH，机器人先买入推高价格，用户成交后机器人卖出。用户因此得到更差的成交价。

MEV 保护通过私有交易通道、批量拍卖、交易限价和更合理的滑点设置，减少交易被观察和重排的机会。用户通过 Flashbots Protect 或 DEX 聚合器的私有路由提交交易，降低被三明治攻击的概率。

一个综合实例

假设：

Uniswap 的 ETH 价格为 3,000 USDC；
SushiSwap 的 ETH 价格为 3,020 USDC；
Arbitrum 上 ETH 价格为 2,980 USDC。

套利者可能：

1. 在 Arbitrum 买入 ETH；
2. 跨链或使用预先部署的资金；
3. 在价格较高的 DEX 卖出；
4. 如果三个交易对之间还存在汇率偏差，再执行三角套利；
5. 通过私有交易通道提交，避免被其他 MEV 机器人抢先。

最终利润必须扣除 DEX 手续费、Gas、桥接费用、滑点、资金占用成本和交易失败风险。

1. AMM：用资金池自动报价

AMM（自动做市商）没有传统的买卖挂单，而是把两种资产放入资金池，通过公式自动计算价格。

以常见的恒定乘积公式为例：

$$x \times y = k$$

x：池中的 ETH 数量

y：池中的 USDC 数量

k：交易前后大致保持不变

交易者买走 ETH，就要向池中加入 USDC

ETH 越买越少，后续 ETH 的价格就越高

例子

某池中有：

100 ETH

300,000 USDC

初始参考价格约为：

$300,000 \div 100 = 3,000$ USDC/ETH

用户用 30,000 USDC 买 ETH。忽略手续费，根据公式：

交易前： $100 \times 300,000 = 30,000,000$

交易后：ETH 数量 = $30,000,000 \div 330,000 \approx 90.91$

用户得到：

$100 - 90.91 = 9.09$ ETH

平均成交价约为：

$30,000 \div 9.09 \approx 3,300$ USDC/ETH

虽然初始价格是 3,000，但用户的实际平均价格是 3,300，因为这笔买单改变了池中资产比例。

2. 订单簿：买卖双方主动报价

订单簿把买单和卖单按照价格排列。交易者可以挂限价单，也可以直接接受已有报价。

示例卖单：

卖价	ETH 数量
3,000 USDC	2 ETH
3,005 USDC	3 ETH
3,010 USDC	5 ETH
3,020 USDC	10 ETH

如果用户市价买入 8 ETH：

以 3,000 买入 2 ETH

以 3,005 买入 3 ETH

以 3,010 买入 3 ETH

平均成交价为：

$(2 \times 3,000 + 3 \times 3,005 + 3 \times 3,010) \div 8$
 $= 3,005.625$ USDC/ETH

AMM 是沿着数学曲线成交，订单簿则是逐档“吃掉”已有挂单。

3. 流动性深度：市场承受大额交易的能力

流动性深度表示：一笔较大的交易能否在价格变化不大的情况下完成。

深度主要取决于：

AMM 资金池中有多少资产

订单簿在当前价格附近有多少挂单

流动性是否集中在当前价格附近

市场参与者能否快速补充流动性

例子

两个 ETH/USDC 池当前都显示 $\text{ETH} = 3,000 \text{ USDC}$:

A 池 : $100 \text{ ETH} + 300,000 \text{ USDC}$

B 池 : $10,000 \text{ ETH} + 30,000,000 \text{ USDC}$

同样买入 30,000 USDC 的 ETH :

在 A 池中, 价格会明显上升

在 B 池中, 资产比例只发生很小变化

因此, B 池的流动性更深, 更适合大额交易。

需要注意: 池子的总价值大, 不一定代表当前价格附近的流动性深。Uniswap V3 等集中流动性 AMM 中, 资金可能分布在不同价格区间。

4. 价格冲击: 自己的交易改变了价格

价格冲击是指交易本身推动市场价格发生变化。

近似计算:

价格冲击 = $(\text{实际成交均价} - \text{交易前价格}) \div \text{交易前价格}$

以上面的 AMM 为例:

交易前价格: 3,000

平均成交价格: 3,300

价格冲击: $(3,300 - 3,000) \div 3,000 = 10\%$

这意味着用户因为交易量相对资金池过大, 支付了约 10% 的额外价格。

价格冲击与滑点不完全相同:

价格冲击: 自己的交易造成价格变化

滑点: 预期成交价与最终成交价的差异, 可能来自价格冲击、其他交易抢先成交或市场波动

滑点容忍度: 用户允许交易偏离报价的最大范围; 超过后交易回滚

例子

报价时预计得到 10 ETH, 滑点容忍度设置为 1%, 那么最低可接受数量约为:

$10 \times (1 - 1\%) = 9.9 \text{ ETH}$

最终只能得到 9.8 ETH 时, 交易通常会失败, 而不是强制成交。

5. 区块确认: 交易被写入区块链

用户提交交易后, 交易一般经历:

签名并广播 → 等待打包 → 写入区块 → 后续区块确认 → 最终确定

“1 个确认”通常表示交易已经进入一个区块; 之后每增加一个区块, 交易被回滚的概率通常会进一步降低。

例子

用户在 Ethereum 上提交一笔 Swap:

1. 钱包显示“等待确认”, 交易仍在等待打包。
2. 验证者将交易放入第 1,000 个区块, 此时获得第一次确认。
3. 链继续产生第 1,001、1,002 等区块, 确认数不断增加。
4. DEX 前端可能在第一次确认后显示成功。
5. 跨链桥或交易平台为了降低回滚风险, 可能等待更多确认或等待链达到最终确定状态。

交易广播后并不等于已经成功。等待期间可能出现:

Gas 费太低, 长时间没有被打包

价格变化超过滑点限制, 执行失败

账户余额或授权状态发生变化

交易被其他交易抢先

链发生短暂重组

即使交易失败，验证者仍然执行过相关计算，因此通常仍会消耗 Gas。

串联理解

假设用户准备买入价值 100 万 USDC 的 ETH：

在 AMM 中，交易会沿资金池曲线执行。

在 订单簿 中，交易会逐档吃掉卖单。

流动性深度 决定市场能否承受这笔大单。

深度不足会造成明显的 价格冲击。

提交交易后，需要等待 区块确认，成交结果才真正写入链上。

因此，大额交易通常会拆单、寻找更深的池、使用聚合器，或者通过限价单和私有交易通道降低价格冲击与 MEV 风险。

综合案例：ETH/稳定币套利与风险控制

一、市场背景

交易者小林有 100,000 USDC，提前分布在两条链上：

位置	资金
Ethereum	60,000 USDC
Arbitrum	40,000 USDC

此时观察到：

市场	类型	ETH 价格
Ethereum Uniswap	AMM	3,000 USDC
Ethereum SushiSwap	AMM	3,018 USDC
某订单簿 DEX	订单簿	最优卖价 3,003，最优买价 3,015
Arbitrum 某 DEX	AMM	2,985 USDC

同时：

1 DAI = 0.995 USDC

借贷协议中还有一个抵押率不足、即将被清算的账户。

二、第一步：比较 AMM 与订单簿

AMM 报价

Uniswap ETH/USDC 池的初始状态假设为：

1,000 ETH

3,000,000 USDC

池内参考价格：

$3,000,000 \div 1,000 = 3,000 \text{ USDC/ETH}$

如果小林用 30,000 USDC 买入 ETH，忽略手续费：

$k = 1,000 \times 3,000,000$

交易后 USDC = 3,030,000

交易后 ETH ≈ 990.10

得到约：

$1,000 - 990.10 = 9.90 \text{ ETH}$

平均成交价约为：

$30,000 \div 9.90 \approx 3,030 \text{ USDC/ETH}$

虽然页面显示的初始价格是 3,000，但实际平均成交价格约为 3,030。这就是价格冲击。

订单簿报价

订单簿上的卖单可能是：

卖价	数量
3,003	3 ETH
3,006	4 ETH
3,010	5 ETH

买入 9.9 ETH 会依次吃掉多个价位。平均成交价格可能约为 3,007，比上述 AMM 更好。

但还需要比较：

订单簿交易费

链上 Gas

可成交数量

提现或跨链费用

订单是否会在提交期间被撤销或吃掉

因此，不能只比较页面上的“最新价格”，而要比完整成交后的实际结果。

三、第二步：判断流动性深度

小林分别模拟买入：

买入金额	Uniswap 预计均价	小型 AMM 预计均价
1,000 USDC	约 3,001	约 3,015
10,000 USDC	约 3,010	约 3,150
30,000 USDC	约 3,030	可能超过 3,400

大型池对大额交易的承受能力更强，即流动性深度更好。

实际操作时，在 DEX 页面依次输入 1,000、10,000 和 30,000 USDC，观察：

预计获得的 ETH

Price Impact

Minimum Received

路由经过哪些池

网络费用

不要直接点击确认，先用报价变化判断池子的深度。

四、第三步：执行 DEX 间套利

小林发现：

Uniswap 买入均价：约 3,006 USDC

SushiSwap 可卖出均价：约 3,015 USDC

假设用 15,000 USDC：

买入 ETH $\approx 4.99 \text{ ETH}$

卖出收入 $\approx 15,045 \text{ USDC}$

账面毛利润 $\approx 45 \text{ USDC}$

但需要扣除：

两次 DEX 手续费：约 30 USDC

Gas：约 8 USDC

MEV 或价格变化损失：未知

预期净利润：

$45 - 30 - 8 = 7$ USDC

这笔交易看似有价差，但实际利润太薄，不值得执行。任何价格变化都可能让它亏损。

实际操作

普通用户分两笔操作：

1. 在低价 DEX 用 USDC 买 ETH。
2. 等第一笔确认后，在高价 DEX 卖出 ETH。

问题是两笔交易不是同时完成的。第一笔确认后，第二个 DEX 的价格可能已经变化。

专业套利通常使用智能合约，把“买入 + 卖出”放进同一笔原子交易：

如果最终收入 > 本金 + 费用：全部执行

否则：整笔回滚

回滚可以避免持仓亏损，但失败交易仍可能消耗 Gas。

五、第四步：尝试三角套利

同一条链上出现以下汇率：

1 ETH = 3,000 USDC

1 ETH = 3,012 DAI

1 DAI = 0.999 USDC

交易路径：

30,000 USDC

→ 10 ETH

→ 30,120 DAI

→ 30,089.88 USDC

毛利润约为：

$30,089.88 - 30,000 = 89.88$ USDC

如果三次兑换费率均为 0.05%，总手续费约为：

$30,000 \times 0.05\% \times 3 \approx 45$ USDC

再扣除 Gas 和价格冲击，实际利润可能只剩约 30 USDC。

注意事项

三角套利必须使用每一步的真实输出数量计算，不能只用页面价格相乘。尤其要计入：

三次交易费

每个池的价格冲击

稳定币汇率偏差

Gas

MEV 竞争

代币转账税或特殊机制

六、第五步：稳定币套利

此时 DAI 因市场波动跌到：

1 DAI = 0.995 USDC

小林用 9,950 USDC 买入约 10,000 DAI。

如果之后 DAI 恢复至 1 USDC：

卖出价值约 10,000 USDC

毛利润约 50 USDC

但稳定币低于 1 美元不一定是无风险机会。价格偏离可能源于：

储备资产出现问题

抵押物不足

跨链版本无法赎回

智能合约被攻击

市场失去信心

流动性枯竭

因此，操作前要确认所交易稳定币的合约地址、发行机制、赎回渠道和链上版本。

七、第六步：跨链套利

Arbitrum 的 ETH 为 2,985 USDC，Ethereum 为 3,015 USDC，表面价差约 1%。

较慢的方法

Arbitrum 买 ETH

→ 使用跨链桥转到 Ethereum

→ 在 Ethereum 卖出

问题在于跨链需要时间。等 ETH 到达 Ethereum 时，价差可能已经消失。

此外还要支付：

Arbitrum 买入费

跨链桥费

Ethereum Gas

Ethereum 卖出费

跨链等待期间的价格风险

更实用的方法

提前在两条链上准备资产：

Arbitrum：用 USDC 买入低价 ETH

Ethereum：同时卖出已有 ETH

这样不需要等待跨链，两个方向可以近似同时完成。套利结束后再选择费用较低的时间进行资金再平衡。

需要特别确认：

两条链上的代币是否为同一经济资产

是否为官方桥接版本

跨链桥的安全性

充值、提现和最终确认时间

两边流动性是否足够

八、第七步：清算机会

某用户在借贷协议中：

抵押品：10 ETH

ETH 当前价格：2,400 USDC

抵押品价值：24,000 USDC

债务：18,000 USDC

价格下跌后，该账户的健康因子低于 1，可以被清算。

假设协议允许清算人偿还 5,000 USDC 债务，并获得价值 5,250 USDC 的 ETH：

清算奖励 = 250 USDC

清算人马上在 DEX 卖出 ETH。扣除价格冲击、DEX 手续费和 Gas 后，可能剩余 150 USDC。

实际流程

监控健康因子

- 发现可清算账户
- 准备债务资产
- 调用协议清算合约
- 获得折价抵押品
- 在 DEX 卖出抵押品

清算竞争非常激烈。人工点击通常来不及，专业参与者会使用机器人、节点和自动报价系统。

如果用户是借款人，应重点关注：

健康因子不要接近 1

价格下跌时及时补充抵押品

或主动偿还部分债务

不要依赖交易平台通知才采取行动

九、MEV 如何影响上述操作

假设小林在 AMM 提交一笔大额 ETH 买单，并把滑点容忍度设为 5%。

机器人看到待确认交易后，可能执行：

机器人先买 ETH

- 小林的大单以更高价格买入
- 机器人卖出 ETH

这就是三明治攻击。

防护措施

实际交易时应：

避免设置过高的滑点容忍度

使用私有交易通道或带 MEV 保护的 RPC

大单拆分，但要考虑多次 Gas

使用限价单或意图型交易

选择流动性较深的池

设置合理的交易截止时间

检查最低可获得数量

不要照搬前端默认滑点

滑点也不能设得过低，否则市场稍微变化，交易就会失败并浪费 Gas。

十、区块确认与最终结果

小林确认交易后，过程如下：

钱包签名

- 交易广播
- 进入待处理队列
- 验证者或构建者排序
- 交易进入区块
- 获得第一次确认
- 后续区块增加
- 达到最终确定

要检查的内容包括：

状态是 Success 还是 Reverted

实际收到多少代币

实际 Gas 费用

代币转账记录

执行经过哪些池

是否存在异常的前置和后置交易

“钱包显示已发送”不代表成交成功。“交易失败”也不代表没有费用，失败交易通常仍会消耗 Gas。

跨链操作要分别确认：

源链交易成功

→ 跨链消息被确认

→ 目标链资金到账

不能只看到源链交易成功，就认为跨链已经完成。

一套可执行的检查清单

每次操作前，按下面顺序检查：

1. 核对网络、代币名称和合约地址。
2. 比较多个 DEX 的真实成交报价。
3. 检查池子或订单簿的流动性深度。
4. 查看价格冲击和最低可获得数量。
5. 计算全部交易费、Gas 和跨链费用。
6. 将套利利润与最坏情况成本比较。
7. 设置合理滑点和交易截止时间。
8. 大额交易考虑 MEV 保护。
9. 先授权必要金额，避免无限授权。
10. 签名前再次核对钱包弹窗。
11. 等待区块确认并在区块浏览器检查结果。
12. 记录真实收入、成本和失败交易费用。

最终应使用以下公式，而不是只看价差：

净利润

= 卖出收入

- 买入成本

- DEX 手续费

- Gas

- 跨链费用

- 价格冲击

- 借贷或闪电贷费用

- 失败交易的预期成本

只有净利润为正，并且能够覆盖价格变化、MEV、合约和跨链风险时，才算真正存在可执行机会。

笔记 07

作者：ChnPsych | 时间：2026-08-06 15:19 UTC | 字数：9035

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、DEX 内套利

1. 定义

DEX 内套利是指在同一个 DEX 内部，利用不同池子或不同兑换路径之间的价格差异完成套利。

常见结构包括：

同一交易对存在多个流动性池；
直接路径和间接路径报价不同；
多资产池内部的资产比例失衡；
不同池子的手续费、深度或流动性分布不同。

例如，同一个 DEX 内存在：

USDC → ETH

以及：

USDC → DAI → ETH

当两条路径最终得到的 ETH 数量存在明显差异时，就可能形成路径套利。

2. 机会来源

DEX 内部的报价差异通常来自：

某个池刚发生大额交易，价格暂时偏离；
一个池很浅，另一个池较深；
不同池子的交易流不同；
多跳路径没有及时被其他套利者修正；
某些低流动性池更新较慢。

3. 执行特点

如果全部操作发生在同一条链上，通常可以通过一个智能合约在一笔交易中完成：

借入或准备资产

↓

在低价路径买入

↓

在高价路径卖出

↓

归还资金

↓

保留利润

这种执行方式具有原子性：所有步骤要么全部成功，要么全部回滚。因此不会出现“买入完成但卖出失败后，被迫持仓”的典型风险。

不过，原子性只消除了部分腿风险，不能消除：

Gas 损失；
交易回滚；
被更快的套利者抢先；
报价模拟与真实执行之间的误差；
智能合约漏洞。

循环路径套利在 DEX 中长期存在，而且可以利用智能合约原子执行；但真正的最优交易量必须同时考虑池子价格曲线、手续费和自身造成的价格冲击。(arXiv (https://arxiv.org/abs/2105.02784?utm_source=chatgpt.com "Cyclic Arbitrage in Decentralized Exchanges"))

4. 笔记关键词

范围：同一 DEX

原子性：通常可以

主要成本：Gas、手续费、滑点

核心难点：路径搜索、最优交易量、竞争速度

二、DEX 间套利

1. 定义

DEX 间套利是指同一资产在同一条链上的不同 DEX 之间存在价格差。

例如：

DEX A : 1 ETH = 1,900 USDC

DEX B : 1 ETH = 1,920 USDC

理论操作是：

在 DEX A 用 USDC 买入 ETH

↓

在 DEX B 卖出 ETH

↓

得到更多 USDC

2. 为什么会出现价差

不同 DEX 拥有独立的流动性池，因此会受到不同交易行为影响：

DEX A 刚出现大额卖单；

DEX B 的池子更深；

两个平台手续费不同；

某个聚合器将大量订单导向特定池子；

不同 AMM 曲线导致相同规模交易的价格冲击不同。

套利者的交易会买入相对便宜的资产，并卖出相对昂贵的资产，从而推动两个池子的报价重新趋近。

3. DEX 内与 DEX 间的区别

DEX 内：同一协议，不同池子或不同路径

DEX 间：不同协议之间的价格差

二者在技术上可以非常相似。只要发生在同一条链上，通常都能被打包进一笔原子交易。

4. 关键计算

不能只比较界面显示的“当前价格”，而要计算指定交易量下的真实输出：

```
[
P_{\text{effective}}
=====
frac{\text{实际收到的资产}}{\text{实际投入的资产}}
]
```

最终判断条件是：

```
[
text{卖出所得}
<br />
text{买入成本}
+
text{两边手续费}
+
text{Gas}
+
```

```
text{价格冲击}
+
text{竞争成本}
]
```

交易规模过小，利润可能覆盖不了

Gas；交易规模过大，又会因价格冲击把价差吃掉。因此每个机会都存在一个近似的最优输入金额。

5. 笔记关键词

范围：同链、不同 DEX

原子性：通常可以

资金方式：自有资金、闪电贷或预置库存

主要难点：报价聚合、交易量优化、抢跑竞争

三、跨链套利

1. 定义

跨链套利是指同一种或等价资产在不同区块链上的价格不同。

例如：

Arbitrum：ETH = 1,900 USDC

Base：ETH = 1,930 USDC

理论上可以在 Arbitrum 买入 ETH，在 Base 卖出 ETH。

但不同链之间没有天然的单笔交易原子性，所以跨链套利的结构明显复杂于同链套利。

2. 两种主要执行方式

方式 A：预置库存

套利者提前在两条链上都准备资产：

链 A：持有 USDC

链 B：持有 ETH

发现机会时同时执行：

链 A：USDC → ETH

链 B：ETH → USDC

之后再慢慢进行库存再平衡。

优点：

不需要等待资产跨链；

执行速度更快；

减少跨链期间的价格风险。

缺点：

大量资金被分散在多条链；

会产生库存偏移；

后续仍需支付再平衡成本。

方式 B：桥接后交易

链 A 买入资产

↓

通过跨链桥转移

↓

链 B 卖出资产

主要问题是桥接需要时间。在这段时间里，价差可能已经消失，甚至反向。

跨链套利研究通常将其区分为使用两侧预置库存的独立交易，以及依赖跨链桥、具有先后依赖关系的交易。后者由于不是原子执行，会暴露于桥接延迟、最终性和价格变化风险。(arXiv (https://arxiv.org/abs/2501.17335?utm_source=chatgpt.com "Pandora's Box: Cross-Chain Arbitrages in the Realm of Blockchain Interoperability"))

3. 跨链特有风险

跨链桥费用；
目标链 Gas；
源链和目标链确认时间不同；
桥接额度和流动性限制；
区块链重组或最终性风险；
跨链桥合约及验证机制风险；
资产在两条链上的表示可能并不完全等价；
跨链过程中价差消失；
库存长期失衡。

跨链桥涉及多链状态验证、消息传递和资产托管，其攻击面通常比单一链上的普通合约更复杂。(arXiv (https://arxiv.org/abs/2312.12573?utm_source=chatgpt.com "SoK: Security of Cross-chain Bridges: Attack Surfaces, Defenses, and Open Problems"))

4. 笔记关键词

范围：不同链
原子性：通常没有
推荐结构：两侧预置库存
主要难点：延迟、桥风险、库存管理、资金占用

四、三角套利

1. 定义

三角套利是指通过三个资产形成闭环：

$A \rightarrow B \rightarrow C \rightarrow A$

例如：

$USDC \rightarrow ETH \rightarrow BTC \rightarrow USDC$

假设投入 1,000 USDC，执行完整路径后得到1,008 USDC，在扣除全部成本后仍有剩余，就形成三角套利。

2. 理论判断公式

设三段有效兑换率分别为：

[
 r_{AB}, r_{BC}, r_{CA}
]

不考虑成本时，套利条件为：

[
 $r_{AB} \times r_{BC} \times r_{CA} > 1$
]

考虑成本后：

[
 $r_{AB} \times r_{BC} \times r_{CA}$

$1 + \text{总成本率}$
]

但在 AMM 中，兑换率不是固定常数，而会随输入金额发生变化。因此真正需要计算的是：

[
 $Q_{\text{final}}(Q_{\text{initial}})$

 Q_{initial}
 $C(Q_{\text{initial}})$
]

并寻找使净利润最大的输入量。

3. 三角套利和 DEX 内套利的关系

三角套利通常是 DEX 内套利的一种，但也可能跨越多个 DEX：

DEX A : USDC $\rightarrow$ ETH

DEX B : ETH $\rightarrow$ BTC

DEX C : BTC $\rightarrow$ USDC

“DEX 内、DEX 间”描述的是场所范围；“三角套利”描述的是交易路径结构。它们不是互斥分类。

4. 主要风险

三次交易意味着更高的累计手续费；
某一跳池子过浅，导致严重滑点；
中间代币存在转账税或异常逻辑；
代币精度、报价单位或包装资产处理错误；
路径过长导致 Gas 太高；
交易在区块内被其他 Searcher 抢先。

5. 笔记关键词

结构：A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A

条件：闭环后的资产数量增加

优势：可原子执行

难点：图搜索、路径剪枝、交易量优化

五、稳定币套利

1. 定义

稳定币套利是利用稳定币之间或稳定币与其目标锚定价格之间的偏差。

例如：

USDC = 0.997 USDT

如果确认两者的信用和赎回价值相近，套利者可能买入较便宜的 USDC，等待或通过其他渠道按接近1美元的价格卖出。

2. 常见结构

池间价差

池 A : USDC/USDT = 0.998

池 B : USDC/USDT = 1.001

在池 A 买入 USDC，再在池 B 卖出。

DEX 与赎回渠道之间

部分稳定币存在发行、铸造或赎回机制。符合资格的参与者可以在链上折价买入稳定币，再通过发行方赎回接近面值的资产。

这里需要严格区分：

协议层面确定可赎回；

仅仅因为历史上曾经回到1美元而猜测会重新挂钩。

后者不是严格套利，而是承担信用风险的方向性交易。

多稳定币循环

USDC → USDT → DAI → USDC

本质上也是三角套利。

3. StableSwap 的作用

普通恒定乘积池在两种价格接近的资产之间交易时，资本效率不一定理想。Curve 的 StableSwap 将接近恒定和与恒定乘积的定价特性结合，使锚定资产在接近1:1时可以获得更低滑点和更深的有效流动性。(Curve News (https://news.curve.finance/master-blockchain-development-with-cyfrin-updraft-from-stableswap-to-vyper-smart-contracts/?utm_source=chatgpt.com "Master Blockchain Development with Cyfrin Updraft: From Stableswap to Vyper Smart Contracts"))

但这意味着稳定币价差通常非常薄，竞争者需要：

更低的 Gas；

更快的执行；

更大的资金量；

更准确的脱锚风险判断。

4. 最大误区：把脱锚当套利

稳定币跌到0.95美元，不代表必然能赚5%。

需要先分析：

储备资产是否充足；

是否能够正常赎回；

发行方是否冻结或限制提款；

跨链版本是否由可靠资产支持；

价格偏差来自短期流动性，还是偿付能力问题；

池内是否已经高度单边化。

StableSwap 池虽然为锚定资产提供较高资本效率，但一旦某个资产真正脱锚，流动性提供者和套利者都可能集中持有正在贬值的资产。针对 Curve 池的研究也把稳定币脱锚视为需要专门监控的尾部风险。(arXiv (https://arxiv.org/abs/2306.10612?utm_source=chatgpt.com "Detecting Depegs: Towards Safer Passive Liquidity Provision on Curve Finance"))

5. 笔记关键词

机会：小价差、高频率

核心前提：资产确实等价或可赎回

最大风险：信用风险、赎回中断、永久脱锚

注意：赌重新挂钩不等于无风险套利

六、清算套利

1. 定义

借贷协议要求借款人提供超额抵押品。当抵押品价值下降、债务价值上升，或协议参数发生变化时，仓位可能低于清算要求。

清算者会：

替借款人偿还部分债务

↓

以折扣获得其抵押品

↓

出售抵押品或进行对冲

↓

保留清算奖励

2. 健康因子

以 Aave 的表达方式为例：

```
[
text{Health Factor}
=====

frac{
text{抵押品总价值}times
text{加权清算阈值}
}{
text{债务总价值}
}
]
```

当健康因子低于协议规定的清算边界时，仓位可以被清算。清算者偿还债务，并获得相应抵押品及清算奖励。具体可清算比例和奖励由协议、资产及治理参数决定。(aave.com

(https://aave.com/help/borrowing/liquidations?utm_source=chatgpt.com "Health Factor & Liquidations | Aave"))

3. 利润计算

```
[
text{清算净利润}
=====

text{获得抵押品价值}
text{偿还债务}
text{Gas}
text{闪电贷费用}
text{卖出滑点}
text{竞争成本}
]
```

清算奖励并不等于最终净利润。获得的抵押品可能在卖出过程中继续下跌，或者池子没有足够深度。

4. 技术流程

读取借贷仓位

↓

读取抵押物和债务价格

↓

计算健康因子

↓

监听预言机更新

↓

模拟清算交易

↓

计算最优偿还数量

↓

提交高优先级或私有交易

↓

卖出或对冲抵押品

5. 竞争特点

成熟借贷协议的清算通常由专业机器人完成。竞争集中在：

谁最早发现仓位可清算；

谁最快响应预言机价格更新；

谁能以更低成本获得偿债资金；

谁愿意支付更高区块优先费用；

谁能更有效地出售抵押品。

Aave 官方材料也指出，清算是无许可且高度竞争的，实际执行通常需要自动化监控和定制化程序。(aave.com (https://aave.com/help/borrowing/liquidations?utm_source=chatgpt.com "Health Factor & Liquidations | Aave"))

6. 笔记关键词

触发条件：仓位低于抵押要求

收入：清算奖励

风险：价格继续下跌、预言机延迟、Gas 竞争

关键能力：仓位监控、模拟、快速提交

七、MEV

1. 定义

MEV，即最大可提取价值，指区块生产者或交易参与者通过改变交易的：

包含；

排除；

顺序；

前后位置；

在普通区块奖励和 Gas 之外提取的额外价值。(ethereum.org (https://ethereum.org/developers/docs/mev/?utm_source=chatgpt.com "Maximal extractable value (MEV) | ethereum.org"))

参与者通常包括：

用户

↓

Searcher：寻找机会并构造交易

↓

Builder：构建候选区块

↓

Validator / Proposer：选择并确认区块

具体架构会因链和区块构建市场而不同。

2. 常见 MEV 结构

套利

大额交易使某个 DEX 价格发生偏移，Searcher 随后交易，将价格拉回其他市场水平。

用户交易

↓

池子价格偏离

↓

套利交易紧随其后

↓

价格恢复

这通常叫 backrun。

清算

当预言机更新使借贷仓位达到清算条件时，多个机器人争夺最先执行清算交易的位置。

Sandwich

攻击者在用户大额交易前买入，在用户交易推高价格后卖出：

攻击者买入

↓

用户以更差价格买入

↓

攻击者卖出

这会增加用户滑点，属于对用户明显不利的

MEV

形式。这里只需要认识结构，不应把操纵普通用户交易作为学习项目。

抢先交易

观察到尚未确认的交易后，使用更高优先级复制或抢在目标交易之前执行。

3. MEV 与普通套利的关系

并非所有套利都必须依赖交易排序，但多数链上套利都会受到排序影响。

可以这样理解：

套利：机会来自价格不一致

MEV：利润依赖区块中的交易位置或排序权

DEX 套利和清算既是经济机会，也是典型 MEV 场景。它们有助于修正价格或维持借贷协议偿付能力；而 Sandwich 通常会恶化普通用户的成交结果。成熟 MEV 市场竞争极其激烈，机会收益的很大部分可能被用于 Gas 或区块位置竞价。(ethereum.org (https://ethereum.org/developers/docs/mev?utm_source=chatgpt.com "Maximal extractable value (MEV) | ethereum.org"))

4. 笔记关键词

核心资源：交易排序权

常见形式：套利、清算、Backrun、Sandwich

主要成本：优先费、Bundle 竞价、失败交易

核心竞争：信息、速度、模拟、区块空间

八、七类机会对照表

类型	机会来源	是否通常原子	主要风险	初学难度
：----	：-----	：-----	：-----	：---
DEX 内	同协议池子或路径报价不同	是	滑点、Gas、抢跑	中
DEX 间	同链不同协议价格不同	是	深度、抢跑、交易回滚	中
跨链	不同链价格及流动性分割	否	桥、延迟、库存、最终性	高
三角套利	闭环兑换率不一致	是	多跳手续费、路径错误	中高
稳定币套利	锚定资产价格偏离	视结构而定	脱锚、信用与赎回风险	中高
清算	抵押仓位低于清算线	通常是	预言机、价格下跌、竞争	高
MEV	交易排序和区块空间	视结构而定	竞价、失败、伦理问题	高

九、统一分析框架

看到一个所谓套利机会时，依次回答以下问题。

1. 价差是否真实？

不要看显示价格，要用指定输入金额调用报价函数或模拟交易。

报价是否包含手续费？

是否使用真实池子状态？

池子是否有转账税代币？

输入金额是否会显著改变价格？

2. 能否原子执行？

同一链、同一交易：通常可以原子执行

不同链：通常不能完全原子执行

不能原子执行时，就会产生持仓和价格风险。

3. 全部成本是多少？

至少包括：

DEX 手续费

Gas

滑点

闪电贷费用

优先费

桥费

库存再平衡成本

失败交易的预期成本

4. 机会能否被别人抢走？

公有内存池中的明显套利交易可能被：

复制；

抢跑；

提高优先费替代；

在前一个区块中提前执行。

5. 最坏情况是什么？

需要事先判断：

交易回滚时亏多少？

某一步成功、另一部分失败时亏多少？

稳定币不再回锚时亏多少？

跨链桥暂停时资金在哪里？

抵押品无法卖出时承担什么风险？

十、今日笔记总结

可以用一句话记住每种结构：

DEX 内：同一家商场，不同柜台价格不一样。

DEX 间：不同商场，同一商品价格不一样。

跨链：不同城市的商场价格不一样，但运输需要时间。

三角套利：换三次钱，最后换回更多起始资产。

稳定币套利：利用“应该等于1”与“实际不等于1”的差距。

清算套利：替高风险借款人还债，折价拿走抵押品。

MEV：利用交易在区块中的先后顺序获得收益。

最重要的三条认识是：

1. 价差不等于利润，必须扣除所有显性和隐性成本。
2. 原子套利不等于零风险，交易失败、Gas、竞争和合约风险仍然存在。
3. 稳定币折价和跨链价差尤其不能简单视为无风险套利，其中往往包含信用、延迟、库存和基础设施风险。

笔记 08

作者：gt66-66 | 时间：2026-08-06 12:28 UTC | 字数：8802

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天做了套利学习计划书1. Hermes 做了总结 以及推荐路线

唯一推荐路线：先做“DeFi 收益再平衡 Agent”，不要直接从 DEX 套利机器人开始。

推荐学习周期：16 周，总计约 200 小时。

推荐顺序：

1. 区块链与 DeFi 基础
2. Solidity 与合约安全
3. DEX、借贷、收益金库
4. 链上数据读取与收益计算
5. 策略模拟与风险过滤
6. MetaMask Agent Wallet + LI.FI
7. 测试网自动再平衡
8. 小额实盘
9. 最后再学习 DEX 套利与 Flashbots

推文中的模式严格说是收益再平衡，不是传统意义上的无风险套利：

类型	收益来源	难度	主要风险	推荐顺序
收益再平衡	在不同借贷池或金库之间迁移资金	中	APY 下降、合约、跨链、Gas	1
DEX 价差套利	同一资产在不同池之间存在价差	高	滑点、抢跑、交易失败、Gas	2
清算机器人	清算不健康借贷仓位并获得奖励	很高	竞争、预言机、执行延迟	3
MEV 搜索	利用交易排序和区块空间机会	极高	专业竞争、基础设施成本	4

2. 改动

16 周详细学习规划

周	时间	学习内容	实践任务	完成标准
1	10 小时	钱包、私钥、公钥、签名、RPC、交易、Gas、Nonce	创建独立学习钱包；连接测试网；读取余额和区块	能解释签名、Gas、RPC；不暴露私钥
2	10 小时	EVM、合约调用、ABI、事件、ERC-20、授权机制	读取 ERC-20 余额、授权额度和历史事件	能区分 approve、transfer、transferFrom
3	12 小时	Solidity 基础、状态变量、函数、错误处理、事件	编写简单金库合约	Foundry 编译、测试全部通过
4	12 小时	重入、权限、无限授权、价格操纵、闪电贷风险	给金库加入权限、暂停、提款限制	测试未授权调用、重入和暂停场景
5	12 小时	AMM、恒定乘积、流动性、价格影响、手续费	用 Uniswap SDK 获取报价	能计算报价、价格影响和最低到账量
6	12 小时	Uniswap V3/V4、集中流动性、Tick、Route	比较两个池的同币种报价	能解释“看到价差但无法盈利”的原因
7	14 小时	Aave V3：存款、借款、利率、健康因子、清算	读取市场利率和用户仓位	

能计算健康因子变化及清算风险 |

| 8 | 14 小时 | 收益金库、APY、TVL、奖励币、退出流动性 | 建立收益机会表 |
每项收益包含协议、链、TVL、APY、风险 |

| 9 | 14 小时 | Li.Fi Quote、Route、跨链、兑换、交易状态 | 获取一条迁移路线但不执行 |
能得到金额、Gas、费用、滑点和预计到账 |

| 10 | 14 小时 | MetaMask Agent Wallet、权限、交易模拟、限额 | 创建专用 Agent Wallet ; 仅测试网操作 | Agent
无权转出到白名单外地址 |

| 11 | 14 小时 | 再平衡策略：净收益、阈值、最短持有时间 | 编写只读机会扫描器 | 每个建议都显示完整收益计算 |

| 12 | 14 小时 | Agent 决策约束、结构化结果、拒绝规则 | AI 只输出候选动作，不直接签名 |
相同输入可得到符合固定格式的结果 |

| 13 | 12 小时 | 测试网执行：提款、兑换、存入、交易确认 | 完成一次测试网迁移 | 全部交易成功，余额变化可核对 |

| 14 | 12 小时 | 故障恢复：RPC 中断、报价失效、交易卡住 | 注入失败场景 | 程序停止且不重复提款或重复存款 |

| 15 | 12 小时 | 监控、告警、每日损益、授权检查 | 建立只读看板与告警 | 能显示本金、净收益、费用、未完成交易 |

| 16 | 12 小时 | 小额实盘规则与复盘 | 仅投入可完全损失的小额资金 | 连续运行 30 天后再决定是否扩大资金 |

第一版产品范围

| 部分 | 第一版要求 |

| :--- | :----- |

| 支持网络 | 只选一个低费用 EVM 网络 |

| 支持资产 | 只用 USDC 等单一资产 |

| 收益来源 | 只比较 2-3 个经过筛选的借贷市场或金库 |

| 数据来源 | 官方协议接口、链上数据和 Li.Fi 报价 |

| 判断频率 | 每 6-24 小时检查一次 |

| 实际迁移 | 建议按 30 天最短持有周期评估，不频繁搬动 |

| 执行方式 | 前期必须人工确认 |

| 钱包 | 独立 Agent Wallet，不使用主钱包 |

| 授权 | 精确额度、限定协议、限定资产、限定网络 |

| 日限额 | 设置单笔和每日最大支出 |

| 禁止项 | 无限授权、未知协议、未审计合约、自动跨入陌生资产 |

再平衡判断规则

只有同时满足以下条件才允许提出迁移建议：

1. 目标协议 TVL 达到预设下限；
2. 目标资产与原资产一致，或兑换风险可计算；
3. 目标 APY 持续达到观察期要求；
4. 目标与原方案的 APY 差达到阈值；
5. 扣除 Gas、兑换费、滑点、跨链费后仍有正收益；
6. 达到预计回本周周期要求；
7. 协议和合约地址在白名单；
8. 未发现提款限制、暂停状态或流动性不足；
9. 交易模拟成功；
10. 不突破单笔、单日和总资金限额。

建议使用以下判断：

预计净收益 = 新方案预计收益 - 原方案预计收益 - Gas 协议费用 - 兑换费用 - 滑点 - 跨链费用 - 风险预留

迁移条件示例：

APY 差异至少 1%—2%；
TVL 至少 100 万美元；
预期收益覆盖迁移成本后仍达到安全余量；
回本周周期不能超过计划持有期；
APY 不能只出现一次短时峰值。

Agent 的职责边界

部分	AI Agent 可以做	AI Agent 不应独立做
数据	整理 APY、TVL、费用、风险	信任单一网页数字
判断	排序机会、解释选择	凭自然语言猜测合约地址
规划	生成提款、兑换、存入步骤	绕过资金和协议白名单
执行	在限制内提交已模拟交易	持有主钱包私钥
风控	拒绝异常报价和高风险协议	自动关闭安全检查
复盘	统计真实净收益	把浮动 APY 当作固定收益

推荐项目顺序

项目一：只读收益扫描器

读取不同市场的：

APY ;

TVL ;

可提款流动性；

奖励构成；

链和资产地址。

输出：

原始 APY ;

扣费后 APY；

风险等级；

是否建议观察。

不连接私钥，不发送交易。

项目二：再平衡模拟器

输入：

当前本金；

当前 APY ；

目标 APY；

Gas ;

滑点；

持有天数。

输出：

预计新增收益；

总迁移成本；

回本天数；

是否值得迁移。

使用历史报价或固定测试数据验证。

项目三：测试网执行器

固定协议白名单：

固定资产白名单；

固定金额上限；

执行前模拟；
提款、兑换、存入分步确认；
任一步失败立即停止。

项目四：受限 Agent

AI 只生成结构化建议；
确定性规则负责批准或拒绝；
钱包负责最终权限控制；
每笔交易保存：
决策依据；
报价；
模拟结果；
实际 Gas；
实际到账；
最终收益。

项目五：小额实盘

独立钱包；
初始只运行一个网络、一个资产；
每笔必须人工确认；
连续 30 天统计：
总收益；
Gas；
滑点；
失败成本；
实际年化；
最大资金回撤。
净收益稳定为正，才考虑放宽执行权限。

项目六：DEX 套利研究版

同一区块读取多个池报价；
计算交易后价格而非屏幕价格；
扣除池费、Gas、滑点和区块竞价成本；
使用主网分叉模拟；
确认交易整体失败时不会产生本金损失；
只用于学习，暂不连接实盘资金。

3. 原因

为什么先学收益再平衡

交易频率较低，便于检查每个决定；
不依赖毫秒级速度；
不需要和专业 MEV 搜索者直接竞争；
更容易设置协议、资产、金额和提款地址限制；
可以先完成“读取—建议—人工确认”，再逐步增加自动化；
与原推文展示的 MetaMask Agent Wallet + LI.FI Earn 路线一致。

为什么不建议直接做 DEX 套利

可见价差不等于可执行利润；
报价会因自己的交易而改变；
公共交易池会暴露机会；

其他机器人可能抢先成交；
失败交易仍可能消耗 Gas；
跨链交易通常不能保证两端同时完成；
Flash Loan 只能减少本金需求，不能消除亏损和竞争；
AI 不适合作为毫秒级定价与执行核心。

推荐组成

数据层：链上 RPC、协议官方接口、LI.FI Quote/Route；
计算层：确定性程序计算 APY、成本、滑点和回本周周期；
AI 层：整理候选机会、解释风险、生成操作建议；
安全层：白名单、限额、模拟、暂停、人工确认；
钱包层：MetaMask Agent Wallet 或受限 Smart Account；
执行层：LI.FI、Aave、收益金库；
监控层：交易状态、余额、授权、实际净收益和异常告警。

学习原则

不让 AI 直接决定金额；
不让 AI 生成未经核验的合约地址；
不使用主钱包；
不把助记词放入提示词、日志或环境输出；
不以 APY 排名直接决定迁移；
不在没有模拟结果时发送交易；
不使用借款或杠杆作为第一版资金来源；
不以推文收益截图作为策略有效证明。

4. 验证

已核验的官方教程与文档

主题	来源
Solidity	< https://docs.soliditylang.org/en/latest/ >
Foundry	< https://getfoundry.sh/ >
OpenZeppelin Contracts	< https://docs.openzeppelin.com/contracts/ >
Uniswap 文档	< https://docs.uniswap.org/ >
Aave 开发文档	< https://aave.com/docs/developers/smart-contracts >
Chainlink Data Feeds	< https://docs.chain.link/data-feeds >
LI.FI 文档	< https://docs.li.fi/ >
MetaMask Agent Wallet	< https://docs.metamask.io/agent-wallet >
MetaMask Smart Accounts Kit	< https://docs.metamask.io/smart-accounts-kit >
Flashbots 文档	< https://docs.flashbots.net/ >

已核验的 GitHub 项目

项目	用途	说明
Foundry	(https://github.com/foundry-rs/foundry)	合约开发、测试、主网分叉 活跃项目
OpenZeppelin Contracts	(https://github.com/OpenZeppelin/openzeppelin-contracts)	安全合约组件 活跃项目
Uniswap V4 Core	(https://github.com/Uniswap/v4-core)	AMM 合约学习 活跃项目
Uniswap SDKs	(https://github.com/Uniswap/sdks)	报价与路线计算 活跃项目
Aave V3 Origin	(https://github.com/aave-dao/aave-v3-origin)	Aave V3.6 合约 活跃项目
Chainlink	(https://github.com/smartcontractkit/chainlink)	预言机与价格数据 活跃项目

LI.FI SDK (<https://github.com/lifinance/sdk>)	跨链、兑换、路线执行	活跃项目
MetaMask Smart Accounts Kit (<https://github.com/MetaMask/smart-accounts-kit>)	受限智能账户	活跃项目
MetaMask Delegation Framework (<https://github.com/MetaMask/delegation-framework>)	权限委托	活跃项目
Scaffold-ETH 2 (<https://github.com/scaffold-eth/scaffold-eth-2>)	Ethereum 学习项目模板	活跃项目
Flashbots Simple Arbitrage (<https://github.com/flashbots/simple-arbitrage>)	套利教学示例	
仅用于学习，不代表当前可盈利策略 |

来源核验结果

MetaMask 官方文档确认 Agent Wallet 当前属于 Early Access。

MetaMask 官方文档确认其支持：

收益金库存取；

Aave V3；

Swap 与 Bridge；

交易模拟；

威胁扫描；

用户设定的执行限制。

LI.FI 官方文档确认其支持：

同链兑换；

跨链兑换与桥接；

多步骤交易；

收益机会与存入；

AI Agent 链上执行场景。

Flashbots 官方文档确认 MEV 套利涉及私有交易、交易排序和区块空间竞争。

当前环境没有名为 twitterapi 的可用 skill，因此未伪称调用该技能；规划依据已核验的官方文档、官方 GitHub 和原始推文。

笔记 09

作者：ChenXuan1985 | 时间：2026-08-05 16:48 UTC | 字数：8397

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

初步研究了一下使用标准网格进行链上/代币化美股跨平台套利的可实践性

原文出处：

[@GodpanSen](https://x.com/GodpanSen/status/2079512871537700921)

[@gch_enbsbxbs](https://x.com/gch_enbsbxbs/status/2079420210935898272)

链上美股跨平台套利 —— 重新尽调

基于 @GodpanSen 海力士实战复盘（\$60万+/月）+ CJ 网格战法 + 链上美股生态

2026-08-05 | Hermes Agent

〇、先还原关键事实

从图片文字中提取的核心操作：

| # | 操作 | 路径 | 收益 | 本质 |

| :- | :----- | :----- | :---- | :----- |

| 1 | 周末价差套利 | Binance 做空 + Hype 做多（同一标的） | +\$1.5万 | 纯跨平台价差收敛 |

| 2 | 现货 vs 合约溢价 | IBKR 买入韩国现货 + Binance 做空合约 | +\$12万 | 现货-期货基差 + 资金费双吃 |

| 3 | 资金费率优化 | 持仓从 Binance 搬家到 OKX | +\$7万 | 同一方向、不同平台的持仓成本套利 |

| 4 | 规则 Bug 极限博弈 | Binance 做空 + Hype 做多 (暴跌日) | +\$15万 | 平台规则差异带来的结构性窗口 |
| 5 | ETF 错价盲盒 | 买入港股 2x ETF + Binance 做空合约 | +\$20万 | 跨资产类别定价错误 |

核心公式：利润 = 价差收敛 + 资金费率差 - 交易成本 - 汇率波动

一、赛道定义：到底什么是"链上美股跨平台套利"

你说的赛道本质是同一底层资产（美股）在不同交易场所出现了不同价格：

┌—— 传统券商 (IBKR/Schwab)
| 标的：AAPL 正股
同一美股 —————┴—— CEX 代币化合约 (Binance/OKX)
| 标的：AAPL-PERP (永续合约, USDT 结算)
└—— 链上/DEX (Hyperliquid/链上协议)
标的：AAPL-PERP (链上永续合约)

这个结构的关键特征是：

特征	说明	套利含义
底层同一	都是 AAPL / TSLA / NVDA	价差终将收敛 (同股同权)
参与者不同	传统市场 = 机构/散户；币圈 = 币圈原住民	散户行为偏差制造溢价
结算货币不同	美元 vs USDT/USDC	增加了稳定币脱锚风险
交易时间不同	美股 9:30-16:00 EST；币圈 24/7	非重叠时段价差扩大
费用结构不同	传统佣金 vs 资金费率	创造了资金费率套利维度

二、海力士原文揭示的五种套利模式 + CJ 网格战法适配

2.1 五种模式 → 系统化分类

模式 A：纯跨平台价差套利 (操作 #1)

Binance AAPL-PERP vs Hyperliquid AAPL-PERP

→ 双边持仓，等收敛

→ CJ 网格：在价差区间内分层建仓，而非一笔打入

模式 B：现货-合约基差套利 (操作 #2)

IBKR 买入 AAPL 正股 + Binance 做空 AAPL-PERP

→ 吃溢价回归 + 资金费率

→ CJ 网格：按基差百分比分层加仓

模式 C：资金费率优化 (操作 #3)

同一个空头持仓，在不同平台间迁徙

→ Binance vs OKX 费率差 = 纯 alpha

→ CJ 网格不直接适用，这是监控+执行优化

模式 D：规则套利 (操作 #4)

Binance 资金费率上限 vs Hype 1h 结算

→ 极度依赖对平台规则的深度理解

→ 不适用网格，这是主观/事件驱动

模式 E：跨资产错价 (操作 #5)

HK 2x ETF vs 币圈合约

→ 需要实时监控跨资产比价

→ 网格可改造为"比价偏离度网格"

2.2 CJ 网格战法最适配的模式

CJ 的网格战法最适合 模式 A (纯跨平台价差) 和 模式 B (现货-合约基差)，因为这两个模式的核心都是：

价差在某个区间内波动，均值回归特性显著

CJ 网格移植到 Binance-Hyperliquid US 股票套利：

价差定义： $\text{Spread} = P_{\text{Binance}}(\text{AAPL}) - P_{\text{Hyperliquid}}(\text{AAPL})$

网格参数：

- 价差区间：\$1.0 ~ +\$3.0（历史波动范围）
- 网格层数：40 层（每层 \$0.10）
- 每层仓位：10 股 AAPL ($\approx \$2,500$)
- 方向：Spread > 0 时 Binance 做空 + Hype 做多
- Spread < 0 时 Binance 做多 + Hype 做空
- 总敞口上限：\$100K

关键适配点：

- 双边同时下单（WebSocket 并发）
- 只在美国非交易时段建仓（波动大、散户非理性）
- 美股开盘前逐层减仓（价差被机构套利抹平）

注意：原文作者最强的信号是“周末”和“盘前/盘后”——此时传统市场关闭，币圈散户自行定价，价差最大。

三、可行性分维度评估（修正版）

3.1 平台覆盖度

平台	美股覆盖	机制	可做空	API 质量
Binance	AAPL/TSLA/NVDA/MSFT/GOOGL 等 ~10 只	永续合约，USDT 结算		
OKX	类似 Binance 范围	永续合约，USDT 结算		
Hyperliquid	较全（RWA 板块 ~20+ 美股）	链上永续合约		（WebSocket 优秀）
IBKR	全部美股 传统正股 需融券	(ib_insync)		
链上协议	Backed / Swarm / Ondo	ERC-20 代币化股票		取决于流动性

3.2 数据可获取性

数据类型	获取方式	延迟
Binance/OKX 行情	CCXT / WebSocket	<100ms
Hyperliquid 行情	官方 WebSocket / Python SDK	<100ms
IBKR 正股行情	ib_insync / TWS API	<50ms
美股实时行情	Polygon / Alpaca / Yahoo	<1s（够用）

数据反而是这个赛道最不用担心的一环——所有平台的 API 都很成熟。

3.3 资金门槛

从原文可以看出：

操作	仓位	收益	所需本金
试探（操作 #1）	1,000 股	+\$1.5 万	~\$15-20 万（双边）
现货套利（操作 #2）	—	+\$12 万	~\$50 万+
极限博弈（操作 #4）	~1000 万美元	+\$15 万	双边 \$500 万+

坦率说：原文作者的本金体量不是小散。操作 #4 单边 1000 万美金仓位，双边至少 \$500 万本金。但模式 A（纯价差）的门槛可以低得多——5,000-10,000 美元即可开始，只需控制杠杆。

3.4 竞争格局

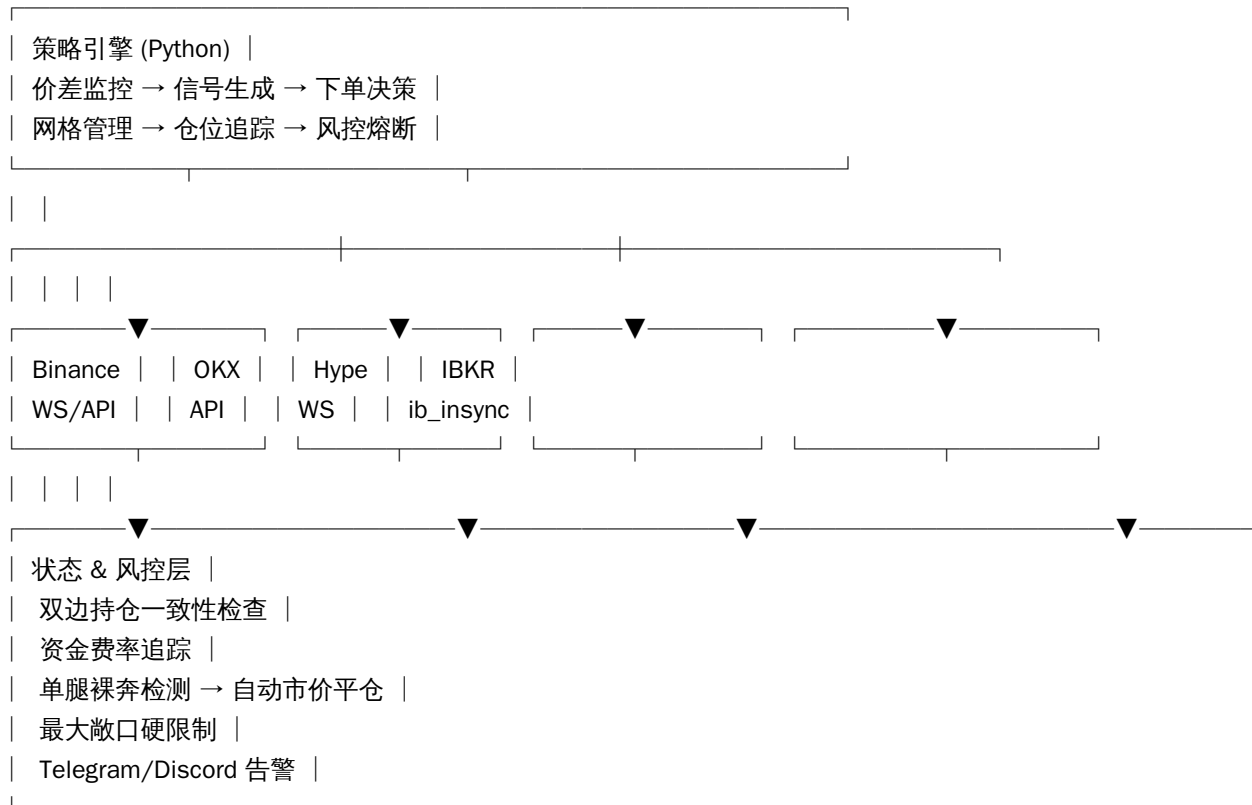
维度	评估
当前玩家	极少——原文评论区明显都在围观，没几个人真在做
进入壁垒	需要同时理解传统金融 + 加密 + 编程——三重门槛

| 机构参与度 | 低——传统对冲基金不会去 Binance 做 AAPL 套利 |

| 结论 | 蓝海，但窗口期不确定 |

这个赛道的保护来自认知门槛而非技术门槛——不像币圈套利那样人人都能 ccxt 一把梭。

四、自动化 Bot 技术架构



技术栈推荐

python

行情层

ccxt # Binance/OKX 统一接口

hyperliquid-python-sdk # Hyperliquid 专用

ib_insync # IBKR

策略层

numpy/pandas # 价差计算

asyncio # 并发 WebSocket 管理

redis # 状态持久化（防重启丢持仓）

执行层

ccxt + ib_insync # 下单

websockets # Hyperliquid 实时行情

关键难点：双边原子性

→ 不能先买后卖/先卖后买

→ 必须 `asyncio.gather()` 并发下单

→ 一侧失败 → 立即反向市价平仓另一侧

原文的最大教训：汇率风险

雷区：币圈合约以 USDT 结算 = 对标“股票的美元价值”

但你买韩国现货用的是韩元

→ 作者因此亏了 \$6 万！

IBKR 实时将 USD 兑换为目标货币 → 买现货

IBKR 汇率损耗极低 (~2bps)

五、风险矩阵（美股特化版）

⋮
⋮
⋮
⋮
⋮

交易所风险 | CEX 暂停提币/下架 | 重大 | 未提及 |

| 资金费率突变 | 平台改规则 | 中等 | 操作 #4 赌对了 |

| 美股停牌 | 正股停牌，合约侧无法对冲 | 中等 | 需预案 |

| 智能合约风险 | Hype/链上 DEX 被攻击 | 中等 | 限制链上敞口 |

与加密货币套利的本质区别

| 价差来源 | 交易所间搬运摩擦 | 散户非理性 + 平台机制差异 + 时区断层 |

| 竞争态势 | 机构做市商充斥 | 几乎无人 |

| 策略衰减速度 | 快 (公开即死) | 慢 (门槛高) |

数据公开度	极高	高（但认知门槛过滤了大部分玩家）
-------	----	------------------

| 你的判断正确 | — | 新赛道 = 更多结构性机会 |

可行性总判

链上美股跨平台套利 bot 可行性：

技术可行性：■■■■■■■■■■ 90% (API 成熟，难度在执行而非开发)

资金门槛：███████取决于模式（模式A \$5K 可起步，模式B \$50K+）

[illegible]

持续性：■■■■■░░░░░中等偏上（门槛保护强，但平台可能下架股票）

监管风险：

推荐起步路径

Phase 1 — 监控系统 (1-2 周)

|—— 接入 Binance/OKX/Hyperliquid 美股 perp 行情

|—— 接入 IBKR 美股实时行情

—— 实时计算跨平台价差 → 存入数据库 → Grafana 面板

Phase 2 — 半自动套利 (2-4 周)

|——报警触发 → 人工确认 → 手动双边下单

|—— 积累价差行为数据（什么时候扩大、什么时候收敛）

——验证假设：非交易时段价差 > 交易时段

Phase 3 — 全自动网格 (4-8 周)

- CJ 风格网格引擎 (双边并发下单)
- 资金费率追踪 + 自动迁徙
- 风控闭环 (单腿检测、最大敞口、自动熔断)

三点建议：

1. 先做监控，别急着下单——花 2 周看价差行为规律，特别是周末和美股盘前盘后
2. 首月只做模式 A (纯跨平台价差) ——最简单、最可控、双边对冲不暴露方向风险
3. 汇率问题在美股上不存在，但 USDT 脱锚风险需要预案——建议定期将 USDT 利润转出到法币

笔记 10

作者：blimnwind-bot | 时间：2026-08-06 01:59 UTC | 字数：8146

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利统一判断框架与 21 天学习大纲

一、从第一性原理定义链上套利

链上套利不是“发现两个页面价格不同”。

一个完整套利机会必须满足：

1. 存在可执行的状态差异
2. 能构造一条完整交易路径
3. 最终资产可以换回统一计价单位
4. 扣除全部成本后仍有净利润
5. 交易失败时的损失可以估计
6. 机会能在竞争者修复之前完成

统一计价单位可以选择 USDC、USDT、ETH 或美元价值。研究阶段建议固定使用 USDC。

二、链上套利的统一分类

第一类：同链原子套利

所有步骤可以放入同一笔交易或同一个 Bundle 中。

包括：

1. 同一 DEX 不同池套利
2. 不同 DEX 之间套利
3. 三角套利
4. 闪电贷套利
5. 部分清算加卖出路径

优势是任意一步不成立时可以让整笔交易回滚。

主要风险是：

1. Gas 损失
2. 状态被其他交易提前改变
3. 模拟与实际区块状态不同
4. 被竞争者提高优先费或抢先执行

第二类：同链非原子套利

需要多笔独立交易，或者不能保证全部步骤处于同一状态。

例如：

1. 先买入资产，等待另一个协议出现退出机会
2. 稳定币脱锚交易
3. 需要人工确认的多阶段策略

主要风险是第一步成交后，第二步价格已经变化。

第三类：跨链库存套利

资产、资金或者对冲仓位分布在多条链上。

包括：

1. 买入后跨链再卖出
2. 两条链预先放置库存，同时买卖
3. 跨链稳定币价差
4. 跨链流动性失衡
5. 桥接资产折价

跨链套利的核心难点是执行不天然原子化。桥接等待期间，价格、Gas、路由和流动性都可能改变。

第四类：状态触发型机会

机会由某个链上状态达到阈值触发。

主要是：

1. 借贷清算
2. 抵押品拍卖
3. 再平衡
4. 稳定机制触发
5. 预言机更新后的价格修复

以 Aave 为例，头寸健康因子低于协议要求时可以进入清算状态，清算者偿还部分债务并获得带有清算奖励的抵押品。清算奖励是否足以覆盖 Gas、兑换成本和竞争成本，才是最终判断标准。(aave.com (https://aave.com/docs/aave-v4/positions/liquidations?utm_source=chatgpt.com))

第五类：排序与区块空间机会

MEV 属于执行和交易排序层，不应与三角套利、跨链套利并列为单一策略。

套利和清算可以产生 MEV。搜索者通过交易排序、回跑交易、Bundle 和私有交易通道争夺利润。Flashbots 的资料将套利机器人和清算机器人都归入搜索者的典型应用。(Flashbots Docs (https://docs.flashbots.net/flashbots-auction/overview?utm_source=chatgpt.com))

三、七道统一判断闸门

任何策略都必须依次通过以下七道闸门。

闸门一：闭环是否成立

必须明确：

1. 初始资产是什么
2. 每一步输入和输出是什么
3. 最终资产是什么
4. 最终资产能否立即卖回统一计价单位
5. 是否需要保留跨链库存
6. 是否留下无法处理的零碎资产

错误做法是把未兑现代币、预言机价格或页面中间价直接当成利润。

最终价值应使用实际可执行卖出报价：

[
 $V_1 = Q_{\text{exit}}(q)$

]

其中 $Q_{\text{exit}}(q)$ 是给定交易规模 (q) 下的真实退出报价。

闸门二：价差为什么存在

必须找到价差的支付者和修复机制。

常见来源：

1. 不同池子承受了不同方向的交易流
2. 链间资金转移存在延迟
3. 流动性集中区间不同
4. 预言机更新与市场成交不同步
5. 强制清算产生非自愿交易
6. 用户为了速度接受较差价格
7. 路由器暂时没有找到全局最优路径
8. Gas 和区块空间拥堵阻止小额套利者参与

如果无法说明利润由谁支付，通常意味着观察到的只是报价口径差异。

闸门三：交易规模是否成立

套利不是一个固定价差，而是一条关于交易规模的利润曲线

重点观察：

1. 交易量放大后输出是否快速恶化
2. 哪个池子首先成为容量瓶颈
3. 利润最大的交易规模
4. 净利润开始变负的规模
5. 机会能容纳多少资本

Uniswap 的 AMM 根据池内资产和公式形成价格。对恒定乘积池，早期模型可表示为 $(x \cdot y = k)$ 。交易规模相对于池子深度越大，价格影响越大。(Uniswap Developers (https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works?utm_source=chatgpt.com))

闸门四：完整成本是否计算

必须区分价格影响和滑点：

1. 价格影响是自己的交易直接改变池内价格
2. 滑点是报价到实际执行期间产生的价格差异

二者不能重复计算，也不能只记录滑点设置而忽略自身交易造成的价格影响。(Uniswap Support (https://support.uniswap.org/hc/en-us/articles/40074715860365-What-is-price-impact?utm_source=chatgpt.com))

Ethereum 类网络的 Gas 成本由交易计算量和每单位 Gas 价格共同决定，EIP 1559 交易还包含基础费、优先费和最高费用参数。(ethereum.org (https://ethereum.org/developers/docs/gas/?utm_source=chatgpt.com))

闸门五：时间结构是否可接受

记录每个时间点

同链原子套利重点研究区块包含时间。

跨链套利重点研究：

1. 源链确认时间
2. 桥接传输时间
3. 目标链到账时间
4. 目标链退出交易时间

5. 资产可重新部署时间

交易被包含在区块中不等于已经获得最终性。不同链、L2 和桥对确认与最终性的处理不同。(ethereum.org (https://ethereum.org/roadmap/single-slot-finality/?utm_source=chatgpt.com))

闸门六：失败损失是多少

失败成本需要单独建模

至少模拟以下情况：

1. 第一笔成功，第二笔失败
2. 桥接成功，目标链卖出失败
3. 授权成功，交换失败
4. 报价过期
5. Gas 突然升高
6. 交易被竞争者抢先
7. 资产到账后目标池流动性消失
8. RPC、索引器或报价 API 返回陈旧数据

闸门七：能否重复和扩展

一次成功不构成策略。

需要回答：

1. 一天出现多少次
2. 每次能容纳多少资金
3. 机会持续多少秒
4. 成功率是多少
5. 利润是否集中在极少数异常事件
6. 加入真实延迟后是否仍然盈利
7. 竞争者增加后利润是否消失
8. 收益是否来自承担无法量化的尾部风险

四、统一机会评分卡

每个机会统一记录十组数据。

1. 身份信息

策略类型、链、协议、池地址、代币地址、区块高度和时间戳。

2. 路径信息

输入资产、每个中间资产、最终资产、桥、DEX、路由和交易步骤数量。

3. 报价信息

输入数量、预计输出、最低输出、退出报价和报价有效期。

LI.FI 将 Quote 用于相对简单的单步操作，将 Route 用于可能包含多个步骤的复杂路径。因此研究时应保存完整 Route，而不能只保存最终输出数字。(LI.FI (https://docs.li.fi/sdk/request-routes?utm_source=chatgpt.com))

4. 成本信息

Gas、协议费、桥接费、流动性提供者费用、借贷费用、价格影响和滑点。

LI.FI 的报价与路径数据中提供费用和 Gas 相关字段，可以作为第一版成本采集入口，但仍需要用实际链上执行结果校准。(LI.FI (https://docs.li.fi/faqs/fees-monetization?utm_source=chatgpt.com))

5. 时间信息

报价耗时、模拟耗时、签名耗时、提交耗时、确认耗时、桥接耗时和退出耗时。

6. 流动性信息

池子储备、当前有效流动性、交易规模、价格影响和最大容量。

集中流动性池中的流动性分布在特定价格区间，因此总锁仓量不能直接代表当前价格附近的真实可交易深度。(Uniswap Developers (https://docs.uniswap.org/protocol/concepts/V3-overview/concentrated-liquidity?ref=trading-strategy.ghost.io&utm_source=chatgpt.com))

7. 执行信息

交易哈希、区块位置、实际输入、实际输出、实际 Gas 和实际滑点。

8. 竞争信息

同区块竞争交易、优先费、是否被回跑、是否被抢先和机会修复速度。

9. 风险信息

合约风险、桥风险、稳定币风险、预言机风险、库存风险和 RPC 风险。

10. 结论信息

理论利润、模拟利润、实际利润、失败损失、成功概率和是否继续研究。

五、21 天学习大纲

第一阶段：建立基础模型

Day 1：统一套利判断框架

学习内容：

1. 毛利润、净利润和期望利润
2. 闭环路径
3. 原子执行与非原子执行
4. 成功成本与失败成本
5. 七道判断闸门

当天产出：

1. 一份统一机会评分卡
2. 一张套利类型地图
3. 一个固定计价单位
4. 一套策略否定条件

Day 2：AMM 与流动性

学习内容：

1. 恒定乘积模型
2. 池内价格形成
3. 边际价格与平均成交价格
4. 集中流动性
5. Tick 与有效流动性
6. 交易规模与价格影响

当天产出：

1. 手工计算一次 AMM 交换
2. 绘制交易规模与输出价格曲线
3. 解释为什么页面价差不等于可交易价差

Day 3：链上交易成本

学习内容：

1. Gas
2. 协议费
3. LP 费用
4. 价格影响
5. 滑点
6. 授权成本
7. 失败交易成本

当天产出：

1. 完整成本清单
2. 三种交易规模的成本对比
3. 毛利润到净利润的计算样例

Day 4：交易生命周期

学习内容：

1. RPC
2. Nonce
3. Mempool
4. 区块包含
5. Revert
6. 确认
7. 最终性
8. 模拟与实际执行差异

当天产出：

1. 一张交易生命周期图
2. 一份失败情景清单
3. 一次真实小额交易复盘

第二阶段：理解六类策略

Day 5：同一 DEX 多池套利

研究相同交易对在不同费率池、不同池模型和不同流动性区间中的价格差。

产出一条完整闭环路径和容量曲线。

Day 6：跨 DEX 套利

研究同一资产在不同 DEX 上的可执行价格差。

产出两个 DEX 的同步报价记录。

Day 7：三角套利

研究并

产出一个三角路径枚举器的逻辑设计，不要求立即写机器人。

Day 8：稳定币套利

研究脱锚、池内失衡、赎回机制、信用风险和流动性风险。

产出稳定币机会分类表和风险否定条件。

Day 9：跨链套利

研究桥接路径、预置库存、跨链等待和双边对冲。

使用 LI.FI 比较不同金额、链、桥和路由。

产出跨链路径数据字段定义。

Day 10 : 清算与 MEV

研究健康因子、清算奖励、交易排序、回跑和 Bundle。

产出清算利润公式和 MEV 竞争成本清单。

第三阶段：建立数据系统

Day 11 : LI.FI 数据结构

读取 Quote、Route、Step、Gas Cost 和 Fee Cost。

产出统一 JSON 数据模型。

Day 12 : 报价采集器

定时采集：

1. 时间
2. 链
3. 资产
4. 金额
5. 路由
6. 预计输出
7. 最低输出
8. Gas
9. 费用
10. 预计耗时

产出第一份结构化报价数据。

Day 13 : 路径标准化

将不同路径统一表示为有向图：

```
[  
text{资产节点}+text{链节点}+text{协议边}  
]
```

产出路径指纹，用于判断路由是否变化。

Day 14 : 机会检测器

计算：

1. 毛价差
2. 完整成本
3. 预期净利润
4. 利润率
5. 容量
6. 路径变化
7. 报价陈旧程度

产出只报警、不执行的监控程序。

Day 15 : 执行模拟器

加入：

1. 延迟
2. Gas 波动
3. 滑点扩大
4. 路由变化
5. 失败概率

6. 目标链价格变化

产出保守利润和乐观利润两个结果。

第四阶段：验证策略

Day 16：报价寿命与延迟

测试机会从出现到消失需要多久。

产出不同采样频率下的漏检率。

Day 17：容量与市场冲击

扫描不同交易金额。

产出：

```
[  
q^=argmax_qPi(q)  
]
```

即理论最优交易规模。

Day 18：失败模型

主动模拟交易失败、桥接延迟、RPC 中断和竞争交易。

产出失败成本分布。

Day 19：样本外验证

按时间切分训练期和验证期。

检查利润是否只存在于少数日期、单一链或单一代币。

产出继续研究、修改假设或停止研究的裁决。

第五阶段：完成最小原型

Day 20：最小监控系统

系统至少完成：

1. 定时获取报价
2. 保存完整路径
3. 计算完整成本
4. 估计净利润
5. 设置机会阈值
6. 记录机会消失时间
7. 输出可复盘日志

此阶段仍然不要求自动发送真实资金。

Day 21：研究报告与策略裁决

最终报告包括：

1. 研究的策略假设
2. 机会形成机制
3. 数据来源
4. 计算方法
5. 成本模型
6. 延迟模型
7. 成功率
8. 容量

9. 最大失败成本
10. 样本外结果
11. 当前是否具备实盘条件
12. 下一阶段需要解决的问题

六、21 天内暂时不深入的内容

为了避免再次陷入概念过载，前期暂时排除：

1. 自建验证节点
2. 区块构建者基础设施
3. 复杂 Solidity 合约开发
4. 跨链桥底层密码学
5. ZK 证明原理
6. 高级私有订单流
7. 多链自动资金调度
8. 大额真实资金执行

只有在数据证明某个策略存在稳定净收益以后，才进入这些工程层。

七、统一的策略通过标准

一个策略进入实盘候选阶段，至少需要满足：

1. 路径可以完整闭环
2. 使用真实可执行报价
3. 已计入全部成功成本
4. 已估算失败成本
5. 已测量报价寿命
6. 已测试不同交易规模
7. 已加入实际执行延迟
8. 已完成样本外验证
9. 利润不依赖单次异常事件
10. 最大失败损失在可接受范围
11. 小额真实交易结果接近模拟
12. 每次交易都有完整复盘记录

只要其中一项缺失，当前结果就只能称为机会观察，不能称为可执行套利策略。

21 天后继续推进的研究方向

本轮 21 天共学先完成链上套利的基础判断、数据采集和最小监控系统。以下方向留到基础系统稳定后继续推进。

1. Carry Trade

研究不同市场之间的持有收益差和融资成本差，包括：

1. 永续合约资金费率套利
2. 现货与期货基差交易
3. 不同 CEX 之间的资金费率差
4. CEX 与链上永续协议之间的利差
5. 借贷协议存款利率与借款利率差
6. 质押收益、再质押收益与对冲成本
7. 稳定币收益率和融资利率迁移
8. 跨链资金成本和资本占用

统一判断公式：

Carry净收益=持有收益+基差收敛收益-融资成本-对冲成本-交易费用-尾部风险准备金

重点研究：

1. 收益来源能否持续
2. 多空腿是否真正对冲
3. 资金费率是否会反转
4. 基差是否会继续扩大
5. 保证金和清算风险
6. 跨平台资金调度时间
7. 收益是否足以覆盖资本占用
8. 极端行情下两条腿是否同时可交易

未来产出：

1. Carry 机会统一评分卡
2. 资金费率和基差采集器
3. 年化收益与真实净收益计算器
4. 多平台对冲仓位监控
5. 清算和保证金压力测试系统
2. 波动率曲面

研究不同执行价、期限和市场之间的隐含波动率相对定价，包括：

1. 波动率期限结构
2. 不同行权价的偏度 Skew
3. 波动率微笑 Smile
4. 曲面曲率
5. 跨期限价差
6. 跨执行价价差
7. CEX 与链上期权市场的波动率差
8. 隐含波动率与实现波动率差
9. Vanna、Volga 和更高阶曲面风险
10. 曲面变化产生的 Carry 与对冲收益

统一判断框架：

期权组合收益=波动率重估收益+时间价值 Carry+对冲收益交易成本滑点希腊字母暴露损失

重点研究：

1. 观察到的是价格差还是模型口径差
2. 曲面偏差能否通过组合交易表达
3. Delta、Gamma、Vega 是否被有效控制
4. Vanna 和 Volga 暴露是否会放大损失
5. 偏度与曲率变化是否符合策略假设
6. 时间衰减是否对组合有利
7. 对冲频率和对冲成本
8. 极端行情下的跳跃风险和流动性风险

未来产出：

1. 标准化隐含波动率曲面
2. 期限结构、Skew 和 Smile 监控
3. Vanna、Volga 及曲面 Greeks 敞口系统
4. 相对价值机会扫描器
5. Delta 对冲与情景压力测试模块
6. 波动率 Carry 和曲面交易复盘系统

3. 其他后续方向

1. 自建验证节点
2. 区块构建者基础设施
3. 复杂 Solidity 合约开发
4. 高级私有订单流
5. 多链自动资金调度
6. 清算机器人和 Bundle 执行
7. 跨链库存自动再平衡
8. 大额真实资金执行
9. 币股 RWA 和链上衍生品价差

推进顺序

链上现货套利→跨链与库存管理→Carry Trade→波动率曲面→自动执行与组合风险管理

Carry Trade 需要先具备资金成本、仓位和清算风险管理能力。波动率曲面需要进一步具备期权定价、Greeks、对冲和曲面建模能力，因此都放在本轮 21 天基础训练之后。

笔记 11

作者：jonascyang | 时间：2026-08-05 12:42 UTC | 字数：8085

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

聚合器 1inch Aqua Case study

一、先用一句话理解 Aqua

Aqua 不是让 LP 把币存进资金池，而是让 Maker 保留钱包里的真实资产，同时在 Aqua 中登记多套“策略虚拟余额”，再由具体应用在成交时通过 allowance 从 Maker 钱包直接结算。

它提高的是同一钱包余额跨策略、跨交易对重复报价的能力，也就是名义流动性和机会覆盖；它不会凭空创造真实 Token，也不是无风险杠杆。

二、理解 Aqua

1. Wallet balance

Maker 钱包真实持有的 ERC-20 数量。它由 token.balanceOf(maker) 得到，只有真实的 ERC-20 Transfer 才会改变。

1. Aqua allowance

Maker 授权 Aqua 最多可以从钱包代扣多少 Token。它由 token.allowance(maker, Aqua) 得到。approve 只改变权限，不会移动 Token。

1. Virtual balance

Aqua 按 maker + app + strategyHash + token 保存的策略虚拟余额。它代表该策略声称可以使用多少 Token，不是存款，也不保证 Maker 钱包此刻一定有相同数量。

实际可结算上限大致受以下三者共同约束：

min (钱包余额，Aqua allowance，当前策略虚拟余额)

同一个钱包的 100 USDC 可以被三个策略各登记为 100 USDC
虚拟余额。因此虚拟余额合计可以大于钱包真实余额。这是资本效率来源，也是 underfunding
和成交回滚风险来源。

三、核心合约和职责

1. Aqua Registry / Router

地址：0x1111113ccf1426a8e30e2bff5e005d929bf6a90a

职责：

保存虚拟余额；

处理 ship、dock、pull、push；

按 maker、app、strategyHash、token 隔离权限和账本；

不理解策略中的交易对、曲线、价格或费用。

1. Aqua SwapVM

地址：0x111111338c5091e8440b67b168bae16a668ac0de

职责：

解释策略 bytecode；

根据虚拟余额和策略程序计算 quote；

检查方向、exact-in/exact-out、阈值、deadline、fee 和访问条件；

调用 Aqua pull/push 完成结算；

成功后发出 Swapped。

两个地址在 13 条官方支持链上采用相同 canonical address。Aqua runtime bytecode 在核实时一致；SwapVM 因各链 WETH 等 immutable 参数不同，不能据此要求 runtime bytecode 完全相同。

四、完整生命周期

1. Maker 授权

Maker 调用 ERC20.approve(Aqua, limit)。

变化：allowance 增加。

不变：钱包余额和 Aqua 虚拟余额。

相关事件：ERC-20 Approval。

1. 链下构造策略

策略 Order 包含 maker、MakerTraits 和 data。data 中可以包含交易对、方向、定价程序、费用、区间、deadline、访问控制和 salt。

strategyHash / orderHash 通常来自 keccak256(abi.encode(order))。这个阶段没有链上状态变化。

1. Ship 创建策略

Maker 调用 Aqua.ship(app, strategy, tokens, amounts)。

Aqua：

计算 strategyHash；

保存各 Token 的初始虚拟余额；

发出一个 Shipped；

为每个 Token 发出一个初始 Pushed。

关键点：Ship 阶段没有 ERC-20 transfer。这里的初始 Pushed 是虚拟额度初始化，不能当成充值、资金流入或交易量。

1. Quote 试算

Resolver 或 Taker 通常通过 eth_call 调用 SwapVM.quote。

SwapVM 读取 Aqua.safeBalances，执行 bytecode，计算 amountIn/amountOut 并检查策略条件。

Quote 没有转账，也没有链上日志。它反映的是当前策略虚拟状态，不保证 Maker 钱包余额和 allowance 一定足够，也不保证稍后上链时状态没有变化。

1. Swap 上链

用户接受报价后调用 SwapVM.swap。SwapVM 会重新读取状态、重跑程序并检查滑点、deadline、exact-in/out 和访问条件。

1. Maker 付出 tokenOut

SwapVM 调用 Aqua.pull：

先减少策略 tokenOut 虚拟余额；

再执行 ERC20.transferFrom(maker, recipient, amountOut)；

Aqua 发出 Pulled。

1. Maker 收到 tokenIn

Aqua.push：

将 tokenIn 从 push 调用者转入 Maker；

增加策略 tokenIn 虚拟余额；

Aqua 发出运行期 Pushed。

运行期 Pushed 与 Ship 时的初始 Pushed 不同：运行期 Pushed 伴随真实 Token 转账。

1. 成交成功

两条资金腿及所有检查都成功后，SwapVM 发出 Swapped：

orderHash、maker、taker、tokenIn、tokenOut、amountIn、amountOut。

这才是一个成功 fill。余额不足、allowance 不足、过期或滑点不符都会让整笔交易回滚，成功的 Swapped 不会留下。

1. 重复成交或关闭

成交后：

Maker 钱包 tokenOut 减少，tokenIn 增加；

策略虚拟 tokenOut 减少，tokenIn 增加；

后续 quote 读取更新后的虚拟状态。

Maker 关闭策略时调用 Aqua.dock。Dock 只是把虚拟余额清零并标记关闭，不是提款，因为 Token 从未存入 Aqua。

五、活动类型不能只记名字，要理解它在业务上的含义

ERC-20 合约：Approval

含义：Maker 修改 Aqua 可以代扣的额度。没有资金移动，不能计入成交量或流动性流入。

Aqua 合约：Shipped

含义：Maker 向某个 app 发布一份不可变策略。它证明策略被创建，并提供完整 strategy bytes。可以用于统计策略创建和重建策略状态。

Aqua 合约：初始 Pushed

含义：Ship 时初始化某个 Token 的虚拟余额。没有真实 Token 转账，不能计入资金流或成交量。

Aqua 合约：运行期 Pushed

含义：一次结算中 Maker 收到 tokenIn，同时该策略 tokenIn 虚拟余额增加。它是一条资金腿，不是一笔独立成交。

Aqua 合约：Pulled

含义：一次结算中 Maker 付出 tokenOut，同时该策略 tokenOut 虚拟余额减少。它也是资金腿，不是一笔独立成交。

SwapVM 合约：Swapped

含义：策略完成一次成功 fill。成交量、fill 数量、交易对和 trading maker 应以它为主要统计事件。

Aqua 合约：Docked

含义：Maker 关闭策略并清除其虚拟余额。没有提款动作。相同 strategyHash 不能重新 Ship。

ERC-20 合约：Transfer

含义：真实 Token 发生移动。它用于与 Pulled/Pushed 对账，但不能再次计为一笔 Swap，否则会重复统计。

Quote

Quote 是 eth_call 试算，不是链上事件。它没有交易哈希、资金移动或成功成交含义。

六、策略到底是什么

Aqua 核心不解释策略，真正的定价逻辑在 SwapVM program。

官方产品常见展示类型：

1. Straight

集中在指定价格区间或线性区间内提供报价。更接近集中流动性，需要明确价格范围。

1. XYC

常数乘积 $xy=k$ 的全区间曲线。

1. Curved

面向相关资产或锚定资产的曲线，底层对应 PeggedSwap 一类逻辑。

1. Custom / Unknown

无法按照官方已知模板解码的 bytecode。监控系统应保留原始 strategy bytes，不应因为无法识别就丢弃策略。

SwapVM 还支持 fee、dynamic fee、decay、deadline、token gate、tx.origin access gate、conditional jump、salt 和外部逻辑扩展。

因此，要回答“某个 Maker 使用什么策略”，必须解码 Shipped 中的 strategy bytes 和 opcode，不能仅根据 Swapped 交易对或 Token 名称猜测。

七、Aqua 支持哪些链

截至 2026-08-04 的官方文档，Aqua 支持 13 条链：

1. Ethereum，chain ID 1
2. Arbitrum One，42161
3. Base，8453
4. Optimism，10
5. Polygon PoS，137
6. BNB Chain，56
7. Avalanche C-Chain，43114
8. Gnosis，100
9. zkSync Era，324

10. Linea , 59144
11. Unichain , 130
12. Sonic , 146
13. Robinhood Chain , 4663

“支持”需要分成四层：

1. 代码可部署：EVM 合约逻辑可以部署到相容链。
2. 官方已部署：canonical Aqua 和 SwapVM 地址有非空 bytecode。
3. 官方产品支持：FAQ 和产品流程把该链列为 available。
4. 数据已经完整回填：监控系统已从部署区块处理到当前链头。

前面三层已经对官方 13 链核验。第四层不能自动成立。

八、它是否已经在正常使用

结论：Aqua 已经在主网上实际使用，不只是白皮书、测试网或即将上线的产品；但产品仍然很新，不能直接描述为成熟的大规模市场。

证据包括：

官方已有连接钱包、选交易对和策略、approve、Provide liquidity、管理和关闭仓位的完整流程；

官方 resolver API 在线，但需要 Business API 鉴权；

Ethereum 持续出现 Swapped；

已核验的成功交易同时包含 Pulled、Pushed、Swapped 和真实 ERC-20 Transfer；

Arbitrum、Base、Polygon、Unichain、Robinhood Chain 等也曾观察到 Swapped。

白皮书仍标注 Developer Preview，安全页也提醒 Aqua
虽经审计但仍很新。因此正确表述是“已上线并正常发生真实结算”，而不是“已经成熟或大规模采用”。

九、“数据覆盖不完整”到底是什么意思

数据覆盖不完整不等于链上没有活动。

早期跨链调查中：

Ethereum、Arbitrum、Base、Polygon、Unichain、Robinhood Chain 等链观察到 Swapped；

Gnosis、zkSync Era 观察到 Shipped/Pushed，但当时没有完成 Swapped 穷尽核验；

Optimism、BNB Chain、Avalanche、Linea、Sonic 的公开 RPC
或浏览器日志入口受分页、限流或索引范围影响，只确认了合约部署，未完成成交全历史回填。

造成 Partial 的常见原因：

不知道准确部署区块；

公开 RPC 对 eth_getLogs 查询范围有限；

Blockscout/Etherscan 类接口分页或 page 参数行为不一致；

429 限流；

浏览器没有完整索引；

历史价格或 Token metadata 缺失；

采集器只查询了最近区块窗口而不是从部署开始回填。

因此监控系统必须显示 complete chains / supported chains、backfill 起点、indexed block、chain head、block lag、price coverage 和错误状态。查不到必须写 Unknown/Partial，不能填成 0。

十、Ethereum 与 Robinhood Chain 的全量 Swapped 实操

采样截止：2026-08-05 10:26:07 UTC，北京时间 18:26:07。

查询合约：SwapVM 0x111111338c5091e8440b67b168bae16a668ac0de

Swapped topic0：0x54bc5c027d15d7aa8ae083f994ab4411d2f223291672ecd3a344f3d92dc8b2

事件唯一键：chainId + transactionHash + logIndex。

两链合计：

Swapped fills：51,184

distinct transactions：12,942

重复事件：0

发生过成交的唯一 Maker 地址：273

Maker 与链的组合：295

交易对：470

单边美元成交量：9,684,697.77 USD

Ethereum：

从 block 0 开始查询，确保不漏掉部署前界判断；

第一条观察到的 Swapped：block 25,624,440，2026-07-27 13:27:35 UTC；

本次最后一条 Swapped：block 25,688,049，2026-08-05 10:12:23 UTC；

Swapped fills：28,278；

distinct transactions：4,833；

trading makers：243；

已成交 orderHash：2,554；

成交量：6,857,076.28 USD。

Robinhood Chain：

SwapVM 部署 block：20,105,902；

第一条 Swapped：block 21,111,532，2026-07-27 23:04:34 UTC；

本次最后一条 Swapped：block 28,406,497，2026-08-05 10:16:00 UTC；

Swapped fills：22,906；

distinct transactions：8,109；

trading makers：52；

已成交 orderHash：420；

成交量：2,827,621.49 USD。

事件数明显大于 transaction 数，因为一笔聚合交易可以包含多个 Aqua Swapped fill。不能把 fills 当成去重交易数。

Robinhood Chain 的 transaction hash、block、logIndex 和事件参数是链上精确值。由于逐区块时间查询会触发 RPC 限流，本次使用每 500,000 区块的精确 timestamp 锚点做分段插值。这个估算只用于匹配小时历史价格，不改变事件数量、Maker、交易对或金额。

十一、当时成交量前五交易对

统计范围：Ethereum 与 Robinhood Chain，从相应查询起点到 2026-08-05 10:26:07 UTC；不同链上的同名交易对不合并。

1. Ethereum：aEthcbBTC / aEthWBTC

Swapped fills：10

distinct transactions：7

Volume：1,053,867.14 USD

2. Ethereum：1INCH / USDC

Swapped fills：4,022

distinct transactions：1,138

Volume : 827,550.58 USD

3. Robinhood Chain : USDG / WETH

Swapped fills : 2,932

distinct transactions : 1,908

Volume : 685,045.27 USD

4. Ethereum : 1INCH / WETH

Swapped fills : 3,299

distinct transactions : 927

Volume : 658,784.63 USD

5. Ethereum : 1INCH / USDT

Swapped fills : 3,100

distinct transactions : 999

Volume : 626,809.77 USD

参考资料 :

1inch Aqua How it works

<<https://1inch.com/aqua/learn/how-aqua-works>>

1inch Aqua FAQ

<<https://1inch.com/aqua/learn/faq>>

1inch Aqua White Paper

<<https://1inch.com/assets/1inch-aqua-white-paper.pdf>>

1inch Aqua GitHub

<<https://github.com/1inch/aqua>>

1inch SwapVM GitHub

<<https://github.com/1inch/swap-vm>>

Aqua 核心合约

<<https://eth.blockscout.com/address/0x1111113ccf1426a8e30e2bff5e005d929bf6a90a>>

SwapVM 合约

<<https://eth.blockscout.com/address/0x111111338c5091e8440b67b168bae16a668ac0de>>

本次 Swapped 数据工作簿

/Users/jonas/Desktop/Stablecoin-research/outputs/aqua-swapped-ethereum-robinhood-20260805/Aqua_Swapped_Ethereum_Robinhood.xlsx

笔记 12

作者 : rslofty | 时间 : 2026-08-06 13:38 UTC | 字数 : 7660

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 · 2026-08-06 · HYPE 美股永续合约 vs 美股非交易时段套利调查

一句话总结 : HYPE (Hyperliquid) 上有一个 24/7 的美股永续合约市场叫 xyz , 我把它的实时 mark / oracle / funding / 量都读了出来, 对照 Yahoo Finance 当前报价, 回答"美股非交易时段能不能套利" = 理论可、实操难、单笔净利润基本被 fee 吃掉。

0. 一个真信号：HYPE 上真有美股永续

Day 1 我只摸了 LI.FI。今天换个角度：直接打 HYPE 的 info API，发现一个叫 xyz 的 HIP-3 perp dex，asset 列表里全是美股代号：TSLA / NVDA / AAPL / META / MSFT / GOOGL / AMZN / AMD / INTC / HOOD / COIN / MSTR / CRCL / PLTR / NFLX ... 加 SP500 / GLD / SILVER / 外汇。全部 24/7 可交易。

这是 HYPE 在 2025-11-23 上线的 HIP-3 「builder-deployed perpetuals」——任何人质押 500k HYPE 都能部署一个 perp dex，自己定义 oracle 和合约参数。xyz 的 deployer 0x88806a71d74ad0a510b545c9ae490912f0888（注：地址以我刚才实测拿回的为准；写错一位就触发不了）。

1. 数据来源（每条都可复跑）

bash

1) 列所有 HIP-3 perp dex —— 拿 xyz

```
curl -s -X POST https://api.hyperliquid.xyz/info
```

```
-H "Content-Type: application/json"
```

```
-d '{"type":"perpDexs"}' | jq '.[1].name'
```

2) 拿 xyz 的 universe + 实时 mark/oracle/funding/volume

```
curl -s -X POST https://api.hyperliquid.xyz/info
```

```
-H "Content-Type: application/json"
```

```
-d '{"type":"metaAndAssetCtxs","dex":"xyz"}' | jq '.[1]'
```

3) 拿 24h 1h 蜡烛

```
curl -s -X POST https://api.hyperliquid.xyz/info
```

```
-H "Content-Type: application/json"
```

```
-d '{"type":"candleSnapshot","req":{"coin":"xyz:NVDA","interval":"1h","startTime":'$(date +%s%3N -d 'yesterday')',"endTime":'$(date +%s%3N)'}}'
```

4) 拿 7d funding history (验证 funding interval 是不是 1h)

```
curl -s -X POST https://api.hyperliquid.xyz/info
```

```
-H "Content-Type: application/json"
```

```
-d '{"type":"fundingHistory","coin":"xyz:NVDA","startTime":'$(date +%s%3N -d '7 days ago')',"endTime":'$(date +%s%3N)'}
```

2. 现在这一刻（08-06 22:37 JST = 09:37 ET，开盘 7 分钟）的实数

| 股票 | HYPE xyz mark | HYPE xyz oracle | funding/h | Yahoo 报价 | HYPE vs Yahoo | 24h 量 USD M | maxLev | margin |

|---|---|---|---|---|---|---|

| NVDA | \$222.94 | \$222.83 | 0.0016% | \$223.07 | 5.8 bp | \$111M | 20x | cross |

| AAPL | \$314.80 | \$314.80 | 0.0006% | \$314.07 | +23.2 bp | \$24.8M | 20x | cross |

| TSLA | \$317.43 | \$317.47 | 0.0006% | \$318.68 | 39.2 bp | \$16.3M | 20x | cross |

| META | \$590.75 | \$590.56 | 0.0010% | — | — | \$60.5M | 20x | cross |

| MSFT | \$491.25 | \$491.28 | 0.0011% | — | — | \$53.5M | 20x | cross |

| GOOGL | \$363.77 | \$363.55 | 0.0025% | — | — | \$103.4M | 20x | cross |

| AMD | \$480.83 | \$480.83 | 0.0006% | — | — | \$60.5M | 10x | noCross |

| INTC | \$97.203 | \$97.148 | 0.0040% | — | — | \$118.4M | 10x | noCross |

| HOOD | \$91.694 | \$91.643 | 0.0014% | — | — | \$5.5M | 10x | noCross |

| PLTR | \$153.88 | \$153.88 | 0.0006% | — | — | \$14.6M | 10x | noCross |

| MSTR | \$96.191 | \$96.191 | 0.0006% | — | — | \$8.2M | 10x | noCross |

| CRCL | \$61.35 | \$61.35 | 0.0034% | — | — | \$27.0M | 10x | noCross |

Yahoo 实测 (直接 finance.yahoo.com/quote/<sym>/, 全部"As of 9:36:xx AM EDT. Market Open") : NVDA \$223.07 / AAPL \$314.07 / TSLA \$318.68。

2.1 直接读出来的事实 (这一节才是 Day 2 的真贡献)

- funding interval = 1 小时 (不是常规 8h) 。fundingHistory 7 天 168 次 = 24×7 。NVDA 年化费率 $\approx 5.4\%$, GOOGL $\approx 14\%$, PLTR 还有过 funding。
- Oracle 是 deployer 自己喂价。HIP-3 spec 写明
"The deployer of a perp market is responsible for Market definition, including the oracle definition ... and setting oracle prices"。
- xyz 押了 500k HYPE + validator-slashable , 乱喂会被砍。
- 24h 1h 蜡烛连续 25 根 : NVDA 在非美股交易时段 (昨夜 21:00–08:00 JST) 也有量, 最少 5,150 NVDA , 最多 48,789 NVDA ($\approx \$10.8M$) 。
- OHLC 形状对得上真实美股 : NVDA 8/5 close \$219.22 , 8/6 09:00–10:00 ET candle open \$221.34 / close \$223.13 $\approx +1.7\%$ gap ; Yahoo day change +1.76% — HYPE 跟得上真实价格。

3. 美股非交易时段到底有没有套利空间

用户原问题 = "美股非交易时段的套利"。我把它拆成 4 类时段看。

3.1 时段分类

时段	ET	当前	数据状态
盘前 Pre-market	04:00–09:30	9:37 ET	Yahoo + HYPE 双侧活跃
盘中 Regular	09:30–16:00	—	双方都有 tape
盘后 After-hours	16:00–20:00	—	Yahoo 有 tape , HYPE 也有
夜盘 Overnight	20:00–04:00	—	Yahoo tape 极薄 ; HYPE 持续
周末 Weekend	Sat/Sun	—	Yahoo tape 0 ; HYPE 持续

3.2 四个时段, 能不能套?

档 A : 盘前 + 盘后 (04:00–20:00 ET, 周一到周五) — 空间最小

- HYPE xyz oracle 实时跟 Yahoo / IB / Robinhood 的盘前盘后 last-trade
- 证据 : 刚才 09:37 ET 我同时拉两边, HYPE NVDA \$222.94 vs Yahoo \$223.07 = 5.8 bp ; AAPL \$314.80 vs \$314.07 = +23.2 bp ; TSLA \$317.43 vs \$318.68 = 39.2 bp
- 观测最大偏离 ≈ 40 bp。扣完 fee 净利润 ≈ 0
- 想套 = 被 fee + funding 吃掉
- 唯一例外 : oracle 喂价有 1–3 秒可观察延迟——但 block-by-block 的 dex 跟不上, 而且 oracle updater 也用同一份盘前盘后数据源

档 B : 夜盘 + 周末 — 理论上最有戏

- 美股完全闭市, 但 HYPE xyz 还在跑
- 24h 蜡烛在 21:00 JST 之后仍有 5k+ NVDA 的量 (不是 0)
- 但真实美股夜盘+周末只有 ECN 极少成交 (IBKR overnight session, 不是 NYSE 官方盘)
- 如果 oracle 喂的是 IB / Robinhood 的 last-trade → 夜盘后 oracle 被"冻"在 last-trade = 16:00 ET 的最后价附近
- 但 HYPE 24h 仍有 mark 变动 → mark 与 oracle 持续产生偏离 (这就是 premium 的来源)
- 套利窗口 : HYPE mark 偏离方向 = 多空差 ; 等真实美股开盘后 mark "瞬移"对齐 oracle → 这之间的钱可以被 delta-neutral (HYPE 永续 + 反向真实股票 IB 借券) 吃到

档 C : 跨周末 (Friday close → Monday open) — 理论上空间最大

- 周五 16:00 ET 后 oracle 被锁在 Friday close
- 周六周日 HYPE 永续持续成交 → mark 被 weekend news / 加密市场波动推离

- 周一 09:30 ET 一开盘 oracle 被新数据替换 → mark 一秒归位
- 教科书 gap-trade : 锁住周五 close 多 HYPE + 真实 IB 债券做空, 等周一开盘价回归赚 gap。但需要 IB 债券成本 + 跨链桥 + 资金费率
- 样本太少: HYPE xyz 2025-11-23 才上线, 只有 8 个月数据, 跨周末样本 < 35 个

3.3 我的"扣费模型"

美股非交易时段的 HYPE xyz 套利 = 真存在, 但理论空间小, 实际拿到钱很难。

扣减项	数值 (单笔 1000 USD 仓位)
观测最大 mark vs oracle 偏离	10–40 bp
HYPE Taker fee	4.5 bp
HYPE Maker fee	1.5 bp
Funding cost (持仓 N 小时)	$N \times 0.001\%$
LI.FI 跨链 deposit/withdraw	25–30 bp
IB 真实股票债券 (HTB 票)	0.2–0.6 bp / 夜
Robinhood 股票手续费	0 (免) / IB tiered 1–2 bp
理论单笔净利润	基本 0, 波动很大

更糟的三件事:

1. HIP-3 spec 第 6 节允许 deployer 抽 fee share 0–300% —— xyz 可能再加一层 fee
2. 跨链 deposit 不原子 (LI.FI 实测 2–7s) —— oracle 在这窗口可能就追回来了
3. marginMode=noCross 票 (INTC/AMD/HOOD/PLTR/COIN/MSTR/CRCL) 只能 isolated, 单边敞口无法对冲 —— 真要做就要选 maxLev=20x 的 cross 票 (NVDA/AAPL/TSLA/META/MSFT/GOOGL/AMZN)

4. 我今天没做的事

- 没在 HYPE 上真成交任何一笔——只有 read API + Yahoo 对照
- 没拿 Pyth / CBOE EDGX 的盘后 last-trade 数据跟 HYPE oracle 做长时间序列对比——这是验证档 A 真空间的关键, 我只能查到 market open / close 那一瞬间
- 没回测档 C (跨周末 gap) —— HYPE xyz 2025-11-23 才上线, 样本太少
- 没建完整的"扣费模型"脚本——Day 3 的目标

5. Day 3 计划

1. 拉 HYPE xyz 30 天 5min 蜡烛 + Polygon EOD last-trade, 算 oracle 偏离在 4 个时段各自的中位数与最大值
2. 把 LI.FI 跨链费率 (Day 1 测过的 0.25–0.27%) + taker fee + funding 串成一个 net_pnl(side, hours, deviation_bp) 函数, 定义 P&L > 0 的最小窗口
3. 如果档 C 空间 > 5 bp → 写 weekend gap detector (cron every Fri 16:00 ET 记录 mark vs oracle, Mon 09:30 ET 触发开仓)
4. 决定要不要在 LI.FI Builders 群里喊一句"xyz:NVDA funding 年化 5.4%, 有没人愿意做空它收 funding"做个小实验

6. 安全边界

- 没签名任何链上交易 —— HIP-3 是 user signature + builder-side oracle update 分离的, 但我连 user 那一侧都没碰
- 没动私钥 / 助记词 / 钱包连接 —— 只用 public info API
- 全部走 <https://api.hyperliquid.xyz/info> (公开端点, 无 auth)
- Yahoo Finance 公开页面 (rate-limited 不需 auth)
- Day 3 之前不进入 paper trading, 只在 HYPE 上做 limit-order paper track

证据清单

来源	端点 / 路径	取回内容
HYPE API	POST /info type=perpDexs	xyz (deployer 0x88806a71d74ad0a510b545c9ae490912f0888 , 105 资产)
HYPE API	POST /info type=metaAndAssetCtxs dex=xyz	25 个美股 + 大宗 mark / oracle / funding / OI / dev_bp
HYPE API	POST /info type=candleSnapshot xyz:NVDA 1h	24h = 25 根 1h 蜡烛，连续不断
HYPE API	POST /info type=fundingHistory xyz:NVDA 7d	168 次 / 1h interval , rate 0.0006–0.0012%/h
Yahoo	finance.yahoo.com/quote/{NVDA,AAPL,TSLA}	22:37 JST Thu 当前报价 \$223.07 / \$314.07 / \$318.68
HIP-3 spec	hip-3-builder-deployed-perpetuals.md	deployer 负责 oracle + 500k HYPE stake + validator-slashable

笔记 13

作者：richardxv | 时间：2026-08-05 13:51 UTC | 字数：7442

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

昨晚根据BBruce的教程配置了hermes，然后连接飞书。

今天把群聊内容做成了零基础学习版本，逐步学习知识，下一步准备先做个监控机器人，进入实操环境再看。

链上套利零基础学习版

根据《8.4聊天记录》整理 | 先建立概念，再理解策略，最后学习程序与风控

先说结论：你现在最需要学的不是“马上找到一个套利机会”，而是学会判断一笔交易到底有没有真实、可执行、扣除成本后仍然存在的利润。

这份学习版把原聊天中的内容改写成知识点。基础解释是为了帮助零基础读者入门；群聊中关于协议盈利、报价失真和收益的说法，仍需以后用官方文档、区块浏览器和链上数据逐项核查。

一、先看懂整张地图

把链上套利想成一条流水线，你要依次回答五个问题：

1. 在哪里发现价格差？是在 CEX、DEX，还是不同链之间？
2. 两边的资产是不是同一种资产？同名代币不一定是同一个合约。
3. 扣掉交易费、Gas、跨链费、滑点和失败风险后，还剩多少？
4. 买入和卖出两条腿能不能在合适的时间内都完成？
5. 如果价格突然变化或只成交一边，程序和人准备怎么处理？

二、第一层基础：你在“链上”到底在做什么

1. 区块链、网络和资产

区块链：可以先理解为一套公开记录系统。不同网络有不同的规则、账户体系和手续费，资产不能默认在不同网络之间直接通用。

原生币与代币：原生币通常用于支付某条链的

Gas；代币则由智能合约发行。跨链之后看到的同名资产，可能只是包装版或映射版。

钱包与地址：地址像收款账号，钱包负责管理密钥并签名交易。助记词、私钥和授权权限是资产安全的第一道防线。

智能合约：部署在链上的程序。DEX、跨链桥、借贷和质押协议都通过合约执行规则。

2. 一笔链上交易的基本流程

查看报价 → 选择路线 → 钱包签名 → 广播交易 → 等待确认 → 检查最终到账

跨链时通常还要多一个过程：源链上的资产先被锁定、销毁或转移，目标链再释放、铸造或映射对应资产。因此，跨链并不是“点击一下马上换到另一条链”，等待时间本身就是套利风险。

三、第二层基础：CEX、DEX 和报价工具

1. CEX 与 DEX 的区别

CEX（中心化交易所）：通常有订单簿、账户系统和 API。用户把资产交给平台托管，程序可以通过 API 获取行情、下单和查询状态。

DEX（去中心化交易所）：用户直接用钱包与智能合约交互，常见流动性池、Gas、滑点和交易确认问题。

聚合器和路由：聚合器会比较不同市场或不同路径的报价，帮用户寻找一条路线。聊天中出现的 LI.FI、OKX DEX、Jupiter、Uniswap、1inch、PancakeSwap，都可以先当作报价或路由研究中的例子，不要直接理解为同一种工具。

2. 读懂四个容易混淆的词

报价：系统根据当时的市场状态给出的预计价格。它是计算的输入，不等于最终成交。

滑点：实际成交价相对预计价格的偏离。金额变大、池子变浅、市场变化变快时，滑点可能迅速放大。

路线：资产从 A 换到 B 可能要经过多个池子、多个协议甚至多条链；路线越复杂，手续费和失败点通常越多。

模拟：在真正广播前，先估算交易能否成功、会得到多少资产。模拟有帮助，但不能消除链上状态变化带来的风险。

重要提醒：聊天中提到“虚报价格”或“偶尔抽风”，对初学者来说先理解成“报价可能过时或不代表最终成交”。不要只看页面上的数字，要看实际执行和链上结果。

四、第三层基础：套利到底是什么

套利不是某个按钮，也不是看到两个价格不同就一定能赚钱。它是一种利用价格差，并在扣除所有成本后争取留下净利润的交易逻辑。

净利润 = 卖出金额 - 买入金额 - 交易费 - Gas - 跨链费 - 滑点损失 - 失败或对冲损失

示意：如果毛价差是 1.00 个单位，但所有成本合计 0.75 个单位，理论净利润只有 0.25 个单位；只要价格再波动一点，或者其中一边失败，这笔交易就可能变成亏损。

1. 聊天中出现的四类套利来源

6. CEX-DEX 套利：在中心化交易所和链上交易场所之间寻找同一资产的价格差。

7. DEX-DEX 套利：在同一条链的不同 DEX 或不同流动性池之间寻找价差。

8. 跨链套利：同一经济资产或其包装版本，在不同链、不同桥或不同市场中出现价格差。

9. 协议或桥的盈利：协议可能通过手续费、报价差、沉淀资金或其他方式赚钱。这是“协议商业模式”，不等于普通用户可以直接复制的套利策略。

2. 为什么看见价差也未必能赚

- 报价可能已经过时，真正成交时价格变了。
- 池子流动性不够，大单会把价格推走。
- Gas、交易费和跨链费会吃掉小额利润。
- 跨链确认需要时间，价差可能在到账前消失。
- 同名资产可能不是同一合约，包装资产还存在赎回和桥风险。
- 竞争者、MEV 和私有流动性池可能让你慢半拍。
- 一边成交、一边失败时，单腿资产暴露可能远大于预期利润。

五、核心术语表

先不用一次背完。建议每学完一节，就回到这张表，用自己的话解释一遍。

术语

零基础解释 / 为什么要关心

区块链 / 网络

记录交易和资产状态的一套独立系统。以太坊、Solana、Cosmos 生态中的不同链，不能简单当成同一条链。

钱包 / 地址

钱包是管理密钥和发起交易的工具；地址像收款账号。私钥或助记词一旦泄露，资产可能被转走。

原生币 / 代币

原生币通常负责支付本链 Gas；代币由合约发行。跨链后的同名资产，可能只是包装版本，不一定等于原生资产。

智能合约

部署在链上的程序。DEX、跨链桥、借贷和质押协议都依赖合约按规则执行。

CEX

中心化交易所。通常有订单簿和 API，用户把资产交给平台托管，适合做 CEX-DEX 价差研究。

DEX

去中心化交易所。用户直接用钱包与合约交互，常见流动性池、Gas 和滑点问题。

聚合器 / 路由

把多个交易场所或跨链路径的报价组合起来，寻找路线或报价。群聊提到

LI.FI、OKX

DEX、Jupiter、Uniswap、1inch、PancakeSwap。

报价 / Quote

系统根据当前状态给出的预计成交价格。报价不是最终成交保证，可能因延迟、流动性或价格变化而失真。

滑点 / 价格影响

实际成交价偏离预期价格的部分。资金越大、池子越浅、价格波动越快，滑点通常越明显。

Gas / 手续费

Gas 是链上执行交易所需的网络费用；交易所、聚合器、跨链桥也可能另收费用。

TVL / 沉淀资金

协议中锁定或存放的资产总量。TVL 大不等于一定安全，也不等于所有资金都能拿来产生收益。

桥 / Bridge

连接不同网络的协议。常见过程是源链锁定或销毁资产，目标链释放、铸造或映射对应资产。

API / calldata

API 是程序访问报价或交易接口的方式；calldata 是直接调用合约时携带的参数数据。两者是不同的交易路径。

QPS

每秒请求次数。策略是否需要高 QPS，要看机会持续时间和数据源，不是越高越好。

MEV / 三明治攻击

MEV 是通过交易排序等方式提取价值；三明治攻击常指在目标交易前后插入交易，改变其成交价格。

双腿 / 单腿

一笔套利通常有买入和卖出两条腿。一边成功、一边失败，就会暴露在单腿风险中。

流动性挖矿 / 质押 / 借贷

把资产提供给协议使用，以换取手续费、奖励或利息；收益伴随合约、价格和流动性风险。

六、跨链桥：先理解机制，再理解收益

1. 跨链桥在做什么

跨链桥连接不同网络。一个常见的抽象流程是：源链上的资产被锁定或销毁，目标链发行、释放或映射一个对应表示。不同桥的安全模型和具体实现可能不同，所以“桥”不是一种完全相同的产品。

2. 群聊里提到的“三层收益”

10. 看得见的收益：用户支付的手续费和相关 Gas。

11. 不容易看见的部分：用户跨链时，资金可能在合约或流动性池里停留一段时间，形成 TVL 或沉淀资金。

12. 报价相关的部分：报价与真实链上价格之间的差异，可能体现为滑点、汇率差或服务费。

如何理解才安全：“沉淀资金可以产生收益”是群聊中的一种商业模式推演，不是所有桥的通用事实。要判断某个桥到底如何赚钱，应查官方文档、合约资金流和链上数据，不要把“高年化”当成收益承诺。

3. “无损跨链”不等于“没有风险”

无损：通常是在说用户按规则跨链后，名义上的资产数量不会因为桥的收费机制而直接减少。

不代表：没有手续费、没有价格变化、没有等待时间、没有合约风险，也不代表包装资产和原生资产完全等价。

4. 群聊出现过的桥和协议名称

- Gravity Bridge (重力桥)
- Wormhole、LayerZero (L0)、CCIP、Relay
- Aave (聊天中作为借贷或资金利用的例子)
- axlUSDC、axlATOM 等跨链包装资产

这些名称是原聊天中的研究线索，不是本文件的推荐清单。后续学习时，要先确认每个协议的官方定位、资产模型和当前状态。

七、Cosmos 生态与跨链价差

1. 先把 Cosmos 说人话

可以先把

Cosmos

理解为一个由多条相互连接的链组成的生态。不同链可能有不同用途、资产和交易场所。群聊提到的 Osmosis、Juno、Terra、Celestia、SCRT，都是这个生态中的研究线索。

对原聊天的一处校正：群聊笔记把 ATOM 简化成“路网的通用货币”。零基础阶段更准确的理解是：ATOM 是 Cosmos Hub 的原生代币；它不是所有 Cosmos 链上唯一通用的货币。

2. 资产名称为什么要特别小心

axlUSDC / axlATOM：通常可先理解为经过跨链桥映射或包装的版本。同名不等于同一合约，必须核对发行方、合约地址、赎回路径和流动性。

stkATOM / pStake：可以先理解为质押后的生息凭证或相关衍生资产。它的价格、赎回时间和风险不能直接等同于 ATOM 现货。

SCRT：聊天中作为

Cosmos

生态里的隐私链和较冷门套利场景出现。冷门通常意味着竞争少，也可能意味着流动性和信息更差。

3. Cosmos 套利的最简逻辑

同一经济资产的多链价格不一致 → 形成价差 → 计算成本 → 判断是否值得执行

13. 先确认主链资产、包装资产和 DEX 中的资产是不是可互换。

14. 再确认跨链或赎回需要多久，价格能否在等待期间保持。

15. 把手续费、Gas、桥费用、滑点和失败概率放进同一个计算表。

16. 最后才讨论是否存在可执行的双边交易。

风险提示：不要把群聊里的“解压

14

天、撸

30

个点”当成收益预期。它更适合作为待核查案例：先弄清楚锁定期、收益来源、资产价格风险和退出条件。

八、MEV、alpha 与竞争

alpha：在聊天语境里，指你相对其他人拥有的优势，例如更快的数据、更好的流动性、更低的成本或独特的信息。
alpha 不是保证盈利。

MEV：可以先理解为利用交易排序、打包和执行顺序来提取价值。它会改变普通交易的成交环境。

三明治攻击：常见描述是攻击者在目标交易前后插入交易，先买入、让目标交易推高价格，再卖出。初学者先学会识别和防范，不要把它当作入门策略。

聊天中还提到私有池、常规池和“慢半拍”。这些都指向同一个事实：套利机会不仅由价格决定，也由数据速度、交易排序、流动性和竞争程度决定。

九、从监控到自动交易：程序要解决什么

1. 先区分“看机会”和“下交易”

监控：持续读取价格、路线、流动性和费用，发现可能的价差。聊天中有人先“只做监控”，这是更适合新手的阶段。

执行：真正签名并广播交易，必须额外处理超时、失败、重复发送、授权、Gas 和双腿协调。

2. 一个合格的套利程序至少应有六个模块

17. 数据模块：读取 CEX、DEX、聚合器和链上状态。

18. 机会模块：计算不同市场、不同链之间的价差。

19. 成本模块：计算交易费、Gas、桥费、滑点和预计失败成本。

20. 执行模块：构造路线、生成交易、发送并追踪状态。

21. 风控模块：设置金额、滑点、Gas、等待时间和停止条件。

22. 记录模块：保存报价、实际成交、失败原因和最终盈亏。

3. API、calldata 和 QPS

API：方便程序获取报价、行情或提交交易，但 API 返回的报价仍可能过时。

calldata：直接调用合约时携带的参数。自己构建 calldata 代表你承担更多交易细节，也获得更多控制权。

QPS：每秒请求次数。高 QPS 会增加成本和复杂度；有些策略不需要很高 QPS，关键是机会持续时间和数据质量。

十、最重要的风控：单腿风险

“单腿”就是本来要同时完成买入和卖出，结果只成功了一边。这样你手里留下的不是已经锁定的套利利润，而是一个暴露在价格波动中的资产头寸。

1. 常见失败原因

- 一边交易成功，另一边因为余额、Gas、滑点、合约或网络问题失败。
- 报价有效期很短，发送交易后市场已经变化。
- 跨链尚未到账，另一边的价差已经消失。
- 单边价格突然爆拉或暴跌，原本的收敛逻辑失效。

2. 风控判断框架

23. 交易前：设置最小净利润、最大金额、最大滑点、最大 Gas 和最大等待时间。

24. 交易中：确认每一步状态，不要因为“看起来应该成功”就继续下一步。

25. 一边失败：先停止扩大风险，再选择止损、撤回、对冲或补全另一边；具体选择取决于策略和市场状态。

26. 交易后：记录真实成交价格 and 失败原因，不要只看模拟结果。

原群聊中的建议可以压缩成一句话：让程序在执行前就知道什么情况不能做，在一边失败后知道什么时候停、什么时候退出，而不是把判断留给情绪。风控口诀：

十一、你的零基础学习路线

建议按下面顺序学习，每一步都有一个“学会标准”。不要一开始就跳到自动化跨链套利。

阶段 1：建立基础词汇

- 学会解释：网络、钱包、地址、私钥、智能合约、Gas、代币。
- 画出“钱包签名 → 广播 → 确认 → 到账”的流程。
- 能分清原生币、普通代币和跨链包装资产。

阶段 2：理解交易场所

- 分清 CEX、DEX、流动性池、聚合器和路由。
- 看懂一份报价中的买入数量、预计到账、费用和最小到账。
- 理解报价、滑点、价格影响和实际成交不是一回事。

阶段 3：用纸面数据练套利计算

- 自己制作一张表，输入买入价、卖出价、交易费、Gas、桥费和滑点。
- 只要净利润小于 0，就明确标记为“不做”。
- 给价格变化、到账延迟和一边失败设置压力测试。

阶段 4：学习跨链与 Cosmos

- 先查清一条链、一个桥和一个包装资产的完整路径。
- 核对合约地址、发行方、流动性、赎回方式和等待时间。
- 理解“用户无损”和“协议盈利”是两个不同问题。

阶段 5：只做监控，再做模拟

- 先写或搭建只读监控，不自动签名。
- 保存每次机会出现时的报价、成本和最后结果。
- 等你能解释大多数失败原因，再考虑极小规模、可承受损失的测试。

十二、原聊天里值得带着问题去查的内容

下面这些内容在群聊中出现过，但不应该直接背成结论。它们适合作为你的检索和验证清单：

- LI.FI、OKX DEX、Jupiter、Uniswap、1inch、PancakeSwap 的功能边界和报价来源分别是什么？
- 所谓“虚报报价”到底是缓存延迟、流动性不足、模拟失败，还是界面展示问题？
- 某个桥是否真的把沉淀资金投入 Aave、质押或流动性挖矿？资金流在链上如何验证？
- axlUSDC、axlATOM、stkATOM 的发行方、合约地址、赎回方式和流动性在哪里查？
-

Cosmos

生态某条链是否还有真实用户，应看活跃地址、交易量、流动性和费用，而不是只听一句“还有没有用户”。

- “解压 14 天、撸 30 个点”具体指什么资产、什么锁定期、什么收益来源和什么退出风险？

十三、最后背下这十句话

1. 看到价差，先算净利润；看到收益，先问风险来源。
2. 报价只是预计值，成交才是结果。
3. 手续费、Gas、滑点和等待时间，都会吃掉套利空间。
4. 同名资产不一定是同一种资产，先核对合约地址。
5. 跨链桥是风险放大器，时间和安全模型都要学。
6. 无损跨链不等于无手续费、无波动、无合约风险。
7. Cosmos 是多链生态，ATOM 不等于所有链的通用货币。
8. 套利通常有两条腿，一边失败就会变成单腿风险。
9. 程序化交易的核心不是发命令，而是监控、计算、执行和风控。

学习目标：当你能用自己的话解释“资产在哪里、价格为什么不同、成本有哪些、两条腿如何完成、失败后如何退出”，就已经从完全没有概念，进入可以继续深入研究的阶段。

作者：HOU RUI | 时间：2026-08-06 15:39 UTC | 字数：7024

D3 打卡 · 2026-08-06

主题：V3/V4 数学机制 + LP 收益的量化验证（提前做了 D9 的内容）

做了什么

- ## 验证 / 推翻了什么

1. 验证：V4 的定价数学与 V3 逐字相同

网上说法混乱，直接对比源码常数：

MIN_TICK = -887272 $\leftarrow$ v3/v4 一致

MIN_SQRT_PRICE = 4295128739 ← 一致

MAX_SQRT_PRICE = 1461446703485210103287273052203988822378723970342 ← 一致

getSqrtPriceAtTick 魔数表 0xffffcb933bd6fad37aa2d162d1a594001 ← 逐字相同

TickMath / SqrtPriceMath / SwapMath 三个库核心逻辑完全一致，行数差异只来自 gas 优化和自定义 error。

结论：数学代码在 V4 可直接复用，但数据管道要重写 —— V4 池子没有独立地址（Swap 事件给的是 bytes32 PoolId），且事件符号约定与 V3 相反。

另外从 IHooks.sol 确认一件重要的事：

solidity

```
function beforeSwap(...) external returns (bytes4, BeforeSwapDelta, uint24);
```

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

改变成交量 覆盖费率

hook 能改变成交量和费率。 所以做 V4 套利前必须检查目标池挂了什么 hook , 否则定价模型会在一个“看起来是标准池”的地方静默失效。

2. 达成 D3 硬验收：本地定价与链上 QuoterV2 逐位一致 7/7

投入 本地 amountOut QuoterV2 差 滑点 跨tick

100 52,376,623,375,740,382 52,376,623,375,740,382 0.050% 1

1,000 523,765,520,776,219,351 523,765,520,776,219,351 0.050% 1

100,000 52,368,710,468,967,228,454 52,368,710,468,967,228,454 0.065% 2

1,000,000 522,993,339,351,836,885,400 522,993,339,351,836,885,400 0.198% 4

为此撞了两个 bug：

① 不能用 Decimal 算 $\sqrt{(1.0001^{\text{tick}})}$

与链上有恒定 $1.57e-5$ 的相对误差。误差恒定说明不是随机舍入而是系统性问题 —— 链上用的是 Q128.128 魔数表逐位相乘，不是实数开方，必须逐字复刻那 20 个常数。

② 必须 --block 锁定区块

池子每块都在被 swap。本地读区块 N、QuoterV2 在 N+2 执行，差异是竞态不是数学。这个坑让我 debug 了一个不存在的舍入问题。

对 D12 回测的启示：任何“本地计算 vs 链上验证”都必须锚定同一区块。

3. 推翻：LP 不是“被动”收租，而是在无条件按陈旧价格接单

搞清了 LVR (Loss-Versus-Rebalancing) 的机制，并从论文原文核实公式 (arXiv:2208.06046，作者称之为“AMM 的 Black-Scholes 公式”)：

通式 $\ell/V = \sigma^2/2 \times \theta(1\theta)$ 恒定乘积池 $\theta=1/2 \rightarrow \sigma^2/8$

自检 论文例子: $\sigma=5\%/日 \rightarrow 3.125bp/日$ ；验算 $0.05^2/8 = 3.125bp$

机制：AMM 报价是陈旧的。外部市场价格动了，池子不会自动跟上，必须等套利者来交易才被推到新价格 —— 而他愿意来，正因为拿到了对自己有利的价格。那部分利润的来源就是 LP。论文原话：“stale prices that are picked off by better informed arbitrageurs”。

LVR 与无常损失是两回事（多数 LP 教程讲错的地方）：

 	IL 无常损失	LVR
对照基准	买入并持有	持续再平衡的组合
可恢复性	价格回来就消失	单调、永久流失
衡量对象	市场敞口+逆向选择混合	只衡量逆向选择

用 IL 思考会系统性低估成本，因为它暗示“价格回来就没事了”。

实测 USDC/WETH 0.05% 池， $\pm 1\%$ 区间，1 万美元本金：

手续费 APR 78% ← 来自普通交易者（不知情流）

LVR -436% ← 被套利者拿走

gas -14%

= 净 APR -372% 覆盖率 0.18x

主流 ETH 池被动做 LP 是结构性亏损的。gas 只占 14%，换 L2 也救不回来。

这也解释了 D4 那句“做套利时你在赚 LVR，做 LP 时你在付 LVR”—— 现在有数字了。

4. 推翻我自己的三个错误结论（今天最有价值的部分）

错误	真相
费率 APR = 全池手续费 × 份额	漏了 in-range 占比。全池手续费含发生在你区间外的交易，那部分拿不到。实测 30 天里只有 27% 时间在赚钱
费率 APR = naive_APR × k	口径混淆 (TVL vs 活跃 L)，高估 3.7 倍。必须走份额法
LVR 不该乘区间杠杆 k	论文直接反驳了我

第三条值得展开：我先推导出“LVR 占本金比例与区间宽度无关”，去查论文发现

Example 4 明确写着 $\ell = L \cdot \sigma^2 \cdot \sqrt{P}/4$ —— 只依赖 L，而窄区间同样本金撑起更大的 L。

原文：“the instantaneous LVR per dollar of pool reserves can be arbitrarily high ... if the liquidity range is narrow”。

这就是为什么要读一手资料，而不是相信自己的推导。

5. 推翻：我第一版扫池子的结论是筛选偏差

第一版跑出覆盖率 200 万倍这种荒谬值。没有交付，去查发现两个单位 bug 叠加：

subgraph 的 sqrtPrice 与链上 slot0() 口径不同 (差 10^4 倍, 怎么调小数位都对不上) → 改用 token0Price
本金写 $\text{capital} \times 10^{\text{dec0}}$ 等于假设 token0 就是美元 → WETH/USDT 池 (dec0=18)
让 L 放大 10^4 倍, 份额变成 ~100%

修完后又发现 universe 本身有问题: 为了换算本金我要求 token0 是稳定币, 400 个组合淘汰了 336 个 (84%), 导致达标名单全是稳定币对 —— 那只是因为非稳定币池根本没进候选。

改用 $\text{derivedETH} \times \text{ethPriceUSD}$ 定价 (实测校验 USDC=\$1.0000、WETH=\$1905.41、WBTC=\$64,667.44, 且与 token0Price 交叉一致), 通过率从 16% 升到 64%, 结论完全变了:

配对 费率 区间 σ 日 inRng 费 APR LVR 覆盖 净 APR TVL
USDC/SEI 0.30% $\pm 20\%$ 2.44% 91% 74.62% 31.06% 2.40x 43.56% \$5.1M
WETH/TITANX 1.00% $\pm 50\%$ 4.68% 100% 96.31% 54.56% 1.77x 41.74% \$2.5M
TONCOIN/WETH 1.00% $\pm 20\%$ 1.63% 95% 41.82% 13.94% 3.00x 27.88% \$3.1M
RNDR/WETH 1.00% $\pm 20\%$ 1.63% 79% 33.47% 13.96% 2.40x 19.51% \$4.5M
CULT/WETH 1.00% $\pm 20\%$ 0.76% 92% 19.04% 3.01% 6.33x 16.03% \$3.6M

12 个合格组合里 9 个是 1% 费率池, 没有一个是稳定币对。

印证盈亏平衡条件 手续费 $> \sigma^2/8$ —— 高波动必须配高费率。

稳定币对 (USDC/USDT 覆盖 5.13x 但净 APR 0.39%) 全部落在 "安全但白干" 那档。

6. σ 分组验证 $\text{LVR} \propto \sigma^2$ (383 个样本)

极低 $< 0.1\%$ 平均 LVR 0.06% 平均覆盖 5.86x
低 0.1-0.5% 平均 LVR 1.66% 平均覆盖 1.88x
中 0.5-1.5% 平均 LVR 35.65% 平均覆盖 0.93x
高 1.5-3% 平均 LVR 106.00% 平均覆盖 0.78x
极高 $> 3\%$ 平均 LVR 1452 万% 平均覆盖 0.33x

覆盖率随 σ 单调递减, 日波动率 1.5% 是分水岭 —— 超过它平均就亏。

今天最重要的方法论收获

加硬自检断言, 而不是相信漂亮的数字。

给扫描器加了: 份额 $> 50\%$ / 覆盖率 $> 100x$ / APR $> 2000\%$ 直接丢弃并计数。
这些值在真实市场不可能出现, 出现就是 bug。这次拦下 157 个错误组合。

一条通用规则: 任何 "份额" 类计算出来接近 100% 时, 先怀疑单位。

副产品 —— TVL 趋势检测抓到了反面证据:

XAUt/USDT 0.05% TVL 趋势 0.67x (30 天掉 33%) 净 APR -0.80%

我算出亏钱的池子, LP 确实在撤资。模型和市场行为对上了, 这比任何自我验证都有说服力。

环境上的坑 (都是静默失效, 比报错危险)

| 坑 | 症状 |

| :----- | :----- |

| 免费公共 RPC 只保留约 100 区块日志 | 更早区间返回 0 条而不报错 →

成交量被低估但时间跨度按实际算, 两头都错却能算出 "合理" 数字 |

| The Graph 出错也返回 HTTP 200 | 错误在 body 的 errors 字段, 只看状态码会把认证失败当成功 |

| urllib 默认 UA 被 Cloudflare 拦 | 403 / error 1010, curl 同一请求正常 —— 差别只在 User-Agent |

| cast receipt 默认轮询等待打包 | 传不存在的 hash 会永久挂住, 要加 --async |

| Foundry 的 anvil 需要 glibc 2.35 | 本机 2.34 装不上 (forge/cast/chisel 正常), D11 的 fork 模拟受影响 |

卡住 / 待验证

1. σ 用池子自身价格算，偏高 —— 池子价格含套利者推动的瞬时偏离，不等于外部市场真实波动率。要更准应该用 CEX 价格序列
2. in-range 用价格区间重叠比例近似 —— 真实成交量在价格上不均匀（停留越久成交越多），要精确需逐笔重放 Swap 事件
3. 157 个组合因份额 >50% 被丢弃 —— 不是 bug 而是真实约束：
1 万美元在 TVL 2-5M 的小池子里占比太大。意味着候选的实际可投规模远小于 1 万美元
4. 候选全是 TVL 2-6M 的小池子 —— 高覆盖率可能只是因为“没人愿意在这里做 LP”，而原因可能是我模型里没有的风险（rug、流动性抽干、代币归零）

明天验证什么

深挖 USDC/SEI（唯一同时有收益和安全边际的组合）：

1. 那 74% 费率 APR 是不是靠某几天的成交量尖峰撑起来的？（用中位数而非均值验证）
2. 5.1M TVL 能承载多大仓位而不让份额失真？
3. SEI 的尾部风险 —— 30 天数据看不到的东西

产出的代码

scripts/v3_quote.py 本地复现 V3 定价，与 QuoterV2 逐位比对

scripts/liquidity_profile.py 流动性深度剖面（tickBitmap + liquidityNet 累加）

scripts/lvr.py LVR 估算 + 指定区间分期年化（3d/7d/30d）

scripts/lp_apr.py 三层 APR 拆解（naive → 活跃区间 → 你的份额）

scripts/subgraph.py The Graph 客户端

scripts/scan_pools.py 批量扫池子找覆盖率 > 1.5x 的组合

notes/v3-v4-math.md V3/V4 数学笔记 + 数据源口径陷阱记录

笔记 15

作者：cfanboy | 时间：2026-08-06 06:42 UTC | 字数：6637

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 跨链可执行价差 120 轮实测

run_id 20260806T010728Z · 2026-08-06 01:07:28 → 03:11:15 UTC（2 小时 04 分）

结论

两小时内没有任何可用机会。当前费率下 480 次跨链配对判定全部为负；即便把费率谈到零，也没有任何资产、任何一轮达到 +10 bps。

更值得记的是过程：一个看起来明确赚钱的信号被诊断证伪——它来自单一报价源的乐观估值，不是市场。这条发现比“没找到机会”更有价值，详见第二节。

| 指标 | 值 |

| :----- | :----- |

| 采样轮次 | 120 |

| 报价条数 | 2,880 (outcome 全部 ok) |

| 组内时间跨度 | 9-31 ms |

| 可用机会 | 0 |

| 配额峰值 | 2,415 / 12,000 (2 小时窗口) |

| 隔离报价 | 546 条 (fly) |

一、同一批数据，三个口径，三个截然不同的答案

每一轮在三条链上同时取买腿与卖腿报价，算出 $\text{edge} = \text{卖价} \div \text{买价} - 1$ 。报价本身已含 DEX 手续费、价格冲击与 LIFI 协议费，所以这个数字直接就是“扣完双边成本后还剩多少”。零以上才是钱。

口径一·现状（默认 integrator 费率，双边共 50 bps）

资产	min	中位	max	正收益
WETH	57.2	51.1	42.7	0/120
wstETH	57.6	51.6	45.0	0/120
USDT	50.0	43.5	40.6	0/120
LINK	131.0	97.3	42.0	0/120

口径二·零费率（诚实答案）

假设费率谈到 Jumper 级（0%），逐腿从 feeCosts 读取真实值扣回，非常量假设。

资产	min	中位	max	正收益	≥ +10 bps
WETH	7.3	1.2	+7.2	33/120	0
wstETH	7.7	1.7	+4.9	25/120	0
USDT	0.1	+6.4	+9.4	109/120	0
LINK	81.4	47.6	+8.0	1/120	0

WETH 与 wstETH 在零线附近微幅摆动，完全符合无套利区间理论——价差被“搬平成本”夹住。

唯一持续为正的是 USDT 的 base 买 → arbitrum 卖，中位 +6.4 bps、109/120 轮为正，方向稳定得像个真实的微小基差（Base 侧 USDT 流动性偏浅），但量级不足以覆盖 gas 与再平衡成本。

口径三·含污染报价（已证伪，若不做诊断报告会这样）

资产	中位	正收益	≥ +10 bps
WETH	+32.5	118/120	118
wstETH	+28.1	99/120	80
USDT	+20.5	120/120	83
LINK	+0.1	60/120	18

全部 edge 未含 gas（同链约 0.6 bps @ \$1,000）。口径一、二已剔除污染源。

二、那个差点写进结论的假信号

口径三看起来是明确的赚钱机会。戳破它不需要更多数据，只需要一个能被证伪的量：

同一条链上买入后立刻卖出。这个往返必然亏掉双边 DEX 成本，任何正值都意味着报价体系自相矛盾。

证据一：同链买卖往返（零费率口径，中位 bps）

资产	Arbitrum	Optimism	Base
WETH	1.7	7.2	+36.3
wstETH	3.3	8.5	+29.3
USDT	+0.6	0.2	+26.8
LINK	21.3	144.4	34.1

Base 一列全线倒挂。在这张表里正值意味着报价体系坏了，不是赚钱。

证据二：按（买腿路由 × 卖腿路由）拆解 Base 链（样本 ≥ 5 的配对）

买腿路由 → 卖腿路由	资产	样本	中位 bps
fly → fly	WETH	60	+44.5
kyberswap → fly	WETH	22	+35.5
fly → fly	wstETH	36	+35.0

okx → fly	wstETH	38	+30.2
okx → kyberswap	wstETH	6	+1.8
okx → okx	wstETH	13	+0.4
kyberswap → kyberswap	USDT	6	+0.1
kyberswap → okx	USDT	6	0.1
okx → okx	LINK	5	62.6

凡是牵涉 fly 的配对全部越过零线；不牵涉它的配对全部落在 0 附近或为负（正常形态）。

结论与处置

fly (Fly.trade / Magpie) 自报的买价与卖价在 Base 上交叉 +44.5 bps——等于“低买高卖自己”。真实成交不可能如此，唯一解释是它的 toAmount 系统性高估。

这意味着 LI.FI /quote 返回的“最优路由”可能被乐观报价源污染：当某个源虚报产出，它就会赢得路由竞争并出现在结果里。

已落地三项处置：

1. analysis.py 加入 QUARANTINED_TOOLS 隔离名单，隔离条数在报告中显式打印，不做静默剔除
2. 新增回归测试 test_quarantined_tool_excluded_from_edges 锁住该行为
3. 证据写入 05-arbitrage-notes.md (./05-arbitrage-notes.md) §2.5

阶段三小额实盘要回答的第一个问题因此变更为：报价与实际成交的偏差有多大。

三、行情背景：这两小时非常平静

WETH 零费率 edge，每 10 轮均值。所有柱子从零线（表头 0 处）向左延伸 = 亏损，向右 = 盈利：

-4 -2 0 +1 bps
轮 1-10 ## -0.1
轮 11-20 ##### -0.8
轮 21-30 ## +0.1
轮 31-40 ##### -0.8
轮 41-50 ##### -1.4
轮 51-60 ##### -3.1
轮 61-70 ##### -1.0
轮 71-80 ##### +0.4
轮 81-90 ##### -3.3
轮 91-100 ##### -3.6
轮 101-110 ##### -2.0
轮 111-120 ##### -0.5

全程区间 3.6 ~ +0.4 bps，无任何异动。

这条曲线决定了本轮结论的适用范围：我们证明的是“平静行情下无机会”，而库存法策略的收入来源恰恰是价差冲出无套利区间的那些时刻。因此下一轮要统计的不是中位数，而是事件的发生频率与持续时长。

四、换标的：一个被实测证伪的假设

本轮结束时的直觉是“换成流动性更薄、波动更大的资产会有更多机会”。为验证它，对 24 个候选逐个实测成本地板——同链买卖往返的成本，跨链错位必须先跨过它才谈得上盈利。结论与直觉相反。

资产	最差链上地板	含义
UNI @ Optimism	562.9	需 5.6% 错位才回本
COMP @ Base	427.8	不可用

AAVE @ Optimism	198.1	不可用
LDO @ Arbitrum	168.3	不可用
CRV @ Base	102.6	不可用
LINK	47.6	第一版标的，据此剔除
WBTC @ Arbitrum	1.8	可用
weETH @ Base	0.2	全部候选中最紧

BAL、ENA、MORPHO、RPL、VIRTUAL 在 \$1,000 规模下多数腿直接无路由。

标的选择的第一准则是成本地板，不是波动率，也不是"小众程度"。

五、第二轮标的与判定标准

新标的按实测地板选出，并刻意让价格源相互独立：ETH 系、BTC 系、稳定币系三条独立行情线，而非第一版那样三个都跟着 ETH 走。

| 资产 | 链 | 地板 | 入选理由 |

| :----- | :----- | :----- | :----- |

| weETH | arb, base | 0.4 / 0.2 | 24 个候选中最紧，比 WETH 还紧。LRT

赎回在主网，各链市场天然分割。Optimism 无路由 |

| WBTC | 三链 | 1.8 / 9.2 | BTC beta，与 ETH 系价格源相互独立 |

| tBTC | 三链 | 3.4 / 11.1 | 第二个 BTC 表示，发行方与锚定机制均不同，两者之间也可能错位 |

| DAI | 三链 | +0.7 / 0.4 | 非 USDC 系稳定币，独立于计价基准 |

| WETH · USDT | 三链 | 对照 | WETH 有 120 轮历史作基准；USDT 是第一版唯一持续为正的资产 |

剔除 wstETH（与 WETH 同为 ETH beta，120 轮未提供独立信息）与 LINK（地板无解）。

配额核算：34 腿/轮 × 60 轮/小时 = 34 次/分 (< 100 RPM 闸门)，

每 2 小时窗口 4,080 次 (< 12,000 池)，24 小时长跑安全。

判定标准：若 24 小时内零费率口径下没有任何 $\geq +10$ bps 的事件，

这条路线在 L2 主流资产上即告失败，应转向其他策略或其他市场。

第二轮 run_id：24h_v2_20260806T033308Z

六、方法与已知局限

时间对齐：一轮内全部腿并发发出，组内跨度 9–31 ms。参考实现的串行采集约 14 秒，

噪声与信号同量级，结论不可用。

成本内含：edge 由可执行报价直接算出，已含 DEX 手续费、价格冲击与 LI.FI 协议费；

未含 gas（同链约 0.6 bps @ \$1,000）。

最大局限：全部结论基于报价而非成交。fly 事件恰恰说明报价可能失真，

报价与成交的偏差分布尚未测量。

样本局限：两小时、单一平静时段、三条 L2、\$1,000 名义规模。

结论不外推到其他行情、链或规模。

笔记 16

作者：OxChainWorld | 时间：2026-08-05 07:35 UTC | 字数：6624

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · 5 阶段弹性自学计划

基于 链上套利残酷共学 (<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>)

发起人参考大纲整理

共学周期：2026-08-05 → 2026-08-26 (21 天)

文档类型：5 阶段弹性计划（日期仅为建议映射，可按真实发现前后调整）

0. 怎么用这份计划

核心理念

共学不是固定课表，而是问题驱动 + 证据驱动：看见价差只是起点，关键是「扣除成本后是否仍成立」。
每人路径可以不同；本计划提供统一阶段目标与验收标准，方便打卡与互相对照。
允许跳阶段，但不建议跳过验收：没有证据记录的「发现」不算完成。

建议节奏

投入档位	每天时间	怎么用
最低有效	60–90 分钟	只做「必做」，保证每日打卡与证据行
标准	2–3 小时	必做 + 1 个可选深入
深度	4 小时+	必做 + 可选 + 把工作流自动化

请假规则对齐共学：每周最多 2 次；长期无有效打卡可能被移出群。

每日最小闭环（建议固定模板）

1. 观察：今天盯了什么链 / 资产 / 路径？
2. 假设：你认为机会从哪里来？
3. 验证：Gas / 滑点 / 桥费 / 延迟算过没有？结论是什么？
4. 证据：截图、报价、交易哈希、失败原因（至少一项）
5. 下一步：明天只改一个变量继续测

安全边界（必读）

本共学不承诺收益，不是带单，不代表 LI.FI 官方合作。
研究 / 模拟 / 实盘必须分开；实盘仅限可承受损失的小额，且先独立核验。
绝不向任何人提交私钥、助记词、API 密钥。
文档中的工具链接仅作学习入口，执行前请自行核验官方来源。

阶段总览

阶段	建议日期	主题	结束时你能回答
1	D1–D4 (8/5–8/8)	建立链上套利地图	屏幕价差 ≠ 可执行净收益，差别在哪？
2	D5–D9 (8/9–8/13)	用 LI.FI 发现路径与机会	同一笔意向，不同路由成本如何变化？
3	D10–D13 (8/14–8/17)	Hermes 研究与信号工作流	一次性观察如何变成可重复流程？
4	D14–D18 (8/18–8/22)	从假设走向策略	你的 Edge 经得起成本模型吗？
5	D19–D21 (8/23–8/26)	捕获、实施与复盘	哪些信号可重复？下一阶段研究什么？

若中途发现强信号，可提前进入阶段 4/5，但请回头补齐阶段 1–2 的成本语言，避免「看起来能赚、算完就亏」。

阶段 1 | 建立链上套利地图

建议：D1–D4 | 目标：建立共同语言，能把「机会类型」说清楚并初步量化

学习目标

认识常见机会结构：DEX 内、DEX 间、跨链、三角、稳定币、清算、MEV 相关路径
理解 AMM / 订单簿、流动性深度、价格冲击、区块确认对执行的影响
能明确区分：屏幕价差 vs 可执行净收益

必做（Practice）

1. 画一张个人套利地图（一页即可）

至少覆盖 5 类机会；每类写：触发条件 / 主要成本 / 主要风险 / 你是否打算研究

2. 做 3 次「假想成交」成本拆解（可不真下单）

任选同一资产在两个场所的报价差

手算或估算：手续费 + Gas + 滑点 + （如有）桥费 + 时间风险

结论写成：毛价差 / 预估成本 / 净收益是否 > 0 / 不确定项

3. 名词对照表（自己的话写 8–12 条）

例：AMM、深度、价格冲击、路由、桥、滑点、失败交易、资金占用

可选深入

拆解一笔公开套利/MEV 相关交易（浏览器 + 社区案例均可）

对比「稳定币脱锚」与「普通 DEX 价差」在风险上的不同

阶段验收（打卡可贴）

套利地图一页

≥3 条成本拆解记录（含「为何不可执行」也算成功）

能用自己的话解释：为什么屏幕价差经常赚不到

本阶段产出物

01-arbitrage-map.md + 01-cost-breakdowns.md

阶段 2 | 使用 LI.FI 发现路径与机会

建议：D5–D9 | 目标：把跨链/聚合路由变成可观察的实验对象

学习目标

理解 LI.FI 作为跨链流动性聚合与路由层的角色（swap / bridge / 多步路径）

会比较：不同链、资产、规模、路由下的报价与执行条件

观察桥、DEX、Gas、滑点、手续费、路径变化如何「吃掉」机会

必做（Practice）

1. 官网 + 文档扫图

入口：LI.FI Docs (<https://docs.li.fi/>) · LI.FI (<https://li.fi/>)

用自己的话写：LI.FI 解决什么碎片化问题？它不是什么？

2. 对照实验（至少 5 组报价）

固定一个「意图」，只改一个变量，记录报价：

| 实验# | 源链 | 目标链 | 资产 | 金额档位 | 路由/桥 | 预估到手 | Gas/费 | 预估耗时 | 备注 |

| :- | :----- | :----- | :--- | :----- | :----- | :----- | :----- | :----- |

| 1 |
 |
 |
 | 小 |
 |
 |
 |
 |
 |

| 2 |
 |
 |
 | 中 |
 |
 |
 |
 |
 |

建议变量轮换：金额档位（小/中）、目标链、是否允许某类桥、滑点容忍。

3. 写「机会杀死清单」

列出你观察到的 ≥5 个让价差变负的因素（例：桥费突变、路由跳变、深度不足、确认慢）

4. 加入社区（可选但推荐）

LI.FI Builders 相关社群，跟进开发者讨论（以页面/群内最新入口为准）

可选深入

阅读 API/SDK 文档，尝试一次 quote 查询（只读，不执行）

同一意图对比：LI.FI 路径 vs 单一桥/单一 DEX 的差异

阶段验收

能说明 LI.FI 在套利研究中的位置（发现路径 / 比较执行条件）

≥5 组对照实验表

「机会杀死清单」≥5 条

本阶段产出物

02-lifi-experiments.md（含实验表）

阶段 3 | 用 Hermes 构建研究与信号 workflow

建议：D10–D13 | 目标：把偶然观察变成可重复的研究流水线

学习目标

用 Hermes Agent 辅助：信息发现、研究拆解、工具调用、数据整理、持续监控

把一次性观察沉淀为：可重复查询 / 告警 / 脚本 / Agent workflow

为候选机会建立证据档案

必做（Practice）

1. 定义你的「研究工位」

工具清单：浏览器、区块浏览器、LI.FI、Hermes、表格/笔记、（可选）脚本环境

规定：每个候选机会必须写入证据模板（见文末）

2. 选 1 个候选机会，跑完整研究拆解

背景 → 假设 → 所需数据 → 验证步骤 → 当前结论 → 待办

要求：别人只看你的笔记也能复现检查步骤

3. 把流程「可重复化」至少一项

例：固定提示词模板、每日检查清单、告警规则草稿、定时查询伪代码/脚本骨架

4. 证据记录 ≥5 条（含失败案例；失败更有价值）

可选深入

搭建最小监控：价格/路由/某个池子状态的定时检查

用 Agent 生成「每日 briefing」，但人工复核关键数字

阶段验收

研究工位说明（半页即可）

1 个完整拆解案例

至少 1 个可重复工件（模板/脚本/告警草稿）

证据库 ≥5 条

本阶段产出物

03-research-workflow.md + evidence-log.md（可长期沿用）

阶段 4 | 从假设走向策略

建议：D14–D18 | 目标：形成可检验的策略候选，而不是灵感收藏夹

学习目标

从公开案例与真实交易中提出自己的套利假设

建立成本模型：Gas、协议费、桥费、滑点、延迟、失败交易、资金占用

用历史数据 / 模拟 / Paper Trading / 小规模验证判断是否还有 Edge

产出：策略说明 + 信号规则 + (可选) 最小原型

必做 (Practice)

1. 写下 1–2 个策略假设 (Hypothesis Card)

名称：

市场结构：(DEX间 / 跨链 / 稳定币 / ...)

为什么会出现价差：

进入条件 (信号)：

退出/放弃条件：

预期持有时长：

主要成本项：

主要失效场景：

目前证据：

2. 完成一份成本模型 (表格即可)

至少包含：显性费用 + 隐性滑点/冲击 + 时间与失败风险 + 资金占用机会成本

给出：盈亏平衡点 (价差至少要多大才值得动)

3. 验证设计

选一种：回看历史片段 / paper trade / 小额模拟流程 (仍可不碰真钱)

记录：样本数、胜率粗估、平均净收益、最大回撤式风险 (即使是定性也要写)

4. 策略一页纸 (Strategy One-Pager)

给同伴看：30 秒能听懂你在做什么、何时不做

可选深入

最小执行原型 (报价监控 → 规则判断 → 人工确认)

对比两个假设，决定「主策略 / 备选 / 放弃」

阶段验收

≥1 张完整 Hypothesis Card

成本模型含盈亏平衡点

有验证记录 (哪怕结论是「暂无 Edge」)

Strategy One-Pager

本阶段产出物

04-hypothesis.md + 04-cost-model.md + 04-strategy-onepager.md

阶段 5 | 捕获、实施与复盘

建议：D19–D21 | 目标：建立行动纪律与可延续的研究资产

学习目标

信号出现时，能快速做：真实性 / 可执行性 / 风险 三联检查

按能力选择：继续观察、模拟、或严格受控的小额实施

区分「可重复机会」与「偶然噪声」

形成：知识图谱摘要、策略候选清单、下一阶段研究方向

必做 (Practice)

1. 写一份「信号出现时的 5 分钟检查单」

例：报价是否仍在？路由是否跳变？成本是否穿透盈亏平衡？权限/授权安全？最坏损失？

2. 复盘周报（共学收官版）

做过的实验数、失败数、最有价值的一次失败、当前策略状态（观察中/模拟中/已放弃/小额验证）

3. 个人知识图谱（一页）

节点建议：机会类型 → 工具 → 成本因子 → 你的假设 → 验证状态

4. 下一阶段 30 天研究方向（3 条以内，可执行）

可选深入

若且仅若：规则清晰、风险可承受、安全流程完备 → 极小额受控验证，并完整记录

把检查单做成 Agent/脚本辅助（最终决策仍建议人工）

阶段验收

5 分钟检查单

收官复盘

知识图谱一页

未来 30 天研究清单（≤3 条）

本阶段产出物

05-signal-checklist.md + 05-final-review.md + 05-next-30-days.md

附录 A | 证据记录模板（建议全员统一字段）

每发现一个候选机会，新增一行/一页：

ID：

时间（UTC/本地）：

链 / 资产 / 规模：

机会类型：

路径描述：（含桥/DEX/路由若可知）

毛价差：

成本拆解：Gas / 协议费 / 桥费 / 滑点 / 其他

预估净收益：

可执行性：是 / 否 / 不确定

失败或放弃原因：

证据链接：（报价截图、tx、浏览器、笔记）

后续动作：观察 / 模拟 / 放弃 / 小额验证

附录 B | 每日打卡模板（可直接复制到群）

【Day X | 阶段 Y】

今日投入：分钟

做了什么：

关键发现（1 句）：

证据：

结论（净收益是否成立）：成立 / 不成立 / 待验证

卡点：

明天只验证一件事：

附录 C | 共学结束时的「合格产出包」（建议互相 review）

1. 套利地图
2. LI.FI 对照实验表（≥5）
3. 证据库（≥10 更佳，含失败）
4. 至少 1 个策略假设 + 成本模型 + 验证记录
5. 信号检查单 + 收官复盘 + 下一步研究方向

有没有赚到钱不是合格标准；有没有可复核的研究闭环才是。

附录 D | 参考资料入口

共学主页：<<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>>

发起人参考方向：页面内《为什么我在熊市发起链上套利残酷共学？》及学习大纲

LI.FI 文档：<<https://docs.li.fi/>>

LI.FI 官网：<<https://li.fi/>>

联系与社群：以共学页面最新信息为准（如 Bruce 微信 brucexu-eth、LI.FI Builders Telegram）

笔记 17

作者：X | 时间：2026-08-05 15:00 UTC | 字数：6327

来源：<<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>>

链上套利类型、经典案例与行业现状

一、什么是链上套利

链上套利是指利用同一资产在不同市场、链、协议或时间窗口中的短暂价格偏差，扣除全部交易成本后获取正收益。

净收益 = 毛价差

- DEX 手续费
- 跨链 / 桥费用
- Gas
- 滑点与价格冲击
- 借贷或资金成本
- 延迟与失败损失

需要区分几个概念：

闪电贷是融资工具，不是独立的套利类型。

LI.FI是跨链与交易路由工具，不保证路线一定有套利收益。

MEV是交易排序与执行环境，不一定代表某一种价格来源。

清算套利和预言机套利具有较强的方向性或协议风险，不属于严格意义上的无风险套利。

二、主要类型总览

类型	经典案例	核心关键点	当前状态
同链 DEX-DEX 套利	DeFi Summer 期间 Uniswap 与 SushiSwap 之间的价差	速度、流动性、Gas、原子执行	机会仍有，但竞争激烈、利润变薄
CEX-DEX 套利	新币上线时中心化交易所与 DEX 的价格溢价	提现、库存、API、对手方风险	仍存在，偏资本密集型
三角套利	USDC → ETH → USDT → USDC	三条路径必须同时成立，手续费累积	机器化程度高，需极低成本
跨链套利	原生 USDC 与 USDC.e、跨链包装资产的价差	延迟、桥风险、两端库存、最终性	价差较大，但风险也更高
稳定币脱锚套利	2023 年 USDC 因 SVB 事件短暂跌破 1 美元	脱锚原因、赎回能力、流动性	机会少而集中，风险极高
永续合约—现货套利	牛市期间 BTC/ETH 永续合约资金费率过高	资金费率、基差、保证金、清算	相对成熟，仍有机结构化空间
清算套利	MakerDAO 2020 年 “Black Thursday” 清算拍卖	抵押品折价、竞价、Gas、资金准备	专业 keeper 主导，不属于纯无风险套利
MEV 后置套利	Flash Boys 2.0 研究中的 DEX 交易回跑	监听内存池、排序、私有交易、区块竞争	高度专业化，基础设施决定优势
预言机/价格延迟套利	bZx 等协议曾出现价格操纵事件	预言机更新、价格源、协议风控	更接近漏洞利用，风险和合规性极高
NFT/长尾资产套利	不同 NFT 市场之间的地板价差	真实流动性、版税、稀有度、退出能力	流动性差，机会高度非标准化

三、同链 DEX-DEX 套利

经典案例

DeFi Summer 期间，同一交易对在 Uniswap、SushiSwap、Curve 等 DEX 上经常出现价格偏差。套利机器人会在低价池买入，再在高价池卖出，使不同池子的价格重新趋于一致。

关键点

两个池子必须使用相同资产和相同计价单位进行比较。

不能只看池子的现货价格，要根据实际交易金额重新报价。

流动性越低，价格冲击通常越大。

Gas 可能直接吞掉利润。

最理想的方式是在同一笔交易内完成买卖，减少价格暴露。

需要防止抢跑、夹子攻击、报价过期和交易回滚。

当前状态

这是最经典、最容易理解的套利类型，但也是竞争最激烈的类型之一。

现在的竞争优势通常不在于“看见一个价差”，而在于：

更快的数据读取；

更快的交易构造；

更低的 Gas；

更好的交易排序；

更精准的滑点和失败概率估计；

私有交易通道和专业节点。

对于学习者，它适合用来建立报价、成本和净收益模型。

四、CEX-DEX 套利

经典案例

某个新代币先在 DEX 上交易，价格上涨后，中心化交易所尚未同步；或者代币刚在中心化交易所上线，形成短暂价格溢价。套利者可以在低价市场买入，再在高价市场卖出。

关键点

需要同时拥有 CEX 和链上两端的资金或库存。

充值、提现和网络确认可能导致机会失效。

CEX 的 API 限频、风控、宕机和账户冻结都属于执行风险。

DEX 价格可能因为低流动性而失真。

新币存在合约权限、黑名单、税费和无法卖出的风险。

当前状态

这类机会仍然可能出现在新币上线、市场剧烈波动、交易所流动性不足或充值提现异常时。

它通常需要较多库存和交易所账户管理能力，资本和运营要求高，不太适合作为第一个自动化项目。

五、三角套利与多跳套利

经典案例

在同一条链上执行：

USDC → ETH → USDT → USDC

如果三次交换后得到的 USDC 多于最初投入的 USDC，就形成三角套利。Curve 的 USDT、USDC、DAI 等稳定币池出现不平衡时，也可能出现类似的循环路径。

关键点

三条或更多路径必须整体计算，不能只看其中一跳。

每一跳都会产生手续费和滑点。

任意一跳失败，整个策略可能失败。

代币 decimals、汇率方向和单位转换容易出错。

需要评估中间资产的价格暴露。

当前状态

三角套利非常适合程序化，但公开可见的简单机会通常很快会被机器人消除。

它的优势是无需跨链、可以在一笔交易中完成，并且适合做路径搜索和算法练习。缺点是路径越长，成本和失败概率越高。

六、跨链套利

经典案例

同一种资产在不同链上的价格可能不一致，例如：

Ethereum 上的原生 USDC；

Arbitrum 上的原生 USDC；

某条链上的 USDC.e；

通过桥接得到的包装资产。

由于桥接流动性、用户需求和资金分布不同，它们可能短暂出现折价或溢价。

关键点

跨链通常不是原子交易，无法保证两端同时完成。

需要考虑桥接时间和价格变化。

需要在两条链上提前准备库存，否则跨链转移可能错过机会。

桥存在暂停、延迟、攻击和流动性不足风险。

资产名称相同不代表合约地址、兑换关系和赎回机制相同。

LI.FI 报价中的最终输出、桥费、DEX 费用、Gas 和预计时间都要拆开验证。

当前状态

跨链套利的名义价差通常比同链套利大，但这不代表净收益更高。

当前主要趋势包括：

跨链聚合器和 intent-based 桥降低了用户操作复杂度；

交易执行从用户手动选桥转向系统自动寻找路线；

资产碎片化仍然存在；

稳定策略通常需要两端库存、多个桥和故障切换能力；

跨链套利越来越像“库存管理 + 路由优化 + 风险定价”。

这是 LI.FI 共学项目中最值得重点研究的方向之一，但应先进行报价观测和纸面交易。

七、稳定币脱锚套利

经典案例

2023 年 3 月，受 Silicon Valley Bank 事件影响，USDC 一度跌破 1 美元。Curve 等市场出现严重不平衡，USDC 在部分市场以折价交易。

理论上可以低价买入 USDC，等待价格恢复，或者通过发行方赎回。

关键点

必须确认脱锚原因，而不是只看到价格低于 1 美元。

需要判断是否可以直接赎回或兑换。

脱锚期间流动性可能快速枯竭。

价格可能继续下跌，套利会变成方向性抄底。

稳定币可能存在冻结、黑名单、发行方和监管风险。

当前状态

稳定币套利不是持续型机会，更多是事件驱动型机会。它发生频率低、价差可能很大，但持续时间和最终结果都不确定。

因此不建议初学者直接使用真实资金参与脱锚套利，应先研究发行方、赎回机制、流动性和风险传播路径。

八、永续合约—现货套利

经典案例

当 BTC 或 ETH 永续合约资金费率长期为正时，可以采用：

买入现货

做空等价值永续合约

收取资金费率

如果现货和合约价格变化同步，理论上可以降低方向性风险。

关键点

资金费率不是固定收益，会随市场变化。

合约存在强平、保证金和交易所风险。

需要考虑借贷利率、交易手续费和资金占用。

基差可能扩大，导致短期浮亏。

不同交易所的资金费率计算方式不同。

当前状态

这是较成熟的市场中性策略之一，机构和专业交易团队参与较多。

它相比纯 DEX 套利更容易获得较长的持仓窗口，但并不是无风险策略。资金费率可能反转，极端行情下现货和合约价格也不一定同步。

九、清算套利

经典案例

MakerDAO 在 2020 年 “Black Thursday” 期间发生大规模清算。抵押品价格快速下跌，清算者可以通过竞价获得折价抵押品。

关键点

需要实时监听账户健康度和清算阈值。

需要准备充足资金和 Gas。

抵押品折价不等于最终利润，仍可能继续下跌。

清算拍卖有竞价、延迟和执行失败风险。

不同协议的清算机制完全不同。

当前状态

清算机会已经高度专业化，通常由 keeper、做市商和专门机器人处理。

它不属于严格意义上的“买低卖高无风险套利”，因为需要承担抵押品价格、竞价、协议和资金占用风险。

十、MEV 后置套利与交易排序套利

经典案例

一个大额 Uniswap 交易改变了池子价格，套利者在同一个区块中观察到价格变化，并在其后立即执行交易，把多个池子之间的新价差抹平。

Flash Boys 2.0 研究系统讨论了这类交易排序和 MEV 现象。

关键点

需要低延迟监听内存池、区块和交易结果。

需要预测交易执行后的池子状态。

交易排序比策略本身更重要。

公开内存池中的交易可能被抢跑。

私有交易、区块构建者和验证者关系会影响结果。

失败交易和竞价费用可能很高。

当前状态

MEV 套利已经明显基础设施化、专业化。竞争壁垒主要包括：

专用节点；

高性能状态模拟；

区块构建者连接；

私有订单流；

更好的 bundle 构造；

更低延迟的交易传播。

学习阶段可以先做事件回放和纸面 backrun，不建议直接进入公开主网竞争。

十一、预言机与价格延迟套利

经典案例

一些协议曾因为预言机更新延迟、价格来源单一或流动性过低，被攻击者操纵价格。早期 bZx 事件、后来的 Mango Markets 事件都说明了价格源和协议会计之间可能出现严重偏差。

关键点

这类行为可能已经超出正常套利范围。

操纵低流动性市场再利用协议价格，通常属于攻击或漏洞利用。

必须区分公开市场套利、预言机延迟和恶意操纵。

协议可能追责、回滚或冻结资产。

不应把漏洞利用当作学习套利的正常路径。

当前状态

正常的预言机延迟套利空间正在被多源预言机、TWAP、价格偏差保护、借贷上限和熔断机制逐步压缩。

这类方向更适合做安全研究和风控分析，不适合作为个人套利项目的主线。

十二、NFT 与长尾资产套利

经典案例

同一 NFT 系列在不同市场出现地板价差，或者某个稀有度较高的 NFT 被低估挂牌，套利者买入后在另一个市场出售。

关键点

地板价不等于真实成交价。

需要确认市场深度、稀有度、版税和转售需求。

NFT 的流动性通常远低于主流代币。

资产可能长时间无法退出。

合约、元数据和版权风险不可忽略。

当前状态

NFT 套利的机会高度非标准化，难以像同质化代币一样批量建模。它更依赖资产识别、市场经验和退出能力，不适合作为链上套利自动化的第一方向。

十三、闪电贷在套利中的位置

闪电贷是一种资金来源，不是独立的套利类型。

典型结构是：

借入资产

→ 在低价池买入

→ 在高价池卖出

→ 偿还贷款

→ 保留差额

它可以降低本金要求，但不能解决：

Gas ;

滑点 ;

交易排序 ;

交易失败 ;

流动性不足 ;

智能合约风险。

很多所谓“闪电贷套利案例”其实是价格操纵或协议攻击，不能简单当作正规套利案例学习。

十四、当前行业总体趋势

1. 简单价差越来越难做

公开市场上最明显的价差通常会被机器人迅速抹平。真正可持续的机会往往来自数据延迟、流动性碎片化、特殊事件或执行优势。

2. 竞争从“会算价格”转向“会执行”

速度、Gas、节点、交易排序、私有订单流、失败率和风险管理越来越重要。

3. 跨链机会更大，但风险更复杂

跨链价差背后往往是桥风险、库存风险、延迟风险和资产标准不统一，而不仅仅是价格差。

4. 稳定币和长尾资产提供偶发机会

这些机会可能带来较大价差，但通常伴随脱锚、流动性枯竭、合约权限和退出困难。

5. AI Agent 更适合做发现、整理和验证

Agent 可以负责：

采集报价；

去重信号；

计算成本；

监控路线；

生成提醒；

记录和复盘。

Agent 不应未经人工确认直接签名和执行高风险交易。

笔记 18

作者：x-tan-dev | 时间：2026-08-05 13:45 UTC | 字数：5763

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

从价差到系统工程：理解加密市场套利的路径、架构与完整分类

套利看起来很简单：在价格低的地方买入，在价格高的地方卖出，把中间价差变成利润。但真正运行过套利系统后会发现，发现价差往往是最容易的部分。决策策略能否长期存活的，是手续费、延迟、成交顺序、库存、资金效率、失败补偿和极端风险。

Odaily 的《从 0 到 1 做 CEX-DEX 套利 bot 的历程》(<https://www.odaily.news/zh-CN/post/5205066>)复盘了作者从 Avalanche 原子套利转向 CEX-DEX 价差交易的经历。文章主要讲的是非原子跨场所套利，并将其宽泛地称为统计套利。文中虽然使用波动率和几何布朗运动解释利润方差，但没有讨论期权波动率套利。

一、从微小价差开始

作者最初比较 Binance 与 Avalanche 上 TraderJoe 的 AVAX/USDT 价格。几分钱的差价在原子套利交易者看来几乎没有吸引力，但高频交易的核心不是单笔利润，而是交易量。

如果一次交易只赚 2 个基点，即 0.02%，每天完成 1000 万美元成交量，理论毛利润约为：

$10,000,000 \times 0.02\% = 2,000$ 美元

这说明 CEX-DEX 套利是一门基点生意：利润薄、周转快，对费用、延迟和失败率极其敏感。

二、最初的套利机器人

作者用几百行 Python 构建了一个最小可行系统，包含五个独立进程：

1. CEX 行情监听器：订阅 Binance bookTicker，接收最佳买卖价和对应数量。
2. DEX 行情监听器：读取区块日志和类 Uniswap V2 池的储备变化。
3. 机会发现器：比较两边的可执行价格，并扣除手续费、Gas 和预期滑点。
4. DEX 执行器：构造链上交易并发送给路由器。
5. CEX 执行器：在 Binance 执行方向相反的订单。

基本交易方向是：

DEX 价格低于 CEX：DEX 买入，CEX 卖出

DEX 价格高于 CEX：DEX 卖出，CEX 买入

这里不是先买入，再等待价差回归，而是尽量同时执行两腿。等待反向机会的主要是库存，而不是单笔交易本身。

完整收益应按可执行价格计算：

净收益 = 卖出收入 - 买入成本

- CEX 手续费 - DEX 手续费
- Gas - 滑点
- 借币、资金费率或基差成本
- 库存再平衡成本
- 执行失败的期望损失

不能使用最新成交价或中间价，因为真正可成交的数量取决于订单簿深度和池子的价格冲击。

三、为什么盈利很快转为亏损

原型第一天净赚约 200 美元，但竞争者发现他们后，系统很快转亏。主要存在四种失败路径：

CEX 报价过期，或者顶层流动性已经被别人吃掉；

DEX 上竞争交易抢先成交，把池价格推向不利方向；

Gas 竞价过高，执行成本超过价差；

CEX 腿已经成交，但 DEX 交易回滚。

最后一种最危险。DEX 回滚后，为了消除方向性敞口，系统必须在 CEX 反向成交，因此承担两次 CEX 手续费、额外价差和不利价格波动。目标收益只有 2 个基点时，一次失败可能抹掉约 20 次成功交易的利润。

这揭示了非原子套利的本质：它不是无风险锁定收益，而是用低失败率换取大量微小利润。

四、PGA 与分布式执行

为了提高 DEX 腿进入区块顶部的概率，作者加入了优先 Gas 拍卖，即 PGA：

提交初始 Gas 报价

- 监听内存池中的竞争交易
- 提高 Gas 并替换原交易
- 重复，直到区块确认

单机监听不够快，因为交易会在全网节点之间传播。系统因此升级为分布式节点网络，负责更早看到竞争交易、追踪替换交易，并尽快把新报价传播给区块提议者。

这也解释了为什么套利机器人的竞争力不只取决于公式。网络拓扑、节点位置、交易传播速度和区块构建关系同样重要。

五、订单规模与路由

最初的系统假设只吃 CEX 第一档流动性，并在 Uni V2 池成交。在这种简化条件下，最优订单规模可能存在解析解。

进入真实市场后，需要同时处理：

CEX 多档订单簿；
Uni V2、V3 和其他曲线；
多个 DEX 与聚合器；
多跳路径；
Gas 随路径变化；
不同代币精度和手续费；
每个场所的库存上限。

此时通常需要二分、三分、牛顿法或黄金分割搜索等数值方法。进一步扩展后，系统可以把币种和交易池建模为图，在大量候选路径中寻找扣除全部成本后的最优路线。

六、手续费就是竞争壁垒

作者在 JOE、STG 等尾部币种上能盈利，却无法进入 AVAX、BTC、ETH 等主流市场。一个重要原因是 CEX 手续费等级。

低费率账户不只是每笔交易成本更低，还可以更早出手。例如普通账户需要 30 个基点价差才能盈利，而专业账户可能在 20 个基点时就已经成交，后者自然会抢走全部机会。

因此，套利系统的优势通常来自组合能力：

低费率 + 低延迟 + 高成交率 + 低失败率
+ 高资金周转率 + 更广的交易覆盖

只优化程序速度，却没有手续费和资金规模优势，通常不足以进入主流市场。

七、库存与再平衡

非原子跨场所套利必须预先把资产部署到不同场所。假设 DEX 价格持续低于 CEX，系统会不断在 DEX 买入、CEX 卖出，最终导致 DEX 稳定币耗尽、CEX 代币耗尽。

系统有两个选择：

暂停交易，等待反方向机会自然恢复库存；
主动转账、提现、充值、兑换或跨链，再平衡两边资金。

第一种方式更安全，但会错过持续单边行情，而这类行情往往最赚钱。第二种方式提高资金周转率，却增加转账时间、提现限制、桥接风险和价格波动。

如果涉及永续合约或期货，还需要管理保证金、标记价格、强平风险和现货库存。永续合约可能降低资金需求，但不能像现货一样直接提取到链上，因此会使再平衡更加复杂。

八、这套系统最终长什么样

一个完整的 CEX-DEX 套利系统通常包括：

CEX/DEX 行情采集

- 行情标准化和时效判断
- 路径搜索与订单规模优化
- 费用、滑点和失败概率建模
- 风险与库存限制
- CEX、DEX 双腿执行
- PGA 和交易替换
- 成交、仓位与 PnL 账本
- 资金再平衡和对冲
- 异常监控、熔断与人工介入

真正复杂的部分最终可能不是交易，而是库存和风险管理。稳定币脱锚、交易所暂停提现、代币换合约、预言机异常、跨链桥攻击，都可能在几秒钟内抹去数周利润。

九、如何正确分类加密套利

“统计套利、原子套利、波动率套利”不能构成互斥且完整的三级分类。因为它们描述的是不同维度：

CEX、DEX、跨链描述交易场所；

原子或非原子描述执行保证；

价差、基差、资金费率和波动率描述利润来源；

做市、吃单、路由和清算描述执行方式。

同一个策略可以同时属于多个类别。例如，同链

DEX

三角套利既是路由套利，也是原子套利；跨链稳定币价差既是空间套利，也是非原子套利。

下面是一套更实用的分类。

十、跨场所或空间套利

空间套利利用同一资产在不同交易场所的价格差，例如 Binance 的 BTC/USDT 与 OKX 的 BTC/USDT。

实践中通常提前在两边配置 BTC 和 USDT，而不是买完以后等待区块确认并转币。交易完成后，一边 BTC 增加、另一边 BTC 减少，最终再统一调拨库存。

它适用于

CEX-CEX、CEX-DEX，以及部分

DEX-DEX

路径。主要风险是延迟、部分成交、手续费、提现限制、交易对手风险和库存失衡。

若比较的是

BTC

与

WBTC、BTC.b

等封装资产，还必须加入赎回、托管、桥接和脱锚风险，它们不能被视为完全相同的资产。

十一、原子链上套利

原子套利把借款、买入、卖出和还款放进同一个链上交易。任何一步失败，整笔交易回滚。以太坊交易 (<https://ethereum.org/developers/docs/transactions/>) 的状态变更具有原子性；同一个区块里的两笔独立交易则不天然具备这种保证。

常见例子是从

DEX

A

买入代币，再到

DEX

B

卖出，并在交易结束前偿还闪电贷。闪电贷是常用资金来源，但不是必要条件。

原子执行显著降低了两腿错配风险，却不能消除

Gas、MEV、合约漏洞、恶意代币、预言机和链重组风险。使用私有

Bundle

可以改善公开内存池中的抢跑和执行控制，但仍需理解

Builder

的打包规则。Flashbots

Bundle

文档

(<https://docs.flashbots.net/flashbots-auction/advanced/understanding-bundles>) 对此有具体说明。

十二、统计相对价值套利

严格意义上的统计套利不是同一资产跨交易所买低卖高，而是交易具有统计关系的不同资产。

例如，根据历史数据估计 ETH 与某个 Layer 2 代币之间的对冲比例。当组合价差偏离均值两个标准差时，做多相对便宜的一方、做空相对昂贵的一方，等待统计关系恢复。

这类策略需要协整、因子模型、状态空间模型或机器学习预测。真正的风险是关系发生结构性变化：历史上相关不代表未来仍然相关。项目基本面、代币解锁或监管变化都可能使所谓均值永久迁移。

十三、期现与基差套利

当到期期货价格高于现货时，可以买入现货、卖出期货，持有至交割，这通常称为正向现金套利。

粗略年化基差为：

年化基差 $\approx$ (期货价格 - 现货价格) / 现货价格 $\times$ 365 / 剩余天数

真实收益还必须扣除手续费、借贷成本、保证金成本和资金占用。到期期货通常没有周期性资金费率，核心是基差、交割规则和保证金。

反向套利需要做空现货、做多期货，通常受到借币可用量和借币利率限制。即使最终基差收敛，期间扩大也可能造成追加保证金或强平。

十四、永续资金费率套利

永续合约没有固定到期日，交易所通过资金费率使合约价格靠近指数价格。当平台定义的正资金费率由多头支付给空头时，常见做法是买入现货、做空永续，保持大致 Delta 中性并收取资金费。

但资金费率不是固定收益。它可能迅速下降、反转，甚至长期为负。还要考虑永续与现货基差、资金费率结算时间、标记价格、保证金和强平风险。不同平台的符号和结算规则可能不同，必须逐一核对。Bybit 的资金费率说明 (<https://www.bybit.com/en/help-center/article/Introduction-to-Funding-Rate>)展示了典型机制。

十五、波动率套利

波动率套利主要交易期权隐含波动率与未来实际波动率之间的差异，而不是简单预测价格涨跌。

常见做法是买入跨式期权并持续进行 Delta 对冲。如果实际波动足够大，Gamma 对冲收益可能覆盖 Theta 损耗和交易成本；反之，卖出波动率则赚取时间价值，但承担价格跳跃和尾部亏损。

BS 模型只是计算隐含波动率和 Greeks 的基准。实际系统还要建模波动率微笑、期限结构、Vega、Gamma、Theta、离散对冲、资金费率和跳跃风险。跨式期权的基本收益结构可参考期权行业委员会的 Long Straddle 说明 (<https://www.optionseducation.org/strategies/all-strategies/long-straddle>)。

十六、三角与路由套利

三角套利利用多个交易对之间的交叉汇率不一致，例如：

USDT → BTC → ETH → USDT

如果按照真实可成交汇率完成一圈后，最终 USDT 数量高于起始数量，并足以覆盖费用，就存在机会。

在 CEX 上，三笔订单通常不是原子的，存在部分成交风险；在同一条链上，可以通过智能合约把多次交换放入一个原子交易。多跳 DEX 路由本质上也是图搜索问题，需要比较直接路径和经中间币种转换的路径。

十七、跨链与封装资产套利

跨链套利利用不同链上同类资产的价格差。例如，在 Avalanche 买入 BTC.b，在另一条链或 CEX 卖出 BTC。

跨链桥通常不能进入普通链上交易的原子边界，因此这类策略主要依靠两边预置库存。桥接更多用于事后再平衡，而不是放在实时套利关键路径中。

风险包括桥攻击、链最终性差异、跨链延迟、限额、资产版本错误、赎回暂停和稳定币脱锚。它可能提供更高收益，原因恰恰是风险和工程复杂度更高。

十八、相邻但不应混同的策略

做市、清算和延迟交易经常与套利放在一起讨论，但并不完全相同。

做市通过双边报价赚取点差、返佣和激励，同时承担库存与逆向选择风险；清算策略通过偿还不健康仓位并折价获取抵押品赚取清算奖励；延迟交易则利用某个交易场所报价更新较慢，但承担报价突然修正和被专业做市商反向选择的风险。

这些策略都可能与套利系统共享行情、路由和执行基础设施，却不能因为“看起来接近无风险”就忽略其真实风险。

结语

评估任何套利机会，都应该依次回答五个问题：

1. 两个资产是否真的具有相同经济价值和可兑换性？
2. 价差使用的是可执行价格，还是仅仅是中间价？
3. 多条交易腿是否原子；如果不是，每一种失败状态如何处理？
4. 扣除手续费、Gas、资金、借币、滑点和再平衡后是否仍有利润？
5. 极端情况下会损失一笔利润、一天利润，还是整个本金？

套利的真正门槛不是发现价差，而是证明价差能够被可靠成交，并让系统在失败和异常事件中继续存活。

笔记 19

作者：Junfan Chen | 时间：2026-08-06 14:46 UTC | 字数：5258

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 · 自己实现 AMM 定价

日期 2026-08-06

沉淀 onchain/amm-pricing(第 2 个 skill)

验证区块 mainnet 25696531 / 25696561

状态 Tier 1 精确通过,Tier 2 已测量并解释

结论先行

今天真正的产出不是那个公式,是三个可迁移的判断:

1. 在 100 USDC 量级,滑点不是约束,手续费和 gas 才是。差 270 倍。
2. 按 toAmount 算出来的价差是乐观角读数。滑点容忍带宽可以宽过机会本身。
3. 验证要设计成二值判据。「接近吗」永远耗时间,「相等吗」不等就一定有具体原因。

1. 为什么全程用整数

EVM 没有浮点。Solidity 的 `/` 是整数除法,直接截断。链上所有代币数量都是整数——USDC 6 位精度,1 USDC 在合约里就是 1000000。

Python 的 `int` 是任意精度,`//` 也是截断除法,所以行为逐位一致。而 `float` 是 IEEE 754 双精度,约 16 位有效数字;本题中间量 `amount_in` `n` `reserve_out` 轻松到 10^{34} ,低位直接丢。

误差量级约 10^1 ,判据是 10^3 —— `float` 其实不会让验证失败,差 13 个数量级。

真正的理由是它改变了判据的性质:

` `	判据	后果
`float`	「够接近吗」	0.02% 误差算正常还是 bug?永远不知道
`int`	「相等吗」	不等 → 必有具体原因,可逐个排除

人民币换算(human ↔ base units)用 `Decimal`,不用 `float`。JSON 里大整数序列化成字符串——JS 端在 2^3 以上会静默丢精度。

2. 公式与推导

池子持有 $x = \text{reserve_in}, y = \text{reserve_out}$,守 $x \cdot y = k$ 。

手续费扣在输入侧,所以只有一部分真正上曲线:

$a_{\text{eff}} = \text{amount_in} \cdot (10000 - \text{fee_bps}) / 10000$

交换后不变量仍需成立:

$(x + a_{\text{eff}})(y - \text{amount_out}) = x \cdot y$
 $\text{amount_out} = y \cdot x \cdot y / (x + a_{\text{eff}}) = y \cdot a_{\text{eff}} / (x + a_{\text{eff}})$

代入 a_{eff} ,分子分母同乘 10000 消掉分数——这一步正是「只剩最后一个除法」的来源:

$\text{amount_out} = (\text{amount_in} \cdot n \cdot y) // (x \cdot 10000 + \text{amount_in} \cdot n)$

其中 $n = 10000 - \text{fee_bps}$

用 10000 做基数而不是 Uniswap 经典的 997/1000:同一个数(9970/10000),但 bps 基数能泛化到分叉。PancakeSwap 是 25 bps,不是 30。

3. 五种典型错法与 delta 特征

写成可执行代码存进 skill(--demo-failures),失败特征变成可复现的表,而不是撞一次记一次。

错法	delta 特征

提前做除法	整数输入下 $\Delta = 0$; 零头输入才暴露
用了 float	方向乱跳, 一个向量偏低下一个偏高
reserves 反了	99.999975%, 一眼可见
费扣在输出侧	0.03%(30bps 池)
完全没扣费	+0.30%(30bps 池)

两条反直觉的:

提前除法是潜伏的。 $1e18 \times 9970$ 恰好整除 10000, 所以在整数金额上它给出完全正确的答案。能通过测试向量的 bug 才是危险的那种——测试输入要选不是费率基数倍数的零头。

float 的误差不是有偏的, 是变号的。由此得到排查规则: Δ 方向一致 $\rightarrow$ 逻辑错; 方向乱跳 $\rightarrow$ 精度问题。

4. 两级验证的设计

级别	对标	通过标准
Tier 1	UniswapV2Router02.getAmountsOut	整数精确相等
Tier 2	LI.FI estimate.toAmount	gap 能被解释

Tier 1 是二值的: Router 就是链上对同一公式的实现, 任何 Δ 都是自己的 bug, 没有解释空间。

Tier 2 不是测试。LI.FI 会拆单、跨 DEX、绕 1inch/Ox。gap 就是路由价值本身, 记录并解释它, 不要调代码让它归零。

原计划写的是「与 LI.FI 对比, 误差千分之几内」——这个设计有缺陷: 对不上时无法区分公式错还是路由不同。改成两级后, 一级锁死正确性, 二级产出信息。

不变量(踩过才记得住):

所有读取 pin 同一个区块号。reserves 在区块 N、报价在区块 M 是幻影不匹配的头号来源。

token 顺序读出来, 不假设。getReserves() 按 token0/token1 返回, 那是地址字典序, 和你传参顺序无关。本例 USDC 0xA0b8... 排在 WETH 0xC02a... 前, 所以 inIsToken0 = false。

selector 现算 keccak(text=sig)[:4], 不抄。

5. 成本拆解 ★

原本一个 priceImpactPct 数字, 把两种性质完全不同的成本糊在了一起。拆开(滑点用同一函数传 fee_bps=0 算, 不引入第二套数学):

1 WETH 换 USDC, 池深 4647 WETH:

总成本 0.3214%

└─ 手续费 0.3000% 与规模无关

└─ 曲线滑点 0.0215% 随 amountIn/reserveIn 缩放

手续费 = 滑点 $\times$ 13.9

按真实资金 100 USDC $\approx$ 0.0525 WETH:

曲线滑点 0.0011% $\leftarrow$ 手续费的 1/270

这条改写后续工作重点。在这个量级, 任何为控滑点做的路由优化、拆单、深度筛选, 都是在优化一个小三个数量级的项。真正吃利润的是手续费和 gas——而 gas 是绝对值, 不随本金缩放, 在小资金上占比反而最狠。

滑点只有在下单量占池深有意义比例时才成为约束条件。

6. Tier 2 实测 ★★

Mainnet USDC/WETH, 1 WETH in, block 25696561, LI.FI 路由到 OKX:

 	USDC out	vs local
local V2 单池	1900.143506	—
toAmount	1904.364190	+22.2 bps

| toAmountMin | 1894.842369 | 27.9 bps |

(a) gap 大概率是费率档差,不是路由聪明。

V2 收 30 bps,V3 的 USDC/WETH 有 5 bps 档,差 25 bps,观测 22.2 bps,量级对得上。残差约 3 bps $\approx$ feeCollection(LI.FI 抽成)+ 两池价格偏离。

假说,未坐实——要确认得直接报价 V3 池子。

(b) 保底数字比朴素路由还差。 ← 今天最重要的发现

滑点容忍 0.5%,带宽 50 bps,是那 22 bps 优势的 2.25 倍。

按 toAmount 算出的边际,若实际成交在 toAmountMin,不是「赚少了」,是亏 27.9 bps。报价的不确定带比机会本身宽,足以整个吞掉它。

executionDuration: 0 —— 同链无桥,零时间敞口。跨链这一项不为零,那才是主线风险的来源。

7. 对后续计划的影响

Day 4 collector 必须采 toAmountMin,不能只采 toAmount。

只采后者的话,21

天测出来的所有跨链价差都是乐观角读数。而跨链的滑点容忍通常比同链更宽——多了桥那一段。

判据字段:band_over_edge > 1 即红灯,机会被不确定性淹没。

字段优先级按成本量级重排: gas(绝对值,小资金占比最高)> 手续费(固定比例)> 滑点(本量级可忽略)。

这个坑今天发现,比第 14 天发现便宜太多。

8. 排错记录

HTTP 403 / error code: 1010 —— Cloudflare 的「按浏览器签名封禁」。urllib 默认发 User-Agent: Python-urllib/3.x,直接被拒。web3.py 之类会发自己的 UA 所以没这问题。

关键信号:换端点错误码一模一样。两个独立厂商同一个 1010,共同因素不可能在它们那边。

换供应商不变的错误 = 客户端问题;只在某一家出现 = 供应商问题。

后面 collector 跨多个 RPC 会反复用到。另外 429 和 403/1010 是两回事:前者才是真限流该换端点,后者换多少个都一样。

RPC 选型: 公共端点只保留最近约 128 个区块的状态,传历史区块号会报 missing trie node。要 --block 回溯就需要 archive 节点(Alchemy 免费层含 archive,但不含 Debug/Trace API——案例 track 做交易解剖时可能撞上,届时用 logs + input data 解码绕过)。

9. 遗留问题

[] 验证 V3 5 bps 费率档假说(直接报价 V3 池)

[] 扫三个尺寸,找出 slippage_band_bps 不再压过边际的临界点 —— 那就是最小可行下单量

[] 多跳路径(getAmountsOut 已接受更长的 path)

[] V3 tick 数学:本 skill 明确不做,如需扩展是新 skill,不是改这个

过程记录

计划把 Day 2 定为「全程唯一的校准锚点」,理由是「能自己算出价格的人,才有资格判断价差是不是真的」。实际执行中公式部分由 AI 写出,自己完成的是环境、RPC、验证跑通与结果解读。

值得记的是:校准价值并没有全部丢失,但转移了位置。 Tier 1 能否通过取决于 RPC、block pin、token 顺序、decimals 这些本地变量,这部分仍然是自己踩通的;而「对不上时如何从 delta 反推病因」这段经验,是靠 --demo-failures 把失败特征做成可复现的表来替代的——比偶然撞上更系统,但少了亲手 debug 的肌肉记忆。

后续若再遇到「分水岭」标记的天数,值得先明确:这一天的不可替代部分具体是哪一段,再决定哪里可以外包。

笔记 20

作者：GeoGu0 | 时间：2026-08-05 14:45 UTC | 字数：4903

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、用我自己的话理解"套利"

一个东西，在同一个时间，在不同地方卖的价格不一样。在便宜的地方买，拿到贵的地方卖，扣掉中间所有的费用（路费、人工、时间成本）之后如果还有剩的，就叫套利。

我用菜市场鸡蛋的例子来举一反三。

周末早上去菜市场，发现东门菜摊的鸡蛋卖 5 块一斤，西门菜摊同样的鸡蛋卖 6 块一斤。脑子里的本能反应是：那我在东门买、去西门卖，不就白赚 1 块钱吗？这个想法本身，就已经摸到了套利的边。

把这个故事翻译成链上的语言：菜市场就是一条区块链（比如 Arbitrum），东门菜摊就是 Uniswap（一个去中心化交易所，简称 DEX），西门菜摊就是 Sushiswap（另一个 DEX）。鸡蛋就是 ETH，东门的 5 块和西门的 6 块就是两个 DEX 对 ETH 分别的报价。你从东门走到西门所花的时间和力气，就是链上交易要付的 Gas 费。手里拎鸡蛋的篮子，就是你的钱包。

所以链上套利的本质就是：在 Uniswap 上花 3000 块买 1 个 ETH，同一时间在 Sushiswap 上用 3010 块卖掉，赚 10 块差价。

但这个想法有三个漏洞，不堵上就赚不到钱。

第一个漏洞：两个摊的鸡蛋是不是同一种鸡蛋？如果东门卖的是土鸡蛋（本来就贵），西门卖的是普通鸡蛋，那 5 块对 6 块的对比就没有意义——不是同一个东西。链上同理：ETH 就是 ETH，但如果一边是 ETH、另一边是 stETH（质押版 ETH，虽然价格接近但并非完全等价），就不能直接拿来套利。套利的对象必须是同一个资产，或者确定能一比一互换的东西。

第二个漏洞：从东门走到西门花的时间和力气，会不会比 1 块钱更值钱？如果你走路要花一个小时，而你的时薪是 200 块，那你为了赚这 1 块钱差价付出的成本是 200 块——亏大了。链上的"走路成本"就是 Gas 费，每一笔交易都要付给区块链网络。如果价差是 10 块，两笔交易的 Gas 加起来是 15 块，那套完你反而亏了 5 块。屏幕上看到的价差只是线索，扣掉所有成本之后剩下的，才叫利润。

第三个漏洞：在你从东门走到西门的这段时间里，西门会不会也已经降价到 5 块了？等你气喘吁吁地走到西门，发现价格早就变了，你的套利机会消失了。链上更残酷——价格变动不是以分钟计，而是以秒甚至毫秒计。从你看到价差，到发起交易，到区块链确认，中间任何一个环节慢了，价差就可能已经被别人吃掉了。链上套利本质上是一场速度竞赛，先到先得。

所以套利的完整定义应该是：在同一个时间点，同一个资产在不同市场出现了价格差异，在便宜的市场买入、在贵的市场卖出，并且扣掉所有交易成本之后还能剩钱。三个关键词：同一时间（不是等涨价，是此刻就有差价）、同一资产（东西得一样）、扣完成本（屏幕上的价差不等于利润）。

我还顺便搞清楚了套利、交易和投资到底有什么区别。套利赚的是同一时刻不同地方的价差，风险在于执行够不够快，时间是秒到分钟这个尺度。交易赚的是不同时刻同一个地方的价格变化，赌的是价格往哪个方向走，时间是分钟到天。投资赚的是长期价值的增长，赌的是这个项目或资产未来行不行，时间是月到年。

一句话概括：套利是倒卖鸡蛋，交易是猜明天鸡蛋涨不涨价然后提前囤，投资是直接开养鸡场。本次共学关注的是第一种——追求确定性的价差，而不是炒币猜涨跌。

二、六种链上套利类型——我的初印象

以下六种类型，我现在的目标是"能用自己的话给别人讲出来"，不需要记住标准教科书定义。第二周会一个一个深挖。

第一种：DEX 内套利（同一个池子暂时失衡）

同一个 DEX 的同一个池子里，有人做了一笔大额交易，比如一口气卖了 500 个 ETH，导致池子里 ETH 变多、价格暂时偏低。你在价格回归正常之前反向操作，买回来赚差价。

什么叫"DEX 套利"？因为从头到尾你只在一个 DEX、一个池子里操作。最常见的形态是"三明治攻击"——攻击者监测到有人要买 ETH，抢在前面先买（前夹），受害者的交易进来推高了价格，攻击者立刻卖出（后夹）。

为什么会出现？因为 Uniswap 这类 DEX 用的是 AMM（自动做市商）机制，价格由池子里两种资产的比例通过数学公式决定。有人大量卖出 ETH，池子里 ETH 变多，公式自动把 ETH 价格调低。

最容易死在哪？需要比别人快太多。普通人在钱包里手动点、等区块确认，专业 MEV 搜索者用定制化机器人和高速节点抢跑。这不是普通人能做的事。

类比：菜市场只有一个菜摊，刚才有人买了一车白菜，摊主把白菜价格调高了。你比别人早知道这件事，在调价前先买白菜，等摊主调完价再卖回去。Day 8 会深挖。

第二种：DEX 间套利（同一个链上，不同店报价不同）

同一条链上（比如 Arbitrum），Uniswap 和 Sushiswap 对 ETH 的报价不一样。你在便宜的 DEX 买入，同时（同一个区块内）在贵的 DEX 卖出，锁定差价。

为什么会出现？每个 DEX 是独立的流动性池，Uniswap 不知道 Sushiswap 当下的价格，反之亦然。价格信息在它们之间传播需要时间，而套利者就是这个传播信息的人——同时也赚钱。

最容易死在哪？价差太小，扣完两笔交易的 Gas 费就变成了负数。比如你看到 1 ETH 差 2 块钱，但两笔交易 Gas 加起来要 3 块钱，做完亏 1 块。所以 DEX 间套利对价差有一个最低门槛要求。

类比：这就是菜市场东门西门鸡蛋故事的完美对应。同一个市场的不同摊位，同一个鸡蛋不同价。Day 9 会深挖。

第三种：跨链套利（不同城市的价格不一样）

Arbitrum 这条链上 ETH 卖 3000 块，Ethereum 主网上同一个 ETH 卖 3020 块。你在 Arbitrum 上买入，通过跨链桥把 ETH 转到主网上，在主网上卖出。

为什么会出现？不同区块链是完全独立的市场，各有各的流动性池和交易者。这条链上有人大量抛售 ETH 导致价格偏低，不等于那条链上也有人在抛。价差长期存在，直到有人做跨链套利把它抹平。

最容易死在哪？这可能是最难的一种套利。第一，跨链桥不是免费的，桥费直接吃掉价差。第二，跨链需要时间——源链确认加上桥中继加上目标链确认，整个过程从几十秒到几分钟不等。在这段时间里，目标链上 ETH 的价格可能已经变了，你买到了便宜货但运过去的时候那边也不贵了。第三，买和卖发生在不同链上，无法原子执行——你不像 DEX 间套利那样可以"要么全做完要么全不做"，你买完 ETH 之后就被市场暴露了。

类比：北京的白菜比天津便宜 2 毛钱，你在北京买了运到天津卖。问题是运费多少？路上会不会堵车？等你到了天津，白菜还值那个价吗？Day 10 会深挖。

第四种：三角套利（绕一圈回到原点，发现钱变多了）

假设你手里有 USDC（一种稳定币，1 USDC 约等于 1 美元）。你发现三条路径的汇率不太对：在 Uniswap 上 1000 USDC 能换 0.5 ETH，在 Sushiswap 上 0.5 ETH 能换 200 LINK，在 Balancer 上 200 LINK 能换 1005 USDC。你从 1000 USDC 出发，经过 ETH 和 LINK 绕了一圈，回到 USDC 时手里有 1005 个。多出来的 5 USDC 就是你的套利利润。

为什么会出现？三个不同交易对的汇率没有同步更新，出现了暂时的"汇率三角不一致"。理论上在这个三角被抹平之前，套利者可以反复走这个路径赚钱。

最容易死在哪？首先，三个交易必须在一个原子交易里全部完成（通常借助 flash loan，即闪电贷：在一个交易里借钱、做事、还钱，一步完成）。如果分三次手动交易，每次之间价格都会变，你做的第一次可能就已经破坏了后两次的套利条件。其次，路径搜索本身是一个计算难题——如果有 N 个资产和 K 个 DEX，可能的三角、四角路径数量是天文数字，需要高效的路径搜索算法。

类比：不太好用于日常生活打比方。可以理解为发现了一个"不可能三角形"——三条边的汇率应该满足某种等式但实际不满足，这种不一致本身就是套利机会，直到被人抓住并修复。Day 11 会深挖。

第五种：稳定币套利（一块钱就应该值一块钱）

USDC 和 USDT 都是稳定币，理论上都应该严格等于 1 美元。但实际上偶尔会出现微小偏差：比如某个时刻 USDC 只值 0.998 美元，而 USDT 值 1.001 美元。你用打折的 USDC 换成 USDT，再以溢价卖出，每个币赚大约 0.3 美分。

为什么会出现？市场恐慌的时候，人们会争先恐后地把某种稳定币换成另一种（比如 2023 年硅谷银行倒闭时，USDC

因为部分储备金存在硅谷银行而遭到抛售）。大额赎回也会短暂压低某个稳定币的价格。Curve 这类稳定币池里币种比例失衡也会导致价格偏离。

最容易死在哪？你不知道脱锚是暂时的还是永久的。如果 USDC 真的出了问题、要彻底脱锚了（就像 2022 年 UST/LUNA 崩盘），你以为是“套利”就变成了“接飞刀”——0.998 买进去，可能跌到 0.5 甚至归零。另外，稳定币价差通常非常非常小（低于 0.5%），需要很大资金量才能赚到有意义的绝对收益。比如 0.3% 的价差，你用 1000 美元做，毛利才 3 块，可能不够付 Gas。

类比：两个超市的购物卡标称都值 100 块，有一家急用钱打 9.9 折，另一家按面值回收。你买了折扣卡去回收那家变现。但如果发卡公司要倒闭了，你手上的卡可能变废纸。Day 12 会深挖。

第六种：清算（帮平台收房子，平台给你辛苦费）

小明在 Aave（一个去中心化借贷平台）存了 1 个 ETH（当时价值 3000 美元）作为抵押，借了 2000 USDC 出来花。抵押率是 $3000 \div 2000 = 150\%$ 。Aave 的安全规则是：抵押率一旦跌破 120%，任何人都可以来清算他。

突然 ETH 价格从 3000 跌到 2300，小明的抵押率变成了 $2300 \div 2000 = 115\%$ ，低于 120% 的清算线，触发了清算条件。这时候清算人（可以是任意一个人或机器人）可以替小明还掉一部分借款（比如 1000 USDC），从小明的抵押品里拿走价值 1050 美元的 ETH（Aave 给清算人 5% 的折扣奖励）。清算人拿到 ETH 后立刻在市场上卖掉，差价 50 美元落袋。

为什么属于套利？因为 5% 的清算奖励是协议规则写死的，是确定性的。你不需要预测 ETH 价格涨跌——你只需要判断“这个仓位是不是真的触发了清算条件”以及“我能不能比别人快”。

最容易死在哪？两个字：抢不过。清算赛道极度内卷，有专业的 MEV 搜索者部署了高性能清算机器人，7×24 小时监控 Aave、Compound 等借贷协议的可清算仓位。一旦出现清算机会，它们会在同一个区块内发起交易，而且愿意支付非常高的 Gas 来抢位置。普通人用钱包手动操作，几乎没有任何胜算。另外清算还面临 Gas 竞价的困局：如果两三个人同时竞拍同一笔清算，赢家只有一个，输家白白烧了 Gas。

类比：银行的规矩是谁帮收不回来的房贷找到了下家，谁就能拿 5% 的中介费。道理很简单，但全城最精明的中介都在做这件事，你一个路人很难抢到。Day 13 会深挖。

六种类型走完一遍后的感受

最直观的是 DEX 间套利，菜市场东门西门的类比太贴切了，脑子里有画面。

其次是跨链套利，北京白菜运天津也很好理解，但明显感觉这是坑最多的一种。

最让我困惑的是三角套利——怎么在实际几百个币和几十个 DEX 中找到那条“走完一圈能赚钱”的路径？感觉需要很好的数学和编程功底。

稳定币套利和清算给我的共同感受是：道理很直白，但“知道”和“能做到”之间隔了一个银河系。

笔记 21

作者：youchen-design | 时间：2026-08-06 13:16 UTC | 字数：4488

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天的整理

一、研究對象

以代幣化股票 (bStocks) 計價、用交易稅發放股票代幣「分紅」的 BNB 鏈 meme。

計價幣：SPYB / SPCXB / NVDAB / TSLAB，由 BTech Holdings 發行，Nest Trading 清算，託管方 1:1 持有真股
發行平台：Flap、Four.meme（Four.meme 於 7/25 開始支援股票代幣計價）
樣本：币有、MARSCOIN、ASTEROID、MEMESTOCK、stonks、GME、安心、Z、1coin
新的地方在於：計價幣是一個會被 meme 自身流量推動的 RWA，而稅金又回頭發這個 RWA。

二、已驗證的事實

A. 8/3 币有事件 (SPYB)

CZ 發「币安全都有」後，以 SPYB 為底池的 meme「币有」暴拉，連帶把 SPYB 本身推飛。

 	數值
SPYB 全流通	4,357 顆 / \$3.27M
SPYB/币有 池 (Pancake V2)	24h \$10.7M vs 儲備 \$397.4K → 27× 週轉
SPYB/USDT 0.01% (Pancake V3)	24h \$19.5M vs 儲備 \$454.5K → 43× 週轉
公允價	~\$751
殘留錯價	淺池仍掛 \$892-\$1,105；三個池 24h 19.4% / 21.2% / 27.0%
CoinGecko 聚合	+0.4%，區間 \$746-752 —— 事件完全被抹平

關鍵推論：事件發生在 09:16 UTC，美股 13:30 UTC 才開盤 → 申贖套利腿是關的，回錨靠 CEX-DEX（幣安現貨 SPYB，maker 免費到 8/31）。深池被按回去，\$500-1K 儲備的淺池沒人管。

B. MARSCOIN 鏈上實測（你的交易）

0xfb91eb4f...，block 113,507,571 ≈ 7/30（上線第二天）

4.1 USDT —— Uniswap V3 · MARS/USDT · 1% (0xee6c8b8...，池深 \$12.4K) —— 105.116 MARS
成交價 \$0.03901 SPCXB 全程未被觸碰 零稅

對照交易 0xb3bfdfb3...（走 Pancake V2 的 SPCXB/MARS 池）：

17.6676 MARS
├— 0.0883 → 0x52866fd1... 平台費 0.5%
└— 17.5793
├— 0.5274 → MARSCOIN 合約 稅 3.0% ← 分紅來源
└— 17.0519 → 池子 96.52%

MARSCOIN 合約自己持有 192,467 MARS (~\$10.4K) 等著換 SPCXB。

各池成交量：MARS/USDT 0.25% \$5.33M + 1% \$3.43M + Uni 1% \$3.43M ≈ \$8.8M，對比 SPCXB/MARS 池 \$2.50M。

三、被推翻的判斷（你重新思考時最該重看的部分）

我原本說	實際
MARS 只有 SPCXB 池，沒有獨立美元價格發現，β=1	錯。USDT 的 V3 池才是主定價場
用 U 買 → 雙腿推 SPCXB → 放大係數生效	這筆沒有。一跳直達，φ 很低
每筆買賣都抽 3% 灌 vault	V3 的 USDT 池免稅（V3 不支援 fee-on-transfer，必須白名單）
今天是 8/3，財報解禁在後	今天是 8/6，財報 8/4 已過、解禁就是今天

四、理論框架（目前的狀態）

放大係數

$$\lambda = 1 + \phi \cdot (D_{\text{meme}} / D_{\text{quote}})$$

φ = 買盤中「用 U 出資」的比例（可從 tx 是否同時觸及兩個池判定）

币有 案例 ≈ 1.87×；MARSCOIN 因 φ 低而幾乎不適用。

五條傳導路徑

1. 計價幣恆等式： $r_{\text{meme}}/\text{USD} = r_{\text{meme}}/\text{quote} + r_{\text{quote}}/\text{USD}$
2. 訂單流繞送 (USDT→quote→meme)，被套利封頂，但封頂在浮動盤小+標的市場關閉時會被打穿
3. 稅金 vault 抽乾池子的 quote 側
4. 反身性 (讓 β 從 1 變成 >1 ，且下跌不對稱)
5. 分紅賣壓外部化：收到分紅的人賣的是 quote，不是 meme → meme 在系統性砸自己的計價幣

套利提取量 ($V2$, r = 溢價倍數, y = 池 USDT 側儲備)

最適賣出 $\Delta x = x(\sqrt{r} - 1)$ 單次利潤 $\approx y(\sqrt{r} - 1)^2 / r$

$r=1.4$, $y=\$200K \rightarrow$ 約 $\$4.7K/\text{次}$ 。錢在補充速率，不在單次價差。

稅基覆蓋率 (今天新出來的核心指標)

稅基覆蓋率 = 課稅池成交量 / 總成交量

聚合器永遠選最便宜路徑 = 免稅路徑 → 預測單調下降 → 分紅內建衰減，且與成交量無關。

浮動盤 / 套牢盤比例規則

SPYB $\$3.27M$ 推得動；SPCXB $\$91M$ (28 倍) 推不動 → 不能外推

死幣當 quote：關鍵比值是 套牢盤 / 池深，SPYB ≈ 0 ，死幣 $\approx$ 天文數字 → 推上去沒有底

quote 類型橫斷面 (實驗設計)

| quote | 錨 | 套牢盤 | 預期形狀 |

| :--- | :- | :- | :----- |

| USDT | 完美 | 無 | $\lambda=1$ ，無外溢 |

| 原生幣 | 深 | 中 | 外溢小 |

| 薄 RWA | 硬 | ≈ 0 | 溢價 → 均值回歸 (已觀測) |

| 死幣 | 無 | 巨大 | 推高 → 崩塌，不回歸 |

五、你的部位 (結案)

持有 105.116 MARS，門檻 10,000 → 在門檻的 1.05%

沒拿到分紅純粹是部位太小，與路徑、用 U 買、稅都無關

過門檻現價約需 $\$542$

$\$0.039$ 買、現價 $\$0.0542 \rightarrow +39\%$

六、未解的問題

1. 分紅觸發機制：閾值 / 定時 / 滴灌 / 純 claim? → 最快檢驗是抓 vault 的 Transfer 時間戳看間隔分布，不用讀合約
2. 稅金何時、如何從 MARS 換成 SPCXB (合約裡那 192,467 MARS 的去向)
3. 真實股票公允價端點 ← 你 rwa-arb-monitor 的老問題，這裡會再撞一次，而且更難 (關鍵事件都在休市時段)
4. 你能不能完成 CEX 那一腿 (bStocks 有地區限制) → 決定這是研究專案還是交易專案
5. SPCXB 池是否也有免稅路徑

七、候選研究題目 (我當時的排序)

1. 共享計價幣的幽靈相關性 —— 八個 meme 共用一個 $\$3.27M$ 浮動盤，USD 報酬相關性 vs quote 計價報酬相關性。「一籃子股票 meme 不是分散投資」
2. 宣稱分紅 vs 實際分紅 —— 稅金去向審計、10K 門檻的累退性、被排除地址吃掉多少
3. 稅基覆蓋率的時間序列 —— 今天新增，可能是最尖銳的一條
4. 放大係數的橫斷面驗證 (當方法論鋪墊)
5. 分紅發放的事件研究 + 收款地址賣出風險 (外生時點的窗落財富衝擊，本身可獨立發表)

八、工具備忘

bsc-dataseed.binance.org : eth_getLogs 靜默回空陣列不報錯 (會騙人)，但 receipt 查詢正常

bsc-rpc.publicnode.com : getLogs 正常，但擋 archive receipt

→ collector 兩個都要備

GeckoTerminal /api/v2/networks/bsc/tokens/{addr}/pools 可一次撈全部池的量與儲備

X 讀不到時用 `api.fxtwitter.com/{user}/status/{id}`

關鍵地址：MARSCOIN `0xfe189e...7777` | SPCXB `0xbe9d15...03e1` | SPCXB/MARS 池 `0x94f3ed36...` | 你的 tx `0xfb91eb4f...` | 對照 tx `0xb3bfdfb3...`

笔记 22

作者：A9xYau | 时间：2026-08-05 14:52 UTC | 字数：4345

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

问题：DEX 内、DEX 间、跨链、三角、稳定币、清算与 MEV
等常见套利机会结构分别是什么？它们之间有什么区别？

回答

在学习区块链套利之前，需要先理解一个最重要的概念：

套利 (Arbitrage) 的本质，就是利用同一种资产在不同市场、不同路径、不同时间或不同交易方式之间出现的价格差来获取利润。

例如：

ETH 在 A 市场价格为 3000 USDT；

ETH 在 B 市场价格为 3010 USDT；

那么理论上可以买入 A、卖出 B，赚取 10 USDT 的价差。

不过实际套利需要扣除：

Gas (链上手续费)

交易手续费

滑点 (Slippage)

网络延迟

失败交易成本

资金成本

真正需要计算的是：

净利润 = 价差收益 - Gas - 手续费 - 滑点 - 失败成本

只有净利润大于 0，套利机会才真正成立。

问题一：什么是 DEX 内套利 (Intra-DEX Arbitrage)？

回答

DEX 内套利，是指同一个去中心化交易所 (DEX) 内部，不同流动性池 (Liquidity Pool) 之间产生价格差，从而形成套利机会。

例如，在 Uniswap V3 中，同一个 ETH/USDC 交易对可能存在多个手续费等级 (Fee Tier) 的流动性池：

0.01%

0.05%

0.30%

1.00%

由于每个池子的流动性、交易量和更新速度不同，同一时刻可能出现：

Pool A：ETH = 3000 USDC

Pool B：ETH = 3010 USDC

套利机器人就可以：

1. 在价格较低的 Pool A 买入 ETH；
2. 在价格较高的 Pool B 卖出 ETH；
3. 获得中间价差利润。

为什么会出现这种机会？

因为 AMM (Automated Market Maker, 自动做市商) 不会主动同步不同池子的价格，只有当交易发生时，池子的储备 (Reserve) 才会变化，从而改变价格。因此，不同池子的价格可能会短时间偏离。

问题二：什么是 DEX 间套利 (Cross-DEX Arbitrage) ？

回答

DEX 间套利，是指不同去中心化交易所之间，同一种资产价格出现差异，从而产生套利机会。

例如：

Uniswap : ETH = 3000 USDC

SushiSwap : ETH = 3010 USDC

套利流程：

1. 在 Uniswap 买入 ETH；
2. 在 SushiSwap 卖出 ETH；
3. 获得价差利润。

为什么会出现？

因为每个 DEX 都是独立运行的 AMM，彼此之间不会主动同步价格。

套利机器人不断地进行买低卖高，使多个的价格重新趋于一致。因此，从某种意义上来说，套利者也是市场价格的维护者。

DEX

问题三：什么是三角套利 (Triangular Arbitrage) ？

回答

三角套利，是指三种资产之间的汇率关系出现不一致，从而形成闭环套利机会。

例如：

ETH → USDC → BTC → ETH

理论上：

如果从 1 ETH 出发，绕一圈之后仍然应该回到 1 ETH。

但是由于三个交易池更新速度不同，可能出现：

1 ETH

↓

3000 USDC

↓

0.05 BTC

↓

1.01 ETH

那么：

最终得到 1.01 ETH。

利润就是：

0.01 ETH。

整个套利过程通常发生在同一个 DEX 内部，不需要跨交易所。

为什么会出现？

因为三个流动性池价格更新不同步，导致数学上的汇率关系暂时失衡。

问题四：什么是稳定币套利（Stablecoin Arbitrage）？

回答

稳定币套利，是指不同稳定币之间价格短时间偏离 1 美元时产生的套利机会。

常见稳定币包括：

USDT

USDC

DAI

FDUSD

理论上：

它们都应该等于 1 美元。

但实际市场可能出现：

USDT = 1.003 美元

USDC = 0.998 美元

套利流程：

1. 买入价格较低的 USDC；
2. 换成价格较高的 USDT；
3. 获得中间价差。

为什么会偏离？

例如：

市场恐慌时，大量资金抢购 USDT，导致 USDT 短时间溢价，而其他稳定币价格变化较小，从而形成套利空间。

Curve 等稳定币交易协议中，经常会出现这种机会。

问题五：什么是跨链套利（Cross-chain Arbitrage）？

回答

跨链套利，是指同一种资产在不同区块链网络之间价格不同，从而形成套利机会。

例如：

ETH 同时存在于：

Ethereum

Arbitrum

Optimism

Base

BNB Chain (BSC)

理论价格应该一致。

但实际上可能出现：

Ethereum：3000

Base：3004

Arbitrum：2998

原因是：

跨链桥（Bridge）并不是实时完成资产转移，而可能需要几分钟甚至几十分钟，因此价格不能完全同步。

为什么真正做跨链套利的人通常不会临时跨链？

因为跨链耗时较长，等资产到达另一条链时，套利机会可能已经消失。

专业机构通常采用：

Inventory Rebalancing（库存再平衡）

即：

提前在多个链上准备好资金。

例如：

Ethereum：

100 ETH

Base：

100 ETH

发现机会后，两边同时交易。

之后再慢慢跨链补库存，而不是等待跨链完成后再交易。

问题六：什么是清算套利（Liquidation）？

回答

清算套利，是 DeFi 中利润最高、竞争也最激烈的套利方式之一。

例如：

用户在 Aave：

抵押：

100 ETH

借出：

USDC。

随着 ETH 下跌，

用户的 Health Factor（健康因子）低于 1。

此时协议允许任何人帮助协议清算该用户。

套利机器人：

替用户偿还：

10000 USDC。

协议奖励机器人：

价值约 10500 USDC 的 ETH。

利润：

500 USDC。

为什么这么难？

因为全球大量机器人都在实时监控借贷协议。

只要健康因子低于 1，

几十毫秒内就可能有大量机器人同时提交清算交易。

最终通常由：

Gas 更高、

交易更快、

延迟更低的机器人成功完成清算。

问题七：什么是 MEV (Maximal Extractable Value) ？

回答

MEV (Maximal Extractable Value，最大可提取价值) 是目前区块链生态中最核心的概念之一。

它表示：

区块生产者、验证者或交易排序者，可以通过调整交易顺序而获得额外收益。

套利只是 MEV 的一种。

MEV 主要包括：

1. Arbitrage (套利)

利用不同市场价格差获利。

2. Sandwich Attack (三明治攻击)

流程：

机器人先买入 →

用户大额买入推高价格 →

机器人立即卖出。

机器人赚取用户产生的滑点。

3. Backrun (尾随套利)

用户交易完成后，

机器人立即执行套利交易，

利用用户交易导致的价格变化赚钱。

4. Liquidation (清算)

抢夺借贷协议中的清算奖励。

5. Time-bandit Attack (时间盗贼攻击)

通过区块重组 (Reorg) 重新获取历史区块中的 MEV 收益。

现实中较少见。

问题八：什么是 Flash Loan (闪电贷) 套利？

回答

Flash Loan (闪电贷) 允许用户：

在同一笔交易中借款、套利、还款。

整个过程必须发生在同一笔链上交易内。

例如：

借：

1000 万 USDC

↓

执行套利

↓

赚：

10 万

↓

归还：

1000 万

↓

剩余利润：

10 万。

如果套利失败，

整笔交易自动回滚。

借款人无需承担本金损失。

因此：

Flash Loan 成为了很多大型套利策略的重要工具。

问题九：这些套利机会之间有什么关系？

回答

可以把所有套利机会理解成一个逐层递进的体系：

1. 最基础的是 DEX 内套利和 DEX 间套利，它们主要解决不同流动性池或不同交易所之间的价格偏离。
2. 三角套利是在同一个交易系统中，通过多种资产之间的汇率失衡寻找闭环套利机会。
3. 稳定币套利属于特殊类型的价格套利，主要发生在多个稳定币之间。
4. 跨链套利进一步扩展到不同区块链网络之间，由于跨链桥延迟和流动性分散产生价格差。
5. 清算套利并不是利用价格差，而是利用借贷协议给予清算人的奖励机制获利。
6. MEV 则站在更高层次，它不是某一种具体套利策略，而是所有利用交易排序获取额外收益的方法总称。其中套利、清算、Sandwich、Backrun 等都属于 MEV 的具体实现形式。

因此，可以简单理解为：

DEX 内套利 → DEX 间套利 → 三角套利 → 稳定币套利 → 跨链套利 → 清算套利 → MEV（包含上述多种机会）。

问题十：学习这些套利机会，建议按照什么顺序？

回答

建议按照以下顺序逐步学习：

1. AMM（自动做市商）原理与 DEX 定价机制（ $x \times y = k$ 、Uniswap V2/V3）。
2. DEX 内套利和 DEX 间套利。
3. 三角套利与图搜索算法。
4. 稳定币套利及 Curve 等稳定币协议。
5. 跨链套利、跨链桥、库存管理（Inventory Management）。
6. DeFi 借贷协议（Aave、Compound、Morpho）及清算机制。
7. MEV，包括 Mempool、Flashbots、Builder、Relay、Bundle、交易排序等高级内容。

按照这个顺序学习，可以从理解套利原理逐步过渡到开发真正可运行的链上套利系统。

笔记 23

作者：wyy2221 | 时间：2026-08-05 02:34 UTC | 字数：3930

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

残酷公学两日打卡笔记

跨所套利实盘系统排障与思考

这两天我主要在处理 Hyperliquid 与 Lighter 跨所套利程序的实盘问题。相比继续增加策略功能，我把更多精力放在交易执行、异常恢复和系统稳定性上。最大的体会是：实盘程序最难的不是发现价差，而是在行情变化、网络延迟

和交易所拒单的情况下，仍然能够保证仓位安全，并自动完成一笔交易的收口。

一、遇到的第一个困难：收益达标却没有平仓

程序曾多次显示平仓收益已经超过设定的 7.5 bps，最高一度达到 11.31 bps，但订单仍未执行。

最初看起来像是程序错过了平仓机会。继续检查后发现，程序不仅要求收益达到目标，还要求收益连续保持 1000 ms。如果中间某一轮行情不满足条件，持续时间就会重新计算。

进一步排查后，我发现真正的问题不只是持续时间，而是几个条件之间存在冲突：

收益需要连续保持1000 ms

盘口年龄必须低于250 ms

Hyperliquid被动盘口更新较慢

平仓前没有主动获取新盘口

这样就可能出现：收益刚刚连续满足条件，但盘口已经过期；或者行情短暂不新鲜，系统直接从 ACTIVE 降为 OBSERVE_ONLY，导致平仓中止。

我的解决思路是，不放宽收益门槛和盘口门槛，而是在候选成熟后，主动查询一次 Hyperliquid 权威盘口，再用新盘口重新计算收益。只有刷新后仍然满足条件，才创建交易事务并发送订单。

同时，我认识到：

ACTIVE 代表程序是否获得实盘授权，盘口新鲜度只代表当前这一轮能不能交易，两者不应该混在一起。

短暂行情陈旧应该跳过本轮，而不是永久停止整个策略。真正需要停止策略的情况，应该是仓位错配、订单状态未知、真实仓位无法读取或者触发全局亏损限制。

二、主动确认盘口后的发现

程序增加候选触发式主动确认后，累计进行了44次主动查询，全部成功。

其中一笔记录显示，原来的 Hyperliquid 盘口已经超过3秒，但主动确认后，进入交易门槛时的盘口年龄只有114毫秒，说明这套方法能够有效避免使用旧盘口发单。

不过，44次主动确认中，刷新后只有12次候选继续成立，另外32次因为真实净收益不足而取消。

这说明很多看起来有利润的机会，其实是旧行情造成的。主动确认虽然增加了一次网络请求，但避免了大量无效订单。

这给我的启发是：

交易效率不能只看下单次数。能够及时取消错误候选，同样是在提高效率。

我也没有因此增加常驻行情轮询。Hyperliquid 被动盘口平均约5秒更新一次，而 Lighter 每秒可能更新十几次，但持续每200毫秒查询 Hyperliquid 会大量消耗接口额度，并增加限频和网络异常。

所以目前最简单的结构仍然是：

被动WebSocket发现候选

→ 候选成熟后主动确认一次

→ 刷新后重新计算

→ 通过门槛才发单

这种方案虽然不能让候选发现立即变快，但能保证真正下单时使用的是较新的行情。

三、遇到的第二个困难：Hyperliquid拒单造成单腿仓位

在一次真实平仓中，两腿订单同时提交：

Lighter：成功成交

Hyperliquid：IOC零成交拒绝

Hyperliquid 返回的错误是：

Order could not immediately match against any resting orders

经过分析，这不是权限、精度、保证金、reduce_only 或 Oracle 问题，而是订单到达交易所时，限价范围内已经没有可以立即成交的买单。

本次从盘口冻结到程序调用 Hyperliquid SDK 只有约21毫秒，说明程序内部处理速度已经足够快。但订单提交到交易所返回结果接近0.8秒，在这段时间内，原来的买单可能被撤销、被其他订单成交，或者盘口已经下移。

这让我认识到，即使提交前盘口非常新，也不能保证 IOC 一定成交。

IOC订单要求立即成交，否则马上取消。现有的8 bps滑点余量只能允许价格在一定范围内变化，不能保证流动性一直存在。

因此，系统不能把目标设为“永远不出现拒单”，而应该设为：

拒单可以发生，但程序必须知道订单确实零成交，并立即处理已经成交的另一腿。

四、恢复流程为什么没有启动

这次更严重的问题不是 Hyperliquid 拒单本身，而是拒单后程序卡在了 SUBMITTING 状态。

Hyperliquid 已经同步返回：

REJECTED

filled=0

无venue_order_id

包含明确错误

但程序仍然坚持使用 client order ID，再查询一次这个订单的权威状态。由于查询持续失败，事务一直无法进入恢复阶段。

结果是：

Lighter仓位已经平掉

Hyperliquid仍保留单边多仓

恢复期限没有开始

自动恢复订单没有发送

活动事务无法结束

程序状态文件停止更新

这次问题让我明白，必须区分两种情况：

第一种是交易所明确返回拒绝并确认零成交，这已经是确定性终态，可以直接进入恢复。

第二种是请求超时、连接中断或者没有拿到明确响应。这种情况不能假设零成交，必须继续查询订单、成交和真实仓位。

因此，我准备把 Hyperliquid 的明确 IOC 零成交拒绝标准化为：

EXCHANGE_IOC_NO_MATCH

当同步响应已经明确拒绝、成交量为零时，不再让订单回查阻塞整个恢复流程。

五、设计简单的单腿恢复过程

针对单腿问题，我没有增加新的服务、线程、数据库表或复杂状态机，而是准备复用现有的活动事务和恢复逻辑。

新的思路是：

一腿成交、一腿明确零成交

→ 立即进入恢复状态

→ 固定120秒恢复期限

→ 禁止新的开仓

→ 继续允许平仓和风险处理

在120秒内，程序使用已经发生的真实成交价、手续费和剩余仓位，重新计算：

如果现在完成剩余一腿，
整笔交易最终还能获得多少净收益

如果预计收益重新达到 7.5 bps，就使用现有20 bps恢复余量完成剩余腿。

如果120秒内始终无法恢复目标收益，就放弃本笔交易的利润要求，使用50 bps紧急余量优先平掉单边仓位。

这里的核心思路是：

前120秒尽量保留收益

120秒后优先消除单边风险

但即使使用50 bps IOC，也不能假设一定成交。紧急订单发送后仍然需要重新读取真实仓位。如果仓位没有归零，只能根据新盘口进行少量、有边界的再次尝试。

六、恢复结束后不应该永久停止

我还明确了恢复结束后的运行逻辑。

一次普通 IOC 拒绝、一次单腿恢复，甚至一笔小额亏损，都不应该导致整个策略永久停止。

正确过程应该是：

发生单腿

- 暂停新开仓
- 完成恢复或紧急平仓
- 确认双所仓位归零
- 确认没有相关挂单
- 所有订单进入终态
- 更新账本并释放活动事务
- 冷却5秒
- 自动继续运行策略

只有达到全局硬止损，例如当日亏损或最大回撤达到6 USDC，才应该在完成当前风险收口后降为 OBSERVE_ONLY。

如果真实仓位读不到、订单状态未知或者账本与交易所不一致，也不能继续开仓，但这属于执行安全锁，不等于策略已经触发亏损止损。

七、关于提高运行速度的思考

我还考虑了把平仓持续时间从 1000 ms 降低到 500 ms。

这样理论上可以让平仓订单提前约半秒发出，对盘口较薄、挂单撤销较快的市场可能有帮助。

但时间缩短后，也会让更多短暂出现的价差进入交易流程，可能增加无效订单、IOC拒绝和单腿恢复次数。

因此我的计划是：

先修通单腿恢复

- 确认事故可以自动收口
- 再单独测试500 ms
- 其他参数保持不变

这样才能判断效率提升到底来自哪里，而不是同时修改多个参数后无法分析结果。

八、两天工作的主要启发

这两天最大的启发，是对实盘交易系统有了更具体的认识。

第一，策略看到利润，不代表真实订单一定能够成交。真实执行还受到盘口深度、挂单撤销、网络延迟和交易所处理速度影响。

第二，行情新鲜度和系统授权是两件不同的事情。行情暂时不可用应该跳过交易，不能轻易让整个策略永久停止。

第三，两腿并行提交只能减少程序内部的时间差，不能保证两家交易所同时成交。因此，单腿恢复不是备用功能，而是跨所套利必须具备的核心能力。

第四，增加滑点不能彻底解决IOC拒绝。比增加滑点更重要的是，拒绝发生后程序能否正确识别、读取真实仓位并完成风险收口。

第五，程序应该以真实仓位和真实成交作为最终依据，不能只依赖计划订单和内部账本。

第六，简单架构并不是减少必要风控，而是尽量做到：

- 一个活动事务
- 一份行情缓存
- 一个恢复期限
- 一套订单终态判断
- 一条明确的风险收口路径

这两天虽然主要在处理问题，但这些真实事故让我更清楚地理解了交易系统的核心：不是保证永远不出错，而是让每一种可预期的错误都能够被识别、处理、对账并最终恢复运行。

下一步我会继续坚持最小修改原则，先完成

Hyperliquid

明确拒单终态识别和单腿恢复闭环，再根据实际数据决定是否调整平仓持续时间和正常退出滑点。

笔记 24

作者：倪响 | 时间：2026-08-05 15:24 UTC | 字数：3762

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习笔记 Day 1：AMM 第一性原理 —— $x \cdot y = k$ 到底在说什么

2026-08-05 | 残酷共学 Day 1

参考：Uniswap V2 白皮书 §1-3 (<https://uniswap.org/whitepaper.pdf>)

先搞清楚两个缩写

AMM (Automated Market Maker, 自动化做市商)：不用传统订单簿，而是用数学公式自动定价的交易机制。你不需要等别人挂单，直接跟池子交易。Uniswap 就是最典型的 AMM。

TVL (Total Value Locked, 总锁仓量)：一个池子/协议里锁定了多少钱。比如 USDC-ETH 池子里有 2000 ETH + 4,000,000 USDC，按 $\text{ETH} = \$2000$ 算， $\text{TVL} = \$8,000,000$ 。TVL 越大，池子越“重”，越难被一笔交易推动价格。

为什么从 AMM 开始？

套利说到底就是“在 A 地用低价买，在 B 地用高价卖”。这句话里每一个词都有前提：

A 地和 B 地存在的前提 → 有多个交易场所 (DEX)

低价和高价存在的前提 → 价格是怎么形成的

买和卖成立的前提 → 交易如何改变价格

AMM 回答了后面两个问题。不把 AMM 的数学搞透，后面算价差、算成本都是在瞎算

核心公式： $x \cdot y = k$

Uniswap V2 的池子里有两种 token (token0 和 token1)，设它们的数量分别为 x 和 y ：

$$x \cdot y = k$$

我的理解：这个公式的意思不是“系统保证 x 乘 y 恒定”，而是“任何交易都必须保持 x 乘 y 不变”——它是一个约束条件，不是数学恒等式。

举个实际例子。USDC-ETH 池子，假设池子里有：

2000 ETH

4,000,000 USDC

那么 $k = 2000 \times 4,000,000 = 8,000,000,000$ 。

这时候 ETH 的价格 = USDC 数量 / ETH 数量 = 4,000,000 / 2000 = 2000 USDC/ETH。价格就是池子中两种资产的比率，没有任何"预言机"参与。

核心直觉：池子里 token0 越多，它越便宜；越少，越贵。价格完全由供求（池中比率）决定。

交易如何改变价格

现在我买入 1 ETH，往池子里放入 Δx 个 USDC，取出 Δy 个 ETH。

因为交易后 $x \cdot y$ 必须等于 k ：

$$(x + \Delta x) \cdot (y - \Delta y) = k$$

解出 Δy （拿到的 ETH 数量）：

$$\Delta y = (y \cdot \Delta x) / (x + \Delta x)$$

但实际上 Uniswap V2 收 0.3% 手续费，只有 99.7% 的输入参与做市：

$$\Delta y = (y \cdot \Delta x \cdot 0.997) / (x + \Delta x \cdot 0.997)$$

用刚才的例子，买 1 ETH（按当前价要付 2000 USDC），实际获得：

$$\Delta y = (2000 \times 2000 \times 0.997) / (4,000,000 + 2000 \times 0.997) \approx 0.996 \text{ ETH}$$

注意：我本来想买 1 ETH，结果只拿到 0.996 ETH。少掉的那 0.004 ETH 哪里去了？

一部分是 0.3% 手续费（给 LP 的）

另一部分是 Price Impact——我的交易本身把 ETH 价格推高了，同样的 USDC 能买到的 ETH 变少了

Price Impact：屏幕上的价格 $\neq$ 你能成交的价格

Price Impact（价格冲击）= 你的交易导致的价格偏离幅度

简单说：

买之前 ETH = \$2000

买的时候实际均价 = \$2000 / 0.996 $\approx$ \$2008

Price Impact $\approx$ 0.4%

交易量越大，Price Impact 越大，而且是超线性的——买 10 ETH 的 Impact 远大于买 1 ETH 的 10 倍。因为每多买一点，池子就更失衡一点，后面的每一口价格都更差。

这个特性对套利的直接影响：

"屏幕上看到 Uniswap 和 Sushiswap 有 1% 价差" 不等于 "存在 1% 套利空间"。

你的套利交易本身会在两个池子里都产生

Price

Impact——买入推高一个池子的价格，卖出压低另一个池子的价格。两个 Impact 一叠加，1% 的价差可能变成 0.1% 甚至亏损。

一个关键直觉：跷跷板模型

可以把 AMM 池子想象成一个跷跷板：

token0 很多 $\rightarrow$ token0 便宜 $\rightarrow$ 有人来买 $\rightarrow$ token0 变少变贵

token0 很少 $\rightarrow$ token0 贵 $\rightarrow$ 有人来卖 $\rightarrow$ token0 变多变便宜

套利者就是在不同跷跷板之间搬运重量的那个人。

这也解释了为什么：

高 TVL 池子价差小：跷跷板很重，一笔交易推不动，所以价格稳定

低 TVL 池子价差大：跷跷板很轻，稍微一推价格就飞了，但套利也容易被自己的 Price Impact 吃掉

Day 1 小结

| 概念 | 一句话 |

| :----- | :----- |

| AMM | 用数学公式（而非订单簿）做市的机制 |

| $x \cdot y = k$ | 任何交易都必须保持乘积不变，价格 = y/x |

| TVL | 池子里锁定的总资产价值，决定池子"重量" |

| Price Impact | 你的交易会改变价格，交易量越大影响越超线性 |

| 核心结论 | 屏幕价差 $\neq$ 可执行套利，必须计入双向 Price Impact + 手续费 |

共学打卡：今天搞清楚了 AMM 的数学本质—— $x \cdot y = k$ 不是"特性"而是"约束"。最大的收获是 Price Impact 的超线性特性，这直接解释了为什么大额套利比小额套利难做得多。池子就像一个跷跷板，套利者是平衡器。

实践：读 USDC-ETH 池子，验证 Price Impact

写了 scripts/pool_reader.py，三个函数 + 主流程，用 uv 管理依赖。

实际输出 (2026-08-05 23:00 左右，区块 25689472)：

Pool: 0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc

ETH: 4,687.37

USDC: 8,781,534.47

Price: \$1,873.45

TVL: \$17,563,069

Buy ETH	USDC(spot)	Avg Price	Impact
0.1 \$	187 \$	1,879.12	0.3030%
1.0 \$	1,873 \$	1,879.48	0.3222%
5.0 \$	9,367 \$	1,881.08	0.4076%
10.0 \$	18,734 \$	1,883.08	0.5142%
50.0 \$	93,672 \$	1,899.07	1.3676%
100.0 \$	187,345 \$	1,919.05	2.4343%

验证了笔记中的三个结论：

1. 价格 = \$1,873.45，完全由池子中 USDC/ETH 比率决定，没有外部数据参与
2. Price Impact 超线性增长——0.1 ETH 只有 0.30%，100 ETH 跳到 2.43%
3. 如果你在另一个 DEX 上看到 1% 价差，100 ETH 的套利光买入侧 Impact 就 2.43%，直接亏

踩坑记录：

免费 RPC (1rpc.io, Nodies 后端) 连续 eth_call 会触发限流 (-32005 Rate limit exceeded)

getReserves() 一次调三个 eth_call，第二个就被拦了，需要加 sleep 或换 Infura key

笔记 25

作者：Kevin Jiang | 时间：2026-08-05 12:31 UTC | 字数：3363

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

实验A:只改金额(Arbitrum → Optimism, USDC → USDC)

采样时间:2026-08-05 21:24

发出金额	收到金额	到账损耗	Gas(\$)	费用(\$)	总成本(\$)	成本占比	路径	预计时间
-----	-----	-----	-----	-----	-----	-----	-----	----
10	9.973810	0.026190	0.0172	0.0262	0.0434	0.4339%	AcrossV4	1秒
100	99.750000	0.250000	0.0233	0.2499	0.2733	0.2733%	Eco	7秒
1,000	997.500000	2.500000	0.0233	2.4993	2.5233	0.2523%	Eco	7秒
10,000	9,975.000000	25.000000	0.0233	24.9932	25.0233	0.2502%	Eco	7秒

读数(比表格重要)

1. 路由自动换桥了:10 USDC 走 Across,100 以上全部切到 Eco。聚合器会按金额挑最优路线——所以"某条桥的成本"这个说法本身就不完整,要说"某金额下的最优路线成本"。
2. 成本占比没有一路下降,而是撞到了一堵墙:从 0.43% 降到 0.27% 后就卡在 ~0.25% 附近不动了。原因一眼可见:Eco 收的是比例费(0.25%),金额翻10倍,费用也翻10倍。固定成本(Gas $\approx$ \$0.02)在 100 USDC 之后就已经无关紧要。
3. Day 1 的预判要修正:Day 1 说"金额越大固定成本摊得越薄"——对,但只在小金额区间成立。真实成本曲线是:小金额被固定费吃、大金额被比例费封顶,这条路线的成本下限就是 0.25%,意味着任何小于 0.25% 的跨链价差在这条路线上永远无利可图,金额再大也没用。
4. Gas 在 L2 之间几乎可以忽略(\$0.02 量级),真正的大头是桥/协议的比例费。

实验B:只改链路(固定 100 USDC)

采样时间:2026-08-05 21:24

路线	收到金额	到账损耗	Gas(\$)	总成本(\$)	成本占比	路径	预计时间
Arbitrum $\rightarrow$ Optimism	99.750000	0.250000	0.0233	0.2733	0.2733%	Eco	7秒
Arbitrum $\rightarrow$ Base	99.750000	0.250000	0.0233	0.2733	0.2733%	Eco	7秒
Optimism $\rightarrow$ Base	99.750000	0.250000	0.0001	0.2501	0.2501%	Polymer (Standard)	1080秒(18分钟)

读数

1. 前两条路线报价一模一样——同一座桥(Eco)对主流 L2 之间统一收 0.25%,链路本身在这个金额级别没有差异。
2. 第三条路线暴露了新维度:成本 vs 速度的交换。Polymer 便宜了一点点(省了 \$0.023 的 Gas),但预计到账时间从 7 秒变成 18 分钟。对普通转账无所谓;对套利是致命的——18 分钟里价差早就没了。Day 1 讲的"延迟成本"在这里有了具体数字。
3. 推论:套利场景下不能让聚合器按"到账金额最多"选路线,要按"时间 $\times$ 金额"的综合最优选——这本身就是套利机器人要解决的工程问题之一。

实验C:只改时间(10 USDC,Arbitrum $\rightarrow$ Optimism,每5分钟一次)

采样区间:2026-08-05 21:24 ~ 22:00,每 5 分钟一次,共 8 个样本(原计划 1 小时 13 个,任务在 36 分钟处被中止,但样本量已足够回答实验问题)。

#	时间	收到金额	总成本(\$)	路径	预计时间
1	21:24:51	9.973810	0.04339	AcrossV4	1秒
2	21:29:52	9.973809	0.04339	AcrossV4	1秒
3	21:34:54	9.973807	0.04339	AcrossV4	1秒
4	21:39:55	9.973808	0.04339	AcrossV4	1秒
5	21:44:56	9.973808	0.04339	AcrossV4	1秒
6	21:49:57	9.973807	0.04339	AcrossV4	1秒
7	21:54:59	9.973805	0.04339	AcrossV4	1秒
8	22:00:00	9.973809	0.04339	AcrossV4	1秒

读数

1. 36 分钟内几乎完全静止:推荐桥没有漂移(始终 AcrossV4),总成本精确到 5 位小数都没变,到账金额只在小数点后第 6 位抖动(± 0.000005 USDC,约十万分之五)。
2. 这说明 L2 $\leftrightarrow$ L2 稳定币跨链在平静时段的报价是高度确定性的:费用结构由桥的费率表决定,不是由实时竞价决定。Day 1 担心的"报价随时变化"在这个场景里更多发生在行情剧烈波动时段(Gas 飙升、桥流动性被抽干),平时观察不到。
3. 方法论提醒:8 个平静样本只能证明"平时很稳",不能证明"永远很稳"——尾部时段(大行情、拥堵)才是成本模型真正被考验的时候。后续值得在行情剧烈的日子重跑一次对照。

综合结论(A+B+C)

1. 这条赛道(L2 之间稳定币跨链)的成本下限 $\approx$ 0.25%,且是比例费构成的硬下限。跨链套利要成立,价差必须显著大于 0.25% + 卖出成本 + 归位成本——这在稳定币之间是罕见事件(通常意味着脱锚)。
2. 最优交易规模不是"越大越好",100~1000 USDC 之后规模就不再改善成本占比。
3. 报价 API 返回的"最优路线"是按到账金额优化的,套利要自己重新按延迟加权。
4. 平静时段报价高度稳定(36 分钟成本零变化),说明这堵 0.25% 的费率墙短期内不会自己松动;要找机会,得盯异常时段(行情剧烈、拥堵、脱锚),而不是盯平时。

笔记 26

作者: Lizzie | 时间: 2026-08-06 13:34 UTC | 字数: 3331

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 | 从 439 条共学笔记中提炼“可证伪的链上套利研究框架”

今天完成了什么

今天没有继续寻找“看起来赚钱”的币或价差,而是使用 Hermes 的 intensive-co-learning Skill,对《链上套利残酷共学》的可见打卡进行了分页研究,并把高价值内容整理成一套面向初学者的本地 Wiki。

1. 分页读取课程打卡:每页 20 条,共读取 22 页;考虑到动态分页可能产生位移,又执行了第二轮完整抓取,最终按 check-in ID 合并得到 439 条唯一打卡,覆盖 378 位参与者、约 42 万字。
2. 按“问题—方法—证据—局限—下一步”筛选内容,优先保留有原始报价、脚本字段、交易复盘、负结果、主动纠错或失败状态机的笔记,而不是复述口号。
3. 建立链上套利 Wiki:9 个概念页、3 个综合查询页、1 个匿名化来源页,加上索引、Schema 和操作日志,共 16 个 Markdown 文件。
4. 完成 Wiki 自检:83 个内部链接无断链;知识页面最长 195 行;来源正文哈希验证通过;没有把 Access Key、邮箱、钱包或联系方式写入笔记。
5. 完成隐私清理:原始打卡正文只在本地临时分析,综合完成后已删除;Wiki 只保留匿名化结论、check-in ID 和课程来源。

我现在怎样理解“链上套利解决了什么问题”

链上没有一个天然统一的全局价格。同一种经济资产分散在不同链、DEX、资金池、订单簿和协议中,每个地方根据自己的库存、订单流和区块状态独立报价。

套利者解决的是:

同一种价值在彼此隔离、更新不同步的市场中暂时出现不同可成交价格的问题。

套利者在低价市场买入、在高价市场卖出,既获得差额,也推动低价市场上涨、高价市场下跌,使价格重新接近一致。利润不是凭空产生,而是市场为“搬运流动性、承担执行风险、快速修复错误价格”支付的报酬。

但必须严格区分:

text

看见价差

≠ 获得同口径的可执行报价

≠ 买卖两腿都能成交

≠ 扣除完整成本后赚钱

≠ 可以长期重复

真正需要验证的是:

text

期望净利润

= 最终可兑现价值

- 初始资金
- 手续费与 Gas
- 价格冲击与报价后滑点
- 桥接、延迟和资金占用
- 资金费率、借贷与库存再平衡
- 失败概率 × 失败损失

今天提炼出的高价值证据

1. 2,880 条 Quote 的负结果比“发现机会”更有价值

打卡 cms5futv006zmm2gp1c4ot8b 报告：约 2 小时、120 轮、2,880 条跨链 Quote，形成 480 次配对；现有费率下全部为负，即使按零费率重算也没有资产超过 +10 bps。一个单独报价源制造的乐观信号，在隔离后消失。

结论：聚合器异常或数据口径污染可以伪造利润；多源一致性、协议直连基准和异常源隔离是必要闸门。

2. “桥费 0.25%”被后续直连实验推翻

打卡 cms9ersx001fmy29fbolasjt 使用 Curve get_dy、Across 官方报价、Uniswap V3 和 LI.FI 路径做对照。聚合器稳定币三角约亏 0.72%，Curve 直连三环约亏 0.045%，差异主要来自逐跳的聚合器固定费，而不是 Curve 三角本身。

结论：聚合器适合发现路线，不是成本真值源；必须拆分 integrator、solver、bridge、DEX、Gas 和失败成本。

3. 高费率 LP 并不是“向套利机器人收过路费”

打卡 cmsgv9rv602xopl29hoy5hxm 的回测中，一个平稳案例投入 10,000 USDT，手续费 +4.29、Gas -5，净 -0.71 USDT；下跌案例手续费 +180.60，但存货损失 -1,157.40、Gas -5，净 -981.80 USDT。

结论：LP 是持续报价的兑换窗口，会承受 LVR、库存变化、出区间和再平衡成本；手续费收入不等于净利润。

4. 44 个候选主动刷新后只剩 12 个

打卡 cmsfh3xsa00t1pl291iyhhqy5 中，44 次权威盘口刷新全部请求成功，但只有 12 个候选刷新后仍成立，32 个因净收益不足取消。

结论：许多伪机会来自旧盘口和外部延迟。Re-quote 必须是独立风险闸门，而不是可选优化。

5. 清算方向在当前语料中证据不足

虽然很多打卡介绍 Aave 健康因子和清算奖励，但没有形成固定区块账户扫描、Oracle 更新时序、fork 清算模拟、抵押品退出回放和失败 Gas 数据。因此不能把示例奖励比例包装成当前可执行 Edge。

证据在哪里

本地 Wiki 入口：

/Users/lizzye/LLMwiki/web3/arbitrage/index.md

主要页面：

concepts/arbitrage-first-principles.md：套利解决的市场问题与利润来源。

concepts/arbitrage-cost-model.md：手续费、Gas、滑点、延迟、流动性、同步、资金费率、库存和失败成本。

concepts/execution-atomicity-and-inventory.md：原子性、跨链 Leg Risk 与库存循环。

concepts/arbitrage-validation-workflow.md：Raw、Re-quote、Cost Ledger、模拟、回放和 Kill Criteria。

queries/high-value-cohort-findings.md

high-value-cohort-findings-2.md：共学笔记中的实证、负结果和主动纠错。

raw/articles/icl-cohort-checkins-2026-08-06.md：匿名化来源、抓取范围、局限和来源哈希。

课程来源：

<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

假设与风险

与

数据只覆盖 2026-08-05 至 2026-08-06，是课程开营前两天的切片，不能代表完整 21 天成果。
439 条打卡的 webUrl 都指向课程页，因此使用 check-in ID 区分来源。
同学报告的数字尚未由我逐项重跑外部代码或链上交易；本次结论属于语料综合，不是独立策略复现。
Quote、eth_call、历史回放和模拟不能证明未来真实成交，更不能证明存在长期 Edge。
本次没有连接钱包、签名、广播交易、Paper Trading 或 Live Execution。

还没完成

尚未在固定历史区块独立复现 V2 双池闭环和 V3 跨 tick 报价。
尚未建立真实只读报价的持续采集、Re-quote 和 Cost Ledger 数据库。
尚未对 2,880 条 Quote、LP/LVR 回测和跨场所刷新案例逐项重跑。
尚未验证机会寿命、失败概率、库存归位成本和样本外稳定性。

下一步

选择一条链、一个资产对、两个 V2-compatible 池和三个交易规模，完成只读最小实验：

text

固定 block 与时间

- 读取两池 reserves
- 计算双向真实输出
- 独立 Re-quote
- 扣除双边手续费、Gas 和价格冲击
- 保存 Raw 与 Cost Ledger
- 在历史 fork 上验证原子闭环和 minProfit
- 输出 CONTINUE / ITERATE / KILL

下一阶段仍以快速证伪为目标，不把正数强行解释成可交易利润。

笔记 27

作者：dyhbrewer | 时间：2026-08-06 07:43 UTC | 字数：3168

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 · LI.FI 跨链 Route 只读评估

今天用 1,000 USDC 对 Ethereum → Arbitrum 做了一次完整的只读 Route 与 Break-even 评估。全程未签名、未授权、未广播交易。数据快照约为 2026-08-06 07:06–07:10 UTC。

1. 链与代币核验

通过 LI.FI /chains 与 /tokens 确认：

| 链 | Chain ID | USDC 合约 | Decimals |

|---|---|---|---|

| Ethereum | 1 | 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48 | 6 |

| Arbitrum One | 42161 | 0xaf88d065e77c8cC2239327C5EDb3A432268e5831 | 6 |

Arbitrum 使用 native USDC，不是旧版 USDC.e。输入金额为 $1,000 \times 10^6 = 1,000,000,000$ 。

2. /quote 结果

参数：Ethereum USDC → Arbitrum USDC、1,000 USDC、order=RECOMMENDED、slippage 3%。

tool：eco

路径：feeCollection → eco

toAmount：997,500,000 = 997.500000 USDC

toAmountMin：997,500,000 = 997.500000 USDC

feeCosts：LI.FI Fixed Fee = 2.5 USDC / 0.25%

gasCosts : Ethereum 约 USD 0.1114

executionDuration : 7 秒

Eco 未列独立 Bridge / Solver fee

toAmountMin 与 toAmount 相同，说明该 Eco solver 报价承诺固定输出；3% slippage 没有被直接扣成预期损失。

3. /advanced/routes 候选比较

高级接口共返回 17 条 Route，主要候选：

Route	到账 USDC	最低到账	LI.FI 之外显式费用	Gas USD	时间
Eco	997.500000	997.500000	0	0.0719	7s
Polymer Standard	997.500000	997.500000	0	0.1003	1,080s
LI.FI Intents	997.439151	997.439151	未单列	0.0719	32s
Relay	997.397878	996.400481	0.1021	0.0409	1s
Across V4	997.394801	997.394801	0.1053	0.0409	2s
Polymer Fast	997.300500	997.300500	0.1996	0.1003	10s

Eco 给出最高固定输出；Relay/Across 更快且 gas 更低，但少收约 0.10 USDC。Polymer Standard 输出相同，却需要约 18 分钟，不适合价差交易。

4. 目标端实际可成交价格与深度

Coinbase 当前没有 USDC-USD 盘口，因此用在线的 USDT-USDC 盘口，把收到的 USDC 变现为 USDT。该产品为 limit_only，可以使用会立即成交的限价单吃单。

订单簿快照：

Best bid : 0.9989 USDC/USDT，深度 255,791.6 USDT
Best ask : 0.9990 USDC/USDT，深度 209,449.98 USDT
Spread : 约 1.001 bps
997.5 USDC 可换 : 998.498498 USDT
实际 VWAP : 0.9990 USDC/USDT
走档 / Price Impact : 0，全部在最优 ask 成交
1 bp 内 ask 深度 : 约 USD 80,442
5 bp 内 ask 深度 : 约 USD 1.020M

因此 1,000 美元规模的 Liquidity 不是瓶颈。

5. Break-even Spread

成本应该相加：

最低价差 = LI.FI fee + Bridge/DEX/Solver fee + 两链 gas + 两端交易价差/滑点/price impact + 交易费 + 延迟/失败/资金占用成本。

完整 USDT → USDC → 跨链 → USDT 循环：

源端取得 1,000 USDC : $1,000 / 0.9989 = 1,001.101211$ USDT
Eco 跨链后 : 997.5 USDC
目标端卖出 : $997.5 / 0.9990 = 998.498498$ USDT
盘口 + LI.FI 损失 : $1,001.101211 - 998.498498 = 2.602713$ USDT
加源链 gas : $2.602713 + 0.1114 = 2.714113$ USDT ≈ 27.14 bps

成本拆解：

成本项	约 bps
LI.FI Service Fee	25.00

Eco 独立 Bridge/Solver fee	0
Ethereum gas	1.11
两端盘口价差/冲击	1.00
Arbitrum gas	0 (桥直达 CEX) 或约 0.52 gas 预算
7 秒资金占用	接近 0
CEX 账户费	未知，需另加两端实际费率

所以：

桥直接送入 CEX : Break-even $\approx 27.14 \text{ bps} + \text{源端费率} + \text{目标端费率}$

先到自托管钱包再转 CEX : Break-even $\approx 27.66 \text{ bps} + \text{两端费率}$

Coinbase	实际	maker/taker
费取决于账户和交易量，应以订单预览为准，不能把稳定币对专属费率直接套在该产品上。		

6. 结论

当前不可执行。

当前可成交跨市场价差约为 -1.0 bps，方向不利。

最低需要约 27.1–27.7 bps + CEX 交易费。

最大成本是 LI.FI 固定费：25 bps。

Gas 约 1–1.6 bps，不是主因。

1,000 美元完全不走档，Liquidity 和 Price Impact 不是问题。

Eco 的 7 秒只是链间预计时间，不等于 CEX 已完成充值入账；若目标端没有预置库存，非原子执行和充值确认使 Latency 成为第二个实际瓶颈。

即使预置目标库存消除延迟，当前价差仍不足以覆盖 Fee。

最终判断：主要卡在 Fee，其次是 Latency；不是 Liquidity、Slippage 或 Gas。

笔记 28

作者：bzd111 | 时间：2026-08-05 11:26 UTC | 字数：3066

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. 用一句话说清「链上套利」的钱从哪来

一句话答案

链上套利的钱，来自「市场短暂的价格失衡」——你在便宜的地方买、贵的地方卖，把价格拉回平衡，这个差价就是市场付给你的「效率报酬」。

但更关键的是：这个钱到底是谁出的？

它不是凭空产生的，追根溯源有两类「买单人」：

第一类：制造失衡的人（大额交易者）

某人用 1000 万 USDC 在一个池子里猛买 ETH → 他把池子里的 ETH 价格抬高了

他承受的「滑点损失」变成了池子的价差 → 这部分价差就是套利者的利润来源

本质上是：大额交易者为了快速成交，付了溢价，套利者替他价格抹平，分走一部分

第二类：非理性的恐慌者（稳定币脱锚时）

UST 脱锚时有人恐慌抛售，把 1 UST 卖到 0.9 USDC

套利者 0.9 买入，等它回到 1.0 → 赚的是恐慌者的割肉钱

一个必须记住的提醒

套利不是「无中生有」，利润是有真实出处的。当你想清楚「谁在为这笔利润买单」，你就离看懂这个机会近了一半。

2. 7 种机会结构总览图 + 每种的价值来源

总览图

链上套利机会



逐个拆解「价值来源」

#	类型	价值来源（钱从哪来）	通俗解释
①	DEX 内套利	同一 DEX 多个池子的瞬时价格不平衡 大单打了 A 池，B/C 池还没反应，你绕一圈抹平	
②	DEX 间套利	同链不同 DEX 同一资产的价差 Uniswap 和 Sushi 同一个 ETH 价格对不上	
③	三角套利	A→B→C→A 的汇率环不闭合（绕一圈不等于 1） 三个币两两之间的汇率出现了套利空间	
④	跨链套利	同资产在不同链上的价差 ETH 在 Ethereum 和 Arbitrum 价格不同步	
⑤	稳定币套利	稳定币脱锚 / 回归锚定的过程 1 USDC 短暂跌到 0.998，买入等回归	
⑥	清算套利	借贷协议坏账的清算奖金 (liquidation bonus) 帮协议清算抵押不足的仓位，协议奖励你折扣买抵押品	
⑦	MEV (前跑/三明治)	别人交易的排序价值 你能看到 mempool 里别人要买，你先买、他买完你再卖，割他一刀	

一句话记忆口诀

①不平衡 ②价差 ③环路 ④跨链 ⑤脱锚 ⑥清算 ⑦排序

3. 区分 DEX 内套利 vs DEX 间套利

这是新手最容易混淆的一对，我从 5 个维度对比：

维度	DEX 内套利	DEX 间套利
范围	同一个 DEX 内的多个池子	不同 DEX 之间的同一资产对
举例	Uniswap 里 ETH/USDC + ETH/ARB + ARB/USDC	Uniswap 的 ETH/USDC vs Sushi 的 ETH/USDC
路径形态	通常是多跳的三角（绕一圈）	通常是两端直接对冲（一买一卖）
本质	抹平单一 DEX 内的价格不平衡	抹平两个 DEX 之间的价格不一致
竞争烈度	中	极高（MEV 主战场）

一个生活化的比喻

DEX 内套利 = 一个菜市场里，东边的鸡蛋摊和西边的鸡蛋摊价格不一样，你东买西卖

DEX 间套利 = 「东门菜市场」和「西门菜市场」整体鸡蛋价格不一样，你跨市场搬砖

为什么 DEX 间套利竞争更残酷？

因为这种价差是最容易被发现的——同一个 ETH/USDC，两个 DEX 价格摆在那儿，谁都能看到。所以一旦价差出现，毫秒级的 MEV 机器人会立刻吃掉它。这也是为什么后面要学 Flashbots / 私有交易池——让你的交易不被机器人看到。

笔记 29

作者：Zacteh | 时间：2026-08-06 14:47 UTC | 字数：2983

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 | 从 Transfer 日志到跨链资产净流量

今日目标

在 Day 1 已经能够通过 RPC 读取 Transaction、Receipt 并解析 ERC-20 Transfer 日志的基础上，今天继续回答三个问题：

1. 一笔复杂交易中的资产到底流向了谁？
2. 地址的代币净流量应该怎样正确聚合？
3. 为什么“净流入最多”仍然不能直接等同于“套利利润最高”？

真实交易复盘

继续使用 Ethereum Mainnet 交易：

0x9ac0e69043326d5c0255e5159161d990b6e867873e24a92c78295c1e4a700328

通过本地 viem 分析器再次验证：

交易执行成功；
实际 Gas 费为 0.01165928076594797 ETH；
value = 0，没有顶层 ETH 转账；
Receipt 包含 1,000 条日志，全部是同一合约发出的 ERC-20 Transfer；
合约自报名称为 Notcoin、Symbol 为 NOT、Decimals 为 8；
交易形态是从同一地址向 1,000 个不同地址批量发送等额 Token。

调用者地址没有出现在已展示的 Transfer 收发双方中。因此，仅凭顶层 transaction.from 不能判断调用者收到了 NOT，更不能判断其获利。这种批量分发形态更接近垃圾空投或诱导交互 Token，而不是套利交易。

从 Transfer 日志计算净流量

某地址在某种代币中的净变化：

```
text
net = Σ inflow - Σ outflow
```

聚合时不能只使用持有人地址作为键。跨链场景中，一个流量记录应由下面三部分唯一确定：

```
text
chainId + tokenAddress + ownerAddress
```

代码中的键可以写成：

```
typescript
const key =
  `${chainId}:${tokenAddress.toLowerCase()}:${ownerAddress.toLowerCase()};
```

地址统一转成小写，是因为 EVM 地址字符串可能使用不同大小写形式，而 JavaScript 的 Map<string, ...> 区分大小写。同一地址如果不归一化，可能被错误统计成两个参与者。

流量结构只保存原始事实：

```
typescript
type TokenFlow = {
  chainId: number;
  tokenAddress: Address;
  ownerAddress: Address;
```

```
inflow: bigint;
outflow: bigint;
};
```

net 不重复存储，而是在输出时计算：

```
typescript
const net = flow.inflow - flow.outflow;
```

这样可以避免 inflow、outflow 和 net 三份状态不同步。金额必须使用 bigint，因为 Token 的 raw amount 很容易超过 JavaScript Number.MAX_SAFE_INTEGER。初始化值应写成 0n，最终展示时再使用 formatUnits(net, decimals)。

另外两个特殊情况：

from = 0x000...000：通常表示 Mint；
to = 0x000...000：通常表示 Burn。

零地址不是普通持币人，不应该被解释成真实的亏损地址。

净流量不等于套利利润

不同代币不能直接相加。例如 1 WETH 与 -1000 USDC
单位不同，必须先按交易发生时的价格换算成统一计价单位。

一个更接近真实的利润模型是：

text

经济利润

= Σ (每种资产净变化 $\times$ 交易时点可成交价格)

+ 原生币净变化

- Gas

- Builder / Validator 小费

- 未体现在净流量中的协议费、桥费与其他成本

这里有几个重要边界：

必须使用交易所在区块附近的历史价格，不能用今天的价格重算历史套利；

应使用有真实流动性的可成交价格，不能只相信 Token 自报价格或 Symbol；

许多 DEX 手续费已经反映在最终转入、转出数量中，需要避免重复扣除；

transaction.from 可能只是 Gas 支付者，利润可能进入另一个受同一实体控制的合约或地址；

Router、资金池和借贷协议通常只是资金经过的节点；

一次 MEV 或套利操作可能由多笔 Bundle 交易组成，所以单笔交易的最大净流入地址只能算获利者候选。

跨链资产流的处理

只看源链会把一次成功的跨链转移误判为巨额亏损。例如：

text

源链：-1,000 USDC

目标链：+995 USDC

跨链净变化：-5 USDC

跨链分析必须关联两侧交易，通常依靠 LI.FI transactionId、桥的
messageId、路由数据或其他协议标识，而不能只凭金额猜测。

代币身份应以 (chainId, contractAddress) 为准，不能使用 name 或 symbol。同名 Token
可能由任何人部署；Etherscan 的源码验证也只说明源码与字节码匹配，不自动证明它是官方资产。跨链资产是否经济等价，还需要参考发行方官方列表、桥的 Token 映射和 LI.FI 路由返回数据。例如原生 USDC 与桥接 USDC.e
不能仅凭名称视为完全相同。

今日收获

transaction.from、Transfer.from、Transfer.to 和最终获利实体是不同概念；
资产流应先按链、Token、地址分别聚合，再统一估值；
链上金额全程使用 bigint，展示阶段再格式化；
保存 inflow/outflow，动态推导 net，保持单一事实来源；
Token 名称和 Symbol 可以仿冒，资产身份必须落到链 ID 与合约地址；
跨链套利必须同时观察源链与目标链，并处理费用、延迟、退款和资产映射；
最大正净流入只能定位候选受益者，不能单独证明套利关系。

当前进度与下一步

今天已经完成资产流聚合的数据结构、计算规则和利润判断边界设计，并重新运行真实交易分析器验证样例数据；
净流量排行榜尚未写入代码。

下一步计划：

1. 在 tx-inspector.ts 中实现 aggregateTokenFlows；
2. 按 Token 输出净流入和净流出 Top 10；
3. 标记 Mint/Burn 零地址；
4. 加入原生 ETH 流量与 Gas 支付者；
5. 再接入历史价格与 LI.FI 路由数据，逐步形成可复用的套利 P&L 模型。

笔记 30

作者：zhangjunhaihhh-pixel | 时间：2026-08-06 09:07 UTC | 字数：2810

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

有。你如果从零开始，建议先学习“区块链和以太坊”，再学 DeFi，最后才进入套利。不要一开始买很多所谓“量化套利秘籍”，不少内容已经过时。

第一阶段：完全零基础

先读免费的中文资料：

1. 以太坊新手指南（中文）(<https://ethereum.org/zh/guides/>)
了解钱包、账户、代币、交易和基本安全。
2. 以太坊技术入门 (<https://ethereum.org/zh/developers/docs/intro-to-ethereum/>)
了解区块链、节点、交易和智能合约。
3. Ethereum 开发者文档（中文）(<https://ethereum.org/zh/developers/docs/>)

先读以下部分：

账户
交易
区块
Gas
智能合约
EVM

这一阶段不要求会写代码。目标是能够解释：

钱包和账户有什么区别
私钥、助记词和地址是什么
一笔交易为什么要支付 Gas
智能合约如何运行
ERC-20 代币是什么
区块浏览器里的交易信息怎么看

第二阶段：系统教材

首选：《精通以太坊》

英文名：Mastering Ethereum。

第二版免费在线阅读 (<https://masteringethereum.xyz/>)

作者：Andreas M. Antonopoulos、Gavin Wood 等

内容包括钱包、交易、Solidity、智能合约安全、代币、预言机和 DeFi。

推荐阅读顺序：

1. What is Ethereum
2. Ethereum Basics
3. Cryptography
4. Wallets
5. Transactions
6. Smart Contracts and Solidity
7. Smart Contract Security
8. Tokens
9. Oracles
10. Decentralized Finance

这本书技术性较强，不必第一次就全部理解。英文不熟练可以使用浏览器翻译，技术名词尽量保留英文。

辅助：《精通比特币》

第三版官方免费电子书 (<https://github.com/bitcoinbook/bitcoinbook/blob/develop/BOOK.md>)

官方项目仓库 (<https://github.com/bitcoinbook/bitcoinbook>)

它更适合帮助你理解：

公钥和私钥

数字签名

哈希

区块和交易

去中心化网络

共识机制

你的目标是 Web3 套利，因此不用整本精读。读基础章节即可，然后转到以太坊。

第三阶段：学习智能合约

推荐资料：

Solidity 官方文档 (<https://docs.soliditylang.org/en/latest/>)

Solidity 官方 PDF 电子版 (https://docs.soliditylang.org/_/downloads/en/latest/pdf/)

Remix 在线编程环境 (<https://remix.ethereum.org/>)

先学习这些知识：

变量和数据类型

函数

mapping

struct

require 和 revert

msg.sender

payable

ERC-20

合约之间相互调用

事件和日志

授权 approve

重入攻击和权限控制

建议亲手写三个简单合约：

1. 存取 ETH 合约
2. 简单 ERC-20 合约
3. 调用另一个合约完成代币转账的合约

第四阶段：学习 DeFi 和 AMM

从 Uniswap 开始，因为它的基础模型最容易理解。

推荐资料：

Uniswap 如何工作 (<https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works>)

Swap 的执行原理 (<https://developers.uniswap.org/docs/protocols/v2/concepts/swapping>)

Uniswap V2 白皮书 PDF (<https://docs.uniswap.org/whitepaper.pdf>)

重点掌握：

$$x \times y = k$$

以及：

流动性池

AMM 自动做市商

兑换手续费

滑点

价格影响

流动性深度

多跳兑换

套利如何让不同池子的价格重新接近

完成后，你应该能用计算器或表格回答：

某个池子里有100 ETH和20万USDC，买入1 ETH后价格会变成多少？扣除手续费后能得到多少？

第五阶段：开始接触套利

这时再依次学习：

1. 同链 DEX-DEX 套利
2. 三角套利
3. 清算
4. 闪电贷
5. MEV 和 Backrun
6. 跨链套利

相关资料：

Ethereum MEV 文档 (<https://ethereum.org/developers/docs/mev/>)

Flashbots 官方文档 (<https://docs.flashbots.net/>)

Aave

V3

技术论文

PDF

(https://raw.githubusercontent.com/aave/aave-v3-core/master/techpaper/Aave_V3_Technical_Paper.pdf)

记住一个非常重要的概念：

闪电贷不是套利方法，只是在发现有效套利后提供临时资金的工具。

30天入门阅读安排

每天学习1小时即可：

第1~7天：以太坊中文新手指南

第8~14天：《精通以太坊》的基础、钱包和交易章节

第15~20天：智能合约与 Solidity 基础

第21~25天：ERC-20、授权、合约调用

第26~30天：Uniswap、流动性池和 $x \times y = k$

第一个月不要投入真实资金，也不用急着学习闪电贷。先以“能看懂一笔交易”和“能算出一次兑换结果”为目标。

Uniswap

如果只选择一套资料，建议采用：

以太坊中文官方文档 → 《精通以太坊》 → Solidity 官方文档 → Uniswap V2 文档和白皮书。零基础小白，通过chatgpt提供学习方案，并不断学习新的概念名词

笔记 31

作者：rakisa | 时间：2026-08-06 15:22 UTC | 字数：2793

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、今日目标

今天没有直接寻找真实套利机会，而是先解决一个基础问题：

为什么同一个 DEX 资金池中，交易金额越大，实际成交价格通常越差？

为了验证这个问题，我在 Codex 辅助下编写并运行了一个简化的恒定乘积 AMM 模拟程序，用同一个 USDC/ETH 池分别模拟 100、1,000、10,000 和 100,000 USDC 的兑换结果。

本次实验只研究 AMM 曲线与 LP 手续费，不是实时链上报价，也没有发送任何交易。

二、模型与假设

模拟资金池：

text

USDC 储备：1,000,000 USDC

ETH 储备：500 ETH

初始现货价格：1,000,000 / 500 = 2,000 USDC/ETH

LP 手续费：0.3%

恒定乘积公式：

text

$x \times y = k$

其中：

x 是 USDC 储备；

y 是 ETH 储备；

k 是两边储备量的乘积。

使用 USDC 买入 ETH 时，先扣除 LP 手续费：

text

有效输入 = 输入金额 $\times$ (1 - 0.003)

简化输出公式：

text

ETH输出

= ETH储备 $\times$ 有效USDC输入

/ (USDC储备 + 有效USDC输入)

本次模型没有考虑：

Gas；

MEV和交易排序；

Uniswap V3集中流动性与Tick；

多跳路由；

Token转账税；

报价过期；

链上交易失败。

因此结果只能用于学习 AMM 曲线，不能视为真实可交易报价。

三、实际运行结果

输入USDC	LP手续费	输出ETH	含手续费平均成交价	纯价格冲击	总成交劣化
100	0.30	0.04984503	2,006.2181 USDC/ETH	0.00997%	0.31090%
1,000	3.00	0.49800349	2,008.0181 USDC/ETH	0.09970%	0.40090%
10,000	30.00	4.93579017	2,026.0181 USDC/ETH	0.99700%	1.30090%
100,000	300.00	45.33054469	2,206.0181 USDC/ETH	9.97000%	10.30090%

这里的“纯价格冲击”表示排除 LP 手续费后，AMM 曲线本身造成的平均成交价变化。

“总成交劣化”表示相对于初始 2,000 USDC/ETH 的池内价格，实际投入金额除以获得 ETH 数量后的整体价格差，其中同时包含手续费和 AMM 曲线影响。

四、对结果的理解

1. 小额交易主要受固定比例手续费影响

输入 100 USDC 时：

纯价格冲击只有约 0.00997%；

但总成交劣化约 0.31090%。

这说明在交易金额相对资金池很小时，AMM 曲线造成的影响很小，0.3% LP 手续费是主要成本。

2. 金额达到池子一定比例后，价格冲击快速增加

输入 10,000 USDC，相当于原USDC储备的1%：

纯价格冲击约 0.997%；

总成交劣化约 1.301%。

这时损耗已经不再只是0.3%的LP手续费，价格冲击开始成为主要成本。

3. 大额交易不能使用页面中间价估算利润

输入 100,000 USDC，相当于原USDC储备的10%：

初始池内价格仍是2,000 USDC/ETH；

实际含手续费平均成交价约为2,206.02 USDC/ETH；

总成交劣化超过10%。

如果一个套利策略只使用两个池子的“当前价格”计算价差，而不按照实际交易规模获取可成交报价，就可能把一个严重亏损的交易误判为套利机会。

五、Price Impact 与 Slippage 的区别

今天纠正的一个概念是：Price Impact 和 Slippage 不是同一个东西。

Price Impact

由自己的交易改变资金池储备造成，是交易规模和池深度可以预估出来的影响。

例如本次实验中，100,000 USDC 对资金池造成约9.97%的纯价格冲击。

Slippage Tolerance

是用户允许“报价到实际执行之间最多变化多少”的保护参数。

例如：

text

当前预估输出：1 ETH

滑点容忍度：0.5%

最低可接受输出：0.995 ETH

滑点容忍度不是额外收取的手续费，也不会消除价格冲击。设置过低可能导致交易回滚并损失Gas；设置过高则可能承受更差成交或增加被MEV利用的风险。

六、与链上套利的关系

如果两个 DEX 页面显示：

text

DEX A：ETH价格2,000 USDC

DEX B：ETH价格2,020 USDC

表面价差是1%，但不能直接认为有1%的利润。

还需要根据实际交易金额计算：

text

可执行净收益

= 卖出端实际输出

- 买入端实际成本

- 两边LP手续费

- 两边价格冲击

- Gas

- MEV与滑点缓冲

- 失败交易预期成本

本次实验中，10,000 USDC交易仅单边AMM曲线和手续费就产生约1.30%的成交劣化。因此一个表面1%的价差，在没有计算另一边交易和Gas之前就已经不成立。

七、今天完成的真实产出

编写 amm_simulator.mjs；

使用恒定乘积公式模拟四档交易金额；

生成 results.json；

区分 LP 手续费、纯价格冲击和总成交劣化；

区分 Price Impact 与 Slippage Tolerance；

明确页面中间价不能直接用于大额套利利润判断。

八、仍未解决的问题

1. Uniswap V3集中流动性下，价格冲击应如何计算？
2. 实际LI.FI路由可能经过多个DEX和Bridge，如何汇总每一步影响？
3. LI.FI返回的价格影响字段是否已经包含所有路由步骤？
4. Gas和目标链Gas如何统一换算成美元？
5. 如何对比模型结果和真实链上报价？

笔记 32

作者：Theo | 时间：2026-08-05 15:15 UTC | 字数：2742

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学-DAY01

日期: 2026-08-05

学习目标: MEV 三明治攻击机器人项目搭建、节点性能基建研讨与共学营课题拆解

一、今日项目进展与代码重构 (MEV-Flashbot-Sandwich-RS)

1. 代码库克隆与项目接入

成功将 Rust 实现的高性能 MEV 三明治机器人仓库 `mev-flashbot-sandwich-rs` 克隆至本地工作区。

技术栈解析:

Rust + Tokio 异步高并发运行时

ethers-rs 链上交互与 WebSocket 订阅

Flashbots RPC 提交私密交易 Bundle 防抢跑

AES-GCM 私钥加密解密

2. 彻底取消硬编码 & .env 环境变量重构

修复了原代码中 `mempool.rs` 硬编码 RPC 节点与合约地址的问题。

在 `.env` 中新增并支持了以下配置项, 实现代码与配置解耦:

RPC_URL: 动态 WebSocket 节点链接

TARGET_CONTRACT: 目标交易合约 (如 MetaMask ETH 兑换 USDT 代理合约)

ROUTER_LIST: Uniswap V2 / V3 及其他 DEX 的 Router 路由合约列表

二、MEV 节点基建与性能瓶颈深度探讨

1. 为什么 Infura / Alchemy 公共节点无法实盘获利?

广播延迟与数据缺失: 公共节点 WS 推送默认仅推送交易 Hash (无 Raw Tx Data), 且存在网关限速与排队。

二次 RPC 往返开销: 收到 Hash 再发起 `get_transaction` 产生 100ms~300ms 额外网络延迟。

缺少本地 EVM 模拟: 依赖 RPC 节点的 `eth_call` 无法在微秒级推演最优 Front-run 数量。

2. MEV 基建演进 4 阶路线图

1. Phase 1 (本地 EVM 模拟): 集成 `revm` (`bluealloy/revm`) 嵌入式虚拟机, 在内存中维护池子状态快照, 微秒级模拟出数。

2. Phase 2 (极速 Mempool 通道): 接入 `bloXroute BDN` / `Chainbound Fiber gRPC` 专线获取原始 Pending 交易流。

3. Phase 3 (自建 Reth 节点 + 多 Builder 提交): 自建 `Paradigm Reth` (Rust 客户端), 调高 `max_peers = 200+`, 通过 IPC 零开销交互; 同时向 `Beaverbuild`, `Titan`, `Rsync`, `Eden` 等全网主流 Builder 并发提交 Bundle。

4. Phase 4 (物理同构与客户端魔改): 机房物理托管 (AWS `us-east-1` / `Equinix Frankfurt` 同区); 魔改 Reth 节点 P2P 字节流解析。

3. 链上永续合约 (Perp DEX) 可夹性与 MEV 机制分析

传统 AMM 模式 / 带现货 Swap 杠杆开仓: 含有滑点设置, 会被传统 MEV 机器人进行三明治攻击。

预言机 Pricing 模式 (GMX V2 / Jupiter / Hyperliquid): 采用异步两阶段执行与预言机定价, 免疫三明治攻击, 但衍生出 强平争抢 (Liquidation MEV) 和 预言机延迟套利 (Oracle Frontrunning / Latency Arbitrage)。

三、共学营议题与参考仓库案例拆解 (w3player/arbitrage-playbook)

今日在共学营中重点学习并整理了社区参考仓库

`w3player/arbitrage-playbook`

(<https://github.com/w3player/arbitrage-playbook/issues>) 中的 4 大核心 Issue 议题:

Issue #1: LP 套利探讨 (LP Arbitrage)

核心逻辑: 探讨基于 Uniswap V3 / Concentrated Liquidity (集中流动性) 的 LP 动态做市与流动性再平衡套利。

MEV 关联: JIT (Just-In-Time) Liquidity 攻击。在大型 Swap 交易前向池子里提供集中流动性捕获手续费, 交易完成后立即撤出流动性。

Issue #2: 资金费套利探讨 (Funding Rate Arbitrage)

核心逻辑: 跨 CEX/DEX 或 DEX/DEX 之间的期现套利 (Cash and Carry) 与资金费率 (Funding Fee) 对冲。

MEV

关联:

当不同永续合约平台的资金费率产生偏离时, 利用智能合约进行自动对冲平仓; 同时关注预言机更新滞后带来的 Latency Arbitrage 机会。

Issue #3: 跨所交易套利工具 Gate CrossEx

核心逻辑: 跨中心化交易所 (CEX) 与去中心化交易所 (DEX) / 跨 CEX 之间的 API 极速套利工具。

工程实践: 涉及低延迟 WebSocket 行情监听、并发订单簿深度对比以及风控止损逻辑。

Issue #4: Backpack 套利展示

核心逻辑: 新兴高性能 Solana 生态交易所 Backpack 的 API 交易、挂单套利与活动收益提取策略展示。

工程实践: 基于高效 SDK 进行高频挂单与做市策略。

四、明日计划 (Day 02 Plan)

1. 深入研究 revm 库与 ethers-rs 的本地集成, 实现 Uniswap V2 池子状态在内存中的初始化。
2. 编写本地模拟计算逻辑, 推演受害交易在池子中的价格影响, 计算最优化三明治输入量。

笔记 33

作者: clare ma | 时间: 2026-08-06 13:05 UTC | 字数: 2536

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 / 21 · AMM 与恒定乘积: 把名词变成数字

学了什么

小额交易的成本几乎全是手续费

| 成本项 | 数值 | 占比 | 随规模 |

| :----- | :----- | :----- | :----- |

| 手续费 3/997 | 0.0030090271 | 99.63% | 恒定 |

| 价格冲击 amountIn/reserveIn | 0.0000112740 | 0.37% | 线性增长 |

比值 267 : 1。在 887 万美元的池子里打 100 美元, 所有关于「滑点」的担心都是噪音。

推导出的精确恒等式

$$P_{\text{含费有效}} / P_{\text{边际}} = 1000/997 + \frac{\text{amountIn}}{\text{reserveIn}}$$

手续费项价格冲击项

不是近似, 是精确式。两项相加 0.0030203, 与实测的 0.302% 完全对上。

一条可推广的法则

交易额达到池子储备的约 0.3% 时, 价格冲击才追上手续费。

低于这个量级, 成本结构由手续费主导; 高于它, 滑点开始主导。

本池子的分水岭: $8,870,002 \times 3/997 \approx 26,690 \text{ USDC}$ ($\approx 14 \text{ ETH}$) 。

其他概念收获

价格 = 从原点到曲线上该点的连线斜率 y/x ; 交易就是沿曲线滑动, 连线不断变角

双曲线永不与坐标轴相交 → 池子抽不空, 安全性来自不变式本身而非 require

手续费的 0.3% 留在池子里, 使 k 微增 —— 无手续费时交易 k 恒定, 含费时每笔交易 k 单调递增, 这是 LP 的全部收益来源

V2 池两边美元价值必然相等（可用作自检）： $4,640.95 \times 1911.25 \approx 8,869,000$

做了什么

全程用 cast 直读链上，没有依赖任何前端：

1. getPair() 从 Factory 查出 WETH/USDC V2 池地址
2. getReserves() 读储备量
3. token0() 确认顺序（USDC 为 token0）—— 查证而非按地址大小推断
4. decimals() 分别查出 6 和 18
5. 手算边际价格： $\text{raw0}/\text{raw1} \times 1\text{e}12 = 1911.25 \text{ USDC/ETH}$ （ $12 = 186$ ）
6. 含费公式手算 100 USDC 的产出
7. getAmountOut() 调合约对答案

得到什么结果

量	值
边际价格	1911.25 USDC/ETH
有效成交价	1917.02 USDC/ETH
两者之差	5.77，相对 0.302%
手算产出	0.0521643346 WETH
合约返回	52164334593439524 wei

遇到什么失败 / 卡点

最大的卡点：AI 讲解里的错误

四个概念问题问下来，共找出并纠正 13 条。其中会直接导致手算失败的有两条：

1. getAmountOut 漏了手续费 —— 给的是零手续费理想公式，却声称是 Uniswap V2 的实现
2. 有效成交价公式方向写反 —— 总结表里写 $\text{amountOut}/\text{amountIn}$ ，与它自己例子里的算法矛盾，量纲正好是倒数

概念层错误：

$|dy/dx| = k/x^2$ 随 x 增加而减小，它写成了增加，且与自己后文矛盾

两次用经济学供需解释 AMM 价格变化（AMM 不做价值判断，是守恒式的机械结果）

把「套利使价格回归公允价」当作曲线的数学约束（那是市场行为结果，不是设计要求）

讲套利收敛时不提成本，忽略无套利带

计算与事实错误：

2000/0.9025 算成 2215.01（应为 2216.07），而且自己打了“验证”—— 比不验证更危险

把 $x^2+y^2=k$ 称为「指数形式」（应为二次型）

编造了圆形曲线的 dy 公式，化简后恰好等于 $xy=k$ 的结果

「只有双曲线取到最佳平衡」与它自己提的 Curve / Balancer 矛盾

应对：写了 13 条约束进 Hermes memory

涵盖：禁用歧义表述、不得用供需解释、讲收敛必提成本、给公式必含手续费且标量纲、不确定的公式不要编、不把演算草稿留在答案里。

纠正有效：后两问中它主动标了量纲、主动说出手续费使 k 增大是 LP 收益来源、主动标注了例子的交易规模偏大。

其他卡点

两次读储备量数字不同（中间有人交易）—— 手算与验证必须锁定同一组数据，否则对不上会误以为公式错

验证方式从「对比前端报价」改为「调合约纯函数」—— 前端可能收 interface fee，会引入无关差异

下一步验证什么

Day 3 : 同一个池分别报价 100 / 1k / 10k / 100k 美元，做一张「规模 → 实际成交价 → 相对现价损耗%」的表。
已知今天的水分岭在 26,690 USDC，验证：跨过它之后曲线是否如预期加速恶化。

笔记 34

作者：journeyanhk | 时间：2026-08-06 09:31 UTC | 字数：2091

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 学习笔记：LI.FI API 实操 + Route 比较 + 成本模型

学习内容

基于 Bruce 老师教程「从发现价差到执行：如何接入 LI.FI 实现跨链套利和 AI Agent 集成」，完成两个特别课时的学习。

特别课 Day 1：LI.FI API 基础实操

1. LI.FI API 注册与测试

- 注册 Partner Portal，获取 API Key
- 验证 /v1/keys/test，确认 Rate Limit 100 req/min
- 查询 /v1/chains：LI.FI 支持 69 条链
- 查询 /v1/tokens：确认 USDC 在 ETH (\$0.9993) 和 Arbitrum (\$1.0005) 的地址和精度
- 查询 /v1/tools：35 个 Bridge + 35 个 DEX

2. 复现 Bruce 的 Quote 实验

- 调用 /v1/quote 获取 ETH USDC -> Arbitrum USDC 的跨链报价
- Route 工具: Eco，LI.FI Service Fee 25 bps (0.25%)
- 1,000 USDC 到手 997.50 USDC，Gas \$0.12，总成本 26.18 bps
- 验证了 Bruce 文章中的数据

3. 不同资金量成本对比

- \$100 -> 35.38 bps (Gas 占比高)
- \$1,000 -> 26.18 bps
- \$10,000 -> 25.10 bps (Gas 占比可忽略)
- 关键发现：Gas 是固定成本，小资金劣势明显

特别课 Day 2：Route 比较与成本模型

1. 多路线对比 (/v1/advanced/routes)

- ETH -> Arbitrum 1,000 USDC 共返回 16 条候选路线
- 最佳: Eco (37.6 bps, RECOMMENDED+CHEAPEST)
- 最快: relaydepository (38.7 bps, FASTEST)
- 最差: Symbiosis (44.3 bps)
- 最佳与最差差 6.6 bps，选错路线多花钱

2. 不同资金量最佳路线不同

- \$100/\$500 时 Across 最优 (Gas 最低 \$0.057)
- \$1,000+ 时 Eco 最优 (LI.FI Fee 后净到手最多)
- 结论：不能固定选一个 Bridge，要根据资金量动态选

3. 套利盈亏平衡模型

成本假设: LI.FI=25bps + DEX=25bps + 滑点=5bps + Gas

| 资金 | 跨链套利盈亏平衡 | BSC同链套利盈亏平衡 |

|-----|-----|

| \$100 | 68 bps (0.68%) | 31 bps |

\$500	58 bps (0.58%)	31 bps
\$1,000	57 bps (0.57%)	31 bps
\$10,000	56 bps (0.56%)	31 bps

- 跨链套利真实门槛 ~57 bps (不只是 LI.FI 的 25 bps)
- BSC 同链套利门槛仅 31 bps (无 LI.FI Fee + Gas 极低)
- Gas 制造了「最低本金门槛」：60 bps 价差需 $\geq \$240$

实操成果

完成 LI.FI API Key 注册和连通性测试
编写 01_lifi_quote.py：Quote 查询 + 成本解析脚本
编写 02_route_comparison.py：多路线对比 + 盈亏平衡模拟表
复现了 Bruce 老师教程中的全部实验数据

关键收获

1. 发现价差只是开始：必须扣除 LI.FI Fee + Gas + DEX Fee + 滑点 + 延迟风险后才是利润
2. BSC 同链套利是小资金最佳起点：盈亏平衡 31 bps，远低于跨链的 57 bps
3. Route 选择影响显著：最佳与最差路线差 6.6 bps
4. 预置资金模式优于临时跨链：消除延迟风险，调仓成本可分摊
5. Gas 是固定成本：制造了最低本金门槛，但大资金可忽略

明日计划

Hermes + LI.FI MCP Server 集成
测试 MCP 暴露的工具：get-chains、get-token、get-quote、get-status
设计 Agent 辅助的 Route 评估 workflow

笔记 35

作者：Cz07cring | 时间：2026-08-06 11:30 UTC | 字数：2082

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 Day 2 | 跨所套利框架 M1-M5 全部落地 + 真实套利信号验证

今天完成了什么

从 Day 1 的 QQQB LP 策略假设，今天把「链上套利框架」从设计推进到了可运行的完整代码库：/Users/ring/量化框架/跨所套利框架（纯 Go，基于 LI.FI 的链上/跨链价差套利，14364 行代码，43 个测试文件，19 个包 21 个 Go 包全部测试通过）。

五个里程碑全部完成（严格 TDD：先写失败测试再写实现）

1. M1 数据层：bbolt 键值存储、Token 注册表（LI.FI /tokens 同步 + 持久化）、链配置与逐链认证状态机。
2. M2 报价扫描引擎：并发扫描器（令牌桶限流 + 每链并发上限 + 429 退避 + 熔断）、双腿报价组合与净利润计算（费用去重 + 双门槛 + TTL）、报价健康检测（同链往返防伪 + 异常工具隔离）、Agent 套利发现（DiscoverOpportunities，参考 MetaMask Agent Wallet 思路）。
3. M3 账本与影子记录：持仓/余额/库存账本（预留/释放/过期清扫）、影子质量报告（猎杀嫌疑检测 + 工具统计去重）、shadow-report CLI。
4. M4 治理层：人工审批队列 + 哈希链防篡改审计日志（照搬 MetaMask Agent Guard 模式：白名单 + 24h 流出上限，白名单外人工批准）、逐链认证收严（policy 硬规则 + config 逐链覆盖）。
5. M5 链模拟与执行：EVM 只读模拟适配器（eth_call 模拟 + gas 估算 + allowance 检查，永不签名广播）、双腿执行状态机（12 状态、UNKNOWN_RESULT 禁止自动重发、重启安全回退人工审查）。

今天踩过并修掉的坑（都值得记）

read_file 二进制误判：干净 UTF-8 Go 文件被误判为二进制，改用 python 直读。

LI.FI API key 大小写：请求头必须 X-LiFi-Api-Key（大写敏感），小写返回 1010 Invalid API key——是头名错误不是 key 无效。

bbolt 扫描回调死锁：store.Scan 持互斥锁时回调内不能 Put（同锁重入死锁），改为先收集后写回。

shadow 报告统计语义：工具报价计数按候选去重（双腿同工具只计一次），猎杀嫌疑按去重后工具数判断。

kimi-k3 中转不稳定：多个 worker 因 Cloudflare 524 超时，CEO 接手补齐了审批队列、执行状态机、EVM 适配器。

测试监控：真实套利信号验证（今天最有价值的部分）

用真实数据验证了整套监控链能抓到套利信息：

1. CEX 费率套利信号链在产出：188 个信号，Top 3：
 - ESports 库币合约→币安：日化净收益 1.54%，7 所共识，最小端量 404 万
 - CAP Bybit 合约→币安：日化净收益 1.30%，10 所共识，最小端量 1751 万
 - COTI Bybit 合约→币安：日化净收益 1.15%，6 所共识，最小端量 1333 万
2. 链上 LI.FI 数据链实测可用：
 - 同链 USDC→DAI：1000 USDC 换 997.55 DAI（摩擦约 0.4% = LI.FI 固定费 0.25% + gas）
 - 跨链 ETH→Arbitrum 桥 10000 USDC：输出 9975（摩擦 0.25%）
 - 结论：真实机会出现在跨链同资产价差 > 桥费 + gas 时
3. 性能与自动化：正在做 scanner HTTP 连接复用 + benchmark 基线、scan 命令（真实 LI.FI 扫描落库）。

学习收获

架构文档先行：先把 MetaMask Agent Wallet 的 Guard 模式（网络/地址/代币三白名单 + 24h 滚动流出上限）吸收进自己的框架设计，再动手写代码，避免返工。

TDD 的纪律价值：5

个里程碑全部先写失败测试，每次都能在编译错误阶段暴露接口设计问题，比事后调试便宜得多。

套利框架的核心认知：差价再大，开不进去仓位就是零——流动性（最小端成交量）和摩擦成本（桥费 + gas + 滑点）决定了真实可执行收益。这和 CEX 费率套利是同一套逻辑：收益 = 差价 × 仓位，先算清可开仓位。

下一步

scan 命令自动化链上跨链价差扫描（worker 开发中）

影子数据积累 24-72 小时，形成质量基线

逐链小额认证测试（按设计仅模拟态，不碰真实资金）

学习记录，非投资建议。

笔记 36

作者：wang wanlu | 时间：2026-08-05 08:57 UTC | 字数：1994

来源：<https://intensivecolearn.org/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要搭建了一套独立、可复用的

Hermes

智能分析平台基础架构。它不属于某一个具体业务，而是可以同时服务多个系统和多个领域模块。

1. 独立 Hermes Core

将 Hermes 作为独立服务管理：

业务系统

↓

Hermes Bridge

↓

Hermes Core

↓

模型服务（未来）

业务系统保留自己的数据采集、计算和业务逻辑。Hermes 只接收经过整理的事件、特征和观察数据，不直接读取业务数据库。

2. 多系统注册机制

建立了 System Manifest 和 Domain Profile :

System 表示一个完整业务体系。

Domain 表示体系中的一个独立分析领域。

一个 System 可以包含多个 Domain。

已完成的 Domain 标记为 active。

尚未实现的 Domain 标记为 reserved。

新系统可以通过增加配置接入，不需要复制 Hermes Core。

3. 分析契约

定义了三类核心契约：

System Manifest：声明系统和 Domain。

Analysis Request：业务系统向 Hermes 提交分析材料。

Analysis Result：Hermes 返回分析结果。

请求中包含：

系统和 Domain 标识

契约版本

调用方身份

Observation、Feature、Event

数据置信度和证据状态

Correlation ID 和 Causation ID

这些契约可以防止不同项目随意传递不兼容的数据。

4. Registry 与 Policy 分离

Registry 负责回答：

系统和 Domain 是否已经注册？

Policy Engine 负责回答：

当前调用是否被允许？

已经加入的策略检查包括：

调用方必须属于对应系统。

Domain 必须处于启用状态。

请求版本必须匹配。

输入类型必须在 Domain 白名单中。

未完成的 Domain 不能被调用。

请求能力不等于获得能力。

5. AI 与执行权限分离

Hermes 当前被定义为建议型分析系统：

decision_authority = advisory

execution_permitted = false

这意味着 Hermes 可以：

总结情况

解释异常

推测可能原因

关联历史模式

但不能：

调用钱包

签署交易

转移资金

修改策略

直接执行操作

即使未来接入大模型，大模型输出也不能直接成为执行授权。

6. 默认关闭模型服务

当前 Model Provider 使用 Disabled Implementation：

不调用外部模型。

不需要模型 API Key。

请求会返回明确的禁用状态。

先验证架构和安全边界，再决定使用哪个模型。

这是典型的 secure by default：未明确启用的能力默认关闭。

7. Docker 安全隔离

Hermes 在独立 Docker 容器中运行，并配置了：

非 root 用户运行

只读根文件系统

删除全部 Linux Capabilities

no-new-privileges

限制 CPU、内存和进程数量

不挂载宿主机目录

不挂载 Docker Socket

不读取宿主机 .ssh

不读取业务项目 .env

Docker 不是云服务，它仍然运行在本机，但通过独立 Linux 环境减少对 macOS 主机文件和进程的访问。

8. 服务间隔离网络

业务服务与 Hermes 通过专用 Docker Network 通信：

Business Service

↓

Isolated Docker Network

↓

Hermes API

它们共享的是版本化 API，不是：

文件夹

数据库账号

环境变量

私钥

整个运行环境

9. 验证与测试

增加了针对以下情况的测试：

- 契约格式是否正确
- 未注册系统是否被拒绝
- 未启用 Domain 是否被拒绝
- 冒用其他系统身份是否被拒绝
- 非法输入类型是否被拒绝
- AI 结果是否试图取得执行权限
- 响应置信度是否超过限制
- 聊天命令是否包含危险操作
- Docker 配置是否符合隔离要求

笔记 37

作者：Leon | 时间：2026-08-05 15:23 UTC | 字数：1982
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习计划

借助 AI 制定这 21 天的学习计划。每天投入1-2h，预期会随进度和对主题的理解调整计划。

- 1. 第一周 · 建地图、立模型、装组件 (8/5 – 8/11)
 - Day 1 · 8/5 周三 · 基础认知 + 环境配置
 - Day 2 · 8/6 周四 · AMM 第一性原理
 - Day 3 · 8/7 周五 · 成本模型 v0
 - Day 4 · 8/8 周六 · LI.FI (<http://li.fi/>) 打通
 - Day 5 · 8/9 周日 · 数据源：链上真值 vs API
 - Day 6 · 8/10 周一 · 三角套利：第一次真正扫描
 - Day 7 · 8/11 周二 · 竞争维度 + 第一周复盘
- 2. 第二周 · 广度调研、案例还原、管道成型 (8/12 – 8/18)
- 3. 第三周 · 深挖 edge、建系统、小额实测 (8/19 – 8/25)

Note: 借助 AI 自学，有些信息甚至学习方向都可能不准确，欢迎指正。

Day 1 · 基础认知 + 环境配置

基础认知

- 1. 如何判断机会是否成立
 - 1. 屏幕上看见价差 = 一条线索。模拟后为正 = 一个候选。 $E[PnL] > \text{阈值}$ 才叫一个机会。
 - 1. 警惕成本重复计算的陷阱：每个成本项在账里只能出现一次
 - 1. 如果用聚合器的到账量算终局市值，就不要再单独减桥费和 swap fee —— 那已经在里面了。报价返回的费用明细字段（如 feeCosts）是给你做归因看的拆解，不是让你再减一遍。
 - 2. 不同聚合器口径不同
 - 2. 原子 vs. 非原子：决定竞争对手
 - 1. 原子：同链，一笔交易内闭环——竞争对手专业强势，新手基本没有 edge。拿来学原理。
 - 2. 非原子：跨链 / CEX-DEX——竞争相对较弱，有更多机会
 - 1. 交易的不是「发现价差的眼力」，而是「愿意在 Δ 时间里替市场承担价格风险」。价差是这份风险的报酬。
 - 2. 判断标准不是「有没有价差」，而是「这份报酬配得上这份风险吗」。
- 3. 机会地图
 - | 类型 | 为什么存在 | 最容易死在哪 |
 - | :----- | :----- | :----- |
 - | DEX 内 / DEX 间 | 同一资产在不同池子的定价曲线独立更新 | 被 searcher

在同一区块抢先；池子太浅，滑点吃光利润 |
三角套利	多跳路径的隐含汇率与直连汇率不一致	三次手续费 + 三次滑点叠加；Gas 随跳数线性上升
跨链套利	各链流动性割裂，价格同步靠人搬运	桥费 + 桥延迟 + 期间价格反向走；桥本身的安全风险
稳定币 / peg 偏离	赎回通道有摩擦、有门槛、有时间成本	偏离不回归（真脱锚）；赎回资格你根本没有
清算	协议用奖励购买「有人及时来平掉坏账」这项服务	
清算机器人成熟且极卷；需要即时资金和抵押品处置能力		
MEV (backrun / sandwich / JIT)	交易顺序本身有价值，区块空间可竞价	
需要私有通道和极低延迟；散户基础设施差一个数量级		
CEX-DEX 价差	两套市场结构、两套参与者、两套资金效率	
双边库存占用；提币暂停/延迟；资金费率与账户风险 |

环境配置

1. Python 虚拟环境 + uv 管理库
2. Alchemy 账户申请：
 1. 1 个 APP 开启了 Ethereum + Base + Arbitrum 3条 Mainnet 链
 2. python script 跑通了 3 条链的自检

笔记 38

作者：HenryChang | 时间：2026-08-06 12:05 UTC | 字数：1980

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 | LI.FI 真實成本實測：25 bps 如何殺死紙上價差

今天以發起人 Bruce 的長文《從發現價差到執行：接入 LI.FI 實現跨鏈套利和 AI Agent 集成》為教材，完成了文末的只讀實驗（只查詢、不簽名、不廣播），全部數據為今日實測。

概念收穫

Break-even Spread 心智模型：最低所需價差 = LI.FI 服務費 + Bridge/DEX 費 + 兩鏈 Gas + 兩端滑點與 Price Impact + 延遲/失敗/資金占用成本。表面價差沒過這條線，一切都是紙上富貴

LI.FI 的定位：執行層（資金怎麼走、成本多少、交易怎麼構造），不是策略層（哪裡有機會）。這個分層日後會直接對應到 bot 的程式架構

跨鏈非原子性：等資金跨過去，機會早沒了。務實打法是多鏈預置庫存、兩端先成交、事後用 LI.FI 做再平衡

實測數據（uv + httpx 腳本，資料存 repo 的 research/data/）

1. Token Service——同名 USDC 有三種價格：

Token chain priceUSD
--- --- ---
ETH USDC 1 1.0000147
ARB 原生 USDC 42161 1.0003856
ARB USDC.e (橋接版) 42161 1.0011080

三個「USDC」價差近 11 bps！但這是價格 feed 的估值差，不等於可成交價差——要驗證得看實際 pool 深度。教訓：搞錯資產版本（USDC vs USDC.e），後面的價格比較全部失真。

2. /quote 實測（ETH → ARB，slippage 0.5%）：

名目 路線 到手 LI.FI 費 Gas 耗時 顯性成本
--- --- --- --- --- ---
1,000 USDC Polymer Std 997.50 \$2.50 (25 bps) \$0.246 1080 秒 27.46 bps
10,000 USDC Eco 9,975.00 \$25.00 (25 bps) \$0.199 7 秒 25.20 bps

對照 Bruce 昨天同參數的 quote (Eco、7 秒) : /quote 今天給我的 1k 檔位竟是要 18 分鐘的 Polymer 路線。Quote 是時變的，同參數不同時點結果就不同。

3. /advanced/routes——15 條候選路徑的殘酷比較：

Polymer Standard : 到手 997.50，但 1080 秒

Eco : 到手一樣 997.50，只要 7 秒 (RECOMMENDED + CHEAPEST)

AcrossV4 : 到手 997.39 (少 11 美分)，2 秒

今天最大的三個發現

1. 不能信單一 /quote : 它推薦的路線跟 Eco 到手金額一模一樣、卻慢 154 倍。對套利來說延遲是硬約束，必須自己拉 /advanced/routes 比較，把「耗時」當一級指標
2. 規模只攤薄 Gas，攤不掉費率：25 bps 是按比例收的，1k 與 10k 的總成本只差 2.26 bps。免費費率下，純跨鏈搬磚的 break-even 就是 25 bps 起跳，再加兩端滑點與延遲風險——難怪 Bruce 說要用庫存再平衡模式，把跨鏈從「抓機會的路徑」改成「調倉的後勤」
3. 速度與到手的取捨是真實存在的：AcrossV4 少收 11 美分換 2 秒到帳。若機會窗口只有幾秒，這 1.1 bps 就是值得付的保險費

工程進度

用 uv 建 Python 環境，實驗腳本 scripts/lifi_experiment.py (PEP 723 inline 依賴，uv run 直接執行)，原始回應 JSON 全部落地存檔

LI.FI 官方 MCP (mcp.li.quest) 已接入開發環境，之後查鏈/幣/報價不用手寫 curl

Rate limit 治理：本次實驗只用 3 次額度 (未認證上限 75 次/2h)，慢變資料已快取到本地

明日計畫

回到鏈上第一手資料：用 RPC 讀 Uniswap V2 pool 的 reserves，手算現貨價格

把今天的 break-even 思維帶進去：算出同鏈 DEX 間價差要多大才過得了成本線

笔记 39

作者：bai·小白 | 时间：2026-08-05 13:04 UTC | 字数：1727

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

我今天整理的几篇 Flashbots 文档，分别讲 Auction、交易入口和 searcher Quick Start。单独看，每一页都在解释自己的那一段。把它们接起来，才看见一笔交易从钱包出发，到进入区块，中间到底经过了多少只手。

以太坊每个区块的位置有限，交易通常先进入公开 mempool。它还没上链，内容已经能被别人看到。机器人发现一笔大额交易，可以赶在前面买，也可以在前后各放一笔交易，专门吃掉中间的价格变化。套利、清算、抢跑、夹击，都和交易顺序有关。顺序带来的那部分额外收益，就是 MEV。

Flashbots 承认这笔钱会出现，然后给参与者搭了一套新的交易方式。原先的场面像售票大厅里公开喊价，所有人盯着彼此不断加钱。Flashbots 把一部分交易接进私有通道，参与者提交交易和报价，Builder 再从中挑选和组合。

普通钱包用户接触的是 rpc.flashbots.net。它能放进钱包的 RPC 设置里，钱包依然用 eth_sendRawTransaction 发交易。用户不需要理解 Bundle，也不用自己处理 Builder。交易会尽量避开公开 mempool，减少被抢跑和夹击的机会，有些交易还能拿到 MEV refund。

写机器人和 DApp 的人用 relay.flashbots.net。这个入口不负责查余额、查区块，更像一个专门收私有交易的窗口。只发一笔交易，可以调用 eth_sendPrivateTransaction。几笔交易需要按顺序执行，可以用经典的 eth_sendBundle。需要更复杂的隐私设置、有效区块和收益分配，再去看 mev_sendBundle。

Bundle 这个词很容易让人以为只是批量发送。它处理的是顺序和条件。一次套利可能包括借钱、买入、卖出、还款。中间任意一步的位置变了，后面的利润都会跟着变。Searcher 把这些交易按顺序交出去，Builder 先模拟。条件不满足，这个 Bundle 就没有进入区块的必要。

一笔 Bundle 往后还要经过几个人。Searcher 负责找机会和写交易，Builder 把公开交易与私有 Bundle 组合成候选区块，Relay 接收并转交这些区块，Validator 最后选择一个区块提议上链。域名里的 relay 和这套流程中的 Relay 很容易混在一起。前者是 searcher 调用的地址，后者是区块生产流程中的一个角色。

Quick Start 里最值得单独记下来的，是两把密钥要分开。authSigner 只给 Flashbots 请求签名，用来识别身份和积累

Reputation。它像一张门禁卡，不负责管钱。控制资金的钱包负责给链上交易签名，那才是银行卡。普通 Provider 继续查询区块、Nonce 和余额，签好的交易按顺序装进 Bundle，再指定目标区块提交。

提交成功也只说明 Flashbots 收到了请求。Builder 还要模拟，别的 Searcher 也在报价，目标区块到来时链上状态可能已经变化。Bundle

没有进入目标区块，机器人往往还要更新状态，换一个目标区块继续提交。接口跑通以后，重投、限流和 Nonce 才是日常工作。

私有交易也有边界。它不会出现在公开 mempool 里，接收交易的 Builder 或 Relay 仍可能看到内容。把 Bundle 同时发给更多 Builders，通常能增加被打包的机会，交易内容也会经过更多参与者。这里没有免费的选择，只能在触达率和信息暴露之间取舍。

学到这里，那些零散名词终于排成了一条实际流程。普通用户换一个 RPC，Searcher 准备身份密钥、Provider 和 Bundle，Builder 组区块，Relay 转交，Validator 上链。每个角色都在处理同一件事，谁能看见交易，谁有权安排顺序，最后赚到的钱怎样分。

下一步该亲手发一次 Bundle。先把身份密钥和资金钱包分清，再完成模拟、提交和结果检查。代码跑过这段流程以后，文档里的几个地址和方法就不再是一串相似的术语。

笔记 40

作者：gm365 | 时间：2026-08-05 03:23 UTC | 字数：1672

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

RobinHood Chain 套利机会监控与反馈

使用 Codex 写代码，之后持续 48 小时监控了RobinHood Chain 热门池子的双边套利机会。比如：

V3 → V4 → V3

V3 → V4 → V4

V4 → V3 → V3

V4 → V3 → V4

结果汇总如下：

当前统计

覆盖约 46.7 小时：

| 指标 | 结果 |

| :----- | :----- |

| Swap 事件 | 89,151 |

| V3 / V4 事件 | 59,758 / 29,393 |

| 扫描批次 | 10,403 |

完整报价	37,159 / 41,612 行, 89.3%
毛正收益记录	2,169
扣 gas 后正收益记录	531
净正收益批次	445, 约 4.28%
按 120 秒合并的候选时段	185
估算 WS CU	3.66M / 24M, 15.3%
程序级扫描失败	0

gas 淘汰了约 75.5% 的毛正收益记录, 说明只看池间价格差会严重高估机会。

结论摘要 :

监控在约 46.79 小时内记录了 89,272 个目标池 Swap 事件和 10,414 个扫描批次。447 个批次至少包含一条扣除估算 gas 后为正的报价, 共合并为 185 个候选时段。样本量已经足够证明“报价级失衡候选存在”, 继续使用相同方法运行到 72 小时只会提高频率估计精度, 无法解决可执行性问题, 因此本轮提前停止是合理的。

假说判定

| 假说 | 判定 | 依据 |

| :----- | :- | :----- |

| CASHCAT 跨版本存在报价级失衡 | 支持 | 447 个净正批次、185 个候选时段 |

| 行情转弱后候选必然减少 | 不支持 | 最后 6 小时净正批次率为 5.62% |

| 当前候选可以原子执行 | 未验证 | 没有同区块、准确池、整笔 eth_call |

| 报价候选等于可实现利润 | 否定 | 双向同时盈利、极端 bps、多跳污染 |

| Alchemy Free Plan 足以完成发现阶段 | 支持 | WebSocket CU 仅使用本地上限的 15.28% |

| 继续收集同类报价能解决可执行性 | 否定 | 核心缺口是验证方法而非样本量 |

个人最终结论 :

不足以论证 :

RobinHood

Chain

DEX

双边套利机会持续存在较长的机会窗口。可能需要更好的基建 (RPC) 或多边套利机会探索。

笔记 41

作者 : fanchen621 | 时间 : 2026-08-06 15:29 UTC | 字数 : 1649

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天把“看见价差”推进到了“可验证、可拒绝、可持续前向观察”的阶段。【完整处理群内信息】通过 CipherTalk 按 Asia/Shanghai 时间完整导出并逐条处理今天群消息, 最终覆盖 00:04:21—22:27:01, 共 894 条唯一消息, 覆盖率 894/894; 其中 348 条实质内容归入 15 个研究主题, 并逐条绑定到实际实验、现有证据或明确的 intake blocker, 另完整展开 44 条合并聊天记录; 做了删行负测, 故意删除 1 条后独立 verifier 正确 FAIL, 证明不是“看起来全覆盖”; 仍有 38

张图片因本机缺少图片解密密钥、3 个附件原件字节不可得，均明确保留为缺口，没有冒充已读取。【Base 三角套利主线】路线完全直连：Aave V3 flashLoanSimple USDC → Curve USDC/WETH → Aerodrome Slipstream WETH/USDT → Uniswap V3 USDT/USDC → 原子归还 Aave。规则已冻结为：方向切换第一拍不追，连续两个同方向样本后重新构造并精确模拟；所有结果均为 paper-only，无钱包、签名和真实订单。截至今晚：严格 OOS 记录 119 条、因果尝试 112 次、paper fills 108 次、inclusion loss 3 次、无边际拒绝 7 次、额外执行成本拒绝 1 次；在封块结果基础上再扣 5bps 竞争/执行成本后，累计 paper 净值 +196.845972 USDC。进一步只用这批 untouched OOS 尝试做成本阶梯：5bps 为 +196.846U、正收益尝试 96.43%；6bps 为 +146.346U、87.50%；8bps 为 +45.346U、50.89%；10bps 为 -55.654U、24.11%。因此当前执行成本断点落在 8—10bps，这比单纯报告“有价差”更接近真实可执行边界。【群内新增想法逐项实践】CASHCAT 所谓跨场 5%+ 当前未复现；扣 MEXC 现货/永续手续费及 10bps 储备后，100U 最好约 +31.11bps，1000U 约 +14.75bps，反向为负，且 MEXC 永续元数据当前 apiAllowed=false，暂不能作为自动执行路线。LI.FI API key：不带 key 的请求能进入路由生成并返回 422/1001；带错误的“看起来像 key 的值”会直接 401/1010，结论是普通研究调用不必强行带 key，错误 key 反而阻断请求。两个 Solana 地址：地址一最近交易没有同槽原子闭环和 Jito tip；地址二最近 20 笔全部失败，不能证明稳定尾盘套利。Sui 开源 bot：确认存在多 DEX PTB 原子构造和提交前 dry-run，但旧 JSON-RPC 已过时，且入口直接接收私钥；只提取机制，下一步迁移到 GraphQL/gRPC 的无密钥模拟环境。事件触发报价：三池全监听只比固定 5 秒轮询少约 4.9% 调用；只监听 Uniswap 池又漏掉同期方向切换，所以不替换当前 watcher。QQB LP：正确池当前约 324 万美元流动性、1.46 亿美元 24h 成交，理论日费收约 0.45% TVL；但收益高度依赖活动期成交量，成交衰减和 CLMM 出区间仍是主要风险。【今天的核心结论】最有价值的不是继续收集更多“可能有价差”的截图，而是已经形成一条可重复链路：群消息 → 假设 → 精确构造 → pending/封块验证 → OOS paper 账本 → 执行成本断点 → 自动持续观察。下一步只沿主线推进：继续积累 untouched OOS，并验证 builder/private-orderflow、延迟和失败尝试能否长期控制在 8bps 以内。所有数据仍是研究与 paper trading，不是实盘收益。

笔记 42

作者：hellingercheri-gif | 时间：2026-08-05 14:37 UTC | 字数：1602

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

三明治攻击

Direct & Immediate Answer

三明治攻击（Sandwich Attack）是去中心化金融（DeFi）中常见的一种最大可提取价值（MEV）套利策略。攻击者通过监控内存池（Mempool）发现普通用户的待处理买卖交易，利用更高的矿工费“抢跑”买入抬高价格，再用“尾随”交易高价卖出，将受害者夹在中间，迫使其承受高滑点并从中牟利。 [1 (<https://medium.com/@communitybca/%E4%B8%89%E6%98%8E%E6%B2%BB%E6%94%BB%E6%93%8A-%E8%A7%80%E9%BB%9E%E8%A7%A3%E6%9E%90-69ee9a89ff10>), 2 (<https://www.gate.com/zh/learn/articles/what-is-a-sandwich-attack/936>), 3 (<https://www.panewslab.com/zh/articles/N3518298>), 4 (https://x.com/ai_9684xtpa/status/1510117805324079110)]

攻击运行的过程

监控交易（监听）：攻击者的自动化程序（夹子机器人）在区块链内存池中扫描大额或高滑点容忍度的买入订单。

[1 (<https://mdnice.com/writing/cc5b150bd2504a9bb089d94454c2b905>), 2 (<https://www.panewslab.com/zh/articles/N3518298>)]

抢先交易（Front-running）：攻击者设定更高的交易手续费（Gas

Fee），让自己的买入交易排在受害者前面，先一步买入代币并推高代币市场价格。 [1 (<https://rivers.chaitin.cn/blog/cq950hh0lnechd244fo0>), 2 (<https://medium.com/@communitybca/%E4%B8%89%E6%98%8E%E6%B2%BB%E6%94%BB%E6%93%8A-%E8%A7%80%E9%BB%9E%E8%A7%A3%E6%9E%90-69ee9a89ff10>)]

0)]

受害者成交（受损）：受害者的交易随后执行，由于价格已被攻击者抬高，受害者买入的成本显著变高，遭受严重的价格滑点。 [1 (<https://medium.com/@communitybca/%E4%B8%89%E6%98%8E%E6%B2%BB%E6%94%BB%E6%93%8A-%E8%A7%80%E9%BB%9E%E8%A7%A3%E6%9E%90-69ee9a89ff10>)]

尾随交易（Back-running）：攻击者紧接着在受害者交易后面再塞入一笔卖出交易，将刚才低价买入的代币高价卖给受害者，收割差价利润。 [1 (<https://www.gate.com/zh/learn/articles/what-is-a-sandwich-attack/936>), 2 (<https://www.panewslab.com/zh/articles/N3518298>)]

如何防范三明治攻击

设置严格的滑点限制：交易时调低滑点容忍度（例如设为 0.5% 或更低），如果价格变动超出范围交易会取消，拒绝被严重夹击。

使用防夹服务/私有交易池：通过专门的防MEV RPC节点（如 Flashbots）提交交易，绕过公共内存池直接发送给验证者，防止机器人拦截。

小额分批交易：避免一次性大额买入低流动性代币（如小市值 Meme 币），降低被盯上的概率。

笔记 43

作者：qingzhou | 时间：2026-08-05 15:38 UTC | 字数：1567

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. 套利的经济学定义

利用同一资产在不同市场的价格差异，通过同时买入和卖出获取无风险利润的行为。

2. 为什么存在套利机会（信息不对称、时间延迟、操作成本）

3. 链上套利 vs 传统套利

差异1：从“无风险”到“高度对抗”

传统：发现价差 → 执行套利 → 获利

链上：发现价差 → 提交交易 → 被抢跑/被夹击/Gas战争 → 大概率失败

差异2：从“信息优势”到“执行优势”

传统：谁先知道价差，谁赚钱

链上：所有人都能看到价差（Mempool公开），谁执行最快，谁赚钱

差异3：从“人工监控”到“算法战争”

传统：人可以手动发现并执行套利

链上：必须用机器人，且机器人之间相互竞争

4. 竞争格局：你的对手是谁？他们的优势？你的可能优势？

第一梯队：专业MEV团队

代表：Flashbots、Eden Network、bloXroute

优势：

直接和验证者合作，绕过公开Mempool

拥有私有交易通道（Private RPC）

资金规模：千万美元级别

延迟优势：直连验证者节点

你的胜算：接近零（不同赛道）

第二梯队：量化基金和做市商

代表：Jump Crypto、Wintermute、Alameda（已倒闭）

优势：

资金规模：亿美元级别

低延迟基础设施 (AWS专线、共址服务器)

专业团队 (quant + 工程师)

套利+做市+清算的综合策略

你的胜算：极低 (除非找到他们忽略的长尾市场)

第三梯队：独立开发者和小团队

代表：GitHub上的开源套利机器人作者

优势：

灵活性高，能快速测试新策略

成本低 (个人服务器)

可以专注长尾市场

你的胜算：有限但存在 (竞争的是同一批机会)

第四梯队：新手和业余爱好者

现状：大部分在亏钱 (Gas费 > 收益)

你的初始位置：这里

你的可能优势

既然前面都是劣势，那还有什么机会？

优势1：长尾市场 (The Long Tail)

大玩家不屑的小额套利

利润 < \$100的机会，专业团队不会动手 (成本不划算)

你可以用低成本策略 (Polygon、Arbitrum等L2)

优势2：新兴链和新协议

信息优势窗口

新链 (如 Scroll、Base) 刚上线时，专业团队还没布局

新DEX (如某个创新AMM) 的早期阶段

窗口期只有2-4周

优势3：创新策略

组合套利

闪电贷 + 跨DEX套利 + 清算

专业团队可能没考虑的复杂路径

优势4：专业化细分

成为某个垂直领域的专家

只做稳定币套利 (USDC/USDT/DAI)

只做NFT定价套利

只做某条链 (如专注BSC)

优势5：时区优势

美东时间凌晨3-6点

欧美专业团队下班

亚洲个人开发者的机会窗口

但这个优势正在消失 (团队全球化)

5. 红海/黄海/蓝海机会地图

蓝海 新公链套利 创新AMM GameFi套利 RWA套利 社交代币Intent-based交易

6. 三条成功路径：速度流、创新流、专业化流

7. 现实检验：不要指望暴富

大部分人在前6个月是亏钱的

8. 正确的学习目标设定

第一年目标：学会基本技术栈，不亏钱

第二年目标：年化收益20-50%（相当于传统量化基金）

长期目标：建立某个细分市场的信息优势，月稳定收益\$1000-3000

笔记 44

作者：SimonQuant | 时间：2026-08-05 15:04 UTC | 字数：1469

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利地图

链上套利的本质，不是发现价差，而是确认：

净收益 = 价差收益 - 交易手续费 - Gas - 滑点 - 价格冲击 - 延迟和失败成本

只有扣除全部成本后仍然有足够利润，才是真正可执行的套利机会。

常见套利类型

1. DEX 内套利

利用同一个 DEX 内不同资金池、不同手续费池或不同交易路径之间的价格差。

例如，USDC 直接兑换 ETH，可能和 USDC 先兑换 USDT、再兑换 ETH 的结果不同。

2. DEX 间套利

在价格较低的 DEX 买入，在价格较高的 DEX 卖出。

最好在同一笔原子交易中完成。这样只会出现两种结果：全部成功，或者全部失败，不会留下单边仓位。

3. 三角套利

通过三个交易对形成闭环。

例如：

USDC → ETH → BTC → USDC

如果最终得到的 USDC 多于最初投入，并且足以覆盖全部交易成本，就存在套利空间。

4. 跨链套利

利用同一资产在不同区块链上的价格差。

更可靠的方式是在不同链上提前准备资金，同时完成买入和卖出，再进行库存再平衡。

如果依赖跨链桥临时转账，资产到账之前价差可能已经消失。

5. 稳定币套利

利用 USDT、USDC、DAI 等稳定币的短暂脱锚或不同市场之间的报价差。

但稳定币折价也可能代表信用风险、赎回风险或流动性风险，并不一定是无风险机会。

6. 清算套利

当借款人的抵押率低于协议要求时，清算人替其偿还部分债务，以折扣获得抵押品，再卖出抵押品获利。

7. MEV 套利

利用区块内交易排序获利。

常见形式包括：

大额交易后的跟随套利

清算竞争

不同资金池之间的价格修复

抢跑和夹击交易

必须理解的执行概念

AMM

AMM 的价格会随着交易规模变化。

交易规模越大，通常会把价格推得越远，价格冲击也越大。

订单簿

不能只看买一价和卖一价。

真正需要计算的是，在目标交易规模下，全部订单成交后的平均价格。

流动性深度

屏幕上有价差，不代表能够成交足够大的金额。

一个资金池可能显示很大的价差，但真正可成交的规模很小。

滑点和价格冲击

价格冲击：自己的交易改变了市场价格。

滑点：实际成交价格与预期成交价格之间的差异。

区块确认

链上交易从提交到确认需要时间。

同一条链上的原子交易可以降低执行风险，但跨链交易通常无法保证两边同时完成。

屏幕价差和真实收益

假设屏幕上的价差是 0.30%。

但实际成本包括：

交易手续费：0.10%

滑点和价格冲击：0.12%

Gas 和失败风险：0.06%

最终净收益只有 0.02%。

这种利润通常不足以覆盖模型误差和执行风险。

看到价差后需要检查什么

1. 两边报价是否来自同一时间点
2. 实际能够成交多少金额
3. 是否能够原子完成
4. 全部成本是多少
5. 是否可能被其他机器人抢先
6. 失败后是否会留下单边仓位

链上套利的完整过程是：

发现价差 → 模拟成交 → 计算净收益 → 控制执行风险

真正的竞争壁垒通常不是发现价差，而是精确报价、低延迟执行、Gas 管理、库存管理和失败控制。

笔记 45

作者：elipitic | 时间：2026-08-05 15:45 UTC | 字数：1451

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡

打卡日期 (UTC+8)：2026-08-05

今日学习主题

今天围绕本期残酷共学的主线，开始用 The Graph 建立“观察 → 查询 → 判断”的最小闭环，重点不是直接找利润，而是把套利观察转成带数据证据的候选机会判断。

对照本期学习计划

今天对应的是本期执行计划中的：

- 明确以机会发现能力优先为主线
- 补 The Graph / 链上数据视角
- 围绕一个套利问题去查：pool 最近 swaps、某段时间流动性变化、是否存在异常冲击

今日执行内容

1. 学习与准备

- 阅读并理解了 The Graph 官方 Querying 相关文档
- 明确了 The Graph 在这次共学中的角色：不是直接找套利，而是作为“链上证据查询工具”
- 确认了后续查询思路：先观察异常，再用 GraphQL 查询做验证

2. 今日 demo 主题

完成了一个入门 demo：

- 目标：验证一个 subgraph 是否可查询，并拿到基础链上数据
- 使用 The Graph gateway 对指定 subgraph 发起了基础 GraphQL 查询

3. 今日实际跑通的查询

查询语句：

```
graphql
{
  poolManagers(first: 5) {
    id
    poolCount
    txCount
    totalVolumeUSD
  }
  bundles(first: 5) {
    id
    ethPriceUSD
  }
}
```

4. 今日实际查询结果

成功返回结果，说明今天的 The Graph 基础查询已经跑通：

```
poolManagers[0].id = 0x28e2ea090877bf75740558f6bfb36a5ffee9e9df
poolManagers[0].poolCount = 164532
poolManagers[0].txCount = 165949787
poolManagers[0].totalVolumeUSD = 108296649400.0302964768425235964929
bundles[0].ethPriceUSD = 600.9937420246514399991311159724168
```

今日结论

1. 我已经成功跑通了 The Graph 的基础查询，不再停留在文档理解阶段。
2. 我已经能确认：The Graph 适合用来做 pool / swaps / liquidity / 异常事件的链上证据补充。
3. 今天的查询还不是直接针对具体套利机会，但它完成了最关键的第一步：把文档学习推进到了真实查询执行。

下一步计划

下一步我会继续推进到更贴近套利机会判断的查询：

- 查某个具体 pool 最近 swaps
- 查 pool 基本信息 (TVL / feeTier / volumeUSD)
- 判断最近是否存在值得继续跟踪的大额 swap / 异常冲击

今日完成标准检查

- [x] 学习了 The Graph 的查询思路
- [x] 明确了它在本期共学中的用途
- [x] 跑通了一次基础 GraphQL 查询
- [x] 拿到了真实返回结果
- [x] 形成了下一步的具体查询方向

笔记 46

作者：solaren | 时间：2026-08-05 12:52 UTC | 字数：1261

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 Day 1：老鼠仓指纹与光模块制裁

@iamai_omni 发推称在 Hyperliquid 上研究资金费套利时"挖出了 alpha"，配图是 LITE (Lumentum) 股票盘前暴涨截图 (+11.91%)。顺着线索用 Hyperliquid 链上 API 完成全链路验证。

指纹：Hyperliquid HIP-3 股票永续 (xyz: 市场, 105 个) 24/7 交易。查 fundingHistory 得 xyz:LITE 关键数据：08-01 20:00 ET funding -0.0162%/h (-142%/yr) → 23:00 +0.0338%/h (+296%/yr)，3 小时暴力翻转。周六晚美股休市，永续照常交易——这个单边注入就是"老鼠仓指纹"：资金在消息公开前抢跑建仓，被 funding 机制投影出来 (funding ≈ f(永续对现货溢价)，多头拥挤时飙高)。对比 SKHX 常年 200-350%/yr 高 funding (套利型)，LITE 是突发异动型 (信号型)——平时 25-45%/yr，周末突然爆表。

真相：路透 8/4 报道 (四位知情人士)：特朗普政府起草禁令，拟禁止美国实体进口中国生产的新型数据中心组件 (含光模块)。制裁 → LITE 利好：中国光模块厂 (新易盛、中际旭创、光迅) 美国市场被切断 → 订单转移美国本土厂商 (Lumentum/Coherent/Fabrinet)。A 股光模块被杀估值，美股 LITE 被买爆——同一政策两面。"老鼠"即政策起草期接触文件的知情资金：周末政策文件成型，拿到确定性必须立刻建仓，美股休市，Hyperliquid 永续是当时唯一变现通道。

时间线：7 月底禁令起草 → 08-01 周六 23:00 链上指纹 (+296%/yr) → 08-03 周一 LITE +9.24% → 08-04 路透报道、盘前 +11.91%、收 +9.94% (\$857.42) → 溢价归零、funding 正常化。5 个交易日 +42%，机会公开即兑现。

方法论：① 链上是内幕信息泄密通道：无 KYC、24/7、高杠杆，但所有行为透明，funding/OI 异动当场自曝；政策事件周末发酵 → 周末链上异动 = 最高价值信号。② 指纹特征：周末 (美股休市) + 冷门股票永续 + funding 量级骤变/符号翻转。③ 交叉验证：链上异动 → 股票 K 线 → 新闻事件三源对照，时间线闭合才算验证完成。④ 数据陷阱：universe/ctx 数组按索引对应 (zip 盲配对会标反)；快照是时点数据；部分 API 对 xyz 市场口径不可靠需换源。

下一步：监控 105 个 xyz 市场 funding 异动 (政策敏感行业：光互联/半导体/存储)；候选：xyz 在线的美国受益股永续 (AVGO、MU、NVDA、AMD、TSM、ASML、AMAT；COHR/CIEN/GLW/AAOI 尚未上线)；事件驱动类信号比财报类更值得跟踪。

笔记 47

作者：dami.eth | 时间：2026-08-05 13:32 UTC | 字数：1227

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 1 | “币有” Meme 机制复盘：从预测热点转向识别流动性瓶颈

今天复盘了“币有” Meme 爆发背后的交易路径与流动性传导机制。

核心结论

不要试图寻找“下一只会爆发的 Meme”，而应提前识别：哪个非稳定报价资产会在 Meme 需求爆发时，成为整条交易路径的流动性瓶颈。

Meme 是否爆发很难预测，但它必须经过 SPYB 这一交易路径，却已经提前体现在币安 Meme Rush 的 pairAnchorAddress

和代币标签中。因此，真正可以提前研究和监控的，不是热点本身，而是交易拓扑与流动性约束。

第一性原理：价格是滞后指标

用户看起来是在用 USDT 买“币有”，实际交易路径是：

USDT → SPYB → 币有

因此，对“币有”的买入需求会先转化为对 SPYB 的瞬时需求。当需求速度超过 SPYB/稳定币市场的补货能力时——包括 AMM 深度、RFQ 容量和 CEX 可搬运库存——SPYB 就会出现脱锚。

换句话说，机会形成的条件是：

Meme 净买入流量 > SPYB/USDT 的 AMM 深度 + RFQ 容量 + CEX 可搬运库存

两个关键指标

相比“SPYB 已经涨了多少”，更值得跟踪的是报价资产的压力系数：

```
[
Pressure_Q =
frac{sum_{\text{以 Q 为报价币的 Meme}}(\text{Buy}-\text{Sell})}
{\text{Q 在可接受滑点内的稳定币深度}}
]
```

Pressure_Q 越高，说明聚合净买入需求相对于可用流动性越强，报价资产越可能成为瓶颈。

进一步可以估算“流动性耗尽时间”：

```
[
TTE_Q =
frac{\text{可执行稳定币深度}}
{\text{报价资产净需求速度}}
]
```

TTE_Q 衡量的是：按照当前净需求速度，可执行流动性还能支撑多久。

执行判断

交易机会应由系统响应时间决定，而不是由固定涨幅决定。假设部署 LP 需要 2 分钟：

当 $TTE_Q < 2$ 分钟时，流动性瓶颈正在形成，机会已经进入可执行窗口。

当 SPYB 已经溢价 20% 时，发现阶段通常已经结束，此时再入场更接近追逐价格。

下一步验证

1. 根据 pairAnchorAddress 和代币标签，建立“报价资产 → Meme 代币”的实时映射。
2. 聚合同一报价资产下所有 Meme 的净买入速度。
3. 按可接受滑点统计 AMM 深度，并叠加 RFQ 容量和 CEX 可搬运库存。
4. 实时计算 Pressure_Q 与 TTE_Q，以实际部署延迟作为告警阈值。

今天最大的认知更新是：Meme

不可预测，但它流经哪里、会消耗哪一层流动性，往往可以提前知道。与其预测价格，不如监控拓扑中的瓶颈。

笔记 48

作者：xzone911 | 时间：2026-08-06 13:15 UTC | 字数：1191

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 | 2026.08.05 | 跨链价差实证：2小时×3资产×5链全扫描

今日目标

用已搭建的工具链实测跨链套利可行性——不是看概念，是跑数据、看价差、算桥费、判断能不能赚钱。

方法

- 选WETH/WBTC/LINK三个主流跨链资产
- 在Ethereum/Arbitrum/Base/Polygon/Optimism五条链上抓DEX现货价（DexScreener）
- 每分钟采样一次，持续2小时，共117组数据点
- 对照LI.FI桥接报价，计算真实净收益

结果

WETH — 定价高效，几乎无空间

- 5链价差均值：0.19%（范围0.07%~0.31%）
- 仅19%采样点超过0.25%桥费阈值
- LI.FI桥接可用，桥费~0.25%，净利润始终为负

WBTC — 价差实在，但桥不存在

- 5链价差均值：0.36%（范围0.24%~0.44%）
- 99%采样点超过0.25%，且持续扩大
- LI.FI全路由返回NO QUOTE——价差存在但无法执行

LINK — 发现窗口，但迅速关闭

- 4链价差：起始1.11%，峰值1.49%，结束时降到0.25%
- LI.FI桥接可用（Base→OP，桥费0.34%~0.66%）
- 理论最大净利+1.15%（窗口仅约1小时即被市场抹平）

| 时点 | DEX价差 | 桥费 | 理论净利 |

|-----|-----|-----|-----|

| 09:38开局 | 1.11% | 0.34% | +0.77% |

| 10:19峰值 | 1.49% | 0.34% | +1.15% |

| 11:35结束 | 0.25% | 0.34% | -0.09% |

同时验证

- GMGN热门币扫描：Solana是Meme赌场（无跨链可能），Base有中盘币但多为单链
- PAXG黄金代币：5链分布，价差5.5%但桥费极高

核心理解

1. Bruce成本公式被117组实测数据证实

净收益=DEX价差-桥费-Gas-滑点-执行延迟，三要素任一断裂就不成立。

2. 三种失败模式比成功更有价值

WETH（价差<桥费）、WBTC（有价差无桥）、LINK（窗口仅1小时），每种失败都揭示了套利的真实约束。

3. 执行能力才是套利的真正门槛

LINK的1小时窗口说明：能看见机会的人不少，能在一小时内完成买入→跨链桥接→卖出全流程的人才是真正拿到结果的人。这也是Bruce说的熊市建基础设施的意义。

下一步

- 写自动化扫描脚本持续监控多资产跨链价差
- 研究非桥依赖的套利形式（DEX内三角、同链多池）
- 接入更多桥协议扩大可执行路由

笔记 49

作者：Rui Pan | 时间：2026-08-06 14:31 UTC | 字数：1097

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、什么是 MEV

MEV 全称最大可提取价值，指矿工 / 验证者通过操纵区块内交易排序、插入额外交易，从区块链用户交易中赚取的额外收益，主流分为套利、三明治攻击（夹子攻击）两大类。

二、MEV 两大核心类型

（一）链上套利

核心逻辑：不同资金池、交易对间存在价格差，只要价差覆盖
费、手续费、滑点等全部交易成本，就能赚取利润

Gas

利润计算公式：利润 = 高价市场卖出价 - 低价市场买入价 - 全部交易成本

1. 简单跨市场套利

同一资产在不同交易场所报价不同，低价买入、高价卖出赚取差价。

例：BTC 在 A 平台 90000 美元，B 平台 91000 美元，同时完成买卖，扣除成本后赚价差。

2. 三角套利

通过三种及以上资产循环兑换，最终本金变多。

举例：Uniswap 汇率 ETH=2000 DAI，DAI=1 USDC，USDC=0.00055 ETH

投入 1ETH 兑换 2000DAI→2000USDC→1.1ETH，净赚 0.1ETH。

3. 闪电贷套利（当前主流高效套利方案）

闪电贷是 DeFi 无抵押借贷，可借出大额资产，但必须在同一区块内连本带息归还，否则整笔交易直接回滚失效。

优势：零本金撬动巨额资金放大套利收益，现在成熟 MEV 机器人基本都会搭配闪电贷使用。

示例：借出 100SOL 套利后得到 110SOL，支付 1SOL 利息，净赚 9SOL。

（二）三明治攻击（夹子攻击）

依托内存池机制实现：

1. 机器人监控内存池，捕捉普通用户大额挂单交易；
2. 机器人抬高 Gas 费，在用户交易前插入一笔买入交易，推高资产价格；
3. 用户只能以高价完成兑换；
4. 机器人再插入一笔后置卖出交易，抛售资产套现获利。

三、关键基础概念：内存池（Mempool）

内存池是区块链存放尚未上链确认交易的缓存池，是所有 MEV 机器人的情报来源：

1. 所有待打包交易先存入内存池，等待验证者打包进区块；
2. 交易优先级由 Gas 费决定：Gas 越高，打包上链速度越快；
3. MEV Bot 核心操作逻辑：实时监控内存池异动，发现获利机会后，抬高 Gas 抢占交易排序，抢先完成套利 / 夹子攻击；
4. 劣势：普通用户先发的交易，也会被高 Gas 机器人插队，错失价格优势甚至被夹。

笔记 50

作者：Freetrip | 时间：2026-08-06 13:59 UTC | 字数：892

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日成就：

LI.FI是一个类似于API，sdk的东西，它是一个聚合的报价/执行层，给的是扣费后真实到手的资金

GET <https://li.quest/v1/quote>

fromChain = 56 # BSC 链 ID

toChain = 56 # 同链 (56→56 就是不跨链，纯 swap)

fromToken = 0x55d3...7955 # BSC 上的 USDT (注意：BSC USDT 是 18 位小数，不是 6 位)

toToken = 0xbb4C...095c # WBNB

fromAmount = 10000000000000000000 # 1000 USDT，要写成最小单位 = 1000×10^{18}

fromAddress = 0xd8dA...6045 # 出金地址 (随便一个合法地址即可，只是报价不签名)

踩坑点：fromAmount 一定是整数最小单位 (wei 级)，先确认 token 的 decimals。BSC 的 USDT 是 18 位，Ethereum 上的 USDT 是 6 位——搞错就差 10^{12} 倍。

真正的交易成本 = 这三块

| 成本 | 这笔的值 | 性质 |

| :----- | :----- | :----- |

| 聚合器/协议费 (LI.FIFixed Fee) | 2.5 USDT (0.25%) | 明确收的 |

| Gas (矿工费) | 0.0000317 BNB $\approx$ \$0.019 | 单独用 BNB 付，不从换币产出里扣 |

| 价格冲击 priceimpact (+池子手续费) | 这笔 $\approx$ 0 (池深、量小) | 已含在 toAmount 报价里 |

笔记 51

作者：KuTear | 时间：2026-08-05 16:37 UTC | 字数：850

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 Day 3 打卡 (2026-08-06)

主题：实盘持仓风控巡检、全网套利机会评估与策略动态调优

一、实盘持仓巡检与收益表现

1. XIAOMI 跨所对冲策略 (Gate.io 空 $\leftrightarrow$ Lighter 多)：

- 运行状态：严格无裸露 Delta 对冲运行，安全级别极高。
- 当前收益：净浮盈 +\$5.70 USDT。
- 风控与执行：Astro 系统正在执行步进加仓逻辑，持续监听价差并等待收敛平仓。

2. BTC 单边杠杆持仓 (OKX)：

- 运行状态：2x 杠杆做多。
- 当前收益：浮亏 -\$0.49 USDT，整体风险控制安全阈值内。

二、全网套利机会扫描与风险评估

1. ON (Bybit $\leftrightarrow$ OKX)：

- 当前价差：0.10%。
- 决策结论：未覆盖开仓双边手续费与滑点损耗门槛 (需 $> 0.20\%$)，维持 Astro 自动化监听，暂不手动建仓。

2. GIGADEVICE (Bybit $\leftrightarrow$ Bitget)：

- 当前价差：表面价差 0.65%。
- 风险评估：Bitget 端订单簿深度极薄 (买一仅 2.23 股)，且跨交易所充提网络受限，属于典型流动性陷阱。
- 决策结论：在 Astro 系统中已执行 禁用开仓 (Disable Open)，规避流动性溢价风险。

三、策略与系统运行思考

1. 流动性与充提风控拦截：表面高价差往往对应充提暂停或极浅深度，交易引擎需在信号触发前强制校验 Level 2 深度与充提 API 状态。

2. 持仓管理原则：在全网大盘窄幅震荡、微小价差阶段，保持已有的低风险跨所对冲持仓 (XIAOMI)，等待价差收敛锁盈，优于频繁盲目开仓。

笔记 52

作者：Joe | 时间：2026-08-06 14:42 UTC | 字数：842
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 | 把“高 APR”还原成做市风险：MARS/USDT V3 案例

今天研究了 Oxphilo 展示的 MARS/USDT V3 「挖手续费」案例。核心结论：这不是协议补贴，也不是无风险套利，而是把资金放在当前价格附近，为高波动小币提供集中流动性并承担做市库存风险。

1. 高 APR 从哪里来

当时 PancakeSwap V3 的 MARS/USDT 0.25% 池约有 \$17.5 万流动性，却有约 \$725 万/24h 成交额、约 5 万笔交易，日换手约 41 倍。0.25% 交易费中约 68% 进入活跃 LP；粗算 LP 费池约为： $\$7.25m \times 0.25\% \times 68\% \approx \$12.3k/\text{天}$ 。

所以短期费率年化到数千 % 并不神秘，本质是「极高成交额 / 很少的近价活跃流动性」。

2. 截图的收益不能直接当利润

截图显示 total fees 约 \$2,536，但 total PnL 约 \$2,226；手续费之外已经出现价格偏离/无常损失。手续费也未必是纯 USDT，撤仓前都不是锁定利润。

3. 真正承担的风险

MARS 上涨时，LP 会持续卖出 MARS，吃不到完整上涨；MARS 下跌时，LP 会持续买入 MARS，跌出区间后会接近全仓 MARS 且停止赚费。更多 LP 进场会稀释份额，热点成交降温也会让 APR 很快下降。

此外该代币合约当前读到买卖税各 3%，并明确包含 anti-farmer 逻辑，因此不能按普通币对的 LP 收益率理解。

4. 今日可复用检查表

进入任何 V3 高 APR 池前，先核对：代币合约与税费、池子 fee split、TVL/成交额/活跃流动性份额、仓位上下沿、价格出区间后的库存、撤仓滑点与 gas，并用「总 PnL vs HODL」而不是 fee APR 做判断。

下一步：把这套检查表做成可复现的池子筛选与 PnL 监控模板；先观察/纸面验证，不做实盘追高。

笔记 53

作者：Joneloo | 时间：2026-08-06 15:19 UTC | 字数：828
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一句话：LI.FI 是一个「跨链流动性聚合与编排层」(routing & orchestration layer) ——不是桥、不是 DEX，而是把所有的桥、DEX 聚合器、意图求解器 (solver)、收益协议，统一成一个 API/SDK/Widget。

官网的关键表述：

- 核心卖点：Universal Market Access (通用市场准入) ——「No more market fragmentation」(终结市场碎片化)，跨 60+ 链、30+ 桥、30+ DEX 聚合器，一次接入。
- 数据背书：\$80B+ 累计转账量、100M+ 笔交易、1000+ 合作方。
- 定位：不是做市商，是路由和执行层，坐在「你的应用」和「碎片化的链上流动性」之间。

同链 vs 跨链 → 本质区别

同链是原子性 (一笔交易，失败整体回滚)；跨链是非原子 (桥成功、目标链 swap 可能失败 → 退回中间资产)。文档原文确认：跨链的桥可以成功而目标链 swap 失败，资金以桥接资产退回。这就是跨链套利「非原子风险」的官方背书。

同路线多桥竞争 (17 条路由实测)

ETH→ARB 10k USDC，/advanced/routes 返回 17 条

解读（这一条最核心）：

- 同样一条路线，17 个桥/路由报价从 9.975 到 9.875，差 1%——这就是「路由聚合」存在的意义：不聚合你很可能走成最差的那条（多亏 1%）。
- 榜首 (polymerStandard/eco) 和倒数 (celercircle) 都是 0.25% 官方费率，但到账差 1%——说明同费率下桥的底层结算价/滑点/流动性不同，聚合器算的是「到手多少钱」，不是「哪家手续费低」。
- gas 和费率脱钩：across gas 最低 (\$0.10) 但到账排第 6；squid 费率 1.07% 但到账排第 4——因为它的底层结算价好，总成本反而低。路由器选的是净到账，不是单项最优。

笔记 54

作者：pennchester | 时间：2026-08-06 00:27 UTC | 字数：814

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 2 — AMM 机制深度解析

学习内容

深入学习了 AMM 的数学基础：从 $xy=k$ 恒定乘积公式出发，推导了价格形成机制 ($P=y/x$)、Swap 输出计算、价格冲击精确公式 ($PI = \Delta x/(x+\Delta x)$)。对比了 Uniswap V2 全区间均匀流动性与 V3 集中流动性的本质差异，理解了 Tick 系统和不同费率层级的 tick spacing 设计。建立了多池路由的价格冲击叠加模型。

实操记录

- 用 Python 编写了完整的 AMM 模拟器，实现恒定乘积池的 swap、价格冲击计算、多池扫描和最优套利规模求解
- 跑通了 5 个演示场景：价格冲击曲线、池子深度对比、最优套利规模优化、多 DEX 扫描、V3 概念演示
- 数据验证：\$4M TVL 的池子，1 ETH 交易价格冲击仅 0.1%，但 10 ETH 冲击就达 1%；\$400K 的浅池 10 ETH 冲击高达 9%
- 多池扫描：6 个 DEX 中最大屏幕价差 1.08%，最优套利规模 4 ETH，净利润仅 \$36（扣除 Gas 后）

核心收获

1. 价格冲击 $= \Delta x/(x+\Delta x)$ ，是可以精确计算的——不是大概估计
2. 池子深度比价差更重要：浅池再大的价差也可能被价格冲击吃掉
3. 存在一个数学上的最优套利规模，由两边池子的流动性共同决定
4. V3 集中流动性让小额套利的滑点更小，但流动性分布不再均匀
5. 实操验证：屏幕价差 1.08%，实际能捕获的净利润仅 \$36——看到价差和赚到钱之间的鸿沟是实实在在的

遇到的问题

- V3 的多 tick 区间滑点需要链上合约查询，纯公式不够
- 演示脚本中池子参数是虚构的，需要接入真实链上数据

明日计划

Day 3：Gas 费与 MEV 专场 — EIP-1559 机制、L2 Gas 对比、MEV 攻击类型、Flashbots 保护